

SpaceHub

NIST 800-53 - Compliance report

Wed Mar 25 2026 16:40:26 GMT+0000 (Coordinated Universal Time)

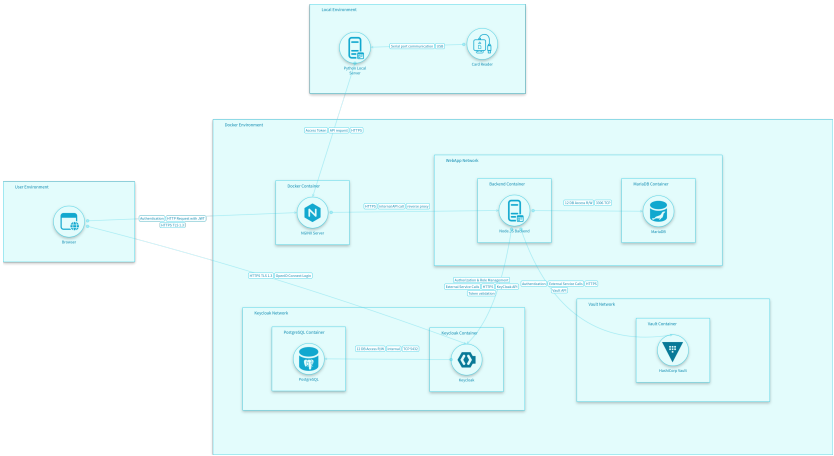
Project description:
Ciao ciao bla bla blob fns

Unique ID: spacehub-1774445306512

Owner: [Annarita Fabiano, Annarita Fabiano, Giuseppe Maglione]

Workflow state: In Progress

Tags: No tags



Compliance summary

See the countermeasure state data categorized by standards:

NIST 800-53: AC-3 ACCESS ENFORCEMENT Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: AC-10 CONCURRENT SESSION CONTROL Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: AC-23 Data Mining Protection Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant
NIST 800-53: AC-4 Information Flow Enforcement Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: AC-21 Information Sharing Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: AC-6 LEAST PRIVILEGE Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant
NIST 800-53: AC-1 POLICY AND PROCEDURES Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: AC-9 PREVIOUS LOGON NOTIFICATION Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: AC-22 Publicly Accessible Content Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant
NIST 800-53: AC-17 Remote Access Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: AC-8 SYSTEM USE NOTIFICATION Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: AC-7 UNSUCCESSFUL LOGON ATTEMPTS Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant
NIST 800-53: AC-20 Use of External Systems Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: AC-18 Wireless Access Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: AU-4 Audit Log Storage Capacity Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant

NIST 800-53: AU-12 Audit Record Generation			NIST 800-53: AU-7 Audit Record Reduction and Report Generation			NIST 800-53: AU-11 Audit Record Retention		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		2	Recommended		2	Recommended		2
Rejected		0	Rejected		0	Rejected		0
Non-compliant		2	Non-compliant		2	Non-compliant		2
NIST 800-53: AU-6 Audit Record Review, Analysis, and Reporting			NIST 800-53: AU-16 Cross-organizational Audit Logging			NIST 800-53: AU-1 Policy and Procedures		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		2	Recommended		2	Recommended		2
Rejected		0	Rejected		0	Rejected		0
Non-compliant		2	Non-compliant		2	Non-compliant		2
NIST 800-53: AU-5 Response to Audit Logging Process Failures			NIST 800-53: CA-3 Information Exchange			NIST 800-53: CA-9 Internal System Connections		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		2	Recommended		5	Recommended		5
Rejected		0	Rejected		0	Rejected		0
Non-compliant		2	Non-compliant		5	Non-compliant		5
NIST 800-53: CM-5 Access Restrictions for Change			NIST 800-53: CM-2 BASELINE CONFIGURATION			NIST 800-53: CM-3 Configuration Change Control		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		11	Recommended		9	Recommended		11
Rejected		0	Rejected		0	Rejected		0
Non-compliant		11	Non-compliant		9	Non-compliant		11
NIST 800-53: CM-9 Configuration Management Plan			NIST 800-53: CM-6 Configuration Settings			NIST 800-53: CM-13 Data Action Mapping		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		11	Recommended		9	Recommended		11
Rejected		0	Rejected		0	Rejected		0
Non-compliant		11	Non-compliant		9	Non-compliant		11

NIST 800-53: CM-4 Impact Analyses			NIST 800-53: CM-12 Information Location			NIST 800-53: CM-7 Least Functionality		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		11	Recommended		11	Recommended		11
Rejected		0	Rejected		0	Rejected		0
Non-compliant		11	Non-compliant		11	Non-compliant		11
NIST 800-53: CM-1 Policy and Procedures			NIST 800-53: CM-14 Signed Components			NIST 800-53: CM-10 Software Usage Restrictions		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		11	Recommended		11	Recommended		11
Rejected		0	Rejected		0	Rejected		0
Non-compliant		11	Non-compliant		11	Non-compliant		11
NIST 800-53: CM-8 System Component Inventory			NIST 800-53: CM-11 User-installed Software			NIST 800-53: CP-11 Alternate Communications Protocols		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		11	Recommended		11	Recommended		2
Rejected		0	Rejected		0	Rejected		0
Non-compliant		11	Non-compliant		11	Non-compliant		2
NIST 800-53: CP-7 Alternate Processing Site			NIST 800-53: CP-6 Alternate Storage Site			NIST 800-53: CP-13 Alternative Security Mechanisms		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		2	Recommended		2	Recommended		2
Rejected		0	Rejected		0	Rejected		0
Non-compliant		2	Non-compliant		2	Non-compliant		2
NIST 800-53: CP-2 Contingency Plan			NIST 800-53: CP-4 Contingency Plan Testing			NIST 800-53: CP-3 Contingency Training		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		2	Recommended		2	Recommended		2
Rejected		0	Rejected		0	Rejected		0
Non-compliant		2	Non-compliant		2	Non-compliant		2

NIST 800-53: CP-1 Policy and Procedures	NIST 800-53: CP-12 Safe Mode	NIST 800-53: CP-9 System Backup
Implemented 0	Implemented 0	Implemented 0
Required 0	Required 0	Required 0
Failed 0	Failed 0	Failed 0
Verified 0	Verified 0	Verified 0
Not-applicable 0	Not-applicable 0	Not-applicable 0
Recommended 2	Recommended 2	Recommended 2
Rejected 0	Rejected 0	Rejected 0
Non-compliant 2	Non-compliant 2	Non-compliant 2
NIST 800-53: CP-10 System Recovery and Reconstitution	NIST 800-53: CP-8 Telecommunications Services	NIST 800-53: IA-5 AUTHENTICATOR MANAGEMENT
Implemented 0	Implemented 0	Implemented 0
Required 0	Required 0	Required 0
Failed 0	Failed 0	Failed 0
Verified 0	Verified 0	Verified 0
Not-applicable 0	Not-applicable 0	Not-applicable 0
Recommended 2	Recommended 2	Recommended 3
Rejected 0	Rejected 0	Rejected 0
Non-compliant 2	Non-compliant 2	Non-compliant 3
NIST 800-53: IA-10 Adaptive Authentication	NIST 800-53: IA-3 Device Identification and Authentication	NIST 800-53: IA-8 Identification and Authentication (non-organizational Users)
Implemented 0	Implemented 0	Implemented 0
Required 0	Required 0	Required 0
Failed 0	Failed 0	Failed 0
Verified 0	Verified 0	Verified 0
Not-applicable 0	Not-applicable 0	Not-applicable 0
Recommended 3	Recommended 3	Recommended 3
Rejected 0	Rejected 0	Rejected 0
Non-compliant 3	Non-compliant 3	Non-compliant 3
NIST 800-53: IA-2 Identification and Authentication (organizational Users)	NIST 800-53: IA-4 Identifier Management	NIST 800-53: IA-12 Identity Proofing
Implemented 0	Implemented 0	Implemented 0
Required 0	Required 0	Required 0
Failed 0	Failed 0	Failed 0
Verified 0	Verified 0	Verified 0
Not-applicable 0	Not-applicable 0	Not-applicable 0
Recommended 3	Recommended 3	Recommended 3
Rejected 0	Rejected 0	Rejected 0
Non-compliant 3	Non-compliant 3	Non-compliant 3
NIST 800-53: IA-1 Policy and Procedures	NIST 800-53: IA-11 RE-AUTHENTICATION	NIST 800-53: PE-5 Access Control for Output Devices
Implemented 0	Implemented 0	Implemented 0
Required 0	Required 0	Required 0
Failed 0	Failed 0	Failed 0
Verified 0	Verified 0	Verified 0
Not-applicable 0	Not-applicable 0	Not-applicable 0
Recommended 3	Recommended 3	Recommended 1
Rejected 0	Rejected 0	Rejected 0
Non-compliant 3	Non-compliant 3	Non-compliant 1

NIST 800-53: PE-4 Access Control for Transmission			NIST 800-53: PE-17 Alternate Work Site			NIST 800-53: PE-20 Asset Monitoring and Tracking		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		1	Recommended		1	Recommended		1
Rejected		0	Rejected		0	Rejected		0
Non-compliant		1	Non-compliant		1	Non-compliant		1
NIST 800-53: PE-22 Component Marking			NIST 800-53: PE-16 Delivery and Removal			NIST 800-53: PE-21 Electromagnetic Pulse Protection		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		1	Recommended		1	Recommended		1
Rejected		0	Rejected		0	Rejected		0
Non-compliant		1	Non-compliant		1	Non-compliant		1
NIST 800-53: PE-12 Emergency Lighting			NIST 800-53: PE-11 Emergency Power			NIST 800-53: PE-10 Emergency Shutoff		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		1	Recommended		1	Recommended		1
Rejected		0	Rejected		0	Rejected		0
Non-compliant		1	Non-compliant		1	Non-compliant		1
NIST 800-53: PE-14 Environmental Controls			NIST 800-53: PE-23 Facility Location			NIST 800-53: PE-13 Fire Protection		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		1	Recommended		1	Recommended		1
Rejected		0	Rejected		0	Rejected		0
Non-compliant		1	Non-compliant		1	Non-compliant		1
NIST 800-53: PE-19 Information Leakage			NIST 800-53: PE-18 Location of System Components			NIST 800-53: PE-6 Monitoring Physical Access		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		1	Recommended		1	Recommended		1
Rejected		0	Rejected		0	Rejected		0
Non-compliant		1	Non-compliant		1	Non-compliant		1

NIST 800-53: PE-2 Physical Access Authorizations Implemented 0 Required 0 Failed 0 Verified 0 Not-applicable 0 Recommended 1 Rejected 0 Non-compliant 1	NIST 800-53: PE-3 Physical Access Control Implemented 0 Required 0 Failed 0 Verified 0 Not-applicable 0 Recommended 1 Rejected 0 Non-compliant 1	NIST 800-53: PE-1 Policy and Procedures Implemented 0 Required 0 Failed 0 Verified 0 Not-applicable 0 Recommended 1 Rejected 0 Non-compliant 1
NIST 800-53: PE-9 Power Equipment and Cabling Implemented 0 Required 0 Failed 0 Verified 0 Not-applicable 0 Recommended 1 Rejected 0 Non-compliant 1	NIST 800-53: PE-8 Visitor Access Records Implemented 0 Required 0 Failed 0 Verified 0 Not-applicable 0 Recommended 1 Rejected 0 Non-compliant 1	NIST 800-53: PE-15 Water Damage Protection Implemented 0 Required 0 Failed 0 Verified 0 Not-applicable 0 Recommended 1 Rejected 0 Non-compliant 1
NIST 800-53: PM-21 Accounting of Disclosures Implemented 0 Required 0 Failed 0 Verified 0 Not-applicable 0 Recommended 1 Rejected 0 Non-compliant 1	NIST 800-53: PM-10 Authorization Process Implemented 0 Required 0 Failed 0 Verified 0 Not-applicable 0 Recommended 2 Rejected 0 Non-compliant 2	NIST 800-53: PM-24 Data Integrity Board Implemented 0 Required 0 Failed 0 Verified 0 Not-applicable 0 Recommended 1 Rejected 0 Non-compliant 1
NIST 800-53: PM-20 Dissemination of Privacy Program Information Implemented 0 Required 0 Failed 0 Verified 0 Not-applicable 0 Recommended 1 Rejected 0 Non-compliant 1	NIST 800-53: PM-25 Minimization of Personally Identifiable Information Used in Testing, Training, and Research Implemented 0 Required 0 Failed 0 Verified 0 Not-applicable 0 Recommended 1 Rejected 0 Non-compliant 1	NIST 800-53: PM-22 Personally Identifiable Information Quality Management Implemented 0 Required 0 Failed 0 Verified 0 Not-applicable 0 Recommended 1 Rejected 0 Non-compliant 1
NIST 800-53: PM-19 Privacy Program Leadership Role Implemented 0 Required 0 Failed 0 Verified 0 Not-applicable 0 Recommended 1 Rejected 0 Non-compliant 1	NIST 800-53: PM-18 Privacy Program Plan Implemented 0 Required 0 Failed 0 Verified 0 Not-applicable 0 Recommended 1 Rejected 0 Non-compliant 1	NIST 800-53: PM-27 Privacy Reporting Implemented 0 Required 0 Failed 0 Verified 0 Not-applicable 0 Recommended 1 Rejected 0 Non-compliant 1

NIST 800-53: PS-6 Access Agreements Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: PS-7 External Personnel Security Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: PS-8 Personnel Sanctions Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant
NIST 800-53: PS-3 Personnel Screening Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: PS-4 Personnel Termination Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: PS-5 Personnel Transfer Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant
NIST 800-53: PS-1 Policy and Procedures Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: PS-9 Position Descriptions Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: PS-2 Position Risk Designation Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant
NIST 800-53: PT-2 Authority to Process Personally Identifiable Information Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: PT-8 Computer Matching Requirements Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: PT-4 Consent Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant
NIST 800-53: PT-3 Personally Identifiable Information Processing Purposes Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: PT-1 Policy and Procedures Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant	NIST 800-53: PT-5 Privacy Notice Implemented Required Failed Verified Not-applicable Recommended Rejected Non-compliant

NIST 800-53: PT-7 Specific Categories of Personally Identifiable Information		
Implemented		0
Required		0
Failed		0
Verified		0
Not-applicable		0
Recommended		1
Rejected		0
Non-compliant		1

NIST 800-53: PT-6 System of Records Notice		
Implemented		0
Required		0
Failed		0
Verified		0
Not-applicable		0
Recommended		1
Rejected		0
Non-compliant		1

NIST 800-53: SA-10 Developer Configuration Management		
Implemented		0
Required		0
Failed		0
Verified		0
Not-applicable		0
Recommended		16
Rejected		0
Non-compliant		16

NIST 800-53: SA-21 Developer Screening		
Implemented		0
Required		0
Failed		0
Verified		0
Not-applicable		0
Recommended		8
Rejected		0
Non-compliant		8

NIST 800-53: SA-15 Development Process, Standards, and Tools		
Implemented		0
Required		0
Failed		0
Verified		0
Not-applicable		0
Recommended		2
Rejected		0
Non-compliant		2

NIST 800-53: SA-8 Security and Privacy Engineering Principles		
Implemented		0
Required		0
Failed		0
Verified		0
Not-applicable		0
Recommended		2
Rejected		0
Non-compliant		2

NIST 800-53: SA-3 System Development Life Cycle		
Implemented		0
Required		0
Failed		0
Verified		0
Not-applicable		0
Recommended		2
Rejected		0
Non-compliant		2

NIST 800-53: SC-22 Architecture and Provisioning for Name/address Resolution Service		
Implemented		0
Required		0
Failed		0
Verified		0
Not-applicable		0
Recommended		1
Rejected		0
Non-compliant		1

NIST 800-53: SC-7 Boundary Protection		
Implemented		0
Required		0
Failed		0
Verified		0
Not-applicable		0
Recommended		1
Rejected		0
Non-compliant		1

NIST 800-53: SC-12 CRYPTOGRAPHIC KEY ESTABLISHMENT AND MANAGEMENT		
Implemented		0
Required		0
Failed		0
Verified		0
Not-applicable		0
Recommended		4
Rejected		0
Non-compliant		4

NIST 800-53: SC-13 Cryptographic Protection		
Implemented		0
Required		0
Failed		0
Verified		0
Not-applicable		0
Recommended		4
Rejected		0
Non-compliant		4

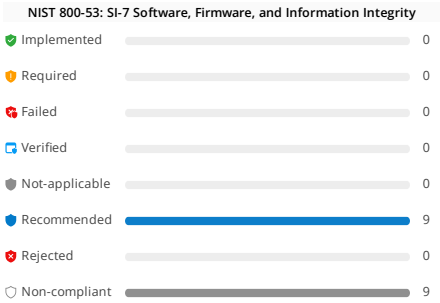
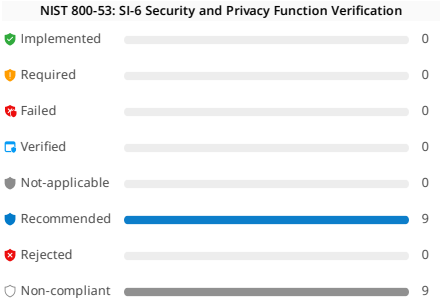
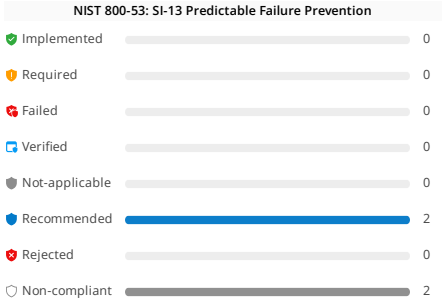
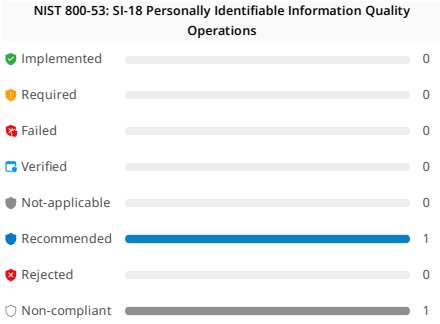
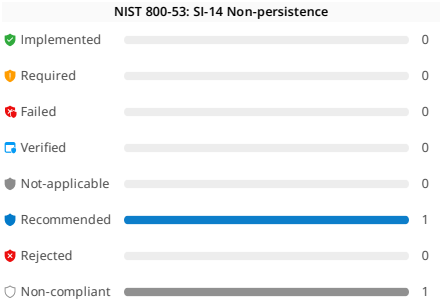
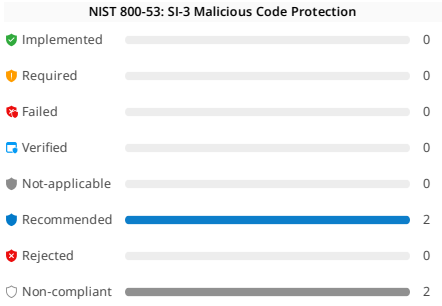
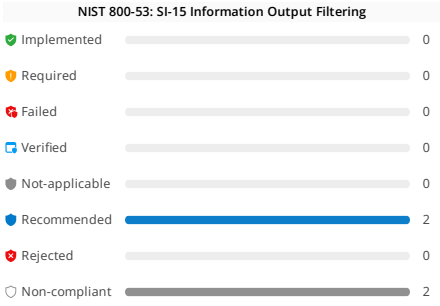
NIST 800-53: SC-5 DENIAL-OF-SERVICE PROTECTION		
Implemented		0
Required		0
Failed		0
Verified		0
Not-applicable		0
Recommended		2
Rejected		0
Non-compliant		2

NIST 800-53: SC-44 Detonation Chambers		
Implemented		0
Required		0
Failed		0
Verified		0
Not-applicable		0
Recommended		1
Rejected		0
Non-compliant		1

NIST 800-53: SC-18 Mobile Code		
Implemented		0
Required		0
Failed		0
Verified		0
Not-applicable		0
Recommended		1
Rejected		0
Non-compliant		1

NIST 800-53: SC-28 PROTECTION OF INFORMATION AT REST		
Implemented		0
Required		0
Failed		0
Verified		0
Not-applicable		0
Recommended		10
Rejected		0
Non-compliant		10

NIST 800-53: SC-39 Process Isolation			NIST 800-53: SC-17 Public Key Infrastructure Certificates			NIST 800-53: SC-6 Resource Availability		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		1	Recommended		4	Recommended		8
Rejected		0	Rejected		0	Rejected		0
Non-compliant		1	Non-compliant		4	Non-compliant		8
NIST 800-53: SC-23 SESSION AUTHENTICITY			NIST 800-53: SC-3 Security Function Isolation			NIST 800-53: SC-32 System Partitioning		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		3	Recommended		1	Recommended		1
Rejected		0	Rejected		0	Rejected		0
Non-compliant		3	Non-compliant		1	Non-compliant		1
NIST 800-53: SC-8 TRANSMISSION CONFIDENTIALITY AND INTEGRITY			NIST 800-53: SC-16 Transmission of Security and Privacy Attributes			NIST 800-53: SC-11 Trusted Path		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		5	Recommended		5	Recommended		5
Rejected		0	Rejected		0	Rejected		0
Non-compliant		5	Non-compliant		5	Non-compliant		5
NIST 800-53: SI-19 De-identification			NIST 800-53: SI-11 Error Handling			NIST 800-53: SI-2 Flaw Remediation		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		1	Recommended		2	Recommended		7
Rejected		0	Rejected		0	Rejected		0
Non-compliant		1	Non-compliant		2	Non-compliant		7
NIST 800-53: SI-10 INFORMATION INPUT VALIDATION			NIST 800-53: SI-22 Information Diversity			NIST 800-53: SI-23 Information Fragmentation		
Implemented		0	Implemented		0	Implemented		0
Required		0	Required		0	Required		0
Failed		0	Failed		0	Failed		0
Verified		0	Verified		0	Verified		0
Not-applicable		0	Not-applicable		0	Not-applicable		0
Recommended		1	Recommended		1	Recommended		1
Rejected		0	Rejected		0	Rejected		0
Non-compliant		1	Non-compliant		1	Non-compliant		1



Content menu

NIST 800-53: PS-6 Access Agreements	
Non-compliant	
Recommended	
NIST 800-53: PS-7 External Personnel Security	
Non-compliant	
Recommended	
NIST 800-53: PS-8 Personnel Sanctions	
Non-compliant	
Recommended	
NIST 800-53: PS-3 Personnel Screening	
Non-compliant	
Recommended	
NIST 800-53: PS-4 Personnel Termination	
Non-compliant	
Recommended	
NIST 800-53: PS-5 Personnel Transfer	
Non-compliant	
Recommended	
NIST 800-53: PS-1 Policy and Procedures	
Non-compliant	
Recommended	
NIST 800-53: PS-9 Position Descriptions	
Non-compliant	
Recommended	
NIST 800-53: PS-2 Position Risk Designation	
Non-compliant	
Recommended	
NIST 800-53: SA-21 Developer Screening	
Non-compliant	
Recommended	
NIST 800-53: SI-6 Security and Privacy Function Verification	
Non-compliant	
Recommended	
NIST 800-53: SI-7 Software, Firmware, and Information Integrity	
Non-compliant	
Recommended	
NIST 800-53: CM-5 Access Restrictions for Change	
Non-compliant	
Recommended	
NIST 800-53: CM-3 Configuration Change Control	
Non-compliant	
Recommended	
NIST 800-53: CM-9 Configuration Management Plan	
Non-compliant	
Recommended	
NIST 800-53: CM-13 Data Action Mapping	
Non-compliant	
Recommended	
NIST 800-53: CM-4 Impact Analyses	
Non-compliant	
Recommended	
NIST 800-53: CM-12 Information Location	
Non-compliant	
Recommended	
NIST 800-53: CM-7 Least Functionality	
Non-compliant	
Recommended	
NIST 800-53: CM-1 Policy and Procedures	
Non-compliant	
Recommended	
NIST 800-53: CM-14 Signed Components	
Non-compliant	
Recommended	
NIST 800-53: CM-10 Software Usage Restrictions	
Non-compliant	
Recommended	
NIST 800-53: CM-8 System Component Inventory	
Non-compliant	
Recommended	
NIST 800-53: CM-11 User-installed Software	

Non-compliant
Recommended
NIST 800-53: SA-10 Developer Configuration Management
Non-compliant
Recommended
NIST 800-53: CM-2 BASELINE CONFIGURATION
Non-compliant
Recommended
NIST 800-53: CM-6 Configuration Settings
Non-compliant
Recommended
NIST 800-53: SC-6 Resource Availability
Non-compliant
Recommended
NIST 800-53: SC-28 PROTECTION OF INFORMATION AT REST
Non-compliant
Recommended
NIST 800-53: AC-6 LEAST PRIVILEGE
Non-compliant
Recommended
NIST 800-53: SI-15 Information Output Filtering
Non-compliant
Recommended
NIST 800-53: SI-2 Flaw Remediation
Non-compliant
Recommended
NIST 800-53: SC-18 Mobile Code
Non-compliant
Recommended
NIST 800-53: SC-12 CRYPTOGRAPHIC KEY ESTABLISHMENT AND MANAGEMENT
Non-compliant
Recommended
NIST 800-53: SC-13 Cryptographic Protection
Non-compliant
Recommended
NIST 800-53: SC-17 Public Key Infrastructure Certificates
Non-compliant
Recommended
NIST 800-53: SC-44 Detonation Chambers
Non-compliant
Recommended
NIST 800-53: SC-39 Process Isolation
Non-compliant
Recommended
NIST 800-53: SC-3 Security Function Isolation
Non-compliant
Recommended
NIST 800-53: SC-32 System Partitioning
Non-compliant
Recommended
NIST 800-53: SI-22 Information Diversity
Non-compliant
Recommended
NIST 800-53: SI-23 Information Fragmentation
Non-compliant
Recommended
NIST 800-53: SI-14 Non-persistence
Non-compliant
Recommended
NIST 800-53: AC-23 Data Mining Protection
Non-compliant
Recommended
NIST 800-53: AC-4 Information Flow Enforcement
Non-compliant
Recommended
NIST 800-53: AC-21 Information Sharing
Non-compliant
Recommended
NIST 800-53: AC-1 POLICY AND PROCEDURES
Non-compliant
Recommended
NIST 800-53: AC-22 Publicly Accessible Content

Non-compliant	
Recommended	
NIST 800-53: AC-20 Use of External Systems	
Non-compliant	
Recommended	
NIST 800-53: PM-10 Authorization Process	
Non-compliant	
Recommended	
NIST 800-53: AC-17 Remote Access	
Non-compliant	
Recommended	
NIST 800-53: AC-18 Wireless Access	
Non-compliant	
Recommended	
NIST 800-53: CA-3 Information Exchange	
Non-compliant	
Recommended	
NIST 800-53: CA-9 Internal System Connections	
Non-compliant	
Recommended	
NIST 800-53: SC-8 TRANSMISSION CONFIDENTIALITY AND INTEGRITY	
Non-compliant	
Recommended	
NIST 800-53: SC-16 Transmission of Security and Privacy Attributes	
Non-compliant	
Recommended	
NIST 800-53: SC-11 Trusted Path	
Non-compliant	
Recommended	
NIST 800-53: AC-9 PREVIOUS LOGON NOTIFICATION	
Non-compliant	
Recommended	
NIST 800-53: AC-8 SYSTEM USE NOTIFICATION	
Non-compliant	
Recommended	
NIST 800-53: AC-7 UNSUCCESSFUL LOGON ATTEMPTS	
Non-compliant	
Recommended	
NIST 800-53: IA-5 AUTHENTICATOR MANAGEMENT	
Non-compliant	
Recommended	
NIST 800-53: IA-10 Adaptive Authentication	
Non-compliant	
Recommended	
NIST 800-53: IA-3 Device Identification and Authentication	
Non-compliant	
Recommended	
NIST 800-53: IA-8 Identification and Authentication (non-organizational Users)	
Non-compliant	
Recommended	
NIST 800-53: IA-2 Identification and Authentication (organizational Users)	
Non-compliant	
Recommended	
NIST 800-53: IA-4 Identifier Management	
Non-compliant	
Recommended	
NIST 800-53: IA-12 Identity Proofing	
Non-compliant	
Recommended	
NIST 800-53: IA-1 Policy and Procedures	
Non-compliant	
Recommended	
NIST 800-53: PE-5 Access Control for Output Devices	
Non-compliant	
Recommended	
NIST 800-53: PE-4 Access Control for Transmission	
Non-compliant	
Recommended	
NIST 800-53: PE-17 Alternate Work Site	
Non-compliant	
Recommended	

NIST 800-53: PE-20 Asset Monitoring and Tracking	Non-compliant	Recommended
NIST 800-53: PE-22 Component Marking	Non-compliant	Recommended
NIST 800-53: PE-16 Delivery and Removal	Non-compliant	Recommended
NIST 800-53: PE-21 Electromagnetic Pulse Protection	Non-compliant	Recommended
NIST 800-53: PE-12 Emergency Lighting	Non-compliant	Recommended
NIST 800-53: PE-11 Emergency Power	Non-compliant	Recommended
NIST 800-53: PE-10 Emergency Shutoff	Non-compliant	Recommended
NIST 800-53: PE-14 Environmental Controls	Non-compliant	Recommended
NIST 800-53: PE-23 Facility Location	Non-compliant	Recommended
NIST 800-53: PE-13 Fire Protection	Non-compliant	Recommended
NIST 800-53: PE-19 Information Leakage	Non-compliant	Recommended
NIST 800-53: PE-18 Location of System Components	Non-compliant	Recommended
NIST 800-53: PE-6 Monitoring Physical Access	Non-compliant	Recommended
NIST 800-53: PE-2 Physical Access Authorizations	Non-compliant	Recommended
NIST 800-53: PE-3 Physical Access Control	Non-compliant	Recommended
NIST 800-53: PE-1 Policy and Procedures	Non-compliant	Recommended
NIST 800-53: PE-9 Power Equipment and Cabling	Non-compliant	Recommended
NIST 800-53: PE-8 Visitor Access Records	Non-compliant	Recommended
NIST 800-53: PE-15 Water Damage Protection	Non-compliant	Recommended
NIST 800-53: AC-3 ACCESS ENFORCEMENT	Non-compliant	Recommended
NIST 800-53: AC-10 CONCURRENT SESSION CONTROL	Non-compliant	Recommended
NIST 800-53: IA-11 RE-AUTHENTICATION	Non-compliant	Recommended
NIST 800-53: SC-23 SESSION AUTHENTICITY	Non-compliant	Recommended
NIST 800-53: SA-15 Development Process, Standards, and Tools	Non-compliant	Recommended

NIST 800-53: SA-8 Security and Privacy Engineering Principles

Non-compliant

Recommended

NIST 800-53: SA-3 System Development Life Cycle

Non-compliant

Recommended

NIST 800-53: SI-3 Malicious Code Protection

Non-compliant

Recommended

NIST 800-53: SC-5 DENIAL-OF-SERVICE PROTECTION

Non-compliant

Recommended

NIST 800-53: AU-4 Audit Log Storage Capacity

Non-compliant

Recommended

NIST 800-53: AU-12 Audit Record Generation

Non-compliant

Recommended

NIST 800-53: AU-7 Audit Record Reduction and Report Generation

Non-compliant

Recommended

NIST 800-53: AU-11 Audit Record Retention

Non-compliant

Recommended

NIST 800-53: AU-6 Audit Record Review, Analysis, and Reporting

Non-compliant

Recommended

NIST 800-53: AU-16 Cross-organizational Audit Logging

Non-compliant

Recommended

NIST 800-53: AU-1 Policy and Procedures

Non-compliant

Recommended

NIST 800-53: AU-5 Response to Audit Logging Process Failures

Non-compliant

Recommended

NIST 800-53: SI-11 Error Handling

Non-compliant

Recommended

NIST 800-53: CP-11 Alternate Communications Protocols

Non-compliant

Recommended

NIST 800-53: CP-7 Alternate Processing Site

Non-compliant

Recommended

NIST 800-53: CP-6 Alternate Storage Site

Non-compliant

Recommended

NIST 800-53: CP-13 Alternative Security Mechanisms

Non-compliant

Recommended

NIST 800-53: CP-2 Contingency Plan

Non-compliant

Recommended

NIST 800-53: CP-4 Contingency Plan Testing

Non-compliant

Recommended

NIST 800-53: CP-3 Contingency Training

Non-compliant

Recommended

NIST 800-53: CP-1 Policy and Procedures

Non-compliant

Recommended

NIST 800-53: CP-12 Safe Mode

Non-compliant

Recommended

NIST 800-53: CP-9 System Backup

Non-compliant

Recommended

NIST 800-53: CP-10 System Recovery and Reconstitution

Non-compliant

Recommended

NIST 800-53: CP-8 Telecommunications Services	Non-compliant
	Recommended
NIST 800-53: SI-13 Predictable Failure Prevention	Non-compliant
	Recommended
NIST 800-53: PM-21 Accounting of Disclosures	Non-compliant
	Recommended
NIST 800-53: PM-24 Data Integrity Board	Non-compliant
	Recommended
NIST 800-53: PM-20 Dissemination of Privacy Program Information	Non-compliant
	Recommended
NIST 800-53: PM-25 Minimization of Personally Identifiable Information Used in Testing, Training, and Research	Non-compliant
	Recommended
NIST 800-53: PM-22 Personally Identifiable Information Quality Management	Non-compliant
	Recommended
NIST 800-53: PM-19 Privacy Program Leadership Role	Non-compliant
	Recommended
NIST 800-53: PM-18 Privacy Program Plan	Non-compliant
	Recommended
NIST 800-53: PM-27 Privacy Reporting	Non-compliant
	Recommended
NIST 800-53: PT-2 Authority to Process Personally Identifiable Information	Non-compliant
	Recommended
NIST 800-53: PT-8 Computer Matching Requirements	Non-compliant
	Recommended
NIST 800-53: PT-4 Consent	Non-compliant
	Recommended
NIST 800-53: PT-3 Personally Identifiable Information Processing Purposes	Non-compliant
	Recommended
NIST 800-53: PT-1 Policy and Procedures	Non-compliant
	Recommended
NIST 800-53: PT-5 Privacy Notice	Non-compliant
	Recommended
NIST 800-53: PT-7 Specific Categories of Personally Identifiable Information	Non-compliant
	Recommended
NIST 800-53: PT-6 System of Records Notice	Non-compliant
	Recommended
NIST 800-53: SI-19 De-identification	Non-compliant
	Recommended
NIST 800-53: SI-18 Personally Identifiable Information Quality Operations	Non-compliant
	Recommended
NIST 800-53: SC-22 Architecture and Provisioning for Name/address Resolution Service	Non-compliant
	Recommended
NIST 800-53: SC-7 Boundary Protection	Non-compliant
	Recommended
NIST 800-53: SI-10 INFORMATION INPUT VALIDATION	Non-compliant
	Recommended

NIST 800-53: PS-6 Access Agreements

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds

C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds

C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds

C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds

C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

 - [Dockerfile Best Practices](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

 - [Dockerfile Best Practices](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

 - [Dockerfile Best Practices](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

 - [Dockerfile Best Practices](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: User Environment

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: Docker Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: Keycloak Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: PostgreSQL Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: Vault Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: Backend Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: MariaDB Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

NIST 800-53: PS-7 External Personnel Security

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

 - [Dockerfile Best Practices](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

 - [Dockerfile Best Practices](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

 - [Dockerfile Best Practices](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:

- [Dockerfile Best Practices](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: User Environment

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: Docker Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: Keycloak Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: PostgreSQL Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: Vault Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: Backend Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: MariaDB Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

NIST 800-53: PS-8 Personnel Sanctions

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best

practices.
References:
• [Dockerfile Best Practices](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:
• [Dockerfile Best Practices](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:
• [Dockerfile Best Practices](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:
• [Dockerfile Best Practices](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:
• [Dockerfile Best Practices](#)

Component: Vault Container

1 Recommended countermeasures

[Rec1. Apply the principle of least privilege in your image builds](#) C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: Backend Container

1 Recommended countermeasures

[Rec1. Apply the principle of least privilege in your image builds](#) C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: MariaDB Container

1 Recommended countermeasures

[Rec1. Apply the principle of least privilege in your image builds](#) C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

[Rec1. Apply the principle of least privilege in your image builds](#) C-DOCKER-CONTAINER-CNT-02

Medium

Component: User Environment

[Rec1. Apply the principle of least privilege in your image builds](#) C-DOCKER-CONTAINER-CNT-02

Medium

Component: Docker Container

[Rec1. Apply the principle of least privilege in your image builds](#) C-DOCKER-CONTAINER-CNT-02

Medium

Component: Keycloak Container

[Rec1. Apply the principle of least privilege in your image builds](#) C-DOCKER-CONTAINER-CNT-02

Medium

Component: PostgreSQL Container

[Rec1. Apply the principle of least privilege in your image builds](#) C-DOCKER-CONTAINER-CNT-02

Medium

Component: Vault Container

[Rec1. Apply the principle of least privilege in your image builds](#) C-DOCKER-CONTAINER-CNT-02

Medium

Component: Backend Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: MariaDB Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

NIST 800-53: PS-3 Personnel Screening

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment 1 Recommended countermeasures Rec1. Apply the principle of least privilege in your image buildsC-DOCKER-CONTAINER-CNT-02MediumNot tested State: Recommended Description: Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively: Define and Use Non-Root Users: Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources. Minimize Package Installations: Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities. Drop Unnecessary Capabilities: Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation. Implement Automated Security Scanning: Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment. Document and Review Build Processes: Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices. References: Dockerfile Best Practices
Component: User Environment 1 Recommended countermeasures Rec1. Apply the principle of least privilege in your image buildsC-DOCKER-CONTAINER-CNT-02MediumNot tested State: Recommended Description: Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively: Define and Use Non-Root Users: Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources. Minimize Package Installations: Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities. Drop Unnecessary Capabilities: Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation. Implement Automated Security Scanning: Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment. Document and Review Build Processes: Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices. References: Dockerfile Best Practices
Component: Docker Container 1 Recommended countermeasures Rec1. Apply the principle of least privilege in your image buildsC-DOCKER-CONTAINER-CNT-02MediumNot tested State: Recommended Description: Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively: Define and Use Non-Root Users: Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources. Minimize Package Installations: Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities. Drop Unnecessary Capabilities: Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation. Implement Automated Security Scanning: Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment. Document and Review Build Processes: Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices. References: Dockerfile Best Practices
Component: Keycloak Container 1 Recommended countermeasures Rec1. Apply the principle of least privilege in your image buildsC-DOCKER-CONTAINER-CNT-02MediumNot tested State: Recommended Description: Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively: Define and Use Non-Root Users: Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources. Minimize Package Installations: Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- **State:**

Recommended
- **Description:**
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:
 - [Dockerfile Best Practices](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- **State:**

Recommended
- **Description:**
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:
 - [Dockerfile Best Practices](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- **State:**

Recommended
- **Description:**
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:
 - [Dockerfile Best Practices](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- **State:**

Recommended
- **Description:**
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:
• [Dockerfile Best Practices](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: User Environment

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: Docker Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: Keycloak Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: PostgreSQL Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: Vault Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: Backend Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: MariaDB Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

NIST 800-53: PS-4 Personnel Termination

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium Not tested

State: Recommended

Description:
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:
• [Dockerfile Best Practices](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium Not tested

State: Recommended

Description:
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:
• [Dockerfile Best Practices](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds

C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- Dockerfile Best Practices

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds

C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- Dockerfile Best Practices

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds

C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- Dockerfile Best Practices

Component: Vault Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds

C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- Dockerfile Best Practices

Component: Backend Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds

C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

- Description:**
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:
 - [Dockerfile Best Practices](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:**

Recommended
- Description:**
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:
 - [Dockerfile Best Practices](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: User Environment

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: Docker Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: Keycloak Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: PostgreSQL Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: Vault Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: Backend Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: MariaDB Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

NIST 800-53: PS-5 Personnel Transfer

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:**

Recommended
- Description:**
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State: Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

 - [Dockerfile Best Practices](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State: Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

 - [Dockerfile Best Practices](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State: Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

 - [Dockerfile Best Practices](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State: Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best

practices.
References:
• [Dockerfile Best Practices](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:
• [Dockerfile Best Practices](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:
• [Dockerfile Best Practices](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:
• [Dockerfile Best Practices](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: User Environment

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: Docker Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: Keycloak Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: PostgreSQL Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: Vault Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: Backend Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: MariaDB Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

NIST 800-53: PS-1 Policy and Procedures

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment 1 Recommended countermeasures Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium Not tested - State: Recommended - Description: Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively: Define and Use Non-Root Users: Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources. Minimize Package Installations: Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities. Drop Unnecessary Capabilities: Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation. Implement Automated Security Scanning: Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment. Document and Review Build Processes: Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices. References: Dockerfile Best Practices
Component: User Environment 1 Recommended countermeasures Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium Not tested - State: Recommended - Description: Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively: Define and Use Non-Root Users: Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources. Minimize Package Installations: Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities. Drop Unnecessary Capabilities: Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation. Implement Automated Security Scanning: Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment. Document and Review Build Processes: Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices. References: Dockerfile Best Practices
Component: Docker Container 1 Recommended countermeasures Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium Not tested - State: Recommended - Description: Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively: Define and Use Non-Root Users: Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources. Minimize Package Installations: Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities. Drop Unnecessary Capabilities: Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation. Implement Automated Security Scanning: Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment. Document and Review Build Processes: Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices. References: Dockerfile Best Practices
Component: Keycloak Container 1 Recommended countermeasures Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium Not tested State: Recommended

- **Description:**
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:
 - [Dockerfile Best Practices](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02  Medium ☐ Not tested

- **State:**  Recommended
- **Description:**
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:
 - [Dockerfile Best Practices](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02  Medium ☐ Not tested

- **State:**  Recommended
- **Description:**
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:
 - [Dockerfile Best Practices](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02  Medium ☐ Not tested

- **State:**  Recommended
- **Description:**
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:
 - [Dockerfile Best Practices](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02  Medium ☐ Not tested

- **State:**  Recommended
- **Description:**
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: User Environment

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: Docker Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: Keycloak Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: PostgreSQL Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: Vault Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: Backend Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: MariaDB Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

NIST 800-53: PS-9 Position Descriptions

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best

practices.
References:
• [Dockerfile Best Practices](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:
• [Dockerfile Best Practices](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:
• [Dockerfile Best Practices](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:
• [Dockerfile Best Practices](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:
• [Dockerfile Best Practices](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: User Environment

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: Docker Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: Keycloak Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: PostgreSQL Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: Vault Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: Backend Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Component: MariaDB Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

NIST 800-53: PS-2 Position Risk Designation

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

 - [Dockerfile Best Practices](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

 - [Dockerfile Best Practices](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

 - [Dockerfile Best Practices](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium Not tested

- State: Recommended
- Description:
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:
 - [Dockerfile Best Practices](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium Not tested

- State: Recommended
- Description:
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:
 - [Dockerfile Best Practices](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium Not tested

- State: Recommended
- Description:
Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:
Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.
Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.
Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.
Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.
Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.
References:
 - [Dockerfile Best Practices](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: User Environment

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: Docker Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: Keycloak Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: PostgreSQL Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: Vault Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: Backend Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

Component: MariaDB Container

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium

NIST 800-53: SA-21 Developer Screening

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02 Medium Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

- [Dockerfile Best Practices](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

☐ Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

 - [Dockerfile Best Practices](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

☐ Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

 - [Dockerfile Best Practices](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

☐ Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

 - [Dockerfile Best Practices](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds C-DOCKER-CONTAINER-CNT-02

Medium

☐ Not tested

- State:

Recommended
- Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:
Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:
Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:
Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:
Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:
Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

 - [Dockerfile Best Practices](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Apply the principle of least privilege in your image builds

C-DOCKER-CONTAINER-CNT-02

Medium

☒ Recommended

☐ Not tested

State: Recommended

Description:

Securing Docker Image Builds by enforcing the principle of least privilege requires meticulous configuration of your Dockerfiles and build processes. This control minimizes risk by ensuring that images include only the essential permissions and components needed for application functionality. Follow the steps below to implement this security control effectively:

Define and Use Non-Root Users:

Modify your Dockerfiles to create and switch to non-root users for running application processes. This reduces the potential impact of a compromised container by preventing unnecessary access to system resources.

Minimize Package Installations:

Include only the packages and dependencies essential for your application. By eliminating unneeded software, you reduce the overall attack surface and limit exposure to potential vulnerabilities.

Drop Unnecessary Capabilities:

Explicitly remove or restrict Linux capabilities that are not required by the application. This practice further limits what a compromised container can do, thereby mitigating the risk of privilege escalation.

Implement Automated Security Scanning:

Integrate security scanning tools into your CI/CD pipeline to continuously verify that Docker images adhere to the least privilege principle. Regular scans help detect and remediate any deviations from secure configuration practices before deployment.

Document and Review Build Processes:

Regularly audit your Dockerfiles and build procedures to ensure ongoing compliance with least privilege guidelines. Update your configurations in line with evolving Docker security recommendations and industry best practices.

References:

Dockerfile Best Practices

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Apply the principle of least privilege in your image builds

C-DOCKER-CONTAINER-CNT-02

Medium

Component: User Environment

Rec1. Apply the principle of least privilege in your image builds

C-DOCKER-CONTAINER-CNT-02

Medium

Component: Docker Container

Rec1. Apply the principle of least privilege in your image builds

C-DOCKER-CONTAINER-CNT-02

Medium

Component: Keycloak Container

Rec1. Apply the principle of least privilege in your image builds

C-DOCKER-CONTAINER-CNT-02

Medium

Component: PostgreSQL Container

Rec1. Apply the principle of least privilege in your image builds

C-DOCKER-CONTAINER-CNT-02

Medium

Component: Vault Container

Rec1. Apply the principle of least privilege in your image builds

C-DOCKER-CONTAINER-CNT-02

Medium

Component: Backend Container

Rec1. Apply the principle of least privilege in your image builds

C-DOCKER-CONTAINER-CNT-02

Medium

Component: MariaDB Container

Rec1. Apply the principle of least privilege in your image builds

C-DOCKER-CONTAINER-CNT-02

Medium

NIST 800-53: SI-6 Security and Privacy Function Verification

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Enhance the security posture of your container images

C-DOCKER-CONTAINER-CNT-07

Very high

☐ Not tested

State: Recommended

Description:

Enhancing the security posture of your container images requires a comprehensive strategy throughout the image build and maintenance lifecycle. By implementing robust security practices, you can mitigate vulnerabilities and reduce the risk of container compromise. Follow the steps below to implement this security control effectively:

Implement Vulnerability Scanning:

Integrate automated security scanning tools into your CI/CD pipeline to detect vulnerabilities in base images and dependencies. Regularly scan images for known security issues and remediate them promptly.

Limit Installed Packages and Dependencies:

Streamline your Dockerfiles by including only essential packages required for the application. Reducing unnecessary software minimizes the attack surface and limits potential exposure to exploits.

Enforce Secure Dockerfile Best Practices:

Adopt secure Dockerfile practices such as using COPY instead of ADD, specifying explicit version constraints, and opting for minimal base images. These steps help maintain consistency and security across your container images.

Apply Least Privilege and Drop Unnecessary Capabilities:

Ensure containers run under non-root users and explicitly drop any Linux capabilities that are not required. This minimizes the risk of privilege escalation in the event of a breach.

Regularly Update and Patch Container Images:

Continuously monitor for updates and patches for base images and application dependencies. Regularly rebuilding images with the latest security updates is crucial to maintaining a secure environment.

Document and Audit Image Build Processes:

Maintain comprehensive documentation of your image build procedures and conduct periodic audits. This practice ensures ongoing adherence to security standards and facilitates prompt identification and correction of misconfigurations.

References:

Dockerfile Best Practices

Component: User Environment

1 Recommended countermeasures

Rec1. Enhance the security posture of your container images

C-DOCKER-CONTAINER-CNT-07

Very high

☐ Not tested

State: Recommended

Description:

Enhancing the security posture of your container images requires a comprehensive strategy throughout the image build and maintenance lifecycle. By implementing robust security practices, you can mitigate vulnerabilities and reduce the risk of container compromise. Follow the steps below to implement this security control effectively:

Implement Vulnerability Scanning:
Integrate automated security scanning tools into your CI/CD pipeline to detect vulnerabilities in base images and dependencies. Regularly scan images for known security issues and remediate them promptly.

Limit Installed Packages and Dependencies:
Streamline your Dockerfiles by including only essential packages required for the application. Reducing unnecessary software minimizes the attack surface and limits potential exposure to exploits.

Enforce Secure Dockerfile Best Practices:
Adopt secure Dockerfile practices such as using COPY instead of ADD, specifying explicit version constraints, and opting for minimal base images. These steps help maintain consistency and security across your container images.

Apply Least Privilege and Drop Unnecessary Capabilities:
Ensure containers run under non-root users and explicitly drop any Linux capabilities that are not required. This minimizes the risk of privilege escalation in the event of a breach.

Regularly Update and Patch Container Images:
Continuously monitor for updates and patches for base images and application dependencies. Regularly rebuilding images with the latest security updates is crucial to maintaining a secure environment.

Document and Audit Image Build Processes:
Maintain comprehensive documentation of your image build procedures and conduct periodic audits. This practice ensures ongoing adherence to security standards and facilitates prompt identification and correction of misconfigurations.

References:

- [Dockerfile Best Practices](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Enhance the security posture of your container images

C-DOCKER-CONTAINER-CNT-07

Very high

☒ Recommended

☐ Not tested

State: ☒ Recommended

Description:

Enhancing the security posture of your container images requires a comprehensive strategy throughout the image build and maintenance lifecycle. By implementing robust security practices, you can mitigate vulnerabilities and reduce the risk of container compromise. Follow the steps below to implement this security control effectively:

Implement Vulnerability Scanning:
Integrate automated security scanning tools into your CI/CD pipeline to detect vulnerabilities in base images and dependencies. Regularly scan images for known security issues and remediate them promptly.

Limit Installed Packages and Dependencies:
Streamline your Dockerfiles by including only essential packages required for the application. Reducing unnecessary software minimizes the attack surface and limits potential exposure to exploits.

Enforce Secure Dockerfile Best Practices:
Adopt secure Dockerfile practices such as using COPY instead of ADD, specifying explicit version constraints, and opting for minimal base images. These steps help maintain consistency and security across your container images.

Apply Least Privilege and Drop Unnecessary Capabilities:
Ensure containers run under non-root users and explicitly drop any Linux capabilities that are not required. This minimizes the risk of privilege escalation in the event of a breach.

Regularly Update and Patch Container Images:
Continuously monitor for updates and patches for base images and application dependencies. Regularly rebuilding images with the latest security updates is crucial to maintaining a secure environment.

Document and Audit Image Build Processes:
Maintain comprehensive documentation of your image build procedures and conduct periodic audits. This practice ensures ongoing adherence to security standards and facilitates prompt identification and correction of misconfigurations.

References:

- [Dockerfile Best Practices](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Enhance the security posture of your container images

C-DOCKER-CONTAINER-CNT-07

Very high

☒ Recommended

☐ Not tested

State: ☒ Recommended

Description:

Enhancing the security posture of your container images requires a comprehensive strategy throughout the image build and maintenance lifecycle. By implementing robust security practices, you can mitigate vulnerabilities and reduce the risk of container compromise. Follow the steps below to implement this security control effectively:

Implement Vulnerability Scanning:
Integrate automated security scanning tools into your CI/CD pipeline to detect vulnerabilities in base images and dependencies. Regularly scan images for known security issues and remediate them promptly.

Limit Installed Packages and Dependencies:
Streamline your Dockerfiles by including only essential packages required for the application. Reducing unnecessary software minimizes the attack surface and limits potential exposure to exploits.

Enforce Secure Dockerfile Best Practices:
Adopt secure Dockerfile practices such as using COPY instead of ADD, specifying explicit version constraints, and opting for minimal base images. These steps help maintain consistency and security across your container images.

Apply Least Privilege and Drop Unnecessary Capabilities:
Ensure containers run under non-root users and explicitly drop any Linux capabilities that are not required. This minimizes the risk of privilege escalation in the event of a breach.

Regularly Update and Patch Container Images:
Continuously monitor for updates and patches for base images and application dependencies. Regularly rebuilding images with the latest security updates is crucial to maintaining a secure environment.

Document and Audit Image Build Processes:
Maintain comprehensive documentation of your image build procedures and conduct periodic audits. This practice ensures ongoing adherence to security standards and facilitates prompt identification and correction of misconfigurations.

References:

- [Dockerfile Best Practices](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Enhance the security posture of your container images

C-DOCKER-CONTAINER-CNT-07

Very high

☒ Recommended

☐ Not tested

State: ☒ Recommended

Description:

Enhancing the security posture of your container images requires a comprehensive strategy throughout the image build and maintenance lifecycle. By implementing robust security practices, you can mitigate vulnerabilities and reduce the risk of container compromise. Follow the steps below to implement this security control effectively:

Implement Vulnerability Scanning:
Integrate automated security scanning tools into your CI/CD pipeline to detect vulnerabilities in base images and dependencies. Regularly scan images for known security issues and remediate them promptly.

Limit Installed Packages and Dependencies:
Streamline your Dockerfiles by including only essential packages required for the application. Reducing unnecessary software minimizes the attack surface and limits potential exposure to exploits.

Enforce Secure Dockerfile Best Practices:
Adopt secure Dockerfile practices such as using COPY instead of ADD, specifying explicit version constraints, and opting for minimal base images. These steps help maintain consistency and security across your container images.

Apply Least Privilege and Drop Unnecessary Capabilities:
Ensure containers run under non-root users and explicitly drop any Linux capabilities that are not required. This minimizes the risk of privilege escalation in the event of a breach.

Regularly Update and Patch Container Images:
Continuously monitor for updates and patches for base images and application dependencies. Regularly rebuilding images with the latest security updates is crucial to maintaining a secure environment.

Document and Audit Image Build Processes:
Maintain comprehensive documentation of your image build procedures and conduct periodic audits. This practice ensures ongoing adherence to security standards and facilitates prompt identification and correction of misconfigurations.

References:

- [Dockerfile Best Practices](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Enhance the security posture of your container images

C-DOCKER-CONTAINER-CNT-07

Very high

☒ Recommended

☐ Not tested

State: ☒ Recommended

- **Description:**
Enhancing the security posture of your container images requires a comprehensive strategy throughout the image build and maintenance lifecycle. By implementing robust security practices, you can mitigate vulnerabilities and reduce the risk of container compromise. Follow the steps below to implement this security control effectively:
Implement Vulnerability Scanning:
Integrate automated security scanning tools into your CI/CD pipeline to detect vulnerabilities in base images and dependencies. Regularly scan images for known security issues and remediate them promptly.
Limit Installed Packages and Dependencies:
Streamline your Dockerfiles by including only essential packages required for the application. Reducing unnecessary software minimizes the attack surface and limits potential exposure to exploits.
Enforce Secure Dockerfile Best Practices:
Adopt secure Dockerfile practices such as using COPY instead of ADD, specifying explicit version constraints, and opting for minimal base images. These steps help maintain consistency and security across your container images.
Apply Least Privilege and Drop Unnecessary Capabilities:
Ensure containers run under non-root users and explicitly drop any Linux capabilities that are not required. This minimizes the risk of privilege escalation in the event of a breach.
Regularly Update and Patch Container Images:
Continuously monitor for updates and patches for base images and application dependencies. Regularly rebuilding images with the latest security updates is crucial to maintaining a secure environment.
Document and Audit Image Build Processes:
Maintain comprehensive documentation of your image build procedures and conduct periodic audits. This practice ensures ongoing adherence to security standards and facilitates prompt identification and correction of misconfigurations.
References:
 - [Dockerfile Best Practices](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07

Very high

Not tested

- **State:**

Recommended
- **Description:**
Enhancing the security posture of your container images requires a comprehensive strategy throughout the image build and maintenance lifecycle. By implementing robust security practices, you can mitigate vulnerabilities and reduce the risk of container compromise. Follow the steps below to implement this security control effectively:
Implement Vulnerability Scanning:
Integrate automated security scanning tools into your CI/CD pipeline to detect vulnerabilities in base images and dependencies. Regularly scan images for known security issues and remediate them promptly.
Limit Installed Packages and Dependencies:
Streamline your Dockerfiles by including only essential packages required for the application. Reducing unnecessary software minimizes the attack surface and limits potential exposure to exploits.
Enforce Secure Dockerfile Best Practices:
Adopt secure Dockerfile practices such as using COPY instead of ADD, specifying explicit version constraints, and opting for minimal base images. These steps help maintain consistency and security across your container images.
Apply Least Privilege and Drop Unnecessary Capabilities:
Ensure containers run under non-root users and explicitly drop any Linux capabilities that are not required. This minimizes the risk of privilege escalation in the event of a breach.
Regularly Update and Patch Container Images:
Continuously monitor for updates and patches for base images and application dependencies. Regularly rebuilding images with the latest security updates is crucial to maintaining a secure environment.
Document and Audit Image Build Processes:
Maintain comprehensive documentation of your image build procedures and conduct periodic audits. This practice ensures ongoing adherence to security standards and facilitates prompt identification and correction of misconfigurations.
References:
 - [Dockerfile Best Practices](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07

Very high

Not tested

- **State:**

Recommended
- **Description:**
Enhancing the security posture of your container images requires a comprehensive strategy throughout the image build and maintenance lifecycle. By implementing robust security practices, you can mitigate vulnerabilities and reduce the risk of container compromise. Follow the steps below to implement this security control effectively:
Implement Vulnerability Scanning:
Integrate automated security scanning tools into your CI/CD pipeline to detect vulnerabilities in base images and dependencies. Regularly scan images for known security issues and remediate them promptly.
Limit Installed Packages and Dependencies:
Streamline your Dockerfiles by including only essential packages required for the application. Reducing unnecessary software minimizes the attack surface and limits potential exposure to exploits.
Enforce Secure Dockerfile Best Practices:
Adopt secure Dockerfile practices such as using COPY instead of ADD, specifying explicit version constraints, and opting for minimal base images. These steps help maintain consistency and security across your container images.
Apply Least Privilege and Drop Unnecessary Capabilities:
Ensure containers run under non-root users and explicitly drop any Linux capabilities that are not required. This minimizes the risk of privilege escalation in the event of a breach.
Regularly Update and Patch Container Images:
Continuously monitor for updates and patches for base images and application dependencies. Regularly rebuilding images with the latest security updates is crucial to maintaining a secure environment.
Document and Audit Image Build Processes:
Maintain comprehensive documentation of your image build procedures and conduct periodic audits. This practice ensures ongoing adherence to security standards and facilitates prompt identification and correction of misconfigurations.
References:
 - [Dockerfile Best Practices](#)

Component: Browser

1 Recommended countermeasures

Rec1. Activate built-in browser security filters C-BROWSER-CNT-02

High

Not tested

- **State:**

Recommended
- **Description:**
Implement and regularly update built-in browser security filters on all client machines to protect against malicious scripts, exploit code, and other types of web-based threats. These filters help detect and block untrusted content and potential attacks that could compromise the browser or steal sensitive data. Regular reviews and updates ensure that the browser's security configurations remain effective against emerging threats.
Implementation Steps:
Review Browser Security Settings:
Verify that the browser's security filters are enabled by default in the configuration settings.
Configure Automatic Updates:
Ensure that browsers are set to automatically update so that the latest security filters and patches are applied.
Enforce Organizational Policies:
Deploy centralized management policies (e.g., via Group Policy or MDM solutions) to enforce and monitor these settings across all client machines.
Test Security Filter Effectiveness:
Regularly perform security audits and simulated attacks to verify that the security filters are properly detecting and blocking malicious content.
References:
 - [OWASP Secure Headers Project](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07

Very high

Component: User Environment

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07 Very high

Component: Docker Container

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07 Very high

Component: Keycloak Container

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07 Very high

Component: PostgreSQL Container

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07 Very high

Component: Vault Container

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07 Very high

Component: Backend Container

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07 Very high

Component: MariaDB Container

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07 Very high

Component: Browser

Rec1. Activate built-in browser security filters C-BROWSER-CNT-02 High

NIST 800-53: SI-7 Software, Firmware, and Information Integrity

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment 1 Recommended countermeasures Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07Very highNot tested State: Recommended Description: Enhancing the security posture of your container images requires a comprehensive strategy throughout the image build and maintenance lifecycle. By implementing robust security practices, you can mitigate vulnerabilities and reduce the risk of container compromise. Follow the steps below to implement this security control effectively: Implement Vulnerability Scanning: Integrate automated security scanning tools into your CI/CD pipeline to detect vulnerabilities in base images and dependencies. Regularly scan images for known security issues and remediate them promptly. Limit Installed Packages and Dependencies: Streamline your Dockerfiles by including only essential packages required for the application. Reducing unnecessary software minimizes the attack surface and limits potential exposure to exploits. Enforce Secure Dockerfile Best Practices: Adopt secure Dockerfile practices such as using COPY instead of ADD, specifying explicit version constraints, and opting for minimal base images. These steps help maintain consistency and security across your container images. Apply Least Privilege and Drop Unnecessary Capabilities: Ensure containers run under non-root users and explicitly drop any Linux capabilities that are not required. This minimizes the risk of privilege escalation in the event of a breach. Regularly Update and Patch Container Images: Continuously monitor for updates and patches for base images and application dependencies. Regularly rebuilding images with the latest security updates is crucial to maintaining a secure environment. Document and Audit Image Build Processes: Maintain comprehensive documentation of your image build procedures and conduct periodic audits. This practice ensures ongoing adherence to security standards and facilitates prompt identification and correction of misconfigurations. References: Dockerfile Best Practices
Component: User Environment 1 Recommended countermeasures Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07Very highNot tested State: Recommended Description: Enhancing the security posture of your container images requires a comprehensive strategy throughout the image build and maintenance lifecycle. By implementing robust security practices, you can mitigate vulnerabilities and reduce the risk of container compromise. Follow the steps below to implement this security control effectively: Implement Vulnerability Scanning: Integrate automated security scanning tools into your CI/CD pipeline to detect vulnerabilities in base images and dependencies. Regularly scan images for known security issues and remediate them promptly. Limit Installed Packages and Dependencies: Streamline your Dockerfiles by including only essential packages required for the application. Reducing unnecessary software minimizes the attack surface and limits potential exposure to exploits. Enforce Secure Dockerfile Best Practices: Adopt secure Dockerfile practices such as using COPY instead of ADD, specifying explicit version constraints, and opting for minimal base images. These steps help maintain consistency and security across your container images. Apply Least Privilege and Drop Unnecessary Capabilities: Ensure containers run under non-root users and explicitly drop any Linux capabilities that are not required. This minimizes the risk of privilege escalation in the event of a breach. Regularly Update and Patch Container Images: Continuously monitor for updates and patches for base images and application dependencies. Regularly rebuilding images with the latest security updates is crucial to maintaining a secure environment. Document and Audit Image Build Processes: Maintain comprehensive documentation of your image build procedures and conduct periodic audits. This practice ensures ongoing adherence to security standards and facilitates prompt identification and correction of misconfigurations. References: Dockerfile Best Practices
Component: Docker Container 1 Recommended countermeasures Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07Very highNot tested State: Recommended Description:

Enhancing the security posture of your container images requires a comprehensive strategy throughout the image build and maintenance lifecycle. By implementing robust security practices, you can mitigate vulnerabilities and reduce the risk of container compromise. Follow the steps below to implement this security control effectively:

- Implement Vulnerability Scanning:**
Integrate automated security scanning tools into your CI/CD pipeline to detect vulnerabilities in base images and dependencies. Regularly scan images for known security issues and remediate them promptly.
 - Limit Installed Packages and Dependencies:**
Streamline your Dockerfiles by including only essential packages required for the application. Reducing unnecessary software minimizes the attack surface and limits potential exposure to exploits.
 - Enforce Secure Dockerfile Best Practices:**
Adopt secure Dockerfile practices such as using COPY instead of ADD, specifying explicit version constraints, and opting for minimal base images. These steps help maintain consistency and security across your container images.
 - Apply Least Privilege and Drop Unnecessary Capabilities:**
Ensure containers run under non-root users and explicitly drop any Linux capabilities that are not required. This minimizes the risk of privilege escalation in the event of a breach.
 - Regularly Update and Patch Container Images:**
Continuously monitor for updates and patches for base images and application dependencies. Regularly rebuilding images with the latest security updates is crucial to maintaining a secure environment.
 - Document and Audit Image Build Processes:**
Maintain comprehensive documentation of your image build procedures and conduct periodic audits. This practice ensures ongoing adherence to security standards and facilitates prompt identification and correction of misconfigurations.
- References:
- [Dockerfile Best Practices](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07

Very high

Not tested

- State:

Recommended
- Description:

Enhancing the security posture of your container images requires a comprehensive strategy throughout the image build and maintenance lifecycle. By implementing robust security practices, you can mitigate vulnerabilities and reduce the risk of container compromise. Follow the steps below to implement this security control effectively:

 - Implement Vulnerability Scanning:**
Integrate automated security scanning tools into your CI/CD pipeline to detect vulnerabilities in base images and dependencies. Regularly scan images for known security issues and remediate them promptly.
 - Limit Installed Packages and Dependencies:**
Streamline your Dockerfiles by including only essential packages required for the application. Reducing unnecessary software minimizes the attack surface and limits potential exposure to exploits.
 - Enforce Secure Dockerfile Best Practices:**
Adopt secure Dockerfile practices such as using COPY instead of ADD, specifying explicit version constraints, and opting for minimal base images. These steps help maintain consistency and security across your container images.
 - Apply Least Privilege and Drop Unnecessary Capabilities:**
Ensure containers run under non-root users and explicitly drop any Linux capabilities that are not required. This minimizes the risk of privilege escalation in the event of a breach.
 - Regularly Update and Patch Container Images:**
Continuously monitor for updates and patches for base images and application dependencies. Regularly rebuilding images with the latest security updates is crucial to maintaining a secure environment.
 - Document and Audit Image Build Processes:**
Maintain comprehensive documentation of your image build procedures and conduct periodic audits. This practice ensures ongoing adherence to security standards and facilitates prompt identification and correction of misconfigurations.

References:

 - [Dockerfile Best Practices](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07

Very high

Not tested

- State:

Recommended
- Description:

Enhancing the security posture of your container images requires a comprehensive strategy throughout the image build and maintenance lifecycle. By implementing robust security practices, you can mitigate vulnerabilities and reduce the risk of container compromise. Follow the steps below to implement this security control effectively:

 - Implement Vulnerability Scanning:**
Integrate automated security scanning tools into your CI/CD pipeline to detect vulnerabilities in base images and dependencies. Regularly scan images for known security issues and remediate them promptly.
 - Limit Installed Packages and Dependencies:**
Streamline your Dockerfiles by including only essential packages required for the application. Reducing unnecessary software minimizes the attack surface and limits potential exposure to exploits.
 - Enforce Secure Dockerfile Best Practices:**
Adopt secure Dockerfile practices such as using COPY instead of ADD, specifying explicit version constraints, and opting for minimal base images. These steps help maintain consistency and security across your container images.
 - Apply Least Privilege and Drop Unnecessary Capabilities:**
Ensure containers run under non-root users and explicitly drop any Linux capabilities that are not required. This minimizes the risk of privilege escalation in the event of a breach.
 - Regularly Update and Patch Container Images:**
Continuously monitor for updates and patches for base images and application dependencies. Regularly rebuilding images with the latest security updates is crucial to maintaining a secure environment.
 - Document and Audit Image Build Processes:**
Maintain comprehensive documentation of your image build procedures and conduct periodic audits. This practice ensures ongoing adherence to security standards and facilitates prompt identification and correction of misconfigurations.

References:

 - [Dockerfile Best Practices](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07

Very high

Not tested

- State:

Recommended
- Description:

Enhancing the security posture of your container images requires a comprehensive strategy throughout the image build and maintenance lifecycle. By implementing robust security practices, you can mitigate vulnerabilities and reduce the risk of container compromise. Follow the steps below to implement this security control effectively:

 - Implement Vulnerability Scanning:**
Integrate automated security scanning tools into your CI/CD pipeline to detect vulnerabilities in base images and dependencies. Regularly scan images for known security issues and remediate them promptly.
 - Limit Installed Packages and Dependencies:**
Streamline your Dockerfiles by including only essential packages required for the application. Reducing unnecessary software minimizes the attack surface and limits potential exposure to exploits.
 - Enforce Secure Dockerfile Best Practices:**
Adopt secure Dockerfile practices such as using COPY instead of ADD, specifying explicit version constraints, and opting for minimal base images. These steps help maintain consistency and security across your container images.
 - Apply Least Privilege and Drop Unnecessary Capabilities:**
Ensure containers run under non-root users and explicitly drop any Linux capabilities that are not required. This minimizes the risk of privilege escalation in the event of a breach.
 - Regularly Update and Patch Container Images:**
Continuously monitor for updates and patches for base images and application dependencies. Regularly rebuilding images with the latest security updates is crucial to maintaining a secure environment.
 - Document and Audit Image Build Processes:**
Maintain comprehensive documentation of your image build procedures and conduct periodic audits. This practice ensures ongoing adherence to security standards and facilitates prompt identification and correction of misconfigurations.

References:

 - [Dockerfile Best Practices](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07

Very high

Not tested

- **State:** 🟢 Recommended
- **Description:**

Enhancing the security posture of your container images requires a comprehensive strategy throughout the image build and maintenance lifecycle. By implementing robust security practices, you can mitigate vulnerabilities and reduce the risk of container compromise. Follow the steps below to implement this security control effectively:

Implement Vulnerability Scanning:
Integrate automated security scanning tools into your CI/CD pipeline to detect vulnerabilities in base images and dependencies. Regularly scan images for known security issues and remediate them promptly.

Limit Installed Packages and Dependencies:
Streamline your Dockerfiles by including only essential packages required for the application. Reducing unnecessary software minimizes the attack surface and limits potential exposure to exploits.

Enforce Secure Dockerfile Best Practices:
Adopt secure Dockerfile practices such as using COPY instead of ADD, specifying explicit version constraints, and opting for minimal base images. These steps help maintain consistency and security across your container images.

Apply Least Privilege and Drop Unnecessary Capabilities:
Ensure containers run under non-root users and explicitly drop any Linux capabilities that are not required. This minimizes the risk of privilege escalation in the event of a breach.

Regularly Update and Patch Container Images:
Continuously monitor for updates and patches for base images and application dependencies. Regularly rebuilding images with the latest security updates is crucial to maintaining a secure environment.

Document and Audit Image Build Processes:
Maintain comprehensive documentation of your image build procedures and conduct periodic audits. This practice ensures ongoing adherence to security standards and facilitates prompt identification and correction of misconfigurations.
- **References:**
 - [Dockerfile Best Practices](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07 Very high ○ Not tested

- **State:** 🟢 Recommended
- **Description:**

Enhancing the security posture of your container images requires a comprehensive strategy throughout the image build and maintenance lifecycle. By implementing robust security practices, you can mitigate vulnerabilities and reduce the risk of container compromise. Follow the steps below to implement this security control effectively:

Implement Vulnerability Scanning:
Integrate automated security scanning tools into your CI/CD pipeline to detect vulnerabilities in base images and dependencies. Regularly scan images for known security issues and remediate them promptly.

Limit Installed Packages and Dependencies:
Streamline your Dockerfiles by including only essential packages required for the application. Reducing unnecessary software minimizes the attack surface and limits potential exposure to exploits.

Enforce Secure Dockerfile Best Practices:
Adopt secure Dockerfile practices such as using COPY instead of ADD, specifying explicit version constraints, and opting for minimal base images. These steps help maintain consistency and security across your container images.

Apply Least Privilege and Drop Unnecessary Capabilities:
Ensure containers run under non-root users and explicitly drop any Linux capabilities that are not required. This minimizes the risk of privilege escalation in the event of a breach.

Regularly Update and Patch Container Images:
Continuously monitor for updates and patches for base images and application dependencies. Regularly rebuilding images with the latest security updates is crucial to maintaining a secure environment.

Document and Audit Image Build Processes:
Maintain comprehensive documentation of your image build procedures and conduct periodic audits. This practice ensures ongoing adherence to security standards and facilitates prompt identification and correction of misconfigurations.
- **References:**
 - [Dockerfile Best Practices](#)

Component: Browser

1 Recommended countermeasures

Rec1. Activate built-in browser security filters C-BROWSER-CNT-02 🔴 High ○ Not tested

- **State:** 🟢 Recommended
- **Description:**

Implement and regularly update built-in browser security filters on all client machines to protect against malicious scripts, exploit code, and other types of web-based threats. These filters help detect and block untrusted content and potential attacks that could compromise the browser or steal sensitive data. Regular reviews and updates ensure that the browser's security configurations remain effective against emerging threats.

Implementation Steps:
Review Browser Security Settings:
Verify that the browser's security filters are enabled by default in the configuration settings.

Configure Automatic Updates:
Ensure that browsers are set to automatically update so that the latest security filters and patches are applied.

Enforce Organizational Policies:
Deploy centralized management policies (e.g., via Group Policy or MDM solutions) to enforce and monitor these settings across all client machines.

Test Security Filter Effectiveness:
Regularly perform security audits and simulated attacks to verify that the security filters are properly detecting and blocking malicious content.
- **References:**
 - [OWASP Secure Headers Project](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07 Very high

Component: User Environment

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07 Very high

Component: Docker Container

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07 Very high

Component: Keycloak Container

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07 Very high

Component: PostgreSQL Container

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07 Very high

Component: Vault Container

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07 Very high

Component: Backend Container

Rec1. Enhance the security posture of your container images C-DOCKER-CONTAINER-CNT-07 Very high

Component: MariaDB Container

- After successful compilation, install the new version with: `sudo make install`. Make sure to backup your existing configuration files before this step.
 - Use `sudo systemctl restart nginx` to restart the server with the new configuration.
6. **Verify the installation:** After installation, confirm the successful compilation and installation of NGINX by checking the version: `nginx -v`, and ensure that it displays the custom-built version.
7. **Regularly Update Your Build:** Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed.

References

- [NGINX Configure Options](#)
- [NGINX Installation Guide](#)
- [NGINX Server Security Tips](#)

Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04 Very high ☐ Not tested

• State: Recommended

• Description:

To enhance the security posture of your NGINX server, it's essential to restrict HTTP methods to only the ones necessary for your application, typically GET and POST. Here's a detailed guide on how to achieve this:

1. Open the NGINX configuration file by using a text editor. You can open the main `nginx` configuration file by executing the following command in your terminal: `sudo nano /etc/nginx/nginx.conf`
2. Locate the server block for your application. You may have multiple server blocks if you're handling multiple domains or applications, so ensure that you are within the correct one.
3. Within the server block, search for or add a location block corresponding to your web application. Here is how a typical location block may look like:
4. Add or modify the `limit_except` directive inside your location block. This directive allows you to specify which HTTP methods are permitted and deny the rest. Configure it to only allow GET and POST methods as shown below:
5. Be sure to test your NGINX configuration for syntax errors before applying the changes. Run the following command to perform the check:
6. If no errors are found, proceed to reload NGINX to apply the configuration changes:
7. Verify the changes have been correctly implemented by making HTTP requests using a tool like `curl` or Postman to methods like TRACE or DELETE and confirm that access is denied.

References

- [NGINX limit_except Directive](#)
- [BeagleSecurity: NGINX Server Security](#)
- [DigitalOcean: Understanding NGINX Server and Location Block Selection](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

• State: Recommended

• Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:

Activate Docker Content Trust by setting the `DOCKER_CONTENT_TRUST` environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:

Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:

Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:

Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

• State: Recommended

• Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:

Activate Docker Content Trust by setting the `DOCKER_CONTENT_TRUST` environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:

Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:

Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:

Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08 Very high ☐ Not tested

• State: Recommended

• Description:

Limit each client's token permissions by explicitly configuring which client scopes it may use:

1. **Open the client's Scope settings**
 - In the Keycloak Admin Console, navigate to **Clients (your client)** **Client Scopes** **Scope**.
2. **Turn off Full Scope Allowed**
 - Disable the **Full Scope Allowed** switch to prevent the client from inheriting all realm-level roles and protocol mapper scopes by default [GitBooks](#).
3. **Assign only required scopes**
 - Under **Assigned Default Client Scopes** and **Assigned Optional Client Scopes**, remove any broad scopes (like roles) and add only those scopes—or associated protocol mappers—that the client truly needs.
4. **Validate configuration**
 - Request an access token for the client and inspect its contents to ensure only the designated roles/claims are present.

By enforcing this control, you prevent over-privileged tokens and adhere to the principle of least privilege for each OAuth2/OpenID Connect client.

References

- [Keycloak Client Scope documentation](#)
- [Keycloak Full Scope Allowed explained](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

• State: Recommended

• Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

State:

Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

State:

Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Component: User Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Component: Docker Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Component: NGINX Server

Rec1. Reconfigure NGINX with only necessary modules C-NGINX-SERVER-CNT-01

Very high

Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04

Very high

Component: Keycloak Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Component: PostgreSQL Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Component: Keycloak

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08

Very high

Component: Vault Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Component: Backend Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Component: MariaDB Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

NIST 800-53: CM-3 Configuration Change Control

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment									
1 Recommended countermeasures									
Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08 Very high Not tested State: Recommended Description: Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively: Enable Docker Content Trust: Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used. Verify Image Signatures: Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source. Enforce Signature Verification in CI/CD Pipelines: Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments. Monitor and Audit Image Registries: Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation. References: - Docker Content Trust Documentation <tr><td>Component: User Environment</td></tr> <tr><td>1 Recommended countermeasures</td></tr> <tr><td> Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08 Very high Not tested State: Recommended Description: Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively: Enable Docker Content Trust: Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used. Verify Image Signatures: Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source. Enforce Signature Verification in CI/CD Pipelines: Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments. Monitor and Audit Image Registries: Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation. References: - Docker Content Trust Documentation <tr><td>Component: Docker Container</td></tr><tr><td>1 Recommended countermeasures</td></tr><tr><td> Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08 Very high Not tested State: Recommended Description: Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively: Enable Docker Content Trust: Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used. Verify Image Signatures: Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source. Enforce Signature Verification in CI/CD Pipelines: Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments. Monitor and Audit Image Registries: Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation. References: - Docker Content Trust Documentation <tr><td>Component: NGINX Server</td></tr><tr><td>2 Recommended countermeasures</td></tr><tr><td> Rec1. Reconfigure NGINX with only necessary modulesC-NGINX-SERVER-CNT-01 Very high Not tested State: Recommended Description: Identify Non-Essential Modules: Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing. Gather Necessary Resources: - Ensure you have administrative access to the server where NGINX is running. - Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: <code>sudo apt-get install build-essential libssl-dev</code>. Download NGINX Source Code: - Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using <code>wget</code>: <code>wget https://nginx.org/download/nginx-x.y.z.tar.gz</code>. - Extract the downloaded tarball: <code>tar -xzf nginx-x.y.z.tar.gz</code>. Configure NGINX Build Options: - Navigate to the extracted directory: <code>cd nginx-x.y.z</code>. - Execute the configuration script with desired modules disabled. Use <code>--without-</code> to disable specific modules. Example: <code>./configure --without-http_autoindex_module --without-http_ssi_module</code>. - Consider optimizing further by enabling only required modules using flags like <code>--with-http_ssl_module</code> if HTTPS support is needed. Compile and Install NGINX: - Build NGINX by running: <code>make</code>. - After successful compilation, install the new version with: <code>sudo make install</code>. Make sure to backup your existing configuration files before this step. - Use <code>sudo systemctl restart nginx</code> to restart the server with the new configuration. Verify the Installation: After installation, confirm the successful compilation and installation of NGINX by checking the version: <code>nginx -v</code>, and ensure that it displays the custom-built version. Regularly Update Your Build: Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed. References - NGINX Configure Options - NGINX Installation Guide </td></tr></td></tr></td></tr>	Component: User Environment	1 Recommended countermeasures	Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08 Very high Not tested State: Recommended Description: Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively: Enable Docker Content Trust: Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used. Verify Image Signatures: Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source. Enforce Signature Verification in CI/CD Pipelines: Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments. Monitor and Audit Image Registries: Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation. References: - Docker Content Trust Documentation <tr><td>Component: Docker Container</td></tr> <tr><td>1 Recommended countermeasures</td></tr> <tr><td> Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08 Very high Not tested State: Recommended Description: Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively: Enable Docker Content Trust: Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used. Verify Image Signatures: Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source. Enforce Signature Verification in CI/CD Pipelines: Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments. Monitor and Audit Image Registries: Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation. References: - Docker Content Trust Documentation <tr><td>Component: NGINX Server</td></tr><tr><td>2 Recommended countermeasures</td></tr><tr><td> Rec1. Reconfigure NGINX with only necessary modulesC-NGINX-SERVER-CNT-01 Very high Not tested State: Recommended Description: Identify Non-Essential Modules: Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing. Gather Necessary Resources: - Ensure you have administrative access to the server where NGINX is running. - Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: <code>sudo apt-get install build-essential libssl-dev</code>. Download NGINX Source Code: - Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using <code>wget</code>: <code>wget https://nginx.org/download/nginx-x.y.z.tar.gz</code>. - Extract the downloaded tarball: <code>tar -xzf nginx-x.y.z.tar.gz</code>. Configure NGINX Build Options: - Navigate to the extracted directory: <code>cd nginx-x.y.z</code>. - Execute the configuration script with desired modules disabled. Use <code>--without-</code> to disable specific modules. Example: <code>./configure --without-http_autoindex_module --without-http_ssi_module</code>. - Consider optimizing further by enabling only required modules using flags like <code>--with-http_ssl_module</code> if HTTPS support is needed. Compile and Install NGINX: - Build NGINX by running: <code>make</code>. - After successful compilation, install the new version with: <code>sudo make install</code>. Make sure to backup your existing configuration files before this step. - Use <code>sudo systemctl restart nginx</code> to restart the server with the new configuration. Verify the Installation: After installation, confirm the successful compilation and installation of NGINX by checking the version: <code>nginx -v</code>, and ensure that it displays the custom-built version. Regularly Update Your Build: Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed. References - NGINX Configure Options - NGINX Installation Guide </td></tr></td></tr>	Component: Docker Container	1 Recommended countermeasures	Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08 Very high Not tested State: Recommended Description: Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively: Enable Docker Content Trust: Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used. Verify Image Signatures: Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source. Enforce Signature Verification in CI/CD Pipelines: Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments. Monitor and Audit Image Registries: Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation. References: - Docker Content Trust Documentation <tr><td>Component: NGINX Server</td></tr> <tr><td>2 Recommended countermeasures</td></tr> <tr><td> Rec1. Reconfigure NGINX with only necessary modulesC-NGINX-SERVER-CNT-01 Very high Not tested State: Recommended Description: Identify Non-Essential Modules: Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing. Gather Necessary Resources: - Ensure you have administrative access to the server where NGINX is running. - Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: <code>sudo apt-get install build-essential libssl-dev</code>. Download NGINX Source Code: - Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using <code>wget</code>: <code>wget https://nginx.org/download/nginx-x.y.z.tar.gz</code>. - Extract the downloaded tarball: <code>tar -xzf nginx-x.y.z.tar.gz</code>. Configure NGINX Build Options: - Navigate to the extracted directory: <code>cd nginx-x.y.z</code>. - Execute the configuration script with desired modules disabled. Use <code>--without-</code> to disable specific modules. Example: <code>./configure --without-http_autoindex_module --without-http_ssi_module</code>. - Consider optimizing further by enabling only required modules using flags like <code>--with-http_ssl_module</code> if HTTPS support is needed. Compile and Install NGINX: - Build NGINX by running: <code>make</code>. - After successful compilation, install the new version with: <code>sudo make install</code>. Make sure to backup your existing configuration files before this step. - Use <code>sudo systemctl restart nginx</code> to restart the server with the new configuration. Verify the Installation: After installation, confirm the successful compilation and installation of NGINX by checking the version: <code>nginx -v</code>, and ensure that it displays the custom-built version. Regularly Update Your Build: Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed. References - NGINX Configure Options - NGINX Installation Guide </td></tr>	Component: NGINX Server	2 Recommended countermeasures	Rec1. Reconfigure NGINX with only necessary modulesC-NGINX-SERVER-CNT-01 Very high Not tested State: Recommended Description: Identify Non-Essential Modules: Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing. Gather Necessary Resources: - Ensure you have administrative access to the server where NGINX is running. - Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: <code>sudo apt-get install build-essential libssl-dev</code>. Download NGINX Source Code: - Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using <code>wget</code>: <code>wget https://nginx.org/download/nginx-x.y.z.tar.gz</code>. - Extract the downloaded tarball: <code>tar -xzf nginx-x.y.z.tar.gz</code>. Configure NGINX Build Options: - Navigate to the extracted directory: <code>cd nginx-x.y.z</code>. - Execute the configuration script with desired modules disabled. Use <code>--without-</code> to disable specific modules. Example: <code>./configure --without-http_autoindex_module --without-http_ssi_module</code>. - Consider optimizing further by enabling only required modules using flags like <code>--with-http_ssl_module</code> if HTTPS support is needed. Compile and Install NGINX: - Build NGINX by running: <code>make</code>. - After successful compilation, install the new version with: <code>sudo make install</code>. Make sure to backup your existing configuration files before this step. - Use <code>sudo systemctl restart nginx</code> to restart the server with the new configuration. Verify the Installation: After installation, confirm the successful compilation and installation of NGINX by checking the version: <code>nginx -v</code>, and ensure that it displays the custom-built version. Regularly Update Your Build: Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed. References - NGINX Configure Options - NGINX Installation Guide
Component: User Environment									
1 Recommended countermeasures									
Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08 Very high Not tested State: Recommended Description: Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively: Enable Docker Content Trust: Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used. Verify Image Signatures: Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source. Enforce Signature Verification in CI/CD Pipelines: Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments. Monitor and Audit Image Registries: Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation. References: - Docker Content Trust Documentation <tr><td>Component: Docker Container</td></tr> <tr><td>1 Recommended countermeasures</td></tr> <tr><td> Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08 Very high Not tested State: Recommended Description: Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively: Enable Docker Content Trust: Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used. Verify Image Signatures: Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source. Enforce Signature Verification in CI/CD Pipelines: Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments. Monitor and Audit Image Registries: Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation. References: - Docker Content Trust Documentation <tr><td>Component: NGINX Server</td></tr><tr><td>2 Recommended countermeasures</td></tr><tr><td> Rec1. Reconfigure NGINX with only necessary modulesC-NGINX-SERVER-CNT-01 Very high Not tested State: Recommended Description: Identify Non-Essential Modules: Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing. Gather Necessary Resources: - Ensure you have administrative access to the server where NGINX is running. - Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: <code>sudo apt-get install build-essential libssl-dev</code>. Download NGINX Source Code: - Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using <code>wget</code>: <code>wget https://nginx.org/download/nginx-x.y.z.tar.gz</code>. - Extract the downloaded tarball: <code>tar -xzf nginx-x.y.z.tar.gz</code>. Configure NGINX Build Options: - Navigate to the extracted directory: <code>cd nginx-x.y.z</code>. - Execute the configuration script with desired modules disabled. Use <code>--without-</code> to disable specific modules. Example: <code>./configure --without-http_autoindex_module --without-http_ssi_module</code>. - Consider optimizing further by enabling only required modules using flags like <code>--with-http_ssl_module</code> if HTTPS support is needed. Compile and Install NGINX: - Build NGINX by running: <code>make</code>. - After successful compilation, install the new version with: <code>sudo make install</code>. Make sure to backup your existing configuration files before this step. - Use <code>sudo systemctl restart nginx</code> to restart the server with the new configuration. Verify the Installation: After installation, confirm the successful compilation and installation of NGINX by checking the version: <code>nginx -v</code>, and ensure that it displays the custom-built version. Regularly Update Your Build: Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed. References - NGINX Configure Options - NGINX Installation Guide </td></tr></td></tr>	Component: Docker Container	1 Recommended countermeasures	Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08 Very high Not tested State: Recommended Description: Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively: Enable Docker Content Trust: Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used. Verify Image Signatures: Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source. Enforce Signature Verification in CI/CD Pipelines: Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments. Monitor and Audit Image Registries: Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation. References: - Docker Content Trust Documentation <tr><td>Component: NGINX Server</td></tr> <tr><td>2 Recommended countermeasures</td></tr> <tr><td> Rec1. Reconfigure NGINX with only necessary modulesC-NGINX-SERVER-CNT-01 Very high Not tested State: Recommended Description: Identify Non-Essential Modules: Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing. Gather Necessary Resources: - Ensure you have administrative access to the server where NGINX is running. - Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: <code>sudo apt-get install build-essential libssl-dev</code>. Download NGINX Source Code: - Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using <code>wget</code>: <code>wget https://nginx.org/download/nginx-x.y.z.tar.gz</code>. - Extract the downloaded tarball: <code>tar -xzf nginx-x.y.z.tar.gz</code>. Configure NGINX Build Options: - Navigate to the extracted directory: <code>cd nginx-x.y.z</code>. - Execute the configuration script with desired modules disabled. Use <code>--without-</code> to disable specific modules. Example: <code>./configure --without-http_autoindex_module --without-http_ssi_module</code>. - Consider optimizing further by enabling only required modules using flags like <code>--with-http_ssl_module</code> if HTTPS support is needed. Compile and Install NGINX: - Build NGINX by running: <code>make</code>. - After successful compilation, install the new version with: <code>sudo make install</code>. Make sure to backup your existing configuration files before this step. - Use <code>sudo systemctl restart nginx</code> to restart the server with the new configuration. Verify the Installation: After installation, confirm the successful compilation and installation of NGINX by checking the version: <code>nginx -v</code>, and ensure that it displays the custom-built version. Regularly Update Your Build: Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed. References - NGINX Configure Options - NGINX Installation Guide </td></tr>	Component: NGINX Server	2 Recommended countermeasures	Rec1. Reconfigure NGINX with only necessary modulesC-NGINX-SERVER-CNT-01 Very high Not tested State: Recommended Description: Identify Non-Essential Modules: Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing. Gather Necessary Resources: - Ensure you have administrative access to the server where NGINX is running. - Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: <code>sudo apt-get install build-essential libssl-dev</code>. Download NGINX Source Code: - Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using <code>wget</code>: <code>wget https://nginx.org/download/nginx-x.y.z.tar.gz</code>. - Extract the downloaded tarball: <code>tar -xzf nginx-x.y.z.tar.gz</code>. Configure NGINX Build Options: - Navigate to the extracted directory: <code>cd nginx-x.y.z</code>. - Execute the configuration script with desired modules disabled. Use <code>--without-</code> to disable specific modules. Example: <code>./configure --without-http_autoindex_module --without-http_ssi_module</code>. - Consider optimizing further by enabling only required modules using flags like <code>--with-http_ssl_module</code> if HTTPS support is needed. Compile and Install NGINX: - Build NGINX by running: <code>make</code>. - After successful compilation, install the new version with: <code>sudo make install</code>. Make sure to backup your existing configuration files before this step. - Use <code>sudo systemctl restart nginx</code> to restart the server with the new configuration. Verify the Installation: After installation, confirm the successful compilation and installation of NGINX by checking the version: <code>nginx -v</code>, and ensure that it displays the custom-built version. Regularly Update Your Build: Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed. References - NGINX Configure Options - NGINX Installation Guide			
Component: Docker Container									
1 Recommended countermeasures									
Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08 Very high Not tested State: Recommended Description: Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively: Enable Docker Content Trust: Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used. Verify Image Signatures: Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source. Enforce Signature Verification in CI/CD Pipelines: Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments. Monitor and Audit Image Registries: Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation. References: - Docker Content Trust Documentation <tr><td>Component: NGINX Server</td></tr> <tr><td>2 Recommended countermeasures</td></tr> <tr><td> Rec1. Reconfigure NGINX with only necessary modulesC-NGINX-SERVER-CNT-01 Very high Not tested State: Recommended Description: Identify Non-Essential Modules: Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing. Gather Necessary Resources: - Ensure you have administrative access to the server where NGINX is running. - Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: <code>sudo apt-get install build-essential libssl-dev</code>. Download NGINX Source Code: - Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using <code>wget</code>: <code>wget https://nginx.org/download/nginx-x.y.z.tar.gz</code>. - Extract the downloaded tarball: <code>tar -xzf nginx-x.y.z.tar.gz</code>. Configure NGINX Build Options: - Navigate to the extracted directory: <code>cd nginx-x.y.z</code>. - Execute the configuration script with desired modules disabled. Use <code>--without-</code> to disable specific modules. Example: <code>./configure --without-http_autoindex_module --without-http_ssi_module</code>. - Consider optimizing further by enabling only required modules using flags like <code>--with-http_ssl_module</code> if HTTPS support is needed. Compile and Install NGINX: - Build NGINX by running: <code>make</code>. - After successful compilation, install the new version with: <code>sudo make install</code>. Make sure to backup your existing configuration files before this step. - Use <code>sudo systemctl restart nginx</code> to restart the server with the new configuration. Verify the Installation: After installation, confirm the successful compilation and installation of NGINX by checking the version: <code>nginx -v</code>, and ensure that it displays the custom-built version. Regularly Update Your Build: Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed. References - NGINX Configure Options - NGINX Installation Guide </td></tr>	Component: NGINX Server	2 Recommended countermeasures	Rec1. Reconfigure NGINX with only necessary modulesC-NGINX-SERVER-CNT-01 Very high Not tested State: Recommended Description: Identify Non-Essential Modules: Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing. Gather Necessary Resources: - Ensure you have administrative access to the server where NGINX is running. - Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: <code>sudo apt-get install build-essential libssl-dev</code>. Download NGINX Source Code: - Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using <code>wget</code>: <code>wget https://nginx.org/download/nginx-x.y.z.tar.gz</code>. - Extract the downloaded tarball: <code>tar -xzf nginx-x.y.z.tar.gz</code>. Configure NGINX Build Options: - Navigate to the extracted directory: <code>cd nginx-x.y.z</code>. - Execute the configuration script with desired modules disabled. Use <code>--without-</code> to disable specific modules. Example: <code>./configure --without-http_autoindex_module --without-http_ssi_module</code>. - Consider optimizing further by enabling only required modules using flags like <code>--with-http_ssl_module</code> if HTTPS support is needed. Compile and Install NGINX: - Build NGINX by running: <code>make</code>. - After successful compilation, install the new version with: <code>sudo make install</code>. Make sure to backup your existing configuration files before this step. - Use <code>sudo systemctl restart nginx</code> to restart the server with the new configuration. Verify the Installation: After installation, confirm the successful compilation and installation of NGINX by checking the version: <code>nginx -v</code>, and ensure that it displays the custom-built version. Regularly Update Your Build: Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed. References - NGINX Configure Options - NGINX Installation Guide						
Component: NGINX Server									
2 Recommended countermeasures									
Rec1. Reconfigure NGINX with only necessary modulesC-NGINX-SERVER-CNT-01 Very high Not tested State: Recommended Description: Identify Non-Essential Modules: Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing. Gather Necessary Resources: - Ensure you have administrative access to the server where NGINX is running. - Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: <code>sudo apt-get install build-essential libssl-dev</code>. Download NGINX Source Code: - Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using <code>wget</code>: <code>wget https://nginx.org/download/nginx-x.y.z.tar.gz</code>. - Extract the downloaded tarball: <code>tar -xzf nginx-x.y.z.tar.gz</code>. Configure NGINX Build Options: - Navigate to the extracted directory: <code>cd nginx-x.y.z</code>. - Execute the configuration script with desired modules disabled. Use <code>--without-</code> to disable specific modules. Example: <code>./configure --without-http_autoindex_module --without-http_ssi_module</code>. - Consider optimizing further by enabling only required modules using flags like <code>--with-http_ssl_module</code> if HTTPS support is needed. Compile and Install NGINX: - Build NGINX by running: <code>make</code>. - After successful compilation, install the new version with: <code>sudo make install</code>. Make sure to backup your existing configuration files before this step. - Use <code>sudo systemctl restart nginx</code> to restart the server with the new configuration. Verify the Installation: After installation, confirm the successful compilation and installation of NGINX by checking the version: <code>nginx -v</code>, and ensure that it displays the custom-built version. Regularly Update Your Build: Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed. References - NGINX Configure Options - NGINX Installation Guide									

- [NGINX Server Security Tips](#)

Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04 Very high ☐ Not tested

- State: Recommended

Description:

To enhance the security posture of your NGINX server, it's essential to restrict HTTP methods to only the ones necessary for your application, typically GET and POST. Here's a detailed guide on how to achieve this:

1. Open the NGINX configuration file by using a text editor. You can open the main nginx configuration file by executing the following command in your terminal: `sudo nano /etc/nginx/nginx.conf`
2. Locate the server block for your application. You may have multiple server blocks if you're handling multiple domains or applications, so ensure that you are within the correct one.
3. Within the server block, search for or add a location block corresponding to your web application. Here is how a typical location block may look like:
4. Add or modify the `limit_except` directive inside your location block. This directive allows you to specify which HTTP methods are permitted and deny the rest. Configure it to only allow GET and POST methods as shown below:
5. Be sure to test your NGINX configuration for syntax errors before applying the changes. Run the following command to perform the check:
6. If no errors are found, proceed to reload NGINX to apply the configuration changes:
7. Verify the changes have been correctly implemented by making HTTP requests using a tool like curl or Postman to methods like TRACE or DELETE and confirm that access is denied.

References

- [NGINX limit_except Directive](#)
- [BeagleSecurity: NGINX Server Security](#)
- [DigitalOcean: Understanding NGINX Server and Location Block Selection](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

- State: Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:

Activate Docker Content Trust by setting the `DOCKER_CONTENT_TRUST` environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:

Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:

Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:

Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

- State: Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:

Activate Docker Content Trust by setting the `DOCKER_CONTENT_TRUST` environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:

Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:

Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:

Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08 Very high ☐ Not tested

- State: Recommended

Description:

Limit each client's token permissions by explicitly configuring which client scopes it may use:

1. **Open the client's Scope settings**
 - In the Keycloak Admin Console, navigate to **Clients (your client)** **Client Scopes** **Scope**.
2. **Turn off Full Scope Allowed**
 - Disable the **Full Scope Allowed** switch to prevent the client from inheriting all realm-level roles and protocol mapper scopes by default [GitBooks](#).
3. **Assign only required scopes**
 - Under **Assigned Default Client Scopes** and **Assigned Optional Client Scopes**, remove any broad scopes (like roles) and add only those scopes—or associated protocol mappers—that the client truly needs.
4. **Validate configuration**
 - Request an access token for the client and inspect its contents to ensure only the designated roles/claims are present.

By enforcing this control, you prevent over-privileged tokens and adhere to the principle of least privilege for each OAuth2/OpenID Connect client.

References

- [Keycloak Client Scope documentation](#)
- [Keycloak Full Scope Allowed explained](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

- State: Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:

Activate Docker Content Trust by setting the `DOCKER_CONTENT_TRUST` environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:

Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:
• [Docker Content Trust Documentation](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

State: Recommended

Description:
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:
• [Docker Content Trust Documentation](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

State: Recommended

Description:
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:
• [Docker Content Trust Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: User Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Docker Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: NGINX Server

Rec1. Reconfigure NGINX with only necessary modules C-NGINX-SERVER-CNT-01 Very high

Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04 Very high

Component: Keycloak Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: PostgreSQL Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Keycloak

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08 Very high

Component: Vault Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Backend Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: MariaDB Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

NIST 800-53: CM-9 Configuration Management Plan

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☒ Recommended

☐ Not tested

State:

☒ Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☒ Recommended

☐ Not tested

State:

☒ Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☒ Recommended

☐ Not tested

State:

☒ Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Component: NGINX Server

2 Recommended countermeasures

Rec1. Reconfigure NGINX with only necessary modules

C-NGINX-SERVER-CNT-01

Very high

☒ Recommended

☐ Not tested

State:

☒ Recommended

Description:

- Identify Non-Essential Modules:** Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing.
- Gather Necessary Resources:**
 - Ensure you have administrative access to the server where NGINX is running.
 - Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: `sudo apt-get install build-essential libssl-dev`.
- Download NGINX Source Code:**
 - Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using `wget`: `wget https://nginx.org/download/nginx-x.y.z.tar.gz`.
 - Extract the downloaded tarball: `tar -xzf nginx-x.y.z.tar.gz`.
- Configure NGINX Build Options:**
 - Navigate to the extracted directory: `cd nginx-x.y.z`.
 - Execute the configuration script with desired modules disabled. Use `--without-` to disable specific modules. Example: `./configure --without-http_autoindex_module --without-http_ssi_module`.
 - Consider optimizing further by enabling only required modules using flags like `--with-http_ssl_module` if HTTPS support is needed.
- Compile and Install NGINX:**
 - Build NGINX by running: `make`.
 - After successful compilation, install the new version with: `sudo make install`. Make sure to backup your existing configuration files before this step.
 - Use `sudo systemctl restart nginx` to restart the server with the new configuration.
- Verify the Installation:** After installation, confirm the successful compilation and installation of NGINX by checking the version: `nginx -v`, and ensure that it displays the custom-built version.
- Regularly Update Your Build:** Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed.

References

- [NGINX Configure Options](#)
- [NGINX Installation Guide](#)
- [NGINX Server Security Tips](#)

Rec2. Restrict HTTP methods

C-NGINX-SERVER-CNT-04

Very high

☒ Recommended

☐ Not tested

State:

☒ Recommended

• **Description:**
To enhance the security posture of your NGINX server, it's essential to restrict HTTP methods to only the ones necessary for your application, typically GET and POST. Here's a detailed guide on how to achieve this:

1. Open the NGINX configuration file by using a text editor. You can open the main nginx configuration file by executing the following command in your terminal: `sudo nano /etc/nginx/nginx.conf`

2. Locate the server block for your application. You may have multiple server blocks if you're handling multiple domains or applications, so ensure that you are within the correct one.

3. Within the server block, search for or add a location block corresponding to your web application. Here is how a typical location block may look like:

4. Add or modify the limit_except directive inside your location block. This directive allows you to specify which HTTP methods are permitted and deny the rest. Configure it to only allow GET and POST methods as shown below:

5. Be sure to test your NGINX configuration for syntax errors before applying the changes. Run the following command to perform the check:

6. If no errors are found, proceed to reload NGINX to apply the configuration changes:

7. Verify the changes have been correctly implemented by making HTTP requests using a tool like curl or Postman to methods like TRACE or DELETE and confirm that access is denied.

References

• [NGINX limit_except Directive](#)

• [BeagleSecurity: NGINX Server Security](#)

• [DigitalOcean: Understanding NGINX Server and Location Block Selection](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☒ Not tested

• **State:** Recommended

• **Description:**
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

• [Docker Content Trust Documentation](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☐ Not tested

• **State:** Recommended

• **Description:**
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

• [Docker Content Trust Documentation](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. Limit client token scopes

C-KEYCLOAK-CNT-08

Very high

☐ Not tested

• **State:** Recommended

• **Description:**
Limit each client's token permissions by explicitly configuring which client scopes it may use:

1. **Open the client's Scope settings**

◦ In the Keycloak Admin Console, navigate to **Clients (your client)** **Client Scopes** **Scope**.

2. **Turn off Full Scope Allowed**

◦ Disable the **Full Scope Allowed** switch to prevent the client from inheriting all realm-level roles and protocol mapper scopes by default [GitBooks](#).

3. **Assign only required scopes**

◦ Under **Assigned Default Client Scopes** and **Assigned Optional Client Scopes**, remove any broad scopes (like roles) and add only those scopes—or associated protocol mappers—that the client truly needs.

4. **Validate configuration**

◦ Request an access token for the client and inspect its contents to ensure only the designated roles/claims are present.

By enforcing this control, you prevent over-privileged tokens and adhere to the principle of least privilege for each OAuth2/OpenID Connect client.
References

• [Keycloak Client Scope documentation](#)

• [Keycloak Full Scope Allowed explained](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☐ Not tested

• **State:** Recommended

• **Description:**
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

Report: SpaceHub

Compliance report - 2026-03-25T16:40:26.994993079Z

55 of 182

- [Docker Content Trust Documentation](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

- State: Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

- State: Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: User Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Docker Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: NGINX Server

Rec1. Reconfigure NGINX with only necessary modules C-NGINX-SERVER-CNT-01 Very high
Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04 Very high

Component: Keycloak Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: PostgreSQL Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Keycloak

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08 Very high

Component: Vault Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Backend Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: MariaDB Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

NIST 800-53: CM-13 Data Action Mapping

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

- State:

Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

- State:

Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

- State:

Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Component: NGINX Server

2 Recommended countermeasures

Rec1. Reconfigure NGINX with only necessary modules C-NGINX-SERVER-CNT-01

Very high

Not tested

- State:

Recommended
- Description:
 - Identify Non-Essential Modules:** Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing.
 - Gather Necessary Resources:**
 - Ensure you have administrative access to the server where NGINX is running.
 - Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: `sudo apt-get install build-essential libssl-dev`.
 - Download NGINX Source Code:**
 - Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using `wget`: `wget https://nginx.org/download/nginx-x.y.z.tar.gz`.
 - Extract the downloaded tarball: `tar -xzf nginx-x.y.z.tar.gz`.
 - Configure NGINX Build Options:**
 - Navigate to the extracted directory: `cd nginx-x.y.z`.
 - Execute the configuration script with desired modules disabled. Use `--without-` to disable specific modules. Example: `./configure --without-http_autoindex_module --without-http_ssl_module`.
 - Consider optimizing further by enabling only required modules using flags like `--with-http_ssl_module` if HTTPS support is needed.
 - Compile and Install NGINX:**
 - Build NGINX by running: `make`.
 - After successful compilation, install the new version with: `sudo make install`. Make sure to backup your existing configuration files before this step.
 - Use `sudo systemctl restart nginx` to restart the server with the new configuration.
 - Verify the Installation:** After installation, confirm the successful compilation and installation of NGINX by checking the version: `nginx -v`, and ensure that it displays the custom-built version.
 - Regularly Update Your Build:** Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed.

References

 - [NGINX Configure Options](#)
 - [NGINX Installation Guide](#)
 - [NGINX Server Security Tips](#)

Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04

Very high

Not tested

- State:

Recommended
- Description:

To enhance the security posture of your NGINX server, it's essential to restrict HTTP methods to only the ones necessary for your application, typically GET and POST. Here's a detailed guide on how to achieve this:

 - Open the NGINX configuration file by using a text editor. You can open the main nginx configuration file by executing the following command in your terminal: `sudo nano /etc/nginx/nginx.conf`
 - Locate the server block for your application. You may have multiple server blocks if you're handling multiple domains or applications, so ensure that you are within the correct one.
 - Within the server block, search for or add a location block corresponding to your web application. Here is how a typical location block may look like:

- 4. Add or modify the `limit_except` directive inside your location block. This directive allows you to specify which HTTP methods are permitted and deny the rest. Configure it to only allow GET and POST methods as shown below:
- 5. Be sure to test your NGINX configuration for syntax errors before applying the changes. Run the following command to perform the check:
- 6. If no errors are found, proceed to reload NGINX to apply the configuration changes:
- 7. Verify the changes have been correctly implemented by making HTTP requests using a tool like curl or Postman to methods like TRACE or DELETE and confirm that access is denied.

- References
- [NGINX limit_except Directive](#)
 - [BeagleSecurity: NGINX Server Security](#)
 - [DigitalOcean: Understanding NGINX Server and Location Block Selection](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

- State:  Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the `DOCKER_CONTENT_TRUST` environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

- State:  Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the `DOCKER_CONTENT_TRUST` environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08 Very high ☐ Not tested

- State:  Recommended
- Description:

Limit each client's token permissions by explicitly configuring which client scopes it may use:

 1. **Open the client's Scope settings**
 - In the Keycloak Admin Console, navigate to **Clients** (your client) **Client Scopes** **Scope**.
 2. **Turn off Full Scope Allowed**
 - Disable the **Full Scope Allowed** switch to prevent the client from inheriting all realm-level roles and protocol mapper scopes by default [GitBooks](#).
 3. **Assign only required scopes**
 - Under **Assigned Default Client Scopes** and **Assigned Optional Client Scopes**, remove any broad scopes (like roles) and add only those scopes—or associated protocol mappers—that the client truly needs.
 4. **Validate configuration**
 - Request an access token for the client and inspect its contents to ensure only the designated roles/claims are present.

By enforcing this control, you prevent over-privileged tokens and adhere to the principle of least privilege for each OAuth2/OpenID Connect client.

References

 - [Keycloak Client Scope documentation](#)
 - [Keycloak Full Scope Allowed explained](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

- State:  Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the `DOCKER_CONTENT_TRUST` environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08

Very high

☐ Not tested

• State: Recommended

• Description:
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08

Very high

☐ Not tested

• State: Recommended

• Description:
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08

Very high

Component: User Environment

Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08

Very high

Component: Docker Container

Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08

Very high

Component: NGINX Server

Rec1. Reconfigure NGINX with only necessary modulesC-NGINX-SERVER-CNT-01

Very high

Rec2. Restrict HTTP methodsC-NGINX-SERVER-CNT-04

Very high

Component: Keycloak Container

Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08

Very high

Component: PostgreSQL Container

Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08

Very high

Component: Keycloak

Rec1. Limit client token scopesC-KEYCLOAK-CNT-08

Very high

Component: Vault Container

Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08

Very high

Component: Backend Container

Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08

Very high

Component: MariaDB Container

Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08

Very high

NIST 800-53: CM-4 Impact Analyses

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08

Very high

☐ Not tested

• State: Recommended

• Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

State:

Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

State:

Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: NGINX Server

2 Recommended countermeasures

Rec1. Reconfigure NGINX with only necessary modules C-NGINX-SERVER-CNT-01

Very high

Not tested

State:

Recommended

Description:

1. **Identify Non-Essential Modules:** Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing.

2. **Gather Necessary Resources:**

- Ensure you have administrative access to the server where NGINX is running.
- Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: `sudo apt-get install build-essential libssl-dev`.

3. **Download NGINX Source Code:**

- Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using `wget`: `wget https://nginx.org/download/nginx-x.y.z.tar.gz`.
- Extract the downloaded tarball: `tar -xzf nginx-x.y.z.tar.gz`.

4. **Configure NGINX Build Options:**

- Navigate to the extracted directory: `cd nginx-x.y.z`.
- Execute the configuration script with desired modules disabled. Use `--without-` to disable specific modules. Example: `./configure --without-http_autoindex_module --without-http_ssi_module`.
- Consider optimizing further by enabling only required modules using flags like `--with-http_ssl_module` if HTTPS support is needed.

5. **Compile and Install NGINX:**

- Build NGINX by running: `make`.
- After successful compilation, install the new version with: `sudo make install`. Make sure to backup your existing configuration files before this step.
- Use `sudo systemctl restart nginx` to restart the server with the new configuration.

6. **Verify the Installation:** After installation, confirm the successful compilation and installation of NGINX by checking the version: `nginx -v`, and ensure that it displays the custom-built version.

7. **Regularly Update Your Build:** Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed.

References

[NGINX Configure Options](#)

[NGINX Installation Guide](#)

[NGINX Server Security Tips](#)

Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04

Very high

Not tested

State:

Recommended

Description:

To enhance the security posture of your NGINX server, it's essential to restrict HTTP methods to only the ones necessary for your application, typically GET and POST. Here's a detailed guide on how to achieve this:

1. Open the NGINX configuration file by using a text editor. You can open the main `nginx` configuration file by executing the following command in your terminal: `sudo nano /etc/nginx/nginx.conf`

2. Locate the server block for your application. You may have multiple server blocks if you're handling multiple domains or applications, so ensure that you are within the correct one.

3. Within the server block, search for or add a location block corresponding to your web application. Here is how a typical location block may look like:

4. Add or modify the `limit_except` directive inside your location block. This directive allows you to specify which HTTP methods are permitted and deny the rest. Configure it to only allow GET and POST methods as shown below:

5. Be sure to test your NGINX configuration for syntax errors before applying the changes. Run the following command to perform the check:

6. If no errors are found, proceed to reload NGINX to apply the configuration changes:

7. Verify the changes have been correctly implemented by making HTTP requests using a tool like `curl` or Postman to methods like `TRACE` or `DELETE` and confirm that access is denied.

References

Report: SpaceHub

Compliance report - 2026-03-25T16:40:26.994993079Z

60 of 182

- [NGINX limit except Directive](#)
- [BeagleSecurity: NGINX Server Security](#)
- [DigitalOcean: Understanding NGINX Server and Location Block Selection](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

- State: Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

- State: Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08 Very high ☐ Not tested

- State: Recommended
- Description:

Limit each client's token permissions by explicitly configuring which client scopes it may use:

 1. **Open the client's Scope settings**
 - In the Keycloak Admin Console, navigate to **Clients** (your client) **Client Scopes** **Scope**.
 2. **Turn off Full Scope Allowed**
 - Disable the **Full Scope Allowed** switch to prevent the client from inheriting all realm-level roles and protocol mapper scopes by default [GitBooks](#).
 3. **Assign only required scopes**
 - Under **Assigned Default Client Scopes** and **Assigned Optional Client Scopes**, remove any broad scopes (like roles) and add only those scopes—or associated protocol mappers—that the client truly needs.
 4. **Validate configuration**
 - Request an access token for the client and inspect its contents to ensure only the designated roles/claims are present.

By enforcing this control, you prevent over-privileged tokens and adhere to the principle of least privilege for each OAuth2/OpenID Connect client.

References

 - [Keycloak Client Scope documentation](#)
 - [Keycloak Full Scope Allowed explained](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

- State: Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

- State: Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

☐ Not tested

State:

Recommended

Description:
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Component: User Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Component: Docker Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Component: NGINX Server

Rec1. Reconfigure NGINX with only necessary modules C-NGINX-SERVER-CNT-01

Very high

Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04

Very high

Component: Keycloak Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Component: PostgreSQL Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Component: Keycloak

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08

Very high

Component: Vault Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Component: Backend Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Component: MariaDB Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

NIST 800-53: CM-12 Information Location

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

☐ Not tested

State:

Recommended

Description:
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

- State: Recommended
- Description:
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:
 - [Docker Content Trust Documentation](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

- State: Recommended
- Description:
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:
 - [Docker Content Trust Documentation](#)

Component: NGINX Server

2 Recommended countermeasures

Rec1. Reconfigure NGINX with only necessary modules C-NGINX-SERVER-CNT-01 Very high ☐ Not tested

- State: Recommended
- Description:
 - Identify Non-Essential Modules:** Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing.
 - Gather Necessary Resources:**
 - Ensure you have administrative access to the server where NGINX is running.
 - Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: `sudo apt-get install build-essential libssl-dev`.
 - Download NGINX Source Code:**
 - Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using `wget`: `wget https://nginx.org/download/nginx-x.y.z.tar.gz`.
 - Extract the downloaded tarball: `tar -xvzf nginx-x.y.z.tar.gz`.
 - Configure NGINX Build Options:**
 - Navigate to the extracted directory: `cd nginx-x.y.z`.
 - Execute the configuration script with desired modules disabled. Use `--without-` to disable specific modules. Example: `./configure --without-http_autoindex_module --without-http_ssi_module`.
 - Consider optimizing further by enabling only required modules using flags like `--with-http_ssl_module` if HTTPS support is needed.
 - Compile and Install NGINX:**
 - Build NGINX by running: `make`.
 - After successful compilation, install the new version with: `sudo make install`. Make sure to backup your existing configuration files before this step.
 - Use `sudo systemctl restart nginx` to restart the server with the new configuration.
 - Verify the Installation:** After installation, confirm the successful compilation and installation of NGINX by checking the version: `nginx -v`, and ensure that it displays the custom-built version.
 - Regularly Update Your Build:** Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed.

References

 - [NGINX Configure Options](#)
 - [NGINX Installation Guide](#)
 - [NGINX Server Security Tips](#)

Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04 Very high ☐ Not tested

- State: Recommended
- Description:
To enhance the security posture of your NGINX server, it's essential to restrict HTTP methods to only the ones necessary for your application, typically GET and POST. Here's a detailed guide on how to achieve this:
 - Open the NGINX configuration file by using a text editor. You can open the main nginx configuration file by executing the following command in your terminal: `sudo nano /etc/nginx/nginx.conf`
 - Locate the server block for your application. You may have multiple server blocks if you're handling multiple domains or applications, so ensure that you are within the correct one.
 - Within the server block, search for or add a location block corresponding to your web application. Here is how a typical location block may look like:
 - Add or modify the `limit_except` directive inside your location block. This directive allows you to specify which HTTP methods are permitted and deny the rest. Configure it to only allow GET and POST methods as shown below:
 - Be sure to test your NGINX configuration for syntax errors before applying the changes. Run the following command to perform the check:
 - If no errors are found, proceed to reload NGINX to apply the configuration changes:
 - Verify the changes have been correctly implemented by making HTTP requests using a tool like curl or Postman to methods like TRACE or DELETE and confirm that access is denied.

References

 - [NGINX limit_except Directive](#)
 - [BeagleSecurity: NGINX Server Security](#)
 - [DigitalOcean: Understanding NGINX Server and Location Block Selection](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

- State: Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

- State: Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08 Very high Not tested

- State: Recommended
- Description:

Limit each client's token permissions by explicitly configuring which client scopes it may use:

 - Open the client's Scope settings**
 - In the Keycloak Admin Console, navigate to **Clients (your client) Client Scopes Scope**.
 - Turn off Full Scope Allowed**
 - Disable the **Full Scope Allowed** switch to prevent the client from inheriting all realm-level roles and protocol mapper scopes by default [GitBooks](#).
 - Assign only required scopes**
 - Under **Assigned Default Client Scopes** and **Assigned Optional Client Scopes**, remove any broad scopes (like roles) and add only those scopes—or associated protocol mappers—that the client truly needs.
 - Validate configuration**
 - Request an access token for the client and inspect its contents to ensure only the designated roles/claims are present.

By enforcing this control, you prevent over-privileged tokens and adhere to the principle of least privilege for each OAuth2/OpenID Connect client.

References

 - [Keycloak Client Scope documentation](#)
 - [Keycloak Full Scope Allowed explained](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

- State: Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

- State: Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☒ Recommended

☐ Not tested

- State:** ☒ Recommended
- Description:**
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:
 - [Docker Content Trust Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: User Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Docker Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: NGINX Server

Rec1. Reconfigure NGINX with only necessary modules C-NGINX-SERVER-CNT-01 Very high
Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04 Very high

Component: Keycloak Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: PostgreSQL Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Keycloak

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08 Very high

Component: Vault Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Backend Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: MariaDB Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

NIST 800-53: CM-7 Least Functionality

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☒ Recommended

☐ Not tested

- State:** ☒ Recommended
- Description:**
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:
 - [Docker Content Trust Documentation](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☒ Recommended

☐ Not tested

State:

☒ Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☒ Recommended

☐ Not tested

State:

☒ Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Component: NGINX Server

2 Recommended countermeasures

Rec1. Reconfigure NGINX with only necessary modules

C-NGINX-SERVER-CNT-01

Very high

☒ Recommended

☐ Not tested

State:

☒ Recommended

Description:

- Identify Non-Essential Modules:** Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing.
- Gather Necessary Resources:**
 - Ensure you have administrative access to the server where NGINX is running.
 - Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: `sudo apt-get install build-essential libssl-dev`.
- Download NGINX Source Code:**
 - Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using `wget`: `wget https://nginx.org/download/nginx-x.y.z.tar.gz`.
 - Extract the downloaded tarball: `tar -xzf nginx-x.y.z.tar.gz`.
- Configure NGINX Build Options:**
 - Navigate to the extracted directory: `cd nginx-x.y.z`.
 - Execute the configuration script with desired modules disabled. Use `--without-` to disable specific modules. Example: `./configure --without-http_autoindex_module --without-http_ssi_module`.
 - Consider optimizing further by enabling only required modules using flags like `--with-http_ssl_module` if HTTPS support is needed.
- Compile and Install NGINX:**
 - Build NGINX by running: `make`.
 - After successful compilation, install the new version with: `sudo make install`. Make sure to backup your existing configuration files before this step.
 - Use `sudo systemctl restart nginx` to restart the server with the new configuration.
- Verify the Installation:** After installation, confirm the successful compilation and installation of NGINX by checking the version: `nginx -v`, and ensure that it displays the custom-built version.
- Regularly Update Your Build:** Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed.

References

- [NGINX Configure Options](#)
- [NGINX Installation Guide](#)
- [NGINX Server Security Tips](#)

Rec2. Restrict HTTP methods

C-NGINX-SERVER-CNT-04

Very high

☒ Recommended

☐ Not tested

State:

☒ Recommended

Description:

To enhance the security posture of your NGINX server, it's essential to restrict HTTP methods to only the ones necessary for your application, typically GET and POST. Here's a detailed guide on how to achieve this:

- Open the NGINX configuration file by using a text editor. You can open the main nginx configuration file by executing the following command in your terminal: `sudo nano /etc/nginx/nginx.conf`
- Locate the server block for your application. You may have multiple server blocks if you're handling multiple domains or applications, so ensure that you are within the correct one.
- Within the server block, search for or add a location block corresponding to your web application. Here is how a typical location block may look like:
- Add or modify the `limit_except` directive inside your location block. This directive allows you to specify which HTTP methods are permitted and deny the rest. Configure it to only allow GET and POST methods as shown below:
- Be sure to test your NGINX configuration for syntax errors before applying the changes. Run the following command to perform the check:
- If no errors are found, proceed to reload NGINX to apply the configuration changes:
- Verify the changes have been correctly implemented by making HTTP requests using a tool like curl or Postman to methods like TRACE or DELETE and confirm that access is denied.

References

- [NGINX limit_except Directive](#)
- [BeagleSecurity: NGINX Server Security](#)
- [DigitalOcean: Understanding NGINX Server and Location Block Selection](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☒ Recommended

☐ Not tested

State:

☒ Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

State:

Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08

Very high

Not tested

State:

Recommended

Description:

Limit each client's token permissions by explicitly configuring which client scopes it may use:

- Open the client's Scope settings**
 - In the Keycloak Admin Console, navigate to **Clients (your client)** **Client Scopes** **Scope**.
- Turn off Full Scope Allowed**
 - Disable the **Full Scope Allowed** switch to prevent the client from inheriting all realm-level roles and protocol mapper scopes by default [GitBooks](#).
- Assign only required scopes**
 - Under **Assigned Default Client Scopes** and **Assigned Optional Client Scopes**, remove any broad scopes (like roles) and add only those scopes—or associated protocol mappers—that the client truly needs.
- Validate configuration**
 - Request an access token for the client and inspect its contents to ensure only the designated roles/claims are present.

By enforcing this control, you prevent over-privileged tokens and adhere to the principle of least privilege for each OAuth2/OpenID Connect client.

References

- [Keycloak Client Scope documentation](#)
- [Keycloak Full Scope Allowed explained](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

State:

Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

State:

Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☒ Recommended

☐ Not tested

• State:

• Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

Component: User Environment

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

Component: Docker Container

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

Component: NGINX Server

Rec1. Reconfigure NGINX with only necessary modules

C-NGINX-SERVER-CNT-01

Very high

Rec2. Restrict HTTP methods

C-NGINX-SERVER-CNT-04

Very high

Component: Keycloak Container

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

Component: PostgreSQL Container

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

Component: Keycloak

Rec1. Limit client token scopes

C-KEYCLOAK-CNT-08

Very high

Component: Vault Container

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

Component: Backend Container

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

Component: MariaDB Container

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

NIST 800-53: CM-1 Policy and Procedures

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☐ Not tested

• State:

• Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☐ Not tested

• State:

- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:

Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:

Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:

Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:

Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

Docker Content Trust Documentation

Component: Docker Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☒ Recommended

☐ Not tested

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:

Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:

Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:

Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:

Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

Docker Content Trust Documentation

Component: NGINX Server

2 Recommended countermeasures

Rec1. Reconfigure NGINX with only necessary modules

C-NGINX-SERVER-CNT-01

Very high

☒ Recommended

☐ Not tested

Description:

1. Identify Non-Essential Modules:

Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing.

2. Gather Necessary Resources:

Ensure you have administrative access to the server where NGINX is running.

Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: `sudo apt-get install build-essential libssl-dev`.

3. Download NGINX Source Code:

Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using `wget`: `wget https://nginx.org/download/nginx-x.y.z.tar.gz`.

Extract the downloaded tarball: `tar -xzf nginx-x.y.z.tar.gz`.

4. Configure NGINX Build Options:

Navigate to the extracted directory: `cd nginx-x.y.z`.

Execute the configuration script with desired modules disabled. Use `--without-` to disable specific modules. Example: `./configure --without-http_autoindex_module --without-http_ssi_module`.

Consider optimizing further by enabling only required modules using flags like `--with-http_ssl_module` if HTTPS support is needed.

5. Compile and Install NGINX:

Build NGINX by running: `make`.

After successful compilation, install the new version with: `sudo make install`. Make sure to backup your existing configuration files before this step.

Use `sudo systemctl restart nginx` to restart the server with the new configuration.

6. Verify the Installation:

After installation, confirm the successful compilation and installation of NGINX by checking the version: `nginx -v`, and ensure that it displays the custom-built version.

7. Regularly Update Your Build:

Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed.

References

NGINX Configure Options

NGINX Installation Guide

NGINX Server Security Tips

Rec2. Restrict HTTP methods

C-NGINX-SERVER-CNT-04

Very high

☒ Recommended

☐ Not tested

Description:

To enhance the security posture of your NGINX server, it's essential to restrict HTTP methods to only the ones necessary for your application, typically GET and POST. Here's a detailed guide on how to achieve this:

1. Open the NGINX configuration file by using a text editor. You can open the main nginx configuration file by executing the following command in your terminal: `sudo nano /etc/nginx/nginx.conf`

Locate the server block for your application. You may have multiple server blocks if you're handling multiple domains or applications, so ensure that you are within the correct one.

Within the server block, search for or add a location block corresponding to your web application. Here is how a typical location block may look like:

Add or modify the limit_except directive inside your location block. This directive allows you to specify which HTTP methods are permitted and deny the rest. Configure it to only allow GET and POST methods as shown below:

Be sure to test your NGINX configuration for syntax errors before applying the changes. Run the following command to perform the check:

If no errors are found, proceed to reload NGINX to apply the configuration changes:

Verify the changes have been correctly implemented by making HTTP requests using a tool like curl or Postman to methods like TRACE or DELETE and confirm that access is denied.

References

NGINX limit_except Directive

BeagleSecurity: NGINX Server Security

DigitalOcean: Understanding NGINX Server and Location Block Selection

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☒ Recommended

☐ Not tested

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:

Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:

Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:

Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Report: SpaceHub

Compliance report - 2026-03-25T16:40:26.994993079Z

69 of 182

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

State: Recommended

Description:
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08

Very high

Not tested

State: Recommended

Description:
Limit each client's token permissions by explicitly configuring which client scopes it may use:

1. Open the client's Scope settings

- In the Keycloak Admin Console, navigate to **Clients (your client)** **Client Scopes** **Scope**.

2. Turn off Full Scope Allowed

- Disable the **Full Scope Allowed** switch to prevent the client from inheriting all realm-level roles and protocol mapper scopes by default [GitBooks](#).

3. Assign only required scopes

- Under **Assigned Default Client Scopes** and **Assigned Optional Client Scopes**, remove any broad scopes (like roles) and add only those scopes—or associated protocol mappers—that the client truly needs.

4. Validate configuration

- Request an access token for the client and inspect its contents to ensure only the designated roles/claims are present.

By enforcing this control, you prevent over-privileged tokens and adhere to the principle of least privilege for each OAuth2/OpenID Connect client.
References

- [Keycloak Client Scope documentation](#)
- [Keycloak Full Scope Allowed explained](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

State: Recommended

Description:
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

State: Recommended

Description:
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

State: Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: User Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Docker Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: NGINX Server

Rec1. Reconfigure NGINX with only necessary modules C-NGINX-SERVER-CNT-01 Very high
Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04 Very high

Component: Keycloak Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: PostgreSQL Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Keycloak

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08 Very high

Component: Vault Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Backend Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: MariaDB Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

NIST 800-53: CM-14 Signed Components

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

State: Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

State: Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

- State: Recommended
- Description:
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:
 - [Docker Content Trust Documentation](#)

Component: NGINX Server

2 Recommended countermeasures

Rec1. Reconfigure NGINX with only necessary modules C-NGINX-SERVER-CNT-01 Very high ☐ Not tested

- State: Recommended
- Description:
 - Identify Non-Essential Modules:** Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing.
 - Gather Necessary Resources:**
 - Ensure you have administrative access to the server where NGINX is running.
 - Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: `sudo apt-get install build-essential libssl-dev`.
 - Download NGINX Source Code:**
 - Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using `wget`: `wget https://nginx.org/download/nginx-x.y.z.tar.gz`.
 - Extract the downloaded tarball: `tar -xzf nginx-x.y.z.tar.gz`.
 - Configure NGINX Build Options:**
 - Navigate to the extracted directory: `cd nginx-x.y.z`.
 - Execute the configuration script with desired modules disabled. Use `--without-` to disable specific modules. Example: `./configure --without-http_autoindex_module --without-http_ssi_module`.
 - Consider optimizing further by enabling only required modules using flags like `--with-http_ssl_module` if HTTPS support is needed.
 - Compile and Install NGINX:**
 - Build NGINX by running: `make`.
 - After successful compilation, install the new version with: `sudo make install`. Make sure to backup your existing configuration files before this step.
 - Use `sudo systemctl restart nginx` to restart the server with the new configuration.
 - Verify the Installation:** After installation, confirm the successful compilation and installation of NGINX by checking the version: `nginx -v`, and ensure that it displays the custom-built version.
 - Regularly Update Your Build:** Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed.
References
 - [NGINX Configure Options](#)
 - [NGINX Installation Guide](#)
 - [NGINX Server Security Tips](#)

Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04 Very high ☐ Not tested

- State: Recommended
- Description:
To enhance the security posture of your NGINX server, it's essential to restrict HTTP methods to only the ones necessary for your application, typically GET and POST. Here's a detailed guide on how to achieve this:
 - Open the NGINX configuration file by using a text editor. You can open the main `nginx` configuration file by executing the following command in your terminal: `sudo nano /etc/nginx/nginx.conf`
 - Locate the server block for your application. You may have multiple server blocks if you're handling multiple domains or applications, so ensure that you are within the correct one.
 - Within the server block, search for or add a location block corresponding to your web application. Here is how a typical location block may look like:
 - Add or modify the `limit_except` directive inside your location block. This directive allows you to specify which HTTP methods are permitted and deny the rest. Configure it to only allow GET and POST methods as shown below:
 - Be sure to test your NGINX configuration for syntax errors before applying the changes. Run the following command to perform the check:
 - If no errors are found, proceed to reload NGINX to apply the configuration changes:
 - Verify the changes have been correctly implemented by making HTTP requests using a tool like `curl` or Postman to methods like TRACE or DELETE and confirm that access is denied.
References
 - [NGINX limit_except Directive](#)
 - [BeagleSecurity: NGINX Server Security](#)
 - [DigitalOcean: Understanding NGINX Server and Location Block Selection](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

- State: Recommended
- Description:
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:
 - [Docker Content Trust Documentation](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☒ Recommended

☐ Not tested

State:

☒ Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. Limit client token scopes

C-KEYCLOAK-CNT-08

Very high

☒ Recommended

☐ Not tested

State:

☒ Recommended

Description:

Limit each client's token permissions by explicitly configuring which client scopes it may use:

- Open the client's Scope settings**
 - In the Keycloak Admin Console, navigate to **Clients** (your client) **Client Scopes** **Scope**.
- Turn off Full Scope Allowed**
 - Disable the **Full Scope Allowed** switch to prevent the client from inheriting all realm-level roles and protocol mapper scopes by default [GitBooks](#).
- Assign only required scopes**
 - Under **Assigned Default Client Scopes** and **Assigned Optional Client Scopes**, remove any broad scopes (like roles) and add only those scopes—or associated protocol mappers—that the client truly needs.
- Validate configuration**
 - Request an access token for the client and inspect its contents to ensure only the designated roles/claims are present.

By enforcing this control, you prevent over-privileged tokens and adhere to the principle of least privilege for each OAuth2/OpenID Connect client.

References

- [Keycloak Client Scope documentation](#)
- [Keycloak Full Scope Allowed explained](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☒ Recommended

☐ Not tested

State:

☒ Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☒ Recommended

☐ Not tested

State:

☒ Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled

C-DOCKER-CONTAINER-CNT-08

Very high

☒ Recommended

☐ Not tested

State:

☒ Recommended

Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: User Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Docker Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: NGINX Server

Rec1. Reconfigure NGINX with only necessary modules C-NGINX-SERVER-CNT-01 Very high
Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04 Very high

Component: Keycloak Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: PostgreSQL Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Keycloak

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08 Very high

Component: Vault Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Backend Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: MariaDB Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

NIST 800-53: CM-10 Software Usage Restrictions

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

State: Recommended

Description:
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

State: Recommended

Description:
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

Report: SpaceHub

Compliance report - 2026-03-25T16:40:26.994993079Z

74 of 182

- [Docker Content Trust Documentation](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

- State: Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Component: NGINX Server

2 Recommended countermeasures

Rec1. Reconfigure NGINX with only necessary modules C-NGINX-SERVER-CNT-01 Very high Not tested

- State: Recommended
- Description:

1. **Identify Non-Essential Modules:** Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSL (Server Side Includes), WebDAV, and autoindexing.

2. **Gather Necessary Resources:**

 - Ensure you have administrative access to the server where NGINX is running.
 - Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: `sudo apt-get install build-essential libssl-dev`.

3. **Download NGINX Source Code:**

 - Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using `wget`: `wget https://nginx.org/download/nginx-x.y.z.tar.gz`.
 - Extract the downloaded tarball: `tar -xvzf nginx-x.y.z.tar.gz`.

4. **Configure NGINX Build Options:**

 - Navigate to the extracted directory: `cd nginx-x.y.z`.
 - Execute the configuration script with desired modules disabled. Use `--without-` to disable specific modules. Example: `./configure --without-http_autoindex_module --without-http_ssl_module`.
 - Consider optimizing further by enabling only required modules using flags like `--with-http_ssl_module` if HTTPS support is needed.

5. **Compile and Install NGINX:**

 - Build NGINX by running: `make`.
 - After successful compilation, install the new version with: `sudo make install`. Make sure to backup your existing configuration files before this step.
 - Use `sudo systemctl restart nginx` to restart the server with the new configuration.

6. **Verify the Installation:** After installation, confirm the successful compilation and installation of NGINX by checking the version: `nginx -v`, and ensure that it displays the custom-built version.

7. **Regularly Update Your Build:** Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed.

References

 - [NGINX Configure Options](#)
 - [NGINX Installation Guide](#)
 - [NGINX Server Security Tips](#)

Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04 Very high Not tested

- State: Recommended
- Description:

To enhance the security posture of your NGINX server, it's essential to restrict HTTP methods to only the ones necessary for your application, typically GET and POST. Here's a detailed guide on how to achieve this:

1. Open the NGINX configuration file by using a text editor. You can open the main nginx configuration file by executing the following command in your terminal: `sudo nano /etc/nginx/nginx.conf`

2. Locate the server block for your application. You may have multiple server blocks if you're handling multiple domains or applications, so ensure that you are within the correct one.

3. Within the server block, search for or add a location block corresponding to your web application. Here is how a typical location block may look like:

4. Add or modify the `limit_except` directive inside your location block. This directive allows you to specify which HTTP methods are permitted and deny the rest. Configure it to only allow GET and POST methods as shown below:

5. Be sure to test your NGINX configuration for syntax errors before applying the changes. Run the following command to perform the check:

6. If no errors are found, proceed to reload NGINX to apply the configuration changes:

7. Verify the changes have been correctly implemented by making HTTP requests using a tool like curl or Postman to methods like TRACE or DELETE and confirm that access is denied.

References

 - [NGINX limit_except Directive](#)
 - [BeagleSecurity: NGINX Server Security](#)
 - [DigitalOcean: Understanding NGINX Server and Location Block Selection](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

- State: Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

- State: Recommended

- **Description:**
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:
 - [Docker Content Trust Documentation](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08 Very high Not tested

- **State:** Recommended
- **Description:**
Limit each client's token permissions by explicitly configuring which client scopes it may use:
 1. **Open the client's Scope settings**
 - In the Keycloak Admin Console, navigate to **Clients (your client)** **Client Scopes** **Scope**.
 2. **Turn off Full Scope Allowed**
 - Disable the **Full Scope Allowed** switch to prevent the client from inheriting all realm-level roles and protocol mapper scopes by default [GitBooks](#).
 3. **Assign only required scopes**
 - Under **Assigned Default Client Scopes** and **Assigned Optional Client Scopes**, remove any broad scopes (like roles) and add only those scopes—or associated protocol mappers—that the client truly needs.
 4. **Validate configuration**
 - Request an access token for the client and inspect its contents to ensure only the designated roles/claims are present.

By enforcing this control, you prevent over-privileged tokens and adhere to the principle of least privilege for each OAuth2/OpenID Connect client.

References
 - [Keycloak Client Scope documentation](#)
 - [Keycloak Full Scope Allowed explained](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

- **State:** Recommended
- **Description:**
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:
 - [Docker Content Trust Documentation](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

- **State:** Recommended
- **Description:**
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:
 - [Docker Content Trust Documentation](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

- **State:** Recommended
- **Description:**
Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:
Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.
Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.
Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.
Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:
 - [Docker Content Trust Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: User Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Docker Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: NGINX Server

Rec1. Reconfigure NGINX with only necessary modules C-NGINX-SERVER-CNT-01 Very high
Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04 Very high

Component: Keycloak Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: PostgreSQL Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Keycloak

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08 Very high

Component: Vault Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Backend Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: MariaDB Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

NIST 800-53: CM-8 System Component Inventory

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment 1 Recommended countermeasures Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☒ Not tested State: Recommended Description: Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively: Enable Docker Content Trust: Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used. Verify Image Signatures: Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source. Enforce Signature Verification in CI/CD Pipelines: Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments. Monitor and Audit Image Registries: Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation. References: Docker Content Trust Documentation
Component: User Environment 1 Recommended countermeasures Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested State: Recommended Description: Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively: Enable Docker Content Trust: Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used. Verify Image Signatures: Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source. Enforce Signature Verification in CI/CD Pipelines: Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments. Monitor and Audit Image Registries: Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation. References: Docker Content Trust Documentation
Component: Docker Container 1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

- State: Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Component: NGINX Server

2 Recommended countermeasures

Rec1. Reconfigure NGINX with only necessary modules C-NGINX-SERVER-CNT-01 Very high ☐ Not tested

- State: Recommended
- Description:
 - Identify Non-Essential Modules:** Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing.
 - Gather Necessary Resources:**
 - Ensure you have administrative access to the server where NGINX is running.
 - Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: `sudo apt-get install build-essential libssl-dev`.
 - Download NGINX Source Code:**
 - Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using `wget`: `wget https://nginx.org/download/nginx-x.y.z.tar.gz`.
 - Extract the downloaded tarball: `tar -zxvf nginx-x.y.z.tar.gz`.
 - Configure NGINX Build Options:**
 - Navigate to the extracted directory: `cd nginx-x.y.z`.
 - Execute the configuration script with desired modules disabled. Use `--without-` to disable specific modules. Example: `./configure --without-http_autoindex_module --without-http_ssi_module`.
 - Consider optimizing further by enabling only required modules using flags like `--with-http_ssl_module` if HTTPS support is needed.
 - Compile and Install NGINX:**
 - Build NGINX by running: `make`.
 - After successful compilation, install the new version with: `sudo make install`. Make sure to backup your existing configuration files before this step.
 - Use `sudo systemctl restart nginx` to restart the server with the new configuration.
 - Verify the Installation:** After installation, confirm the successful compilation and installation of NGINX by checking the version: `nginx -v`, and ensure that it displays the custom-built version.
 - Regularly Update Your Build:** Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed.

References

 - [NGINX Configure Options](#)
 - [NGINX Installation Guide](#)
 - [NGINX Server Security Tips](#)

Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04 Very high ☐ Not tested

- State: Recommended
- Description:

To enhance the security posture of your NGINX server, it's essential to restrict HTTP methods to only the ones necessary for your application, typically GET and POST. Here's a detailed guide on how to achieve this:

 - Open the NGINX configuration file by using a text editor. You can open the main `nginx` configuration file by executing the following command in your terminal: `sudo nano /etc/nginx/nginx.conf`
 - Locate the `server` block for your application. You may have multiple `server` blocks if you're handling multiple domains or applications, so ensure that you are within the correct one.
 - Within the `server` block, search for or add a `location` block corresponding to your web application. Here is how a typical `location` block may look like:
 - Add or modify the `limit_except` directive inside your `location` block. This directive allows you to specify which HTTP methods are permitted and deny the rest. Configure it to only allow GET and POST methods as shown below:
 - Be sure to test your NGINX configuration for syntax errors before applying the changes. Run the following command to perform the check:
 - If no errors are found, proceed to reload NGINX to apply the configuration changes:
 - Verify the changes have been correctly implemented by making HTTP requests using a tool like `curl` or Postman to methods like `TRACE` or `DELETE` and confirm that access is denied.

References

 - [NGINX limit_except Directive](#)
 - [BeagleSecurity: NGINX Server Security](#)
 - [DigitalOcean: Understanding NGINX Server and Location Block Selection](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

- State: Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested

- State: Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08

Very high

Not tested

- State:

Recommended
- Description:

Limit each client's token permissions by explicitly configuring which client scopes it may use:

 - Open the client's Scope settings
 - In the Keycloak Admin Console, navigate to **Clients** (your client) **Client Scopes** **Scope**.
 - Turn off Full Scope Allowed
 - Disable the **Full Scope Allowed** switch to prevent the client from inheriting all realm-level roles and protocol mapper scopes by default [GitBooks](#).
 - Assign only required scopes
 - Under **Assigned Default Client Scopes** and **Assigned Optional Client Scopes**, remove any broad scopes (like roles) and add only those scopes—or associated protocol mappers—that the client truly needs.
 - Validate configuration
 - Request an access token for the client and inspect its contents to ensure only the designated roles/claims are present.

By enforcing this control, you prevent over-privileged tokens and adhere to the principle of least privilege for each OAuth2/OpenID Connect client.

References

- [Keycloak Client Scope documentation](#)
- [Keycloak Full Scope Allowed explained](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

- State:

Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

- State:

Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

- State:

Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: User Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Docker Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: NGINX Server

Rec1. Reconfigure NGINX with only necessary modules C-NGINX-SERVER-CNT-01 Very high
Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04 Very high

Component: Keycloak Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: PostgreSQL Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Keycloak

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08 Very high

Component: Vault Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Backend Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: MariaDB Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

NIST 800-53: CM-11 User-installed Software

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("Req") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment 1 Recommended countermeasures Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08 Very high ☒ Not tested State: Recommended Description: Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively: Enable Docker Content Trust: Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used. Verify Image Signatures: Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source. Enforce Signature Verification in CI/CD Pipelines: Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments. Monitor and Audit Image Registries: Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation. References: Docker Content Trust Documentation
Component: User Environment 1 Recommended countermeasures Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested State: Recommended Description: Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively: Enable Docker Content Trust: Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used. Verify Image Signatures: Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source. Enforce Signature Verification in CI/CD Pipelines: Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments. Monitor and Audit Image Registries: Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation. References: Docker Content Trust Documentation
Component: Docker Container 1 Recommended countermeasures Rec1. Ensure content trust for Docker is enabledC-DOCKER-CONTAINER-CNT-08 Very high ☐ Not tested State: Recommended Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

- [Docker Content Trust Documentation](#)

Component: NGINX Server

2 Recommended countermeasures

Rec1. Reconfigure NGINX with only necessary modules C-NGINX-SERVER-CNT-01 Very high Not tested

- State: Recommended
- Description:
 - Identify Non-Essential Modules:** Begin by reviewing the list of NGINX modules that are included in your current configuration. Consider your specific use case and assess which modules are unnecessary for your deployment. Common non-essential modules might include those related to features such as SSI (Server Side Includes), WebDAV, and autoindexing.
 - Gather Necessary Resources:**
 - Ensure you have administrative access to the server where NGINX is running.
 - Install essential packages for building software, such as GCC (GNU Compiler Collection), make, and OpenSSL development libraries. On a Debian-based system, use: `sudo apt-get install build-essential libssl-dev`.
 - Download NGINX Source Code:**
 - Navigate to the NGINX download page or use the command line. For instance, you can download a specific version using `wget`: `wget https://nginx.org/download/nginx-x.y.z.tar.gz`.
 - Extract the downloaded tarball: `tar -xzf nginx-x.y.z.tar.gz`.
 - Configure NGINX Build Options:**
 - Navigate to the extracted directory: `cd nginx-x.y.z`.
 - Execute the configuration script with desired modules disabled. Use `--without-` to disable specific modules. Example: `./configure --without-http_autoindex_module --without-http_ssl_module`.
 - Consider optimizing further by enabling only required modules using flags like `--with-http_ssl_module` if HTTPS support is needed.
 - Compile and Install NGINX:**
 - Build NGINX by running: `make`.
 - After successful compilation, install the new version with: `sudo make install`. Make sure to backup your existing configuration files before this step.
 - Use `sudo systemctl restart nginx` to restart the server with the new configuration.
 - Verify the Installation:** After installation, confirm the successful compilation and installation of NGINX by checking the version: `nginx -v`, and ensure that it displays the custom-built version.
 - Regularly Update Your Build:** Periodically revisit the list of necessary and optional modules to reflect changes in application requirements or security insights, and recompile NGINX as needed.

- References
- [NGINX Configure Options](#)
 - [NGINX Installation Guide](#)
 - [NGINX Server Security Tips](#)

Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04 Very high Not tested

- State: Recommended
- Description:

To enhance the security posture of your NGINX server, it's essential to restrict HTTP methods to only the ones necessary for your application, typically GET and POST. Here's a detailed guide on how to achieve this:

 - Open the NGINX configuration file by using a text editor. You can open the main `nginx` configuration file by executing the following command in your terminal: `sudo nano /etc/nginx/nginx.conf`
 - Locate the server block for your application. You may have multiple server blocks if you're handling multiple domains or applications, so ensure that you are within the correct one.
 - Within the server block, search for or add a location block corresponding to your web application. Here is how a typical location block may look like:
 - Add or modify the `limit_except` directive inside your location block. This directive allows you to specify which HTTP methods are permitted and deny the rest. Configure it to only allow GET and POST methods as shown below:
 - Be sure to test your NGINX configuration for syntax errors before applying the changes. Run the following command to perform the check:
 - If no errors are found, proceed to reload NGINX to apply the configuration changes:
 - Verify the changes have been correctly implemented by making HTTP requests using a tool like `curl` or Postman to methods like TRACE or DELETE and confirm that access is denied.

- References
- [NGINX limit_except Directive](#)
 - [BeagleSecurity: NGINX Server Security](#)
 - [DigitalOcean: Understanding NGINX Server and Location Block Selection](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

- State: Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:

 - [Docker Content Trust Documentation](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high Not tested

- State: Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:
Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:
Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:
Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:
Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.
References:

- [Docker Content Trust Documentation](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08

Very high

Not tested

- State:

Recommended
- Description:

Limit each client's token permissions by explicitly configuring which client scopes it may use:

 - Open the client's Scope settings
 - In the Keycloak Admin Console, navigate to **Clients** (your client) **Client Scopes** **Scope**.
 - Turn off Full Scope Allowed
 - Disable the **Full Scope Allowed** switch to prevent the client from inheriting all realm-level roles and protocol mapper scopes by default [GitBooks](#).
 - Assign only required scopes
 - Under **Assigned Default Client Scopes** and **Assigned Optional Client Scopes**, remove any broad scopes (like roles) and add only those scopes—or associated protocol mappers—that the client truly needs.
 - Validate configuration
 - Request an access token for the client and inspect its contents to ensure only the designated roles/claims are present.

By enforcing this control, you prevent over-privileged tokens and adhere to the principle of least privilege for each OAuth2/OpenID Connect client.

References
 - [Keycloak Client Scope documentation](#)
 - [Keycloak Full Scope Allowed explained](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

- State:

Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:

Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:

Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:

Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:

Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:
 - [Docker Content Trust Documentation](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

- State:

Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:

Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:

Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:

Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:

Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:
 - [Docker Content Trust Documentation](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Not tested

- State:

Recommended
- Description:

Enhancing the integrity and authenticity of your container images by ensuring Docker Content Trust is enabled is critical for a secure container deployment. This control verifies that all images are digitally signed, preventing the use of tampered or untrusted images. Follow the steps below to implement this security control effectively:

Enable Docker Content Trust:

Activate Docker Content Trust by setting the DOCKER_CONTENT_TRUST environment variable to 1 on your Docker host. This enforces the verification of image signatures during pull and push operations, ensuring that only trusted images are used.

Verify Image Signatures:

Integrate automated processes to verify that all images have valid digital signatures. This step helps ensure that any image used in your deployment has not been altered and comes from a trusted source.

Enforce Signature Verification in CI/CD Pipelines:

Incorporate signature verification into your CI/CD workflows to automatically validate image integrity before deployment. This minimizes the risk of deploying compromised images to production environments.

Monitor and Audit Image Registries:

Regularly review your Docker registries for compliance with content trust policies. Ensure that all images are signed and any unsigned or tampered images are flagged for immediate remediation.

References:
 - [Docker Content Trust Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08

Very high

Component: User Environment

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Docker Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: NGINX Server

Rec1. Reconfigure NGINX with only necessary modules C-NGINX-SERVER-CNT-01 Very high
Rec2. Restrict HTTP methods C-NGINX-SERVER-CNT-04 Very high

Component: Keycloak Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: PostgreSQL Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Keycloak

Rec1. Limit client token scopes C-KEYCLOAK-CNT-08 Very high

Component: Vault Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: Backend Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

Component: MariaDB Container

Rec1. Ensure content trust for Docker is enabled C-DOCKER-CONTAINER-CNT-08 Very high

NIST 800-53: SA-10 Developer Configuration Management

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

2 Recommended countermeasures

Rec1. Harden the container runtime environmentC-DOCKER-CONTAINER-CNT-06

Very high

☐ Not tested

State: Recommended

Description:
Enhancing the security posture of your container runtime environment is vital for preventing exploitation through misconfigurations and insecure defaults. Follow the steps below to implement this security control effectively:
Enforce Mandatory Security Profiles:
Enable and configure security modules such as seccomp, AppArmor, or SELinux to enforce strict policies on system calls and resource access. This reduces the risk of unauthorized actions within containers.
Restrict Privileged Operations:
Avoid running containers in privileged mode and limit Linux capabilities to only those necessary for the container's operation. This minimizes the potential for privilege escalation and host compromise.
Isolate Container Processes:
Utilize user namespaces and other isolation mechanisms to ensure that processes within containers operate with the least privileges and remain segregated from the host system.
Regularly Update and Patch:
Keep the container runtime, host operating system, and associated components up-to-date with security patches. Regular updates reduce exposure to known vulnerabilities.
Audit and Monitor Runtime Configurations:
Implement continuous monitoring and regular audits of container runtime configurations. Automated tools can help detect and alert on deviations from established security baselines.
References:

- [Docker Engine Security Documentation](#)

Rec2. Strengthen network security and segmentationC-DOCKER-CONTAINER-CNT-09

Medium

☐ Not tested

State: Recommended

Description:
Strengthening network security and segmentation is essential to minimize exposure to external threats and ensure that only authorized communications occur between containers and external networks. Follow the steps below to implement this security control effectively:
Define and Isolate Networks:

- Create dedicated Docker networks for different applications or service tiers to isolate container communications and limit lateral movement in case of a breach.

Restrict Network Interfaces:

- Bind container traffic to specific host interfaces and avoid using broad network configurations that expose unnecessary ports or services.

Implement Firewall Rules:

- Use host-based or container-level firewalls to restrict incoming and outgoing traffic to only those ports and protocols necessary for application functionality.

Apply Secure Network Configurations:

- Avoid using Docker's default bridge network (docker0) for production workloads. Configure custom networks with enhanced security settings.

Monitor and Audit Network Traffic:

- Regularly monitor network traffic using logging and intrusion detection systems. Audit network configurations to ensure compliance with security policies.

References:

- [Docker Networking Documentation](#)

Component: User Environment

2 Recommended countermeasures

Rec1. Harden the container runtime environmentC-DOCKER-CONTAINER-CNT-06

Very high

☐ Not tested

State: Recommended

Description:
Enhancing the security posture of your container runtime environment is vital for preventing exploitation through misconfigurations and insecure defaults. Follow the steps below to implement this security control effectively:
Enforce Mandatory Security Profiles:
Enable and configure security modules such as seccomp, AppArmor, or SELinux to enforce strict policies on system calls and resource access. This reduces the risk of unauthorized actions within containers.
Restrict Privileged Operations:
Avoid running containers in privileged mode and limit Linux capabilities to only those necessary for the container's operation. This minimizes the potential for privilege escalation and host compromise.
Isolate Container Processes:
Utilize user namespaces and other isolation mechanisms to ensure that processes within containers operate with the least privileges and remain segregated from the host system.

Regularly Update and Patch:
Keep the container runtime, host operating system, and associated components up-to-date with security patches. Regular updates reduce exposure to known vulnerabilities.

Audit and Monitor Runtime Configurations:
Implement continuous monitoring and regular audits of container runtime configurations. Automated tools can help detect and alert on deviations from established security baselines.

References:

- [Docker Engine Security Documentation](#)

Rec2. Strengthen network security and segmentation C-DOCKER-CONTAINER-CNT-09 Medium Not tested

- State: Recommended
- Description:
Strengthening network security and segmentation is essential to minimize exposure to external threats and ensure that only authorized communications occur between containers and external networks. Follow the steps below to implement this security control effectively:
Define and Isolate Networks:
 - Create dedicated Docker networks for different applications or service tiers to isolate container communications and limit lateral movement in case of a breach.**Restrict Network Interfaces:**
 - Bind container traffic to specific host interfaces and avoid using broad network configurations that expose unnecessary ports or services.**Implement Firewall Rules:**
 - Use host-based or container-level firewalls to restrict incoming and outgoing traffic to only those ports and protocols necessary for application functionality.**Apply Secure Network Configurations:**
 - Avoid using Docker's default bridge network (docker0) for production workloads. Configure custom networks with enhanced security settings.**Monitor and Audit Network Traffic:**
 - Regularly monitor network traffic using logging and intrusion detection systems. Audit network configurations to ensure compliance with security policies.**References:**
 - [Docker Networking Documentation](#)

Component: Docker Container

2 Recommended countermeasures

Rec1. Harden the container runtime environment C-DOCKER-CONTAINER-CNT-06 Very high Not tested

- State: Recommended
- Description:
Enhancing the security posture of your container runtime environment is vital for preventing exploitation through misconfigurations and insecure defaults. Follow the steps below to implement this security control effectively:
Enforce Mandatory Security Profiles:
Enable and configure security modules such as seccomp, AppArmor, or SELinux to enforce strict policies on system calls and resource access. This reduces the risk of unauthorized actions within containers.
Restrict Privileged Operations:
Avoid running containers in privileged mode and limit Linux capabilities to only those necessary for the container's operation. This minimizes the potential for privilege escalation and host compromise.
Isolate Container Processes:
Utilize user namespaces and other isolation mechanisms to ensure that processes within containers operate with the least privileges and remain segregated from the host system.
Regularly Update and Patch:
Keep the container runtime, host operating system, and associated components up-to-date with security patches. Regular updates reduce exposure to known vulnerabilities.
Audit and Monitor Runtime Configurations:
Implement continuous monitoring and regular audits of container runtime configurations. Automated tools can help detect and alert on deviations from established security baselines.
References:
 - [Docker Engine Security Documentation](#)

Rec2. Strengthen network security and segmentation C-DOCKER-CONTAINER-CNT-09 Medium Not tested

- State: Recommended
- Description:
Strengthening network security and segmentation is essential to minimize exposure to external threats and ensure that only authorized communications occur between containers and external networks. Follow the steps below to implement this security control effectively:
Define and Isolate Networks:
 - Create dedicated Docker networks for different applications or service tiers to isolate container communications and limit lateral movement in case of a breach.**Restrict Network Interfaces:**
 - Bind container traffic to specific host interfaces and avoid using broad network configurations that expose unnecessary ports or services.**Implement Firewall Rules:**
 - Use host-based or container-level firewalls to restrict incoming and outgoing traffic to only those ports and protocols necessary for application functionality.**Apply Secure Network Configurations:**
 - Avoid using Docker's default bridge network (docker0) for production workloads. Configure custom networks with enhanced security settings.**Monitor and Audit Network Traffic:**
 - Regularly monitor network traffic using logging and intrusion detection systems. Audit network configurations to ensure compliance with security policies.**References:**
 - [Docker Networking Documentation](#)

Component: Keycloak Container

2 Recommended countermeasures

Rec1. Harden the container runtime environment C-DOCKER-CONTAINER-CNT-06 Very high Not tested

- State: Recommended
- Description:
Enhancing the security posture of your container runtime environment is vital for preventing exploitation through misconfigurations and insecure defaults. Follow the steps below to implement this security control effectively:
Enforce Mandatory Security Profiles:
Enable and configure security modules such as seccomp, AppArmor, or SELinux to enforce strict policies on system calls and resource access. This reduces the risk of unauthorized actions within containers.
Restrict Privileged Operations:
Avoid running containers in privileged mode and limit Linux capabilities to only those necessary for the container's operation. This minimizes the potential for privilege escalation and host compromise.
Isolate Container Processes:
Utilize user namespaces and other isolation mechanisms to ensure that processes within containers operate with the least privileges and remain segregated from the host system.
Regularly Update and Patch:
Keep the container runtime, host operating system, and associated components up-to-date with security patches. Regular updates reduce exposure to known vulnerabilities.
Audit and Monitor Runtime Configurations:
Implement continuous monitoring and regular audits of container runtime configurations. Automated tools can help detect and alert on deviations from established security baselines.
References:
 - [Docker Engine Security Documentation](#)

Rec2. Strengthen network security and segmentation C-DOCKER-CONTAINER-CNT-09 Medium Not tested

- State: Recommended
- Description:
Strengthening network security and segmentation is essential to minimize exposure to external threats and ensure that only authorized communications occur between containers and external networks. Follow the steps below to implement this security control effectively:
Define and Isolate Networks:
 - Create dedicated Docker networks for different applications or service tiers to isolate container communications and limit lateral movement in case of a breach.**Restrict Network Interfaces:**
 - Bind container traffic to specific host interfaces and avoid using broad network configurations that expose unnecessary ports or services.**Implement Firewall Rules:**
 - Use host-based or container-level firewalls to restrict incoming and outgoing traffic to only those ports and protocols necessary for application functionality.**Apply Secure Network Configurations:**
 - Avoid using Docker's default bridge network (docker0) for production workloads. Configure custom networks with enhanced security settings.**Monitor and Audit Network Traffic:**
 - Regularly monitor network traffic using logging and intrusion detection systems. Audit network configurations to ensure compliance with security policies.

- References:
- [Docker Networking Documentation](#)

Component: PostgreSQL Container

2 Recommended countermeasures

Rec1. Harden the container runtime environment C-DOCKER-CONTAINER-CNT-06 Very high ☐ Not tested

- State: Recommended
- Description:

Enhancing the security posture of your container runtime environment is vital for preventing exploitation through misconfigurations and insecure defaults. Follow the steps below to implement this security control effectively:

Enforce Mandatory Security Profiles:
Enable and configure security modules such as seccomp, AppArmor, or SELinux to enforce strict policies on system calls and resource access. This reduces the risk of unauthorized actions within containers.

Restrict Privileged Operations:
Avoid running containers in privileged mode and limit Linux capabilities to only those necessary for the container's operation. This minimizes the potential for privilege escalation and host compromise.

Isolate Container Processes:
Utilize user namespaces and other isolation mechanisms to ensure that processes within containers operate with the least privileges and remain segregated from the host system.

Regularly Update and Patch:
Keep the container runtime, host operating system, and associated components up-to-date with security patches. Regular updates reduce exposure to known vulnerabilities.

Audit and Monitor Runtime Configurations:
Implement continuous monitoring and regular audits of container runtime configurations. Automated tools can help detect and alert on deviations from established security baselines.

References:

 - [Docker Engine Security Documentation](#)

Rec2. Strengthen network security and segmentation C-DOCKER-CONTAINER-CNT-09 Medium ☐ Not tested

- State: Recommended
- Description:

Strengthening network security and segmentation is essential to minimize exposure to external threats and ensure that only authorized communications occur between containers and external networks. Follow the steps below to implement this security control effectively:

Define and Isolate Networks:
 - Create dedicated Docker networks for different applications or service tiers to isolate container communications and limit lateral movement in case of a breach.

Restrict Network Interfaces:
 - Bind container traffic to specific host interfaces and avoid using broad network configurations that expose unnecessary ports or services.

Implement Firewall Rules:
 - Use host-based or container-level firewalls to restrict incoming and outgoing traffic to only those ports and protocols necessary for application functionality.

Apply Secure Network Configurations:
 - Avoid using Docker's default bridge network (docker0) for production workloads. Configure custom networks with enhanced security settings.

Monitor and Audit Network Traffic:
 - Regularly monitor network traffic using logging and intrusion detection systems. Audit network configurations to ensure compliance with security policies.

References:

 - [Docker Networking Documentation](#)

Component: Vault Container

2 Recommended countermeasures

Rec1. Harden the container runtime environment C-DOCKER-CONTAINER-CNT-06 Very high ☐ Not tested

- State: Recommended
- Description:

Enhancing the security posture of your container runtime environment is vital for preventing exploitation through misconfigurations and insecure defaults. Follow the steps below to implement this security control effectively:

Enforce Mandatory Security Profiles:
Enable and configure security modules such as seccomp, AppArmor, or SELinux to enforce strict policies on system calls and resource access. This reduces the risk of unauthorized actions within containers.

Restrict Privileged Operations:
Avoid running containers in privileged mode and limit Linux capabilities to only those necessary for the container's operation. This minimizes the potential for privilege escalation and host compromise.

Isolate Container Processes:
Utilize user namespaces and other isolation mechanisms to ensure that processes within containers operate with the least privileges and remain segregated from the host system.

Regularly Update and Patch:
Keep the container runtime, host operating system, and associated components up-to-date with security patches. Regular updates reduce exposure to known vulnerabilities.

Audit and Monitor Runtime Configurations:
Implement continuous monitoring and regular audits of container runtime configurations. Automated tools can help detect and alert on deviations from established security baselines.

References:

 - [Docker Engine Security Documentation](#)

Rec2. Strengthen network security and segmentation C-DOCKER-CONTAINER-CNT-09 Medium ☐ Not tested

- State: Recommended
- Description:

Strengthening network security and segmentation is essential to minimize exposure to external threats and ensure that only authorized communications occur between containers and external networks. Follow the steps below to implement this security control effectively:

Define and Isolate Networks:
 - Create dedicated Docker networks for different applications or service tiers to isolate container communications and limit lateral movement in case of a breach.

Restrict Network Interfaces:
 - Bind container traffic to specific host interfaces and avoid using broad network configurations that expose unnecessary ports or services.

Implement Firewall Rules:
 - Use host-based or container-level firewalls to restrict incoming and outgoing traffic to only those ports and protocols necessary for application functionality.

Apply Secure Network Configurations:
 - Avoid using Docker's default bridge network (docker0) for production workloads. Configure custom networks with enhanced security settings.

Monitor and Audit Network Traffic:
 - Regularly monitor network traffic using logging and intrusion detection systems. Audit network configurations to ensure compliance with security policies.

References:

 - [Docker Networking Documentation](#)

Component: Backend Container

2 Recommended countermeasures

Rec1. Harden the container runtime environment C-DOCKER-CONTAINER-CNT-06 Very high ☐ Not tested

- State: Recommended
- Description:

Enhancing the security posture of your container runtime environment is vital for preventing exploitation through misconfigurations and insecure defaults. Follow the steps below to implement this security control effectively:

Enforce Mandatory Security Profiles:
Enable and configure security modules such as seccomp, AppArmor, or SELinux to enforce strict policies on system calls and resource access. This reduces the risk of unauthorized actions within containers.

Restrict Privileged Operations:
Avoid running containers in privileged mode and limit Linux capabilities to only those necessary for the container's operation. This minimizes the potential for privilege escalation and host compromise.

Isolate Container Processes:
Utilize user namespaces and other isolation mechanisms to ensure that processes within containers operate with the least privileges and remain segregated from the host system.

Regularly Update and Patch:
Keep the container runtime, host operating system, and associated components up-to-date with security patches. Regular updates reduce exposure to known vulnerabilities.

Audit and Monitor Runtime Configurations:
Implement continuous monitoring and regular audits of container runtime configurations. Automated tools can help detect and alert on deviations from established security baselines.
References:

- [Docker Engine Security Documentation](#)

Rec2. Strengthen network security and segmentation C-DOCKER-CONTAINER-CNT-09 Medium Not tested

- State: Recommended
- Description:
Strengthening network security and segmentation is essential to minimize exposure to external threats and ensure that only authorized communications occur between containers and external networks. Follow the steps below to implement this security control effectively:
Define and Isolate Networks:
 - Create dedicated Docker networks for different applications or service tiers to isolate container communications and limit lateral movement in case of a breach.**Restrict Network Interfaces:**
 - Bind container traffic to specific host interfaces and avoid using broad network configurations that expose unnecessary ports or services.**Implement Firewall Rules:**
 - Use host-based or container-level firewalls to restrict incoming and outgoing traffic to only those ports and protocols necessary for application functionality.**Apply Secure Network Configurations:**
 - Avoid using Docker's default bridge network (docker0) for production workloads. Configure custom networks with enhanced security settings.**Monitor and Audit Network Traffic:**
 - Regularly monitor network traffic using logging and intrusion detection systems. Audit network configurations to ensure compliance with security policies.**References:**
 - [Docker Networking Documentation](#)

Component: MariaDB Container

2 Recommended countermeasures

Rec1. Harden the container runtime environment C-DOCKER-CONTAINER-CNT-06 Very high Not tested

- State: Recommended
- Description:
Enhancing the security posture of your container runtime environment is vital for preventing exploitation through misconfigurations and insecure defaults. Follow the steps below to implement this security control effectively:
Enforce Mandatory Security Profiles:
Enable and configure security modules such as seccomp, AppArmor, or SELinux to enforce strict policies on system calls and resource access. This reduces the risk of unauthorized actions within containers.
Restrict Privileged Operations:
Avoid running containers in privileged mode and limit Linux capabilities to only those necessary for the container's operation. This minimizes the potential for privilege escalation and host compromise.
Isolate Container Processes:
Utilize user namespaces and other isolation mechanisms to ensure that processes within containers operate with the least privileges and remain segregated from the host system.
Regularly Update and Patch:
Keep the container runtime, host operating system, and associated components up-to-date with security patches. Regular updates reduce exposure to known vulnerabilities.
Audit and Monitor Runtime Configurations:
Implement continuous monitoring and regular audits of container runtime configurations. Automated tools can help detect and alert on deviations from established security baselines.
References:
 - [Docker Engine Security Documentation](#)

Rec2. Strengthen network security and segmentation C-DOCKER-CONTAINER-CNT-09 Medium Not tested

- State: Recommended
- Description:
Strengthening network security and segmentation is essential to minimize exposure to external threats and ensure that only authorized communications occur between containers and external networks. Follow the steps below to implement this security control effectively:
Define and Isolate Networks:
 - Create dedicated Docker networks for different applications or service tiers to isolate container communications and limit lateral movement in case of a breach.**Restrict Network Interfaces:**
 - Bind container traffic to specific host interfaces and avoid using broad network configurations that expose unnecessary ports or services.**Implement Firewall Rules:**
 - Use host-based or container-level firewalls to restrict incoming and outgoing traffic to only those ports and protocols necessary for application functionality.**Apply Secure Network Configurations:**
 - Avoid using Docker's default bridge network (docker0) for production workloads. Configure custom networks with enhanced security settings.**Monitor and Audit Network Traffic:**
 - Regularly monitor network traffic using logging and intrusion detection systems. Audit network configurations to ensure compliance with security policies.**References:**
 - [Docker Networking Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Harden the container runtime environment C-DOCKER-CONTAINER-CNT-06 Very high
Rec2. Strengthen network security and segmentation C-DOCKER-CONTAINER-CNT-09 Medium

Component: User Environment

Rec1. Harden the container runtime environment C-DOCKER-CONTAINER-CNT-06 Very high
Rec2. Strengthen network security and segmentation C-DOCKER-CONTAINER-CNT-09 Medium

Component: Docker Container

Rec1. Harden the container runtime environment C-DOCKER-CONTAINER-CNT-06 Very high
Rec2. Strengthen network security and segmentation C-DOCKER-CONTAINER-CNT-09 Medium

Component: Keycloak Container

Rec1. Harden the container runtime environment C-DOCKER-CONTAINER-CNT-06 Very high
Rec2. Strengthen network security and segmentation C-DOCKER-CONTAINER-CNT-09 Medium

Component: PostgreSQL Container

Rec1. Harden the container runtime environment C-DOCKER-CONTAINER-CNT-06 Very high
Rec2. Strengthen network security and segmentation C-DOCKER-CONTAINER-CNT-09 Medium

Component: Vault Container

Rec1. Harden the container runtime environment C-DOCKER-CONTAINER-CNT-06 Very high
Rec2. Strengthen network security and segmentation C-DOCKER-CONTAINER-CNT-09 Medium

Component: Backend Container

Rec1. Harden the container runtime environment C-DOCKER-CONTAINER-CNT-06 Very high

Rec2. Strengthen network security and segmentation C-DOCKER-CONTAINER-CNT-09 Medium

Component: MariaDB Container

Rec1. Harden the container runtime environment C-DOCKER-CONTAINER-CNT-06 Very high
Rec2. Strengthen network security and segmentation C-DOCKER-CONTAINER-CNT-09 Medium

NIST 800-53: CM-2 BASELINE CONFIGURATION

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Implement a hardened container runtime configuration

C-DOCKER-CONTAINER-CNT-01

Very high

☒ Recommended

☐ Not tested

State:

☒ Recommended

Description:

Implementing a hardened container runtime configuration is essential for reducing the attack surface of your Docker deployments and preventing unauthorized actions within your container environment. Follow the steps below to achieve a robust configuration:

Configure a Read-Only Root Filesystem:

- Set the container's root filesystem to read-only to prevent unauthorized modifications.

Restrict Host Device Exposure:

- Do not expose host devices directly to containers, reducing the risk of unauthorized hardware access.

Enforce Secure Mount Propagation:

- Ensure that mount propagation is not set to shared to prevent unintended access to the host's file system.

Enable and Enforce Security Profiles:

- Activate security modules such as seccomp, AppArmor, or SELinux to enforce strict control over system calls and container behavior.

Control Privileged Operations:

- Avoid using privileged containers and restrict the use of docker exec commands with elevated options.
- Limit Linux kernel capabilities to only those necessary for container operations.

Validate cgroup and Resource Isolation:

- Ensure that cgroup usage is correctly configured to manage and isolate container resources.

Avoid Mounting the Docker Socket:

- Do not mount the Docker socket inside containers to prevent escalation of privileges and host compromise.

References:

- [Docker Engine Security Documentation](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Implement a hardened container runtime configuration

C-DOCKER-CONTAINER-CNT-01

Very high

☒ Recommended

☐ Not tested

State:

☒ Recommended

Description:

Implementing a hardened container runtime configuration is essential for reducing the attack surface of your Docker deployments and preventing unauthorized actions within your container environment. Follow the steps below to achieve a robust configuration:

Configure a Read-Only Root Filesystem:

- Set the container's root filesystem to read-only to prevent unauthorized modifications.

Restrict Host Device Exposure:

- Do not expose host devices directly to containers, reducing the risk of unauthorized hardware access.

Enforce Secure Mount Propagation:

- Ensure that mount propagation is not set to shared to prevent unintended access to the host's file system.

Enable and Enforce Security Profiles:

- Activate security modules such as seccomp, AppArmor, or SELinux to enforce strict control over system calls and container behavior.

Control Privileged Operations:

- Avoid using privileged containers and restrict the use of docker exec commands with elevated options.
- Limit Linux kernel capabilities to only those necessary for container operations.

Validate cgroup and Resource Isolation:

- Ensure that cgroup usage is correctly configured to manage and isolate container resources.

Avoid Mounting the Docker Socket:

- Do not mount the Docker socket inside containers to prevent escalation of privileges and host compromise.

References:

- [Docker Engine Security Documentation](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Implement a hardened container runtime configuration

C-DOCKER-CONTAINER-CNT-01

Very high

☒ Recommended

☐ Not tested

State:

☒ Recommended

Description:

Implementing a hardened container runtime configuration is essential for reducing the attack surface of your Docker deployments and preventing unauthorized actions within your container environment. Follow the steps below to achieve a robust configuration:

Configure a Read-Only Root Filesystem:

- Set the container's root filesystem to read-only to prevent unauthorized modifications.

Restrict Host Device Exposure:

- Do not expose host devices directly to containers, reducing the risk of unauthorized hardware access.

Enforce Secure Mount Propagation:

- Ensure that mount propagation is not set to shared to prevent unintended access to the host's file system.

Enable and Enforce Security Profiles:

- Activate security modules such as seccomp, AppArmor, or SELinux to enforce strict control over system calls and container behavior.

Control Privileged Operations:

- Avoid using privileged containers and restrict the use of docker exec commands with elevated options.
- Limit Linux kernel capabilities to only those necessary for container operations.

Validate cgroup and Resource Isolation:

- Ensure that cgroup usage is correctly configured to manage and isolate container resources.

Avoid Mounting the Docker Socket:

- Do not mount the Docker socket inside containers to prevent escalation of privileges and host compromise.

References:

- [Docker Engine Security Documentation](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high ☐ Not tested

- State: Recommended
- Description:

Implementing a hardened container runtime configuration is essential for reducing the attack surface of your Docker deployments and preventing unauthorized actions within your container environment. Follow the steps below to achieve a robust configuration:

Configure a Read-Only Root Filesystem:

 - Set the container's root filesystem to read-only to prevent unauthorized modifications.

Restrict Host Device Exposure:

 - Do not expose host devices directly to containers, reducing the risk of unauthorized hardware access.

Enforce Secure Mount Propagation:

 - Ensure that mount propagation is not set to shared to prevent unintended access to the host's file system.

Enable and Enforce Security Profiles:

 - Activate security modules such as seccomp, AppArmor, or SELinux to enforce strict control over system calls and container behavior.

Control Privileged Operations:

 - Avoid using privileged containers and restrict the use of docker exec commands with elevated options.
 - Limit Linux kernel capabilities to only those necessary for container operations.

Validate cgroup and Resource Isolation:

 - Ensure that cgroup usage is correctly configured to manage and isolate container resources.

Avoid Mounting the Docker Socket:

 - Do not mount the Docker socket inside containers to prevent escalation of privileges and host compromise.

References:

 - [Docker Engine Security Documentation](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high ☐ Not tested

- State: Recommended
- Description:

Implementing a hardened container runtime configuration is essential for reducing the attack surface of your Docker deployments and preventing unauthorized actions within your container environment. Follow the steps below to achieve a robust configuration:

Configure a Read-Only Root Filesystem:

 - Set the container's root filesystem to read-only to prevent unauthorized modifications.

Restrict Host Device Exposure:

 - Do not expose host devices directly to containers, reducing the risk of unauthorized hardware access.

Enforce Secure Mount Propagation:

 - Ensure that mount propagation is not set to shared to prevent unintended access to the host's file system.

Enable and Enforce Security Profiles:

 - Activate security modules such as seccomp, AppArmor, or SELinux to enforce strict control over system calls and container behavior.

Control Privileged Operations:

 - Avoid using privileged containers and restrict the use of docker exec commands with elevated options.
 - Limit Linux kernel capabilities to only those necessary for container operations.

Validate cgroup and Resource Isolation:

 - Ensure that cgroup usage is correctly configured to manage and isolate container resources.

Avoid Mounting the Docker Socket:

 - Do not mount the Docker socket inside containers to prevent escalation of privileges and host compromise.

References:

 - [Docker Engine Security Documentation](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high ☐ Not tested

- State: Recommended
- Description:

Implementing a hardened container runtime configuration is essential for reducing the attack surface of your Docker deployments and preventing unauthorized actions within your container environment. Follow the steps below to achieve a robust configuration:

Configure a Read-Only Root Filesystem:

 - Set the container's root filesystem to read-only to prevent unauthorized modifications.

Restrict Host Device Exposure:

 - Do not expose host devices directly to containers, reducing the risk of unauthorized hardware access.

Enforce Secure Mount Propagation:

 - Ensure that mount propagation is not set to shared to prevent unintended access to the host's file system.

Enable and Enforce Security Profiles:

 - Activate security modules such as seccomp, AppArmor, or SELinux to enforce strict control over system calls and container behavior.

Control Privileged Operations:

 - Avoid using privileged containers and restrict the use of docker exec commands with elevated options.
 - Limit Linux kernel capabilities to only those necessary for container operations.

Validate cgroup and Resource Isolation:

 - Ensure that cgroup usage is correctly configured to manage and isolate container resources.

Avoid Mounting the Docker Socket:

 - Do not mount the Docker socket inside containers to prevent escalation of privileges and host compromise.

References:

 - [Docker Engine Security Documentation](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high ☐ Not tested

- State: Recommended
- Description:

Implementing a hardened container runtime configuration is essential for reducing the attack surface of your Docker deployments and preventing unauthorized actions within your container environment. Follow the steps below to achieve a robust configuration:

Configure a Read-Only Root Filesystem:

 - Set the container's root filesystem to read-only to prevent unauthorized modifications.

Restrict Host Device Exposure:

 - Do not expose host devices directly to containers, reducing the risk of unauthorized hardware access.

Enforce Secure Mount Propagation:

 - Ensure that mount propagation is not set to shared to prevent unintended access to the host's file system.

Enable and Enforce Security Profiles:

 - Activate security modules such as seccomp, AppArmor, or SELinux to enforce strict control over system calls and container behavior.

Control Privileged Operations:

 - Avoid using privileged containers and restrict the use of docker exec commands with elevated options.
 - Limit Linux kernel capabilities to only those necessary for container operations.

Validate cgroup and Resource Isolation:

 - Ensure that cgroup usage is correctly configured to manage and isolate container resources.

Avoid Mounting the Docker Socket:

 - Do not mount the Docker socket inside containers to prevent escalation of privileges and host compromise.

References:

- [Docker Engine Security Documentation](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high ☐ Not tested

- State: ☒ Recommended
- Description:
Implementing a hardened container runtime configuration is essential for reducing the attack surface of your Docker deployments and preventing unauthorized actions within your container environment. Follow the steps below to achieve a robust configuration:
Configure a Read-Only Root Filesystem:
 - Set the container's root filesystem to read-only to prevent unauthorized modifications.**Restrict Host Device Exposure:**
 - Do not expose host devices directly to containers, reducing the risk of unauthorized hardware access.**Enforce Secure Mount Propagation:**
 - Ensure that mount propagation is not set to shared to prevent unintended access to the host's file system.**Enable and Enforce Security Profiles:**
 - Activate security modules such as seccomp, AppArmor, or SELinux to enforce strict control over system calls and container behavior.**Control Privileged Operations:**
 - Avoid using privileged containers and restrict the use of docker exec commands with elevated options.
 - Limit Linux kernel capabilities to only those necessary for container operations.**Validate cgroup and Resource Isolation:**
 - Ensure that cgroup usage is correctly configured to manage and isolate container resources.**Avoid Mounting the Docker Socket:**
 - Do not mount the Docker socket inside containers to prevent escalation of privileges and host compromise.**References:**
 - [Docker Engine Security Documentation](#)

Component: Browser

1 Recommended countermeasures

Rec1. Configure automatic browser updates C-BROWSER-CNT-07 Very high ☐ Not tested

- State: ☒ Recommended
- Description:
Implement and regularly update automatic browser updates on all client machines to ensure that the browser always runs the latest secure version. This control minimizes exposure to vulnerabilities by automatically applying patches and security improvements, reducing the window of opportunity for attackers to exploit known weaknesses.
Implementation Steps:
Enable Automatic Updates:
Configure the browser's settings to allow automatic updates. This can typically be done via built-in options or centralized management tools such as Group Policy or MDM solutions.
Verify Update Channels:
Ensure that the browser is configured to use the appropriate update channel (e.g., stable, beta) that balances security with compatibility for your organization's needs.
Monitor Update Compliance:
Regularly audit update logs and use monitoring tools to confirm that all client machines are receiving and installing updates promptly.
Test Updates in a Controlled Environment:
Before deploying updates organization-wide, test them in a controlled environment to ensure compatibility and avoid disruptions.
References:
 - [Mozilla Firefox Enterprise Policies](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high

Component: User Environment

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high

Component: Docker Container

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high

Component: Keycloak Container

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high

Component: PostgreSQL Container

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high

Component: Vault Container

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high

Component: Backend Container

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high

Component: MariaDB Container

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high

Component: Browser

Rec1. Configure automatic browser updates C-BROWSER-CNT-07 Very high

NIST 800-53: CM-6 Configuration Settings

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high ☐ Not tested

- State:  Recommended
- Description:

Implementing a hardened container runtime configuration is essential for reducing the attack surface of your Docker deployments and preventing unauthorized actions within your container environment. Follow the steps below to achieve a robust configuration:

Configure a Read-Only Root Filesystem:

 - Set the container's root filesystem to read-only to prevent unauthorized modifications.

Restrict Host Device Exposure:

 - Do not expose host devices directly to containers, reducing the risk of unauthorized hardware access.

Enforce Secure Mount Propagation:

 - Ensure that mount propagation is not set to shared to prevent unintended access to the host's file system.

Enable and Enforce Security Profiles:

 - Activate security modules such as seccomp, AppArmor, or SELinux to enforce strict control over system calls and container behavior.

Control Privileged Operations:

 - Avoid using privileged containers and restrict the use of docker exec commands with elevated options.
 - Limit Linux kernel capabilities to only those necessary for container operations.

Validate cgroup and Resource Isolation:

 - Ensure that cgroup usage is correctly configured to manage and isolate container resources.

Avoid Mounting the Docker Socket:

 - Do not mount the Docker socket inside containers to prevent escalation of privileges and host compromise.

References:

 - [Docker Engine Security Documentation](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high ☐ Not tested

- State:  Recommended
- Description:

Implementing a hardened container runtime configuration is essential for reducing the attack surface of your Docker deployments and preventing unauthorized actions within your container environment. Follow the steps below to achieve a robust configuration:

Configure a Read-Only Root Filesystem:

 - Set the container's root filesystem to read-only to prevent unauthorized modifications.

Restrict Host Device Exposure:

 - Do not expose host devices directly to containers, reducing the risk of unauthorized hardware access.

Enforce Secure Mount Propagation:

 - Ensure that mount propagation is not set to shared to prevent unintended access to the host's file system.

Enable and Enforce Security Profiles:

 - Activate security modules such as seccomp, AppArmor, or SELinux to enforce strict control over system calls and container behavior.

Control Privileged Operations:

 - Avoid using privileged containers and restrict the use of docker exec commands with elevated options.
 - Limit Linux kernel capabilities to only those necessary for container operations.

Validate cgroup and Resource Isolation:

 - Ensure that cgroup usage is correctly configured to manage and isolate container resources.

Avoid Mounting the Docker Socket:

 - Do not mount the Docker socket inside containers to prevent escalation of privileges and host compromise.

References:

 - [Docker Engine Security Documentation](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high ☐ Not tested

- State:  Recommended
- Description:

Implementing a hardened container runtime configuration is essential for reducing the attack surface of your Docker deployments and preventing unauthorized actions within your container environment. Follow the steps below to achieve a robust configuration:

Configure a Read-Only Root Filesystem:

 - Set the container's root filesystem to read-only to prevent unauthorized modifications.

Restrict Host Device Exposure:

 - Do not expose host devices directly to containers, reducing the risk of unauthorized hardware access.

Enforce Secure Mount Propagation:

 - Ensure that mount propagation is not set to shared to prevent unintended access to the host's file system.

Enable and Enforce Security Profiles:

 - Activate security modules such as seccomp, AppArmor, or SELinux to enforce strict control over system calls and container behavior.

Control Privileged Operations:

 - Avoid using privileged containers and restrict the use of docker exec commands with elevated options.
 - Limit Linux kernel capabilities to only those necessary for container operations.

Validate cgroup and Resource Isolation:

 - Ensure that cgroup usage is correctly configured to manage and isolate container resources.

Avoid Mounting the Docker Socket:

 - Do not mount the Docker socket inside containers to prevent escalation of privileges and host compromise.

References:

 - [Docker Engine Security Documentation](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high ☐ Not tested

- State:  Recommended
- Description:

Implementing a hardened container runtime configuration is essential for reducing the attack surface of your Docker deployments and preventing unauthorized actions within your container environment. Follow the steps below to achieve a robust configuration:

Configure a Read-Only Root Filesystem:

 - Set the container's root filesystem to read-only to prevent unauthorized modifications.

Restrict Host Device Exposure:

 - Do not expose host devices directly to containers, reducing the risk of unauthorized hardware access.

Enforce Secure Mount Propagation:

 - Ensure that mount propagation is not set to shared to prevent unintended access to the host's file system.

Enable and Enforce Security Profiles:

 - Activate security modules such as seccomp, AppArmor, or SELinux to enforce strict control over system calls and container behavior.

Control Privileged Operations:

 - Avoid using privileged containers and restrict the use of docker exec commands with elevated options.
 - Limit Linux kernel capabilities to only those necessary for container operations.

Validate cgroup and Resource Isolation:

 - Ensure that cgroup usage is correctly configured to manage and isolate container resources.

Avoid Mounting the Docker Socket:

- Do not mount the Docker socket inside containers to prevent escalation of privileges and host compromise.
- References:
- [Docker Engine Security Documentation](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high ☐ Not tested

- State: Recommended
 - Description:
Implementing a hardened container runtime configuration is essential for reducing the attack surface of your Docker deployments and preventing unauthorized actions within your container environment. Follow the steps below to achieve a robust configuration:
Configure a Read-Only Root Filesystem:
 - Set the container's root filesystem to read-only to prevent unauthorized modifications.**Restrict Host Device Exposure:**
 - Do not expose host devices directly to containers, reducing the risk of unauthorized hardware access.**Enforce Secure Mount Propagation:**
 - Ensure that mount propagation is not set to shared to prevent unintended access to the host's file system.**Enable and Enforce Security Profiles:**
 - Activate security modules such as seccomp, AppArmor, or SELinux to enforce strict control over system calls and container behavior.**Control Privileged Operations:**
 - Avoid using privileged containers and restrict the use of docker exec commands with elevated options.
 - Limit Linux kernel capabilities to only those necessary for container operations.**Validate cgroup and Resource Isolation:**
 - Ensure that cgroup usage is correctly configured to manage and isolate container resources.**Avoid Mounting the Docker Socket:**
 - Do not mount the Docker socket inside containers to prevent escalation of privileges and host compromise.
- References:
- [Docker Engine Security Documentation](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high ☐ Not tested

- State: Recommended
 - Description:
Implementing a hardened container runtime configuration is essential for reducing the attack surface of your Docker deployments and preventing unauthorized actions within your container environment. Follow the steps below to achieve a robust configuration:
Configure a Read-Only Root Filesystem:
 - Set the container's root filesystem to read-only to prevent unauthorized modifications.**Restrict Host Device Exposure:**
 - Do not expose host devices directly to containers, reducing the risk of unauthorized hardware access.**Enforce Secure Mount Propagation:**
 - Ensure that mount propagation is not set to shared to prevent unintended access to the host's file system.**Enable and Enforce Security Profiles:**
 - Activate security modules such as seccomp, AppArmor, or SELinux to enforce strict control over system calls and container behavior.**Control Privileged Operations:**
 - Avoid using privileged containers and restrict the use of docker exec commands with elevated options.
 - Limit Linux kernel capabilities to only those necessary for container operations.**Validate cgroup and Resource Isolation:**
 - Ensure that cgroup usage is correctly configured to manage and isolate container resources.**Avoid Mounting the Docker Socket:**
 - Do not mount the Docker socket inside containers to prevent escalation of privileges and host compromise.
- References:
- [Docker Engine Security Documentation](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high ☐ Not tested

- State: Recommended
 - Description:
Implementing a hardened container runtime configuration is essential for reducing the attack surface of your Docker deployments and preventing unauthorized actions within your container environment. Follow the steps below to achieve a robust configuration:
Configure a Read-Only Root Filesystem:
 - Set the container's root filesystem to read-only to prevent unauthorized modifications.**Restrict Host Device Exposure:**
 - Do not expose host devices directly to containers, reducing the risk of unauthorized hardware access.**Enforce Secure Mount Propagation:**
 - Ensure that mount propagation is not set to shared to prevent unintended access to the host's file system.**Enable and Enforce Security Profiles:**
 - Activate security modules such as seccomp, AppArmor, or SELinux to enforce strict control over system calls and container behavior.**Control Privileged Operations:**
 - Avoid using privileged containers and restrict the use of docker exec commands with elevated options.
 - Limit Linux kernel capabilities to only those necessary for container operations.**Validate cgroup and Resource Isolation:**
 - Ensure that cgroup usage is correctly configured to manage and isolate container resources.**Avoid Mounting the Docker Socket:**
 - Do not mount the Docker socket inside containers to prevent escalation of privileges and host compromise.
- References:
- [Docker Engine Security Documentation](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01 Very high ☐ Not tested

- State: Recommended
- Description:
Implementing a hardened container runtime configuration is essential for reducing the attack surface of your Docker deployments and preventing unauthorized actions within your container environment. Follow the steps below to achieve a robust configuration:
Configure a Read-Only Root Filesystem:
 - Set the container's root filesystem to read-only to prevent unauthorized modifications.**Restrict Host Device Exposure:**
 - Do not expose host devices directly to containers, reducing the risk of unauthorized hardware access.**Enforce Secure Mount Propagation:**
 - Ensure that mount propagation is not set to shared to prevent unintended access to the host's file system.**Enable and Enforce Security Profiles:**

- Activate security modules such as seccomp, AppArmor, or SELinux to enforce strict control over system calls and container behavior.
- Control Privileged Operations:**
- Avoid using privileged containers and restrict the use of docker exec commands with elevated options.
 - Limit Linux kernel capabilities to only those necessary for container operations.
- Validate cgroup and Resource Isolation:**
- Ensure that cgroup usage is correctly configured to manage and isolate container resources.
- Avoid Mounting the Docker Socket:**
- Do not mount the Docker socket inside containers to prevent escalation of privileges and host compromise.
- References:**
- [Docker Engine Security Documentation](#)

Component: Browser

1 Recommended countermeasures

Rec1. Configure automatic browser updates C-BROWSER-CNT-07

Very high

Not tested

• State:

Recommended

• Description:
Implement and regularly update automatic browser updates on all client machines to ensure that the browser always runs the latest secure version. This control minimizes exposure to vulnerabilities by automatically applying patches and security improvements, reducing the window of opportunity for attackers to exploit known weaknesses.
Implementation Steps:
Enable Automatic Updates:
Configure the browser's settings to allow automatic updates. This can typically be done via built-in options or centralized management tools such as Group Policy or MDM solutions.
Verify Update Channels:
Ensure that the browser is configured to use the appropriate update channel (e.g., stable, beta) that balances security with compatibility for your organization's needs.
Monitor Update Compliance:
Regularly audit update logs and use monitoring tools to confirm that all client machines are receiving and installing updates promptly.
Test Updates in a Controlled Environment:
Before deploying updates organization-wide, test them in a controlled environment to ensure compatibility and avoid disruptions.
References:

- [Mozilla Firefox Enterprise Policies](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01

Very high

Component: User Environment

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01

Very high

Component: Docker Container

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01

Very high

Component: Keycloak Container

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01

Very high

Component: PostgreSQL Container

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01

Very high

Component: Vault Container

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01

Very high

Component: Backend Container

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01

Very high

Component: MariaDB Container

Rec1. Implement a hardened container runtime configuration C-DOCKER-CONTAINER-CNT-01

Very high

Component: Browser

Rec1. Configure automatic browser updates C-BROWSER-CNT-07

Very high

NIST 800-53: SC-6 Resource Availability

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Mitigate resource exhaustion by setting strict resource limits and policies C-DOCKER-CONTAINER-CNT-03

High

Not tested

• State:

Recommended

• Description:
Mitigating resource exhaustion by setting strict resource limits and policies is crucial to ensure that no single container can monopolize system resources, leading to performance degradation or denial-of-service conditions. Follow the steps below to implement this security control effectively:
Define Memory Limits:

- Configure the container's maximum memory usage using Docker's --memory flag to prevent any container from consuming excessive RAM, which could starve other processes.

Set CPU Quotas and Shares:

- Utilize CPU limits with the --cpus or --cpu-quota options to ensure that containers do not exceed their allocated CPU resources, maintaining balanced processing power across services.

Limit Process Count:

- Implement PID cgroup limits by using the --pids-limit flag, restricting the number of processes a container can spawn to avoid resource exhaustion attacks.

Enforce Ulimit Settings:

- Override the default ulimit settings where necessary to control the maximum number of open files and other system resources available to a container.

Implement Restart Policies:

- Configure container restart policies (e.g., --restart on-failure) to mitigate the impact of resource-intensive failures and avoid continuous, unrestrained restarts that can further exhaust system resources.

Monitor and Adjust Resource Usage:

- Continuously monitor container resource consumption using tools such as Docker stats or integrated monitoring solutions, and adjust resource limits as needed based on workload demands and observed performance.

References:

- [Docker Resource Constraints](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Mitigate resource exhaustion by setting strict resource limits and policies C-DOCKER-CONTAINER-CNT-03 High Not tested

- **State:** Recommended
- **Description:**

Mitigating resource exhaustion by setting strict resource limits and policies is crucial to ensure that no single container can monopolize system resources, leading to performance degradation or denial-of-service conditions. Follow the steps below to implement this security control effectively:

Define Memory Limits:

 - Configure the container's maximum memory usage using Docker's --memory flag to prevent any container from consuming excessive RAM, which could starve other processes.

Set CPU Quotas and Shares:

 - Utilize CPU limits with the --cpus or --cpu-quota options to ensure that containers do not exceed their allocated CPU resources, maintaining balanced processing power across services.

Limit Process Count:

 - Implement PID cgroup limits by using the --pids-limit flag, restricting the number of processes a container can spawn to avoid resource exhaustion attacks.

Enforce Ulimit Settings:

 - Override the default ulimit settings where necessary to control the maximum number of open files and other system resources available to a container.

Implement Restart Policies:

 - Configure container restart policies (e.g., --restart on-failure) to mitigate the impact of resource-intensive failures and avoid continuous, unrestrained restarts that can further exhaust system resources.

Monitor and Adjust Resource Usage:

 - Continuously monitor container resource consumption using tools such as Docker stats or integrated monitoring solutions, and adjust resource limits as needed based on workload demands and observed performance.
- **References:**
 - [Docker Resource Constraints](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Mitigate resource exhaustion by setting strict resource limits and policies C-DOCKER-CONTAINER-CNT-03 High Not tested

- **State:** Recommended
- **Description:**

Mitigating resource exhaustion by setting strict resource limits and policies is crucial to ensure that no single container can monopolize system resources, leading to performance degradation or denial-of-service conditions. Follow the steps below to implement this security control effectively:

Define Memory Limits:

 - Configure the container's maximum memory usage using Docker's --memory flag to prevent any container from consuming excessive RAM, which could starve other processes.

Set CPU Quotas and Shares:

 - Utilize CPU limits with the --cpus or --cpu-quota options to ensure that containers do not exceed their allocated CPU resources, maintaining balanced processing power across services.

Limit Process Count:

 - Implement PID cgroup limits by using the --pids-limit flag, restricting the number of processes a container can spawn to avoid resource exhaustion attacks.

Enforce Ulimit Settings:

 - Override the default ulimit settings where necessary to control the maximum number of open files and other system resources available to a container.

Implement Restart Policies:

 - Configure container restart policies (e.g., --restart on-failure) to mitigate the impact of resource-intensive failures and avoid continuous, unrestrained restarts that can further exhaust system resources.

Monitor and Adjust Resource Usage:

 - Continuously monitor container resource consumption using tools such as Docker stats or integrated monitoring solutions, and adjust resource limits as needed based on workload demands and observed performance.
- **References:**
 - [Docker Resource Constraints](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Mitigate resource exhaustion by setting strict resource limits and policies C-DOCKER-CONTAINER-CNT-03 High Not tested

- **State:** Recommended
- **Description:**

Mitigating resource exhaustion by setting strict resource limits and policies is crucial to ensure that no single container can monopolize system resources, leading to performance degradation or denial-of-service conditions. Follow the steps below to implement this security control effectively:

Define Memory Limits:

 - Configure the container's maximum memory usage using Docker's --memory flag to prevent any container from consuming excessive RAM, which could starve other processes.

Set CPU Quotas and Shares:

 - Utilize CPU limits with the --cpus or --cpu-quota options to ensure that containers do not exceed their allocated CPU resources, maintaining balanced processing power across services.

Limit Process Count:

 - Implement PID cgroup limits by using the --pids-limit flag, restricting the number of processes a container can spawn to avoid resource exhaustion attacks.

Enforce Ulimit Settings:

 - Override the default ulimit settings where necessary to control the maximum number of open files and other system resources available to a container.

Implement Restart Policies:

 - Configure container restart policies (e.g., --restart on-failure) to mitigate the impact of resource-intensive failures and avoid continuous, unrestrained restarts that can further exhaust system resources.

Monitor and Adjust Resource Usage:

 - Continuously monitor container resource consumption using tools such as Docker stats or integrated monitoring solutions, and adjust resource limits as needed based on workload demands and observed performance.
- **References:**
 - [Docker Resource Constraints](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Mitigate resource exhaustion by setting strict resource limits and policies C-DOCKER-CONTAINER-CNT-03 High Not tested

- **State:** Recommended
- **Description:**

Mitigating resource exhaustion by setting strict resource limits and policies is crucial to ensure that no single container can monopolize system resources, leading to performance degradation or denial-of-service conditions. Follow the steps below to implement this security control effectively:

Define Memory Limits:

 - Configure the container's maximum memory usage using Docker's --memory flag to prevent any container from consuming excessive RAM, which could starve other processes.

Set CPU Quotas and Shares:

 - Utilize CPU limits with the --cpus or --cpu-quota options to ensure that containers do not exceed their allocated CPU resources, maintaining balanced processing power across services.

Limit Process Count:

 - Implement PID cgroup limits by using the --pids-limit flag, restricting the number of processes a container can spawn to avoid resource exhaustion attacks.

Enforce Ulimit Settings:

 - Override the default ulimit settings where necessary to control the maximum number of open files and other system resources available to a container.

Implement Restart Policies:

 - Configure container restart policies (e.g., --restart on-failure) to mitigate the impact of resource-intensive failures and avoid continuous, unrestrained restarts that can further exhaust system resources.

Monitor and Adjust Resource Usage:

- Continuously monitor container resource consumption using tools such as Docker stats or integrated monitoring solutions, and adjust resource limits as needed based on workload demands and observed performance.

References:

- [Docker Resource Constraints](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Mitigate resource exhaustion by setting strict resource limits and policies C-DOCKER-CONTAINER-CNT-03 High Not tested

- **State:** Recommended
- **Description:**

Mitigating resource exhaustion by setting strict resource limits and policies is crucial to ensure that no single container can monopolize system resources, leading to performance degradation or denial-of-service conditions. Follow the steps below to implement this security control effectively:

Define Memory Limits:

 - Configure the container's maximum memory usage using Docker's --memory flag to prevent any container from consuming excessive RAM, which could starve other processes.

Set CPU Quotas and Shares:

 - Utilize CPU limits with the --cpus or --cpu-quota options to ensure that containers do not exceed their allocated CPU resources, maintaining balanced processing power across services.

Limit Process Count:

 - Implement PID cgroup limits by using the --pids-limit flag, restricting the number of processes a container can spawn to avoid resource exhaustion attacks.

Enforce Ulimit Settings:

 - Override the default ulimit settings where necessary to control the maximum number of open files and other system resources available to a container.

Implement Restart Policies:

 - Configure container restart policies (e.g., --restart on-failure) to mitigate the impact of resource-intensive failures and avoid continuous, unrestrained restarts that can further exhaust system resources.

Monitor and Adjust Resource Usage:

 - Continuously monitor container resource consumption using tools such as Docker stats or integrated monitoring solutions, and adjust resource limits as needed based on workload demands and observed performance.
- **References:**
 - [Docker Resource Constraints](#)

Component: Backend Container

1 Recommended countermeasures

Rec1. Mitigate resource exhaustion by setting strict resource limits and policies C-DOCKER-CONTAINER-CNT-03 High Not tested

- **State:** Recommended
- **Description:**

Mitigating resource exhaustion by setting strict resource limits and policies is crucial to ensure that no single container can monopolize system resources, leading to performance degradation or denial-of-service conditions. Follow the steps below to implement this security control effectively:

Define Memory Limits:

 - Configure the container's maximum memory usage using Docker's --memory flag to prevent any container from consuming excessive RAM, which could starve other processes.

Set CPU Quotas and Shares:

 - Utilize CPU limits with the --cpus or --cpu-quota options to ensure that containers do not exceed their allocated CPU resources, maintaining balanced processing power across services.

Limit Process Count:

 - Implement PID cgroup limits by using the --pids-limit flag, restricting the number of processes a container can spawn to avoid resource exhaustion attacks.

Enforce Ulimit Settings:

 - Override the default ulimit settings where necessary to control the maximum number of open files and other system resources available to a container.

Implement Restart Policies:

 - Configure container restart policies (e.g., --restart on-failure) to mitigate the impact of resource-intensive failures and avoid continuous, unrestrained restarts that can further exhaust system resources.

Monitor and Adjust Resource Usage:

 - Continuously monitor container resource consumption using tools such as Docker stats or integrated monitoring solutions, and adjust resource limits as needed based on workload demands and observed performance.
- **References:**
 - [Docker Resource Constraints](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Mitigate resource exhaustion by setting strict resource limits and policies C-DOCKER-CONTAINER-CNT-03 High Not tested

- **State:** Recommended
- **Description:**

Mitigating resource exhaustion by setting strict resource limits and policies is crucial to ensure that no single container can monopolize system resources, leading to performance degradation or denial-of-service conditions. Follow the steps below to implement this security control effectively:

Define Memory Limits:

 - Configure the container's maximum memory usage using Docker's --memory flag to prevent any container from consuming excessive RAM, which could starve other processes.

Set CPU Quotas and Shares:

 - Utilize CPU limits with the --cpus or --cpu-quota options to ensure that containers do not exceed their allocated CPU resources, maintaining balanced processing power across services.

Limit Process Count:

 - Implement PID cgroup limits by using the --pids-limit flag, restricting the number of processes a container can spawn to avoid resource exhaustion attacks.

Enforce Ulimit Settings:

 - Override the default ulimit settings where necessary to control the maximum number of open files and other system resources available to a container.

Implement Restart Policies:

 - Configure container restart policies (e.g., --restart on-failure) to mitigate the impact of resource-intensive failures and avoid continuous, unrestrained restarts that can further exhaust system resources.

Monitor and Adjust Resource Usage:

 - Continuously monitor container resource consumption using tools such as Docker stats or integrated monitoring solutions, and adjust resource limits as needed based on workload demands and observed performance.
- **References:**
 - [Docker Resource Constraints](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Mitigate resource exhaustion by setting strict resource limits and policies C-DOCKER-CONTAINER-CNT-03 High

Component: User Environment

Rec1. Mitigate resource exhaustion by setting strict resource limits and policies C-DOCKER-CONTAINER-CNT-03 High

Component: Docker Container

Rec1. Mitigate resource exhaustion by setting strict resource limits and policies C-DOCKER-CONTAINER-CNT-03 High

Component: Keycloak Container

Rec1. Mitigate resource exhaustion by setting strict resource limits and policies C-DOCKER-CONTAINER-CNT-03 High

Component: PostgreSQL Container

Rec1. Mitigate resource exhaustion by setting strict resource limits and policies C-DOCKER-CONTAINER-CNT-03 ^ High

Component: Vault Container

Rec1. Mitigate resource exhaustion by setting strict resource limits and policies C-DOCKER-CONTAINER-CNT-03 ^ High

Component: Backend Container

Rec1. Mitigate resource exhaustion by setting strict resource limits and policies C-DOCKER-CONTAINER-CNT-03 ^ High

Component: MariaDB Container

Rec1. Mitigate resource exhaustion by setting strict resource limits and policies C-DOCKER-CONTAINER-CNT-03 ^ High

NIST 800-53: SC-28 PROTECTION OF INFORMATION AT REST

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Recommendation	Category	Severity	Status
Rec1. Remove sensitive information from Dockerfile or Docker-Compose files	C-DOCKER-CONTAINER-CNT-05	High	Not tested

- **State:**  Recommended
- **Description:**

Removing sensitive information from Dockerfiles or Docker-Compose files is critical to safeguarding your application's confidentiality and preventing unauthorized access. Follow the steps below to implement this security control effectively:

Identify Sensitive Data:

 - Audit your Dockerfiles and Docker-Compose files to locate hardcoded credentials, API keys, tokens, and other sensitive configurations.

Remove or Externalize Secrets:

 - Eliminate sensitive data from your build files. Instead, leverage environment variables, secure secrets management solutions, or dedicated vaults (e.g., HashiCorp Vault or AWS Secrets Manager) to handle credentials.

Implement .dockerignore Files:

 - Use a .dockerignore file to prevent sensitive configuration files or directories from being included in your build context.

Review and Audit Regularly:

 - Periodically review your build files to ensure that no sensitive information is inadvertently added. Integrate automated scanning tools into your CI/CD pipeline to detect and flag sensitive data.

References:

 - [Docker Security Best Practices](#)

Component: User Environment

1 Recommended countermeasures

Rec	Findings	CVSS	Severity	Tested
Rec1.	Remove sensitive information from Dockerfile or Docker-Compose files	C-DOCKER-CONTAINER-CNT-05	High	Not tested

- **State:**  Recommended
- **Description:**

Removing sensitive information from Dockerfiles or Docker-Compose files is critical to safeguarding your application's confidentiality and preventing unauthorized access. Follow the steps below to implement this security control effectively:

Identify Sensitive Data:

 - Audit your Dockerfiles and Docker-Compose files to locate hardcoded credentials, API keys, tokens, and other sensitive configurations.

Remove or Externalize Secrets:

 - Eliminate sensitive data from your build files. Instead, leverage environment variables, secure secrets management solutions, or dedicated vaults (e.g., HashiCorp Vault or AWS Secrets Manager) to handle credentials.

Implement .dockerignore Files:

 - Use a .dockerignore file to prevent sensitive configuration files or directories from being included in your build context.

Review and Audit Regularly:

 - Periodically review your build files to ensure that no sensitive information is inadvertently added. Integrate automated scanning tools into your CI/CD pipeline to detect and flag sensitive data.

References:

 - [Docker Security Best Practices](#)

Component: Docker Container

1 Recommended countermeasures

Rec	Findings	CVSS	Severity	Tested
Rec1.	Remove sensitive information from Dockerfile or Docker-Compose files	C-DOCKER-CONTAINER-CNT-05	High	Not tested

- **State:**  Recommended
- **Description:**

Removing sensitive information from Dockerfiles or Docker-Compose files is critical to safeguarding your application's confidentiality and preventing unauthorized access. Follow the steps below to implement this security control effectively:

Identify Sensitive Data:

 - Audit your Dockerfiles and Docker-Compose files to locate hardcoded credentials, API keys, tokens, and other sensitive configurations.

Remove or Externalize Secrets:

 - Eliminate sensitive data from your build files. Instead, leverage environment variables, secure secrets management solutions, or dedicated vaults (e.g., HashiCorp Vault or AWS Secrets Manager) to handle credentials.

Implement .dockerignore Files:

 - Use a .dockerignore file to prevent sensitive configuration files or directories from being included in your build context.

Review and Audit Regularly:

 - Periodically review your build files to ensure that no sensitive information is inadvertently added. Integrate automated scanning tools into your CI/CD pipeline to detect and flag sensitive data.

References:

 - [Docker Security Best Practices](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Remove sensitive information from Dockerfile or Docker-Compose files	C-DOCKER-CONTAINER-CNT-05	High	Not tested
--	---------------------------	------	------------

- **State:**  Recommended
- **Description:**

Removing sensitive information from Dockerfiles or Docker-Compose files is critical to safeguarding your application's confidentiality and preventing unauthorized access. Follow the steps below to implement this security control effectively:

Identify Sensitive Data:

 - Audit your Dockerfiles and Docker-Compose files to locate hardcoded credentials, API keys, tokens, and other sensitive configurations.

Remove or Externalize Secrets:

 - Eliminate sensitive data from your build files. Instead, leverage environment variables, secure secrets management solutions, or dedicated vaults (e.g., HashiCorp Vault or AWS Secrets Manager) to handle credentials.

Implement .dockerignore Files:

 - Use a .dockerignore file to prevent sensitive configuration files or directories from being included in your build context.

Review and Audit Regularly:

- Periodically review your build files to ensure that no sensitive information is inadvertently added. Integrate automated scanning tools into your CI/CD pipeline to detect and flag sensitive data.

References:

- [Docker Security Best Practices](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Remove sensitive information from Dockerfile or Docker-Compose files C-DOCKER-CONTAINER-CNT-05

High

Not tested

State: Recommended

Description:

Removing sensitive information from Dockerfiles or Docker-Compose files is critical to safeguarding your application's confidentiality and preventing unauthorized access. Follow the steps below to implement this security control effectively:

Identify Sensitive Data:

- Audit your Dockerfiles and Docker-Compose files to locate hardcoded credentials, API keys, tokens, and other sensitive configurations.

Remove or Externalize Secrets:

- Eliminate sensitive data from your build files. Instead, leverage environment variables, secure secrets management solutions, or dedicated vaults (e.g., HashiCorp Vault or AWS Secrets Manager) to handle credentials.

Implement .dockerignore Files:

- Use a .dockerignore file to prevent sensitive configuration files or directories from being included in your build context.

Review and Audit Regularly:

- Periodically review your build files to ensure that no sensitive information is inadvertently added. Integrate automated scanning tools into your CI/CD pipeline to detect and flag sensitive data.

References:

- [Docker Security Best Practices](#)

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Encrypt data at rest using Transparent Data Encryption (TDE) or Volume-Level Encryption C-POSTGRESQL-CNT-POSTGRES-11

High

Not tested

State: Recommended

Description:

Implement encryption for all data stored on disk to protect against physical theft of storage media or unauthorized access to the underlying filesystem. PostgreSQL does not natively support Transparent Data Encryption (TDE) in the community edition (though it is available in some enterprise forks); therefore, this control is typically achieved by using **Volume-Level Encryption** (e.g., LUKS, AWS EBS Encryption, Azure Disk Encryption) or **File-System Level Encryption**. This ensures that the data files, WAL (Write-Ahead Logs), and temporary files are unreadable without the appropriate cryptographic keys. The objective of this control is to ensure **Data Confidentiality at the Storage Layer**. It serves as a critical defense-in-depth measure, ensuring that even if an attacker gains access to a backup tape, a discarded hard drive, or the raw cloud storage volume, the data remains encrypted and inaccessible.

Implementation guidance for developers and DevOps engineers:

- Use Cloud Provider Managed Encryption:** If running on managed services (e.g., AWS RDS, GCP Cloud SQL), enable the "Encrypt Storage" option during instance creation. These services use AES-256 encryption with keys managed by **KMS (Key Management Service)**.
- Implement OS-Level Encryption (Self-Hosted):** For on-premises or IaaS deployments, use **LUKS (Linux Unified Key Setup)** to encrypt the partition where the PostgreSQL data directory (PGDATA) resides.
 - Command Example:* cryptsetup luksFormat /dev/sdb1 and cryptsetup luksOpen /dev/sdb1 pg_data.
- Encrypt Backups:** Ensure that database backups (created via pg_dump or pg_basebackup) are encrypted before being moved to secondary storage. Use tools like **GnuPG** or the built-in encryption features of backup managers like **pgBackRest**.
- Protect the WAL and Temp Files:** Ensure that the encryption scope includes the directories for **Write-Ahead Logs (WAL)** and **temporary files**, as these often contain fragments of sensitive data in plaintext.
- Manage Keys Securely:** Never store encryption keys on the same physical or virtual disk as the data. Use a hardware security module (HSM) or a dedicated secret management service (e.g., **HashiCorp Vault**) to inject keys at boot time.
- Rotate Encryption Keys:** Establish a lifecycle policy to rotate the master encryption keys periodically without requiring a full database re-encryption, which is often supported by KMS and advanced volume drivers.
- Monitor Performance Impact:** While modern CPUs use **AES-NI** hardware acceleration to minimize overhead, perform a baseline test to ensure that encryption-induced latency does not violate your application's SLA.

References:

- [PostgreSQL Documentation: Encryption Options](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Remove sensitive information from Dockerfile or Docker-Compose files C-DOCKER-CONTAINER-CNT-05

High

Not tested

State: Recommended

Description:

Removing sensitive information from Dockerfiles or Docker-Compose files is critical to safeguarding your application's confidentiality and preventing unauthorized access. Follow the steps below to implement this security control effectively:

Identify Sensitive Data:

- Audit your Dockerfiles and Docker-Compose files to locate hardcoded credentials, API keys, tokens, and other sensitive configurations.

Remove or Externalize Secrets:

- Eliminate sensitive data from your build files. Instead, leverage environment variables, secure secrets management solutions, or dedicated vaults (e.g., HashiCorp Vault or AWS Secrets Manager) to handle credentials.

Implement .dockerignore Files:

- Use a .dockerignore file to prevent sensitive configuration files or directories from being included in your build context.

Review and Audit Regularly:

- Periodically review your build files to ensure that no sensitive information is inadvertently added. Integrate automated scanning tools into your CI/CD pipeline to detect and flag sensitive data.

References:

- [Docker Security Best Practices](#)

Component: HashiCorp Vault

1 Recommended countermeasures

Rec1. Disable swap on the system hosting HashiCorp Vault C-HASHICORP-VAULT-CNT-02

High

Not tested

State: Recommended

Description:

To enhance the security of sensitive data handled by HashiCorp Vault, it is advisable to disable swap space on your system. This measure prevents sensitive information from being temporarily written to disk, where it might be accessible by unauthorized users. Here is a step-by-step guide on how you can disable swap on a Linux-based system:

- Check Current Swap Status:**

Open a terminal window and execute the following command to check if swap is currently enabled:

```
sudo swapon --show
```

If there is no output, swap is not enabled. If there is output, proceed with the next steps to disable it.
- Turn Off Swap Temporarily:**

To immediately disable swap without rebooting, run the following command:

```
sudo swapoff -a
```
- Disable Swap Permanently:**

Edit the `/etc/fstab` file to remove or comment out any lines that define swap partitions or swap files. Open the file using a text editor, such as nano:

```
sudo nano /etc/fstab
```

Locate the line that looks something like:

```
/swapfile swap swap defaults 0 0
```

Add a comment symbol `#` at the beginning of the line to disable it:

```
# /swapfile swap swap defaults 0 0
```

Save the file and exit the text editor.
- Verify Swap is Disabled:**

Reboot the system to ensure that swap is not enabled on startup. After rebooting, use the following command to confirm that swap has been successfully disabled:

```
sudo swapon --show
```

You should see no output, indicating that swap is indeed disabled.

By disabling swap, you significantly reduce the risk of sensitive data exposure, thereby strengthening the security posture of your HashiCorp Vault implementation.

Component: Backend Container

1 Recommended countermeasures

Rec1. Remove sensitive information from Dockerfile or Docker-Compose files C-DOCKER-CONTAINER-CNT-05

High

Not tested

State: Recommended

Description:
Removing sensitive information from Dockerfiles or Docker-Compose files is critical to safeguarding your application's confidentiality and preventing unauthorized access. Follow the steps below to implement this security control effectively.
Identify Sensitive Data:

- Audit your Dockerfiles and Docker-Compose files to locate hardcoded credentials, API keys, tokens, and other sensitive configurations.

Remove or Externalize Secrets:

- Eliminate sensitive data from your build files. Instead, leverage environment variables, secure secrets management solutions, or dedicated vaults (e.g., HashiCorp Vault or AWS Secrets Manager) to handle credentials.

Implement .dockerignore Files:

- Use a .dockerignore file to prevent sensitive configuration files or directories from being included in your build context.

Review and Audit Regularly:

- Periodically review your build files to ensure that no sensitive information is inadvertently added. Integrate automated scanning tools into your CI/CD pipeline to detect and flag sensitive data.

References:

- [Docker Security Best Practices](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Remove sensitive information from Dockerfile or Docker-Compose files C-DOCKER-CONTAINER-CNT-05

High

Not tested

State: Recommended

Description:
Removing sensitive information from Dockerfiles or Docker-Compose files is critical to safeguarding your application's confidentiality and preventing unauthorized access. Follow the steps below to implement this security control effectively:
Identify Sensitive Data:

- Audit your Dockerfiles and Docker-Compose files to locate hardcoded credentials, API keys, tokens, and other sensitive configurations.

Remove or Externalize Secrets:

- Eliminate sensitive data from your build files. Instead, leverage environment variables, secure secrets management solutions, or dedicated vaults (e.g., HashiCorp Vault or AWS Secrets Manager) to handle credentials.

Implement .dockerignore Files:

- Use a .dockerignore file to prevent sensitive configuration files or directories from being included in your build context.

Review and Audit Regularly:

- Periodically review your build files to ensure that no sensitive information is inadvertently added. Integrate automated scanning tools into your CI/CD pipeline to detect and flag sensitive data.

References:

- [Docker Security Best Practices](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Remove sensitive information from Dockerfile or Docker-Compose files C-DOCKER-CONTAINER-CNT-05

High

Component: User Environment

Rec1. Remove sensitive information from Dockerfile or Docker-Compose files C-DOCKER-CONTAINER-CNT-05

High

Component: Docker Container

Rec1. Remove sensitive information from Dockerfile or Docker-Compose files C-DOCKER-CONTAINER-CNT-05

High

Component: Keycloak Container

Rec1. Remove sensitive information from Dockerfile or Docker-Compose files C-DOCKER-CONTAINER-CNT-05

High

Component: PostgreSQL Container

Rec1. Remove sensitive information from Dockerfile or Docker-Compose files C-DOCKER-CONTAINER-CNT-05

High

Component: PostgreSQL

Rec1. Encrypt data at rest using Transparent Data Encryption (TDE) or Volume-Level Encryption C-POSTGRESQL-CNT-POSTGRES-11

High

Component: Vault Container

Rec1. Remove sensitive information from Dockerfile or Docker-Compose files C-DOCKER-CONTAINER-CNT-05

High

Component: HashiCorp Vault

Rec1. Disable swap on the system hosting HashiCorp Vault C-HASHICORP-VAULT-CNT-02

High

Component: Backend Container

Rec1. Remove sensitive information from Dockerfile or Docker-Compose files C-DOCKER-CONTAINER-CNT-05

High

Component: MariaDB Container

Rec1. Remove sensitive information from Dockerfile or Docker-Compose files C-DOCKER-CONTAINER-CNT-05

High

NIST 800-53: AC-6 LEAST PRIVILEGE

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Local Environment

1 Recommended countermeasures

Rec1. Ensure sensitive host system directories are not mounted on containers C-DOCKER-CONTAINER-CNT-04

Medium

Not tested

- State:

Recommended
- Description:

Enhancing container security by ensuring that sensitive host system directories are not mounted on containers helps prevent unintended data exposure and minimizes the risk of host compromise. Follow the steps below to implement this security control effectively:

Review and Identify Sensitive Directories:
Identify directories on the host that contain critical system configurations or sensitive data (e.g., /etc, /root, /var/lib) and avoid mounting them directly into containers.

Implement Strict Volume Mount Policies:
Restrict volume mounts to only those directories that are essential for application functionality. Use explicit and well-documented volume configurations in your Docker Compose files or Docker run commands.

Use Read-Only Mounts Where Applicable:
For necessary mounts, configure them as read-only to prevent modifications from within the container, reducing the risk of accidental or malicious changes.

Regularly Audit Container Configurations:
Conduct periodic reviews of container configurations and deployment scripts to ensure that no sensitive host directories are inadvertently mounted.

References:

 - [Docker Volumes Documentation](#)

Component: User Environment

1 Recommended countermeasures

Rec1. Ensure sensitive host system directories are not mounted on containers C-DOCKER-CONTAINER-CNT-04

Medium

Not tested

- State:

Recommended
- Description:

Enhancing container security by ensuring that sensitive host system directories are not mounted on containers helps prevent unintended data exposure and minimizes the risk of host compromise. Follow the steps below to implement this security control effectively:

Review and Identify Sensitive Directories:
Identify directories on the host that contain critical system configurations or sensitive data (e.g., /etc, /root, /var/lib) and avoid mounting them directly into containers.

Implement Strict Volume Mount Policies:
Restrict volume mounts to only those directories that are essential for application functionality. Use explicit and well-documented volume configurations in your Docker Compose files or Docker run commands.

Use Read-Only Mounts Where Applicable:
For necessary mounts, configure them as read-only to prevent modifications from within the container, reducing the risk of accidental or malicious changes.

Regularly Audit Container Configurations:
Conduct periodic reviews of container configurations and deployment scripts to ensure that no sensitive host directories are inadvertently mounted.

References:

 - [Docker Volumes Documentation](#)

Component: Docker Container

1 Recommended countermeasures

Rec1. Ensure sensitive host system directories are not mounted on containers C-DOCKER-CONTAINER-CNT-04

Medium

Not tested

- State:

Recommended
- Description:

Enhancing container security by ensuring that sensitive host system directories are not mounted on containers helps prevent unintended data exposure and minimizes the risk of host compromise. Follow the steps below to implement this security control effectively:

Review and Identify Sensitive Directories:
Identify directories on the host that contain critical system configurations or sensitive data (e.g., /etc, /root, /var/lib) and avoid mounting them directly into containers.

Implement Strict Volume Mount Policies:
Restrict volume mounts to only those directories that are essential for application functionality. Use explicit and well-documented volume configurations in your Docker Compose files or Docker run commands.

Use Read-Only Mounts Where Applicable:
For necessary mounts, configure them as read-only to prevent modifications from within the container, reducing the risk of accidental or malicious changes.

Regularly Audit Container Configurations:
Conduct periodic reviews of container configurations and deployment scripts to ensure that no sensitive host directories are inadvertently mounted.

References:

 - [Docker Volumes Documentation](#)

Component: Keycloak Container

1 Recommended countermeasures

Rec1. Ensure sensitive host system directories are not mounted on containers C-DOCKER-CONTAINER-CNT-04

Medium

Not tested

- State:

Recommended
- Description:

Enhancing container security by ensuring that sensitive host system directories are not mounted on containers helps prevent unintended data exposure and minimizes the risk of host compromise. Follow the steps below to implement this security control effectively:

Review and Identify Sensitive Directories:
Identify directories on the host that contain critical system configurations or sensitive data (e.g., /etc, /root, /var/lib) and avoid mounting them directly into containers.

Implement Strict Volume Mount Policies:
Restrict volume mounts to only those directories that are essential for application functionality. Use explicit and well-documented volume configurations in your Docker Compose files or Docker run commands.

Use Read-Only Mounts Where Applicable:
For necessary mounts, configure them as read-only to prevent modifications from within the container, reducing the risk of accidental or malicious changes.

Regularly Audit Container Configurations:
Conduct periodic reviews of container configurations and deployment scripts to ensure that no sensitive host directories are inadvertently mounted.

References:

 - [Docker Volumes Documentation](#)

Component: PostgreSQL Container

1 Recommended countermeasures

Rec1. Ensure sensitive host system directories are not mounted on containers C-DOCKER-CONTAINER-CNT-04

Medium

Not tested

- State:

Recommended
- Description:

Enhancing container security by ensuring that sensitive host system directories are not mounted on containers helps prevent unintended data exposure and minimizes the risk of host compromise. Follow the steps below to implement this security control effectively:

Review and Identify Sensitive Directories:
Identify directories on the host that contain critical system configurations or sensitive data (e.g., /etc, /root, /var/lib) and avoid mounting them directly into containers.

Implement Strict Volume Mount Policies:
Restrict volume mounts to only those directories that are essential for application functionality. Use explicit and well-documented volume configurations in your Docker Compose files or Docker run commands.

Use Read-Only Mounts Where Applicable:
For necessary mounts, configure them as read-only to prevent modifications from within the container, reducing the risk of accidental or malicious changes.

Regularly Audit Container Configurations:
Conduct periodic reviews of container configurations and deployment scripts to ensure that no sensitive host directories are inadvertently mounted.

References:

 - [Docker Volumes Documentation](#)

Component: Vault Container

1 Recommended countermeasures

Rec1. Ensure sensitive host system directories are not mounted on containers C-DOCKER-CONTAINER-CNT-04

Medium

Not tested

- State: Recommended
- Description:

Enhancing container security by ensuring that sensitive host system directories are not mounted on containers helps prevent unintended data exposure and minimizes the risk of host compromise. Follow the steps below to implement this security control effectively:

Review and Identify Sensitive Directories:
Identify directories on the host that contain critical system configurations or sensitive data (e.g., /etc, /root, /var/lib) and avoid mounting them directly into containers.

Implement Strict Volume Mount Policies:
Restrict volume mounts to only those directories that are essential for application functionality. Use explicit and well-documented volume configurations in your Docker Compose files or Docker run commands.

Use Read-Only Mounts Where Applicable:
For necessary mounts, configure them as read-only to prevent modifications from within the container, reducing the risk of accidental or malicious changes.

Regularly Audit Container Configurations:
Conduct periodic reviews of container configurations and deployment scripts to ensure that no sensitive host directories are inadvertently mounted.

References:

 - [Docker Volumes Documentation](#)

Component: HashiCorp Vault

1 Recommended countermeasures

Rec1. Avoid running HashiCorp Vault as root C-HASHICORP-VAULT-CNT-03

Medium

Not tested

- State: Recommended
- Description:

To enhance the security posture of HashiCorp Vault and reduce the risk of privilege escalation, it is essential to configure Vault to run under a non-privileged user account. This configuration helps in minimizing potential damage in case of a security breach. Follow these steps to implement this control:

 - Create a dedicated user account:**
On the system where Vault is installed, create a non-privileged user account specifically for running Vault. This can typically be done using a command such as:

```
sudo useradd -r -d /var/vault -s /sbin/nologin vaultuser
```


Here, -r creates a system account, -d sets the home directory, and -s specifies no login shell, reducing unnecessary access.
 - Set appropriate permissions:**
Ensure that the Vault executable, configuration files, and working directories are owned by the newly created user. Adjust the file permissions to restrict access to other users. For example:

```
sudo chown -R vaultuser:vaultuser /opt/vault
```



```
sudo chmod -R 750 /opt/vault
```


This ensures only the Vault service can access its files and directories.
 - Update the Vault service configuration:**
Modify the Vault service configuration file to run under the non-privileged user. If using systemd, this can be achieved by editing the Vault service unit file, for instance /etc/systemd/system/vault.service, and adding or modifying the following lines:

```
[Service]
User=vaultuser
Group=vaultuser
```


This specifies the user and group under which the Vault service should operate.
 - Restart the Vault service:**
Apply the changes by restarting the Vault service. For example, use:

```
sudo systemctl daemon-reload
sudo systemctl restart vault
```


This ensures that Vault runs under the appropriate user context without requiring elevated privileges.
 - Review and test:**
Finally, verify that Vault is operating as expected without any issues under the non-privileged account. Conduct a series of tests to ensure it can access necessary resources while maintaining restricted permissions to mitigate potential security risks.

By following these steps, the risk of unauthorized system modifications and privilege escalation is significantly minimized, helping secure the Vault environment against potential threats.

Component: Backend Container

1 Recommended countermeasures

Rec1. Ensure sensitive host system directories are not mounted on containers C-DOCKER-CONTAINER-CNT-04

Medium

Not tested

- State: Recommended
- Description:

Enhancing container security by ensuring that sensitive host system directories are not mounted on containers helps prevent unintended data exposure and minimizes the risk of host compromise. Follow the steps below to implement this security control effectively:

Review and Identify Sensitive Directories:
Identify directories on the host that contain critical system configurations or sensitive data (e.g., /etc, /root, /var/lib) and avoid mounting them directly into containers.

Implement Strict Volume Mount Policies:
Restrict volume mounts to only those directories that are essential for application functionality. Use explicit and well-documented volume configurations in your Docker Compose files or Docker run commands.

Use Read-Only Mounts Where Applicable:
For necessary mounts, configure them as read-only to prevent modifications from within the container, reducing the risk of accidental or malicious changes.

Regularly Audit Container Configurations:
Conduct periodic reviews of container configurations and deployment scripts to ensure that no sensitive host directories are inadvertently mounted.

References:

 - [Docker Volumes Documentation](#)

Component: MariaDB Container

1 Recommended countermeasures

Rec1. Ensure sensitive host system directories are not mounted on containers C-DOCKER-CONTAINER-CNT-04

Medium

Not tested

- State: Recommended
- Description:

Enhancing container security by ensuring that sensitive host system directories are not mounted on containers helps prevent unintended data exposure and minimizes the risk of host compromise. Follow the steps below to implement this security control effectively:

Review and Identify Sensitive Directories:
Identify directories on the host that contain critical system configurations or sensitive data (e.g., /etc, /root, /var/lib) and avoid mounting them directly into containers.

Implement Strict Volume Mount Policies:
Restrict volume mounts to only those directories that are essential for application functionality. Use explicit and well-documented volume configurations in your Docker Compose files or Docker run commands.

Use Read-Only Mounts Where Applicable:
For necessary mounts, configure them as read-only to prevent modifications from within the container, reducing the risk of accidental or malicious changes.

Regularly Audit Container Configurations:
Conduct periodic reviews of container configurations and deployment scripts to ensure that no sensitive host directories are inadvertently mounted.

References:

 - [Docker Volumes Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Local Environment

Rec1. Ensure sensitive host system directories are not mounted on containers C-DOCKER-CONTAINER-CNT-04 Medium

Component: User Environment

Rec1. Ensure sensitive host system directories are not mounted on containers C-DOCKER-CONTAINER-CNT-04 Medium

Component: Docker Container

Rec1. Ensure sensitive host system directories are not mounted on containers C-DOCKER-CONTAINER-CNT-04 Medium

Component: Keycloak Container

Rec1. Ensure sensitive host system directories are not mounted on containers C-DOCKER-CONTAINER-CNT-04 Medium

Component: PostgreSQL Container

Rec1. Ensure sensitive host system directories are not mounted on containers C-DOCKER-CONTAINER-CNT-04 Medium

Component: Vault Container

Rec1. Ensure sensitive host system directories are not mounted on containers C-DOCKER-CONTAINER-CNT-04 Medium

Component: HashiCorp Vault

Rec1. Avoid running HashiCorp Vault as root C-HASHICORP-VAULT-CNT-03 Medium

Component: Backend Container

Rec1. Ensure sensitive host system directories are not mounted on containers C-DOCKER-CONTAINER-CNT-04 Medium

Component: MariaDB Container

Rec1. Ensure sensitive host system directories are not mounted on containers C-DOCKER-CONTAINER-CNT-04 Medium

NIST 800-53: SI-15 Information Output Filtering

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Browser

2 Recommended countermeasures

Rec1. Activate URL filtering mechanisms C-BROWSER-CNT-04

Very high

☐ Not tested

State: Recommended

Description:
Implement and regularly update URL filtering mechanisms on all client machines to protect against phishing attacks and access to deceptive websites. These mechanisms help detect and block access to known malicious domains and prevent users from visiting harmful sites that could compromise credentials or lead to malware infections. Regular updates and reviews ensure that the URL filtering remains effective against evolving threats.
Implementation Steps:
Configure Browser Settings:
Set the browser's URL filtering features via built-in settings or through centralized management policies to block access to high-risk websites.
Integrate with Threat Intelligence:
Leverage reputable threat intelligence feeds to update filtering rules and identify new malicious URLs continuously.
Schedule Regular Audits:
Perform periodic reviews of the URL filtering configuration to ensure it is properly enforced and updated in line with the latest threat data.
Monitor and Alert:
Establish monitoring mechanisms to detect attempts to access blocked URLs and alert administrators for further investigation.
References:

- [Google Safe Browsing API](#)

Rec2. Implement client-side script blockers C-BROWSER-CNT-01

High

☐ Not tested

State: Recommended

Description:
Implement and regularly update client-side script blockers on all client machines to mitigate cross-site scripting (XSS) risks. This control restricts the execution of untrusted scripts within the browser, ensuring that only verified and safe code is run. Developers and DevOps engineers should configure built-in browser options or deploy reputable script-blocking extensions to prevent malicious code execution. Regular audits and updates are essential to maintain the effectiveness of these blockers against emerging threats.
Implementation Steps:
Activate Built-in Script Blocking:
Configure the browser settings to enable any built-in script blocking features that prevent untrusted script execution.
Deploy Trusted Extensions:
For browsers lacking robust built-in capabilities, deploy reputable third-party script-blocking extensions (e.g., NoScript) using centralized management tools.
Enforce Configuration Policies:
Use enterprise management solutions such as Group Policy or MDM to enforce consistent script-blocking settings across all client machines.
Monitor and Audit:
Regularly review and test the effectiveness of script blockers through security audits and simulated attack scenarios, updating configurations as needed.
References:

- [OWASP XSS Prevention Cheat Sheet](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Browser

Rec1. Activate URL filtering mechanisms C-BROWSER-CNT-04 Very high
Rec2. Implement client-side script blockers C-BROWSER-CNT-01 High

NIST 800-53: SI-2 Flaw Remediation

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: NGINX Server

2 Recommended countermeasures

Rec1. Implement appropriate security headers

C-NGINX-SERVER-CNT-05

High

Not tested

State: Recommended

Description:

Configuration snippet for enabling security headers in NGINX

```
server {
    # Add Content-Security-Policy header
    add_header Content-Security-Policy "
default-src 'self';
script-src 'self' 'unsafe-inline' https://trusted-cdn.example.com;
style-src 'self' 'unsafe-inline';
img-src 'self' data;
connect-src 'self';
font-src 'self' https://fonts.googleapis.com;
frame-ancestors 'none';
block-all-mixed-content;
upgrade-insecure-requests;" always;
# Mitigate MIME type sniffing
vulnerabilities
add_header X-Content-Type-Options nosniff always;
# Prevent the page from being loaded in an iframe
add_header X-Frame-Options SAMEORIGIN always;
# Mitigate XSS attacks
add_header X-XSS-Protection "1; mode=block" always;
}
```

To implement these headers, follow these steps:

1. Edit the NGINX Configuration File:

Locate the main nginx.conf file, typically found in /etc/nginx/ on Unix-like systems. You'll want to make changes in the server block configurations, either within the main nginx.conf, or within a site-specific configuration under sites-available or sites-enabled.

2. Add Security Headers:

Insert the configuration snippet provided above within the server block context. Ensure that these headers are configured to apply always to all requests, which ensures that these protections are consistently applied.

3. Test the Configuration:

Before applying your changes, validate the configuration file by running `sudo nginx -t`. This step checks for syntax errors and ensures that the configuration is correct.

4. Reload NGINX:

If the configuration tests successfully, apply changes by reloading NGINX using `sudo systemctl reload nginx`. This process will apply the new configuration without interrupting existing connections.

5. Verify Header Implementation:

To ensure the headers are correctly applied, use browser developer tools or tools like [securityheaders.com](#) to inspect received headers after accessing your site.

References

Beagle Security: NGINX Server Security

NGINX Documentation

MDN Web Docs: HTTP Headers

Rec2. Regularly update NGINX

C-NGINX-SERVER-CNT-06

Very high

Not tested

State: Recommended

Description:

Regular updates are critical to maintaining the security and stability of your NGINX server. By ensuring it is up-to-date, you protect your infrastructure against vulnerabilities and benefit from the latest features and performance improvements. Below are step-by-step instructions for keeping your NGINX server updated:

1. Regularly Check for Updates:

Visit the official [NGINX download page](#) frequently to be informed about new releases.

You can also subscribe to the [NGINX mailing list](#) for notifications on new stable releases.

2. Review the Documentation:

Before applying updates, review the [NGINX documentation](#) concerning any changes or requirements for the new version.

3. Backup Current Configuration:

Create a backup of your current NGINX configuration files to ensure you can restore them if necessary. Use the command:
`cp /etc/nginx/nginx.conf /etc/nginx/nginx.conf.backup`

Do the same for any custom configuration files located in /etc/nginx/conf.d/ or similar directories.

4. Install Updates:

On a system using a package manager (such as apt on Ubuntu):

Update the package index:
`sudo apt update`

Upgrade NGINX to the latest version:
`sudo apt upgrade nginx`

If you built NGINX from source:

Download the latest source code from the [official site](#).

Compile and install it following [the build instructions](#).

5. Test the Updated Configuration:

After updating, test the configuration files for compatibility and errors:
`sudo nginx -t`

6. Restart NGINX:

Once you've confirmed the configuration is valid, restart the server to apply changes:
`sudo systemctl restart nginx`

7. Monitor Server Performance and Logs:

Check error logs and monitor server performance to ensure the update did not introduce any issues: `sudo tail -f /var/log/nginx/error.log`

Check the server status with:
`sudo systemctl status nginx`

References

NGINX Configuration Guide

NGINX Download

NGINX on Debian

How to Install NGINX on Ubuntu

Component: Keycloak

1 Recommended countermeasures

Rec1. Use specific redirect URIs

C-KEYCLOAK-CNT-04

Medium

Not tested

State: Recommended

Description:

Prevent open-redirect and replay attacks by tightly controlling where Keycloak may send authorization responses:

1. Navigate to each client's redirect settings

In the Admin Console, go to **Clients** **[Your Client]** **Settings** **Valid Redirect URIs**.

2. Remove wildcards

Eliminate any entries containing * or other glob patterns. Wildcards increase the risk of unauthorized endpoints being accepted [Keycloak](#).

3. Specify full, exact URIs (or minimal paths)

Enter each allowed callback as a complete URI, including scheme, domain, port (if non-standard), and path.

For example:
`https://app.example.com/oauth/callback`
`https://admin.example.com/auth/keycloak`

If you must allow multiple paths under the same host, prefer separate exact entries or minimal path-based prefixes without wildcards.

4. Validate enforcement

Test by initiating an OAuth2/OpenID Connect flow with both approved and unapproved redirect URIs—Keycloak should reject any redirect not matching exactly.

By enforcing exact string matching on redirect URIs, you adhere to OAuth 2.0 best practices and eliminate open-redirect vulnerabilities. Precise URI registration is the most effective guard against malicious redirection [Keycloak](#).

References

Upgrading Guide

Securing applications with OIDC

Component: PostgreSQL

1 Recommended countermeasures

Report: SpaceHub

Compliance report - 2026-03-25T16:40:26.994993079Z

101 of 182

Rec1. Regularly update postgresql to the latest secure version C-POSTGRESQL-CNT-POSTGRES-05 Very high ☐ Not tested

- State: Recommended
- Description:

Implement and regularly update PostgreSQL to the latest secure version to ensure that all known vulnerabilities are patched and that the database benefits from the latest security enhancements. This control minimizes the risk of exploitation by integrating an effective patch management process that monitors for new releases, tests updates in a controlled environment, and deploys them consistently across all systems. Developers and DevOps engineers should incorporate this practice into their routine maintenance processes, using automated tools where possible to ensure compliance with security best practices.

Implementation Steps:

Monitor for New Releases:
Subscribe to PostgreSQL security bulletins and regularly review release notes to stay informed about patches and updates.

Test Updates in Staging:
Validate new PostgreSQL versions in a controlled environment to ensure compatibility and stability with existing applications before production deployment.

Automate Deployment:
Use automated deployment tools or scripts to ensure that updates are applied consistently and promptly across all PostgreSQL instances.

Audit and Verify Compliance:
Regularly review version inventories and update logs to confirm that all systems are running the latest secure version of PostgreSQL.

References:

 - [PostgreSQL Release Notes](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Harden configurations and apply security updates C-MARIADB-CNT-02 High ☐ Not tested

- State: Recommended
- Description:

1. Disable Remote Root Access:

 - Log in to your MariaDB server using a local account with administrative privileges.
 - Open the MariaDB command-line tool by typing `mysql -u root -p` and entering the password when prompted.
 - To check the current privileges for the root account, execute `SELECT host, user FROM mysql.user WHERE user = 'root';` and look for any non-localhost entries.
 - For any root entries that allow remote access (e.g., host is '%'), revoke the privilege by executing:

`REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'root'@'host_ip_or_hostname';`

Replace `host_ip_or_hostname` with the actual host from which remote access should be removed.

5. Delete any remote root accounts if necessary using:

`DROP USER 'root'@'remote_host_ip';`

6. Finally, flush the privileges to ensure changes are applied immediately:

`FLUSH PRIVILEGES;`

2. Remove Unnecessary Plugins:

 - Identify installed plugins by executing the query:

`SHOW PLUGINS;`
 - Review the list and determine which plugins are not required for your specific environment.
 - Remove unnecessary plugins using the following command for each unneeded plugin:

`UNINSTALL PLUGIN plugin_name;`

3. Apply Secure Default Settings:

 - Edit the MariaDB configuration file, typically located at `/etc/my.cnf` or `/etc/mysql/my.cnf`.
 - Add or modify the following security-focused configurations in the `[mysqld]` section:
 - `local-infile=0` - Disables the LOCAL capability for file loading to prevent unauthorized file access.
 - `skip-symbolic-links` - Prevents the use of symbolic links in file names.
 - `sql_mode=STRICT_TRANS_TABLES,ERROR_FOR_DIVISION_BY_ZERO,NO_AUTO_CREATE_USER,NO_ENGINE_SUBSTITUTION` - Sets a recommended secure SQL mode.
 - Save the configuration file and restart the MariaDB service for changes to take effect:

`sudo systemctl restart mariadb`

4. Regularly Update MariaDB with Security Patches:

 - Ensure that your package manager repositories are configured to receive MariaDB updates. For example, on Debian-based systems, check `/etc/apt/sources.list.d/mariadb.list`.
 - Update the package list by executing:

`sudo apt update`
 - Upgrade the MariaDB server to the latest version using:

`sudo apt upgrade mariadb-server`
 - Verify the upgrade was successful and check the current MariaDB version by logging in to the MariaDB shell and executing:

`SELECT VERSION();`

References

 - [Securing MariaDB](#)
 - [MariaDB Installation](#)
 - [Configuring MariaDB with my.cnf](#)

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement regular firmware updates and secure management C-CARD-READER-CNT-03 Very high ☐ Not tested

- State: Recommended
- Description:

Establish a routine process to check for and apply firmware updates to the Card Reader to trusted sources, ensuring that vulnerabilities are promptly patched. Restrict access to update channels by enforcing strong authentication and role-based access controls, allowing only authorized personnel to perform updates. Additionally, configure logging and auditing of firmware update activities to detect any unauthorized changes. This proactive maintenance approach helps maintain the security posture of the Card Reader and protects it against known vulnerabilities.

References:

 - [NIST Cybersecurity Framework](#)

Component: Browser

1 Recommended countermeasures

Rec1. Apply security hardening measures C-BROWSER-CNT-08 Very high ☐ Not tested

- State: Recommended
- Description:

Implement and regularly update security hardening measures on all client machines to reduce the attack surface and mitigate vulnerabilities within the browser. These measures involve configuring the browser to disable unnecessary features, enforcing secure settings, and applying patches that strengthen the overall security posture against potential threats. Regular audits and updates ensure that the browser remains resilient against evolving attack vectors.

Implementation Steps:

Review and Configure Default Settings:
Examine the browser's default configuration and disable non-essential features that could expose vulnerabilities.

Enforce Secure Configuration:
Apply recommended security settings, such as disabling insecure protocols and enabling strict privacy controls, through centralized management or configuration scripts.

Regularly Update and Patch:
Ensure that the browser and its security settings are updated regularly to include the latest hardening recommendations and patches.

Conduct Security Audits:
Schedule periodic security audits to verify the effectiveness of the hardening measures and adjust configurations based on emerging threats.

References:

- [OWASP Secure Configuration Guide](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: NGINX Server

- Rec1. Implement appropriate security headers C-NGINX-SERVER-CNT-05 High
- Rec2. Regularly update NGINX C-NGINX-SERVER-CNT-06 Very high

Component: Keycloak

- Rec1. Use specific redirect URIs C-KEYCLOAK-CNT-04 Medium

Component: PostgreSQL

- Rec1. Regularly update postgresql to the latest secure version C-POSTGRESQL-CNT-POSTGRES-05 Very high

Component: MariaDB

- Rec1. Harden configurations and apply security updates C-MARIADB-CNT-02 High

Component: Card Reader

- Rec1. Implement regular firmware updates and secure management C-CARD-READER-CNT-03 Very high

Component: Browser

- Rec1. Apply security hardening measures C-BROWSER-CNT-08 Very high

NIST 800-53: SC-18 Mobile Code

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Browser

1 Recommended countermeasures

Rec1. Deploy anti-phishing protection C-BROWSER-CNT-03 Very high Not tested

State: Recommended

Description:
Implement and regularly update anti-phishing protection on all client machines to safeguard users from deceptive websites and phishing attempts. This control leverages browser-integrated security features and threat intelligence to detect and block fraudulent sites, protecting sensitive credentials and personal data. Regular configuration reviews and updates ensure that phishing protection remains effective against emerging threats.
Implementation Steps:
Enable Browser Phishing Protection:
Activate the built-in anti-phishing features available in the browser settings, ensuring that users are warned about potentially deceptive websites.
Integrate Threat Intelligence Feeds:
Connect the browser's security system to reputable threat intelligence services (e.g., Google Safe Browsing) to keep filtering rules current with the latest phishing data.
Enforce Organizational Policies:
Use centralized management tools (e.g., Group Policy or MDM solutions) to enforce anti-phishing configurations across all client machines.
Monitor and Review:
Regularly audit and monitor logs for phishing detection alerts, and adjust settings as necessary to maintain a robust defense against phishing attacks.
References:

- [Google Safe Browsing API](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Browser

- Rec1. Deploy anti-phishing protection C-BROWSER-CNT-03 Very high

NIST 800-53: SC-12 CRYPTOGRAPHIC KEY ESTABLISHMENT AND MANAGEMENT

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Keycloak

1 Recommended countermeasures

Rec1. Use high-iteration PBKDF2 hashing C-KEYCLOAK-CNT-07 Very high Not tested

State: Recommended

Description:
Harden stored user credentials by configuring Keycloak's password policy to use a slow, memory-hard hash with sufficient iteration count:

- Open the Password Policy settings
 - In the Admin Console, go to Authentication Password Policy.
- Select the hashing algorithm

- In the **Add policy** dropdown, choose **Hash Algorithm** and select either **pbkdf2-sha256** or **pbkdf2-sha512** according to your security/performance requirements.
3. **Configure iteration count**
- Add the **Hashing Iterations** policy.
 - Set the number of iterations to **27 500** (Keycloak's default minimum) or higher.
 - For stronger protection, align with OWASP's current guidance (e.g. **600 000** iterations for PBKDF2-HMAC-SHA256) to resist offline cracking [Documentación de Red HatInformation Security Stack Exchange](#).
4. **Save and enforce**
- Click **Save**.
 - New user registrations and password updates will now use the chosen algorithm and iteration count.
 - **Note:** Raising iterations increases CPU costs—benchmark to find the highest tolerable value that doesn't degrade user experience.

By enforcing a PBKDF2-based hash with a high iteration count, you greatly slow down brute-force and dictionary attacks against your credential store.

References

- [Red Hat Build of Keycloak Server Administration Guide](#)
- [OWASP Password Storage Cheat Sheet](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Implement encryption for data in transit and at rest C-MARIADB-CNT-03 High Not tested

- State: Recommended
- Description:

1. Enable TLS for Client-Server Communications:

1. Ensure that you have MariaDB server version 10.1.0 or later, as TLS support is integrated into these versions.
2. Generate SSL certificates and keys for the server and clients using OpenSSL or any other trusted CA. Ensure the following:
 - Server Private Key
 - Server Certificate
 - CA Certificate
3. Configure the MariaDB server to use TLS by editing the my.cnf file to include the following:

```
[mariadb]
ssl-ca=/path/to/ca-cert.pem
ssl-cert=/path/to/server-cert.pem
ssl-key=/path/to/server-key.pem
```

4. Restart the MariaDB server to apply the configuration by executing:

```
sudo systemctl restart mariadb
```

5. For client connections, configure the mysql client or other clients to use TLS by specifying the certificates in the client configuration or command line.

2. Enable Data-at-Rest Encryption:

1. Verify that you are using at least MariaDB version 10.1.3 for table-level encryption support.
2. Create a directory to store your encryption keys securely, ensuring only authorized users have access.
3. Update the my.cnf file to include the encryption settings:

```
[mariadb]
plugin-load-add=file_key_management
file-key-management-data=/path/to/keyfile.enc
file-key-management-encryption-algorithm=aes_cbc
```

4. Generate a key file using a secure method and populate it with your encryption keys. Ensure this file is highly secured and access is restricted.
5. Encrypt tablespaces, redo logs, and binary logs by setting the respective options in my.cnf:

```
[mariadb]
encrypt-tmp-disk-tables
encrypt-tmp-files
encrypt-binlog
```

6. Restart the MariaDB service to implement the new encryption settings by executing:

```
sudo systemctl restart mariadb
```

3. Secure Encryption Key Management:

1. Use a secret management tool or hardware security module (HSM) to manage encryption keys securely.
2. Regularly rotate encryption keys to minimize the risk of key compromise. Create a key rotation policy and implement it.
3. Ensure keys are only accessible by authorized applications and personnel through strict access controls and auditing procedures.

References

- [Securing Connections for Client and Server](#)
- [Data-at-Rest Encryption](#)

Component: Card Reader

1 Recommended countermeasures

Rec1. Enforce secure data transmission C-CARD-READER-CNT-02 High Not tested

- State: Recommended

• Description:

Configure the Card Reader to require secure communication channels by implementing robust encryption protocols, such as TLS, for all inbound and outbound data transmissions. This includes setting up the device to only accept encrypted connections, managing digital certificates securely, and routinely updating encryption settings to align with the latest standards. Additionally, configure the system to log and alert on any unencrypted connection attempts, ensuring continuous monitoring and compliance. This approach helps protect sensitive cardholder data from interception and tampering during transmission.

References:

- [NIST Cryptographic Standards and Guidelines](#)

Component: Browser

1 Recommended countermeasures

Rec1. Enforce strict certificate validation C-BROWSER-CNT-05 Very high Not tested

- State: Recommended

• Description:

Implement and regularly update strict certificate validation on all client machines to ensure that the browser only establishes secure connections with trusted websites. This control verifies that SSL/TLS certificates are valid and issued by recognized Certificate Authorities (CAs), preventing attackers from using forged certificates for man-in-the-middle attacks.

Implementation Steps:

Enable Certificate Validation:

Configure the browser to enforce strict certificate validation, ensuring that invalid, expired, or self-signed certificates trigger warnings or connection blocks.

Leverage Centralized Policies:

Use enterprise management tools (e.g., Group Policy or MDM solutions) to enforce certificate validation settings across all client machines.

Monitor Certificate Revocations:

Ensure that the browser is set to regularly check Certificate Revocation Lists (CRLs) or use Online Certificate Status Protocol (OCSP) to verify certificate validity.

Regularly Audit Configurations:

Periodically review and test the certificate validation process to confirm that it effectively blocks connections to untrusted sites.

References:

- [OWASP Transport Layer Protection Cheat Sheet](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Keycloak

Rec1. Use high-iteration PBKDF2 hashing C-KEYCLOAK-CNT-07 Very high

Component: MariaDB

Rec1. Implement encryption for data in transit and at rest C-MARIADB-CNT-03 High

Component: Card Reader

Rec1. Enforce secure data transmission C-CARD-READER-CNT-02 High

Component: Browser

Rec1. Enforce strict certificate validation C-BROWSER-CNT-05 Very high

NIST 800-53: SC-13 Cryptographic Protection

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Keycloak

1 Recommended countermeasures

Rec1. Use high-iteration PBKDF2 hashing C-KEYCLOAK-CNT-07 Very high Not tested

State: Recommended

Description:

Harden stored user credentials by configuring Keycloak's password policy to use a slow, memory-hard hash with sufficient iteration count:

- Open the Password Policy settings
 - In the Admin Console, go to **Authentication** **Password Policy**.
- Select the hashing algorithm
 - In the **Add policy** dropdown, choose **Hash Algorithm** and select either **pbkdf2-sha256** or **pbkdf2-sha512** according to your security/performance requirements.
- Configure iteration count
 - Add the **Hashing Iterations** policy.
 - Set the number of iterations to **27 500** (Keycloak's default minimum) or higher.
 - For stronger protection, align with OWASP's current guidance (e.g. **600 000** iterations for PBKDF2-HMAC-SHA256) to resist offline cracking [Documentación de Red Hat](#)[Information Security Stack Exchange](#).
- Save and enforce
 - Click **Save**.
 - New user registrations and password updates will now use the chosen algorithm and iteration count.
 - Note:** Raising iterations increases CPU costs—benchmark to find the highest tolerable value that doesn't degrade user experience.

By enforcing a PBKDF2-based hash with a high iteration count, you greatly slow down brute-force and dictionary attacks against your credential store.

References

- [Red Hat Build of Keycloak Server Administration Guide](#)
- [OWASP Password Storage Cheat Sheet](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Implement encryption for data in transit and at rest C-MARIADB-CNT-03 High Not tested

State: Recommended

Description:

- Enable TLS for Client-Server Communications:
 - Ensure that you have MariaDB server version 10.1.0 or later, as TLS support is integrated into these versions.
 - Generate SSL certificates and keys for the server and clients using OpenSSL or any other trusted CA. Ensure the following:
 - Server Private Key
 - Server Certificate
 - CA Certificate
 - Configure the MariaDB server to use TLS by editing the my.cnf file to include the following:

```
[mariadb]
ssl-ca=/path/to/ca-cert.pem
ssl-cert=/path/to/server-cert.pem
ssl-key=/path/to/server-key.pem
```
- Restart the MariaDB server to apply the configuration by executing:

```
sudo systemctl restart mariadb
```
- For client connections, configure the mysql client or other clients to use TLS by specifying the certificates in the client configuration or command line.

- Enable Data-at-Rest Encryption:
 - Verify that you are using at least MariaDB version 10.1.3 for table-level encryption support.
 - Create a directory to store your encryption keys securely, ensuring only authorized users have access.
 - Update the my.cnf file to include the encryption settings:

```
[mariadb]
plugin-load-add=file_key_management
file-key-management-data=/path/to/keyfile.enc
file-key-management-encryption-algorithm=aes_cbc
```
- Generate a key file using a secure method and populate it with your encryption keys. Ensure this file is highly secured and access is restricted.
- Encrypt tablespaces, redo logs, and binary logs by setting the respective options in my.cnf:

```
[mariadb]
encrypt-tmp-disk-tables
encrypt-tmp-files
encrypt-binlog
```

- Restart the MariaDB service to implement the new encryption settings by executing:

sudo systemctl restart mariadb

3. Secure Encryption Key Management:

1. Use a secret management tool or hardware security module (HSM) to manage encryption keys securely.

2. Regularly rotate encryption keys to minimize the risk of key compromise. Create a key rotation policy and implement it.

3. Ensure keys are only accessible by authorized applications and personnel through strict access controls and auditing procedures.

References

• [Securing Connections for Client and Server](#)

• [Data-at-Rest Encryption](#)

Component: Card Reader

1 Recommended countermeasures

Rec1. Enforce secure data transmission

C-CARD-READER-CNT-02

High

Not tested

• State:

Recommended

• Description:

Configure the Card Reader to require secure communication channels by implementing robust encryption protocols, such as TLS, for all inbound and outbound data transmissions. This includes setting up the device to only accept encrypted connections, managing digital certificates securely, and routinely updating encryption settings to align with the latest standards. Additionally, configure the system to log and alert on any unencrypted connection attempts, ensuring continuous monitoring and compliance. This approach helps protect sensitive cardholder data from interception and tampering during transmission.

References:

• [NIST Cryptographic Standards and Guidelines](#)

Component: Browser

1 Recommended countermeasures

Rec1. Enforce strict certificate validation

C-BROWSER-CNT-05

Very high

Not tested

• State:

Recommended

• Description:

Implement and regularly update strict certificate validation on all client machines to ensure that the browser only establishes secure connections with trusted websites. This control verifies that SSL/TLS certificates are valid and issued by recognized Certificate Authorities (CAs), preventing attackers from using forged certificates for man-in-the-middle attacks.

Implementation Steps:

Enable Certificate Validation:

Configure the browser to enforce strict certificate validation, ensuring that invalid, expired, or self-signed certificates trigger warnings or connection blocks.

Leverage Centralized Policies:

Use enterprise management tools (e.g., Group Policy or MDM solutions) to enforce certificate validation settings across all client machines.

Monitor Certificate Revocations:

Ensure that the browser is set to regularly check Certificate Revocation Lists (CRLs) or use Online Certificate Status Protocol (OCSP) to verify certificate validity.

Regularly Audit Configurations:

Periodically review and test the certificate validation process to confirm that it effectively blocks connections to untrusted sites.

References:

• [OWASP Transport Layer Protection Cheat Sheet](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Keycloak

Rec1. Use high-iteration PBKDF2 hashing

C-KEYCLOAK-CNT-07

Very high

Component: MariaDB

Rec1. Implement encryption for data in transit and at rest

C-MARIADB-CNT-03

High

Component: Card Reader

Rec1. Enforce secure data transmission

C-CARD-READER-CNT-02

High

Component: Browser

Rec1. Enforce strict certificate validation

C-BROWSER-CNT-05

Very high

NIST 800-53: SC-17 Public Key Infrastructure Certificates

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Keycloak

1 Recommended countermeasures

Rec1. Use high-iteration PBKDF2 hashing

C-KEYCLOAK-CNT-07

Very high

Not tested

• State:

Recommended

• Description:

Harden stored user credentials by configuring Keycloak's password policy to use a slow, memory-hard hash with sufficient iteration count:

1. Open the Password Policy settings

In the Admin Console, go to **Authentication**

Password Policy.

2. Select the hashing algorithm

In the **Add policy** dropdown, choose **Hash Algorithm** and select either **pbkdf2-sha256** or **pbkdf2-sha512** according to your security/performance requirements.

3. Configure iteration count

Add the **Hashing Iterations** policy.

Set the number of iterations to **27 500** (Keycloak's default minimum) or higher.

For stronger protection, align with OWASP's current guidance (e.g. **600 000** iterations for PBKDF2-HMAC-SHA256) to resist offline cracking [Documentación de Red Hat](#)[Information Security Stack Exchange](#).

4. Save and enforce

Click **Save**.

New user registrations and password updates will now use the chosen algorithm and iteration count.

Note: Raising iterations increases CPU costs—benchmark to find the highest tolerable value that doesn't degrade user experience.

By enforcing a PBKDF2-based hash with a high iteration count, you greatly slow down brute-force and dictionary attacks against your credential store.

References

• [Red Hat Build of Keycloak Server Administration Guide](#)

Report: SpaceHub

Compliance report - 2026-03-25T16:40:26.994993079Z

106 of 182

- [OWASP Password Storage Cheat Sheet](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Implement encryption for data in transit and at rest C-MARIADB-CNT-03 High Not tested

- State: Recommended
- Description:
 - 1. **Enable TLS for Client-Server Communications:**
 - Ensure that you have MariaDB server version 10.1.0 or later, as TLS support is integrated into these versions.
 - Generate SSL certificates and keys for the server and clients using OpenSSL or any other trusted CA. Ensure the following:
 - Server Private Key
 - Server Certificate
 - CA Certificate
 - Configure the MariaDB server to use TLS by editing the my.cnf file to include the following:

```
[mariadb]
ssl-ca=/path/to/ca-cert.pem
ssl-cert=/path/to/server-cert.pem
ssl-key=/path/to/server-key.pem
```
 - Restart the MariaDB server to apply the configuration by executing:

```
sudo systemctl restart mariadb
```
 - For client connections, configure the mysql client or other clients to use TLS by specifying the certificates in the client configuration or command line.
 - 2. **Enable Data-at-Rest Encryption:**
 - Verify that you are using at least MariaDB version 10.1.3 for table-level encryption support.
 - Create a directory to store your encryption keys securely, ensuring only authorized users have access.
 - Update the my.cnf file to include the encryption settings:

```
[mariadb]
plugin-load-add=file_key_management
file-key-management-data=/path/to/keyfile.enc
file-key-management-encryption-algorithm=aes_cbc
```
 - Generate a key file using a secure method and populate it with your encryption keys. Ensure this file is highly secured and access is restricted.
 - Encrypt tablespaces, redo logs, and binary logs by setting the respective options in my.cnf:

```
[mariadb]
encrypt-tmp-disk-tables
encrypt-tmp-files
encrypt-binlog
```
 - Restart the MariaDB service to implement the new encryption settings by executing:

```
sudo systemctl restart mariadb
```
 - 3. **Secure Encryption Key Management:**
 - Use a secret management tool or hardware security module (HSM) to manage encryption keys securely.
 - Regularly rotate encryption keys to minimize the risk of key compromise. Create a key rotation policy and implement it.
 - Ensure keys are only accessible by authorized applications and personnel through strict access controls and auditing procedures.
- References
 - [Securing Connections for Client and Server](#)
 - [Data-at-Rest Encryption](#)

Component: Card Reader

1 Recommended countermeasures

Rec1. Enforce secure data transmission C-CARD-READER-CNT-02 High Not tested

- State: Recommended
- Description:

Configure the Card Reader to require secure communication channels by implementing robust encryption protocols, such as TLS, for all inbound and outbound data transmissions. This includes setting up the device to only accept encrypted connections, managing digital certificates securely, and routinely updating encryption settings to align with the latest standards. Additionally, configure the system to log and alert on any unencrypted connection attempts, ensuring continuous monitoring and compliance. This approach helps protect sensitive cardholder data from interception and tampering during transmission.
- References:
 - [NIST Cryptographic Standards and Guidelines](#)

Component: Browser

1 Recommended countermeasures

Rec1. Enforce strict certificate validation C-BROWSER-CNT-05 Very high Not tested

- State: Recommended
- Description:

Implement and regularly update strict certificate validation on all client machines to ensure that the browser only establishes secure connections with trusted websites. This control verifies that SSL/TLS certificates are valid and issued by recognized Certificate Authorities (CAs), preventing attackers from using forged certificates for man-in-the-middle attacks.

Implementation Steps:

Enable Certificate Validation:
Configure the browser to enforce strict certificate validation, ensuring that invalid, expired, or self-signed certificates trigger warnings or connection blocks.

Leverage Centralized Policies:
Use enterprise management tools (e.g., Group Policy or MDM solutions) to enforce certificate validation settings across all client machines.

Monitor Certificate Revocations:
Ensure that the browser is set to regularly check Certificate Revocation Lists (CRLs) or use Online Certificate Status Protocol (OCSP) to verify certificate validity.

Regularly Audit Configurations:
Periodically review and test the certificate validation process to confirm that it effectively blocks connections to untrusted sites.
- References:
 - [OWASP Transport Layer Protection Cheat Sheet](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Keycloak

Rec1. Use high-iteration PBKDF2 hashing C-KEYCLOAK-CNT-07 Very high

Component: MariaDB

Rec1. Implement encryption for data in transit and at rest C-MARIADB-CNT-03 High

Component: Card Reader

Rec1. Enforce secure data transmission C-CARD-READER-CNT-02 High

Component: Browser

Rec1. Enforce strict certificate validation C-BROWSER-CNT-05 Very high

NIST 800-53: SC-44 Detonation Chambers

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Browser

1 Recommended countermeasures

Rec1. Implement extension whitelisting policies C-BROWSER-CNT-09 Very high Not tested

- State: Recommended
- Description:
Implement and regularly update extension whitelisting policies on all client machines to ensure that only approved and trusted browser extensions are installed. This control prevents the use of unauthorized or malicious extensions that could compromise the browser's security or steal sensitive data. Developers and DevOps engineers should configure management tools to enforce a whitelist of extensions and continuously review it for compliance.
Implementation Steps:
Establish a Whitelist:
Identify and document a list of trusted browser extensions that are approved for use within the organization.
Configure Management Tools:
Use centralized management solutions (e.g., Group Policy, MDM, or browser-specific management consoles) to enforce the extension whitelist on all client machines.
Monitor Extension Usage:
Regularly audit installed extensions to ensure compliance with the whitelist and remove any unauthorized or unapproved extensions.
Review and Update Policies:
Periodically review the whitelist and update it based on emerging threats, changes in business requirements, and updated security guidelines.
References:
 - [Google Chrome Enterprise - Manage Chrome Extensions](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Browser

Rec1. Implement extension whitelisting policies C-BROWSER-CNT-09 Very high

NIST 800-53: SC-39 Process Isolation

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Browser

1 Recommended countermeasures

Rec1. Implement extension whitelisting policies C-BROWSER-CNT-09 Very high Not tested

- State: Recommended
- Description:
Implement and regularly update extension whitelisting policies on all client machines to ensure that only approved and trusted browser extensions are installed. This control prevents the use of unauthorized or malicious extensions that could compromise the browser's security or steal sensitive data. Developers and DevOps engineers should configure management tools to enforce a whitelist of extensions and continuously review it for compliance.
Implementation Steps:
Establish a Whitelist:
Identify and document a list of trusted browser extensions that are approved for use within the organization.
Configure Management Tools:
Use centralized management solutions (e.g., Group Policy, MDM, or browser-specific management consoles) to enforce the extension whitelist on all client machines.
Monitor Extension Usage:
Regularly audit installed extensions to ensure compliance with the whitelist and remove any unauthorized or unapproved extensions.
Review and Update Policies:
Periodically review the whitelist and update it based on emerging threats, changes in business requirements, and updated security guidelines.
References:
 - [Google Chrome Enterprise - Manage Chrome Extensions](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Browser

Rec1. Implement extension whitelisting policies C-BROWSER-CNT-09 Very high

NIST 800-53: SC-3 Security Function Isolation

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Browser

1 Recommended countermeasures

Rec1. Implement extension whitelisting policies

C-BROWSER-CNT-09

Very high

☒ Recommended ☐ Not tested

State: Recommended

Description: Implement and regularly update extension whitelisting policies on all client machines to ensure that only approved and trusted browser extensions are installed. This control prevents the use of unauthorized or malicious extensions that could compromise the browser's security or steal sensitive data. Developers and DevOps engineers should configure management tools to enforce a whitelist of extensions and continuously review it for compliance.

Implementation Steps:

Establish a Whitelist: Identify and document a list of trusted browser extensions that are approved for use within the organization.

Configure Management Tools: Use centralized management solutions (e.g., Group Policy, MDM, or browser-specific management consoles) to enforce the extension whitelist on all client machines.

Monitor Extension Usage: Regularly audit installed extensions to ensure compliance with the whitelist and remove any unauthorized or unapproved extensions.

Review and Update Policies: Periodically review the whitelist and update it based on emerging threats, changes in business requirements, and updated security guidelines.

References:

- Google Chrome Enterprise - Manage Chrome Extensions

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Browser

Rec1. Implement extension whitelisting policies C-BROWSER-CNT-09 Very high

NIST 800-53: SC-32 System Partitioning

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Browser

1 Recommended countermeasures

Rec1. Implement extension whitelisting policies

C-BROWSER-CNT-09

Very high

☒ Recommended ☐ Not tested

State: Recommended

Description: Implement and regularly update extension whitelisting policies on all client machines to ensure that only approved and trusted browser extensions are installed. This control prevents the use of unauthorized or malicious extensions that could compromise the browser's security or steal sensitive data. Developers and DevOps engineers should configure management tools to enforce a whitelist of extensions and continuously review it for compliance.

Implementation Steps:

Establish a Whitelist: Identify and document a list of trusted browser extensions that are approved for use within the organization.

Configure Management Tools: Use centralized management solutions (e.g., Group Policy, MDM, or browser-specific management consoles) to enforce the extension whitelist on all client machines.

Monitor Extension Usage: Regularly audit installed extensions to ensure compliance with the whitelist and remove any unauthorized or unapproved extensions.

Review and Update Policies: Periodically review the whitelist and update it based on emerging threats, changes in business requirements, and updated security guidelines.

References:

- Google Chrome Enterprise - Manage Chrome Extensions

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Browser

Rec1. Implement extension whitelisting policies C-BROWSER-CNT-09 Very high

NIST 800-53: SI-22 Information Diversity

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Browser

1 Recommended countermeasures

Rec1. Implement extension whitelisting policies

C-BROWSER-CNT-09

Very high

☒ Recommended ☐ Not tested

State: Recommended

Description: Implement and regularly update extension whitelisting policies on all client machines to ensure that only approved and trusted browser extensions are installed. This control prevents the use of unauthorized or malicious extensions that could compromise the browser's security or steal sensitive data. Developers and DevOps engineers should configure management tools to enforce a whitelist of extensions and continuously review it for compliance.

Implementation Steps:

Establish a Whitelist: Identify and document a list of trusted browser extensions that are approved for use within the organization.

Configure Management Tools: Use centralized management solutions (e.g., Group Policy, MDM, or browser-specific management consoles) to enforce the extension whitelist on all client machines.

Monitor Extension Usage: Regularly audit installed extensions to ensure compliance with the whitelist and remove any unauthorized or unapproved extensions.

Review and Update Policies: Periodically review the whitelist and update it based on emerging threats, changes in business requirements, and updated security guidelines.

- References:
- [Google Chrome Enterprise - Manage Chrome Extensions](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Browser

Rec1. Implement extension whitelisting policies C-BROWSER-CNT-09 Very high

NIST 800-53: SI-23 Information Fragmentation

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Browser

1 Recommended countermeasures

Rec1. Implement extension whitelisting policies C-BROWSER-CNT-09 Very high Not tested

State: Recommended

Description:

Implement and regularly update extension whitelisting policies on all client machines to ensure that only approved and trusted browser extensions are installed. This control prevents the use of unauthorized or malicious extensions that could compromise the browser's security or steal sensitive data. Developers and DevOps engineers should configure management tools to enforce a whitelist of extensions and continuously review it for compliance.

Implementation Steps:

Establish a Whitelist:
Identify and document a list of trusted browser extensions that are approved for use within the organization.

Configure Management Tools:
Use centralized management solutions (e.g., Group Policy, MDM, or browser-specific management consoles) to enforce the extension whitelist on all client machines.

Monitor Extension Usage:
Regularly audit installed extensions to ensure compliance with the whitelist and remove any unauthorized or unapproved extensions.

Review and Update Policies:
Periodically review the whitelist and update it based on emerging threats, changes in business requirements, and updated security guidelines.

References:

- [Google Chrome Enterprise - Manage Chrome Extensions](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Browser

Rec1. Implement extension whitelisting policies C-BROWSER-CNT-09 Very high

NIST 800-53: SI-14 Non-persistence

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Browser

1 Recommended countermeasures

Rec1. Implement extension whitelisting policies C-BROWSER-CNT-09 Very high Not tested

State: Recommended

Description:

Implement and regularly update extension whitelisting policies on all client machines to ensure that only approved and trusted browser extensions are installed. This control prevents the use of unauthorized or malicious extensions that could compromise the browser's security or steal sensitive data. Developers and DevOps engineers should configure management tools to enforce a whitelist of extensions and continuously review it for compliance.

Implementation Steps:

Establish a Whitelist:
Identify and document a list of trusted browser extensions that are approved for use within the organization.

Configure Management Tools:
Use centralized management solutions (e.g., Group Policy, MDM, or browser-specific management consoles) to enforce the extension whitelist on all client machines.

Monitor Extension Usage:
Regularly audit installed extensions to ensure compliance with the whitelist and remove any unauthorized or unapproved extensions.

Review and Update Policies:
Periodically review the whitelist and update it based on emerging threats, changes in business requirements, and updated security guidelines.

References:

- [Google Chrome Enterprise - Manage Chrome Extensions](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Browser

Rec1. Implement extension whitelisting policies C-BROWSER-CNT-09 Very high

NIST 800-53: AC-23 Data Mining Protection

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: NGINX Server

1 Recommended countermeasures

Rec1. Remove unnecessary server headers

C-NGINX-SERVER-CNT-03

High

Not tested

State: Recommended

Description:
Ensure that the NGINX configuration file is accessible for editing. Typically, this file is located at `/etc/nginx/nginx.conf`. Access to this configuration file should be restricted to a secure user account with appropriate permissions.

1. Open the NGINX configuration file in a text editor with superuser privileges. You can use a command like:`sudo nano /etc/nginx/nginx.conf`

2. Locate the global context in your NGINX configuration file. The global context is outside the http, server, and location blocks.

3. Add the following directive at the global context to disable the display of NGINX version information:`server_tokens off;`

4. After making the necessary changes, save and close the configuration file. If you are using nano, for example, you can save by pressing CTRL + X, then Y to confirm and Enter to exit.

5. Test the new NGINX configuration for any syntax errors with the command:`sudo nginx -t` If any errors are reported, review the configuration file and resolve them before proceeding.

6. Reload the NGINX server to apply the changes. This can be accomplished using the command:`sudo systemctl reload nginx`

7. Verify that the server tokens are set correctly by checking an HTTP response from your server using a command like:`curl -I http://yourserver.com` Ensure there is no version information in the Server header field of the response.

Applying this configuration change helps to obscure potential attack vectors from attackers by limiting the server information available, thereby reducing the risk of exploiting known vulnerabilities specific to your server version.

References

- NGINX Configuration Pitfalls
- Beagle Security: NGINX Server Security
- NGINX Beginner's Guide
- NGINX: Managing Configuration Files

Component: Browser

1 Recommended countermeasures

Rec1. Manage browser extensions securely

C-BROWSER-CNT-10

Very high

Not tested

State: Recommended

Description:
Implement and regularly update secure management of browser extensions on all client machines to prevent unauthorized or malicious extensions from compromising browser security. This control involves enforcing approved extension policies, monitoring for unauthorized installations, and ensuring that all extensions are updated and configured according to security best practices.
Implementation Steps:
Enforce Extension Policies:
Utilize centralized management tools (e.g., Group Policy, MDM, or browser-specific management consoles) to control which extensions can be installed and used.
Monitor and Audit Extensions:
Regularly review installed extensions to ensure compliance with organizational policies. Remove or block any unapproved or suspicious extensions.
Regular Updates and Reviews:
Ensure that all approved extensions are kept up-to-date and configure automatic updates where possible. Periodically review security guidelines and adjust policies as needed.
User Education:
Educate users about the risks of installing unapproved extensions and provide guidelines for verifying extension authenticity.
References:

- Google Chrome Enterprise - Manage Chrome Extensions

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: NGINX Server

Rec1. Remove unnecessary server headers C-NGINX-SERVER-CNT-03 High

Component: Browser

Rec1. Manage browser extensions securely C-BROWSER-CNT-10 Very high

NIST 800-53: AC-4 Information Flow Enforcement

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: NGINX Server

1 Recommended countermeasures

Rec1. Remove unnecessary server headers

C-NGINX-SERVER-CNT-03

High

Not tested

State: Recommended

Description:
Ensure that the NGINX configuration file is accessible for editing. Typically, this file is located at `/etc/nginx/nginx.conf`. Access to this configuration file should be restricted to a secure user account with appropriate permissions.

1. Open the NGINX configuration file in a text editor with superuser privileges. You can use a command like:`sudo nano /etc/nginx/nginx.conf`

2. Locate the global context in your NGINX configuration file. The global context is outside the http, server, and location blocks.

3. Add the following directive at the global context to disable the display of NGINX version information:`server_tokens off;`

4. After making the necessary changes, save and close the configuration file. If you are using nano, for example, you can save by pressing CTRL + X, then Y to confirm and Enter to exit.

5. Test the new NGINX configuration for any syntax errors with the command:`sudo nginx -t` If any errors are reported, review the configuration file and resolve them before proceeding.

6. Reload the NGINX server to apply the changes. This can be accomplished using the command:`sudo systemctl reload nginx`

7. Verify that the server tokens are set correctly by checking an HTTP response from your server using a command like:`curl -I http://yourserver.com` Ensure there is no version information in the Server header field of the response.

Applying this configuration change helps to obscure potential attack vectors from attackers by limiting the server information available, thereby reducing the risk of exploiting known vulnerabilities specific to your server version.

References

- NGINX Configuration Pitfalls
- Beagle Security: NGINX Server Security
- NGINX Beginner's Guide
- NGINX: Managing Configuration Files

Component: Browser

1 Recommended countermeasures

Report: SpaceHub

Compliance report - 2026-03-25T16:40:26.994993079Z

111 of 182

Rec1. Manage browser extensions securely

C-BROWSER-CNT-10

Very high

☐ Not tested

• State: 🟢 Recommended

• Description:
Implement and regularly update secure management of browser extensions on all client machines to prevent unauthorized or malicious extensions from compromising browser security. This control involves enforcing approved extension policies, monitoring for unauthorized installations, and ensuring that all extensions are updated and configured according to security best practices.
Implementation Steps:
Enforce Extension Policies:
Utilize centralized management tools (e.g., Group Policy, MDM, or browser-specific management consoles) to control which extensions can be installed and used.
Monitor and Audit Extensions:
Regularly review installed extensions to ensure compliance with organizational policies. Remove or block any unapproved or suspicious extensions.
Regular Updates and Reviews:
Ensure that all approved extensions are kept up-to-date and configure automatic updates where possible. Periodically review security guidelines and adjust policies as needed.
User Education:
Educate users about the risks of installing unapproved extensions and provide guidelines for verifying extension authenticity.
References:

- [Google Chrome Enterprise - Manage Chrome Extensions](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: NGINX Server

Rec1. Remove unnecessary server headers C-NGINX-SERVER-CNT-03 ⬆️ High

Component: Browser

Rec1. Manage browser extensions securely C-BROWSER-CNT-10 Very high

NIST 800-53: AC-21 Information Sharing

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: NGINX Server

1 Recommended countermeasures

Rec1. Remove unnecessary server headers

C-NGINX-SERVER-CNT-03

⬆️ High

☐ Not tested

• State: 🟢 Recommended

• Description:
Ensure that the NGINX configuration file is accessible for editing. Typically, this file is located at `/etc/nginx/nginx.conf`. Access to this configuration file should be restricted to a secure user account with appropriate permissions.

- Open the NGINX configuration file in a text editor with superuser privileges. You can use a command like:`sudo nano /etc/nginx/nginx.conf`
- Locate the global context in your NGINX configuration file. The global context is outside the `http`, `server`, and `location` blocks.
- Add the following directive at the global context to disable the display of NGINX version information:`server_tokens off`;
- After making the necessary changes, save and close the configuration file. If you are using `nano`, for example, you can save by pressing `CTRL + X`, then `Y` to confirm and `Enter` to exit.
- Test the new NGINX configuration for any syntax errors with the command:`sudo nginx -t`If any errors are reported, review the configuration file and resolve them before proceeding.
- Reload the NGINX server to apply the changes. This can be accomplished using the command:`sudo systemctl reload nginx`
- Verify that the server tokens are set correctly by checking an HTTP response from your server using a command like:`curl -I http://yourserver.com`Ensure there is no version information in the Server header field of the response.

Applying this configuration change helps to obscure potential attack vectors from attackers by limiting the server information available, thereby reducing the risk of exploiting known vulnerabilities specific to your server version.
References

- [NGINX Configuration Pitfalls](#)
- [Beagle Security: NGINX Server Security](#)
- [NGINX Beginner's Guide](#)
- [NGINX: Managing Configuration Files](#)

Component: Browser

1 Recommended countermeasures

Rec1. Manage browser extensions securely

C-BROWSER-CNT-10

Very high

☐ Not tested

• State: 🟢 Recommended

• Description:
Implement and regularly update secure management of browser extensions on all client machines to prevent unauthorized or malicious extensions from compromising browser security. This control involves enforcing approved extension policies, monitoring for unauthorized installations, and ensuring that all extensions are updated and configured according to security best practices.
Implementation Steps:
Enforce Extension Policies:
Utilize centralized management tools (e.g., Group Policy, MDM, or browser-specific management consoles) to control which extensions can be installed and used.
Monitor and Audit Extensions:
Regularly review installed extensions to ensure compliance with organizational policies. Remove or block any unapproved or suspicious extensions.
Regular Updates and Reviews:
Ensure that all approved extensions are kept up-to-date and configure automatic updates where possible. Periodically review security guidelines and adjust policies as needed.
User Education:
Educate users about the risks of installing unapproved extensions and provide guidelines for verifying extension authenticity.
References:

- [Google Chrome Enterprise - Manage Chrome Extensions](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: NGINX Server

Rec1. Remove unnecessary server headers C-NGINX-SERVER-CNT-03 ⬆️ High

Component: Browser

Rec1. Manage browser extensions securely C-BROWSER-CNT-10 Very high

Report: SpaceHub

Compliance report - 2026-03-25T16:40:26.994993079Z

112 of 182

NIST 800-53: AC-1 POLICY AND PROCEDURES

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: NGINX Server

1 Recommended countermeasures

Rec1. Remove unnecessary server headers

C-NGINX-SERVER-CNT-03

High

Not tested

State: Recommended

Description:

Ensure that the NGINX configuration file is accessible for editing. Typically, this file is located at `/etc/nginx/nginx.conf`. Access to this configuration file should be restricted to a secure user account with appropriate permissions.

1. Open the NGINX configuration file in a text editor with superuser privileges. You can use a command like:`sudo nano /etc/nginx/nginx.conf`

2. Locate the global context in your NGINX configuration file. The global context is outside the `http`, `server`, and `location` blocks.

3. Add the following directive at the global context to disable the display of NGINX version information:`server_tokens off`;

4. After making the necessary changes, save and close the configuration file. If you are using `nano`, for example, you can save by pressing `CTRL + X`, then `Y` to confirm and `Enter` to exit.

5. Test the new NGINX configuration for any syntax errors with the command:`sudo nginx -t` If any errors are reported, review the configuration file and resolve them before proceeding.

6. Reload the NGINX server to apply the changes. This can be accomplished using the command:`sudo systemctl reload nginx`

7. Verify that the server tokens are set correctly by checking an HTTP response from your server using a command like:`curl -I http://yourserver.com`Ensure there is no version information in the `Server` header field of the response.

Applying this configuration change helps to obscure potential attack vectors from attackers by limiting the server information available, thereby reducing the risk of exploiting known vulnerabilities specific to your server version.

References

- NGINX Configuration Pitfalls
- Beagle Security: NGINX Server Security
- NGINX Beginner's Guide
- NGINX: Managing Configuration Files

Component: Browser

1 Recommended countermeasures

Rec1. Manage browser extensions securely

C-BROWSER-CNT-10

Very high

Not tested

State: Recommended

Description:

Implement and regularly update secure management of browser extensions on all client machines to prevent unauthorized or malicious extensions from compromising browser security. This control involves enforcing approved extension policies, monitoring for unauthorized installations, and ensuring that all extensions are updated and configured according to security best practices.

Implementation Steps:

Enforce Extension Policies:
Utilize centralized management tools (e.g., Group Policy, MDM, or browser-specific management consoles) to control which extensions can be installed and used.

Monitor and Audit Extensions:
Regularly review installed extensions to ensure compliance with organizational policies. Remove or block any unapproved or suspicious extensions.

Regular Updates and Reviews:
Ensure that all approved extensions are kept up-to-date and configure automatic updates where possible. Periodically review security guidelines and adjust policies as needed.

User Education:
Educate users about the risks of installing unapproved extensions and provide guidelines for verifying extension authenticity.

References:

- Google Chrome Enterprise - Manage Chrome Extensions

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: NGINX Server

Rec1. Remove unnecessary server headers

C-NGINX-SERVER-CNT-03

High

Component: Browser

Rec1. Manage browser extensions securely

C-BROWSER-CNT-10

Very high

NIST 800-53: AC-22 Publicly Accessible Content

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: NGINX Server

1 Recommended countermeasures

Rec1. Remove unnecessary server headers

C-NGINX-SERVER-CNT-03

High

Not tested

State: Recommended

Description:

Ensure that the NGINX configuration file is accessible for editing. Typically, this file is located at `/etc/nginx/nginx.conf`. Access to this configuration file should be restricted to a secure user account with appropriate permissions.

1. Open the NGINX configuration file in a text editor with superuser privileges. You can use a command like:`sudo nano /etc/nginx/nginx.conf`

2. Locate the global context in your NGINX configuration file. The global context is outside the `http`, `server`, and `location` blocks.

3. Add the following directive at the global context to disable the display of NGINX version information:`server_tokens off`;

4. After making the necessary changes, save and close the configuration file. If you are using `nano`, for example, you can save by pressing `CTRL + X`, then `Y` to confirm and `Enter` to exit.

5. Test the new NGINX configuration for any syntax errors with the command:`sudo nginx -t` If any errors are reported, review the configuration file and resolve them before proceeding.

6. Reload the NGINX server to apply the changes. This can be accomplished using the command:`sudo systemctl reload nginx`

7. Verify that the server tokens are set correctly by checking an HTTP response from your server using a command like:`curl -I http://yourserver.com`Ensure there is no version information in the `Server` header field of the response.

Applying this configuration change helps to obscure potential attack vectors from attackers by limiting the server information available, thereby reducing the risk of exploiting known vulnerabilities specific to your server version.

References

- NGINX Configuration Pitfalls
- Beagle Security: NGINX Server Security

Report: SpaceHub

Compliance report - 2026-03-25T16:40:26.994993079Z

113 of 182

- [NGINX Beginner's Guide](#)
- [NGINX: Managing Configuration Files](#)

Component: Browser

1 Recommended countermeasures

Rec1. Manage browser extensions securely C-BROWSER-CNT-10 **Very high** ☐ Not tested

- **State:**  Recommended
- **Description:**
Implement and regularly update secure management of browser extensions on all client machines to prevent unauthorized or malicious extensions from compromising browser security. This control involves enforcing approved extension policies, monitoring for unauthorized installations, and ensuring that all extensions are updated and configured according to security best practices.
Implementation Steps:
Enforce Extension Policies:
Utilize centralized management tools (e.g., Group Policy, MDM, or browser-specific management consoles) to control which extensions can be installed and used.
Monitor and Audit Extensions:
Regularly review installed extensions to ensure compliance with organizational policies. Remove or block any unapproved or suspicious extensions.
Regular Updates and Reviews:
Ensure that all approved extensions are kept up-to-date and configure automatic updates where possible. Periodically review security guidelines and adjust policies as needed.
User Education:
Educate users about the risks of installing unapproved extensions and provide guidelines for verifying extension authenticity.
References:
 - [Google Chrome Enterprise - Manage Chrome Extensions](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: NGINX Server

Rec1. Remove unnecessary server headers C-NGINX-SERVER-CNT-03  **High**

Component: Browser

Rec1. Manage browser extensions securely C-BROWSER-CNT-10 **Very high**

NIST 800-53: AC-20 Use of External Systems

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: NGINX Server

1 Recommended countermeasures

Rec1. Remove unnecessary server headers C-NGINX-SERVER-CNT-03  **High** ☐ Not tested

- **State:**  Recommended
- **Description:**
Ensure that the NGINX configuration file is accessible for editing. Typically, this file is located at `/etc/nginx/nginx.conf`. Access to this configuration file should be restricted to a secure user account with appropriate permissions.
 1. Open the NGINX configuration file in a text editor with superuser privileges. You can use a command like:`sudo nano /etc/nginx/nginx.conf`
 2. Locate the global context in your NGINX configuration file. The global context is outside the http, server, and location blocks.
 3. Add the following directive at the global context to disable the display of NGINX version information:`server_tokens off;`
 4. After making the necessary changes, save and close the configuration file. If you are using nano, for example, you can save by pressing CTRL + X, then Y to confirm and Enter to exit.
 5. Test the new NGINX configuration for any syntax errors with the command:`sudo nginx -t`If any errors are reported, review the configuration file and resolve them before proceeding.
 6. Reload the NGINX server to apply the changes. This can be accomplished using the command:`sudo systemctl reload nginx`
 7. Verify that the server tokens are set correctly by checking an HTTP response from your server using a command like:`curl -I http://yourserver.com`Ensure there is no version information in the Server header field of the response.
Applying this configuration change helps to obscure potential attack vectors from attackers by limiting the server information available, thereby reducing the risk of exploiting known vulnerabilities specific to your server version.
References
 - [NGINX Configuration Pitfalls](#)
 - [Beagle Security: NGINX Server Security](#)
 - [NGINX Beginner's Guide](#)
 - [NGINX: Managing Configuration Files](#)

Component: Browser

1 Recommended countermeasures

Rec1. Manage browser extensions securely C-BROWSER-CNT-10 **Very high** ☐ Not tested

- **State:**  Recommended
- **Description:**
Implement and regularly update secure management of browser extensions on all client machines to prevent unauthorized or malicious extensions from compromising browser security. This control involves enforcing approved extension policies, monitoring for unauthorized installations, and ensuring that all extensions are updated and configured according to security best practices.
Implementation Steps:
Enforce Extension Policies:
Utilize centralized management tools (e.g., Group Policy, MDM, or browser-specific management consoles) to control which extensions can be installed and used.
Monitor and Audit Extensions:
Regularly review installed extensions to ensure compliance with organizational policies. Remove or block any unapproved or suspicious extensions.
Regular Updates and Reviews:
Ensure that all approved extensions are kept up-to-date and configure automatic updates where possible. Periodically review security guidelines and adjust policies as needed.
User Education:
Educate users about the risks of installing unapproved extensions and provide guidelines for verifying extension authenticity.
References:
 - [Google Chrome Enterprise - Manage Chrome Extensions](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: NGINX Server

Rec1. Remove unnecessary server headers C-NGINX-SERVER-CNT-03 High

Component: Browser

Rec1. Manage browser extensions securely C-BROWSER-CNT-10 Very high

NIST 800-53: PM-10 Authorization Process

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: NGINX Server

1 Recommended countermeasures

Rec1. Remove unnecessary server headers C-NGINX-SERVER-CNT-03 High Not tested

State: Recommended

Description:

Ensure that the NGINX configuration file is accessible for editing. Typically, this file is located at `/etc/nginx/nginx.conf`. Access to this configuration file should be restricted to a secure user account with appropriate permissions.

1. Open the NGINX configuration file in a text editor with superuser privileges. You can use a command like:`sudo nano /etc/nginx/nginx.conf`

2. Locate the global context in your NGINX configuration file. The global context is outside the http, server, and location blocks.

3. Add the following directive at the global context to disable the display of NGINX version information:`server_tokens off;`

4. After making the necessary changes, save and close the configuration file. If you are using nano, for example, you can save by pressing CTRL + X, then Y to confirm and Enter to exit.

5. Test the new NGINX configuration for any syntax errors with the command:`sudo nginx -t` If any errors are reported, review the configuration file and resolve them before proceeding.

6. Reload the NGINX server to apply the changes. This can be accomplished using the command:`sudo systemctl reload nginx`

7. Verify that the server tokens are set correctly by checking an HTTP response from your server using a command like:`curl -I http://yourserver.com` Ensure there is no version information in the Server header field of the response.

Applying this configuration change helps to obscure potential attack vectors from attackers by limiting the server information available, thereby reducing the risk of exploiting known vulnerabilities specific to your server version.

References

- NGINX Configuration Pitfalls
- Beagle Security: NGINX Server Security
- NGINX Beginner's Guide
- NGINX: Managing Configuration Files

Component: Browser

1 Recommended countermeasures

Rec1. Manage browser extensions securely C-BROWSER-CNT-10 Very high Not tested

State: Recommended

Description:

Implement and regularly update secure management of browser extensions on all client machines to prevent unauthorized or malicious extensions from compromising browser security. This control involves enforcing approved extension policies, monitoring for unauthorized installations, and ensuring that all extensions are updated and configured according to security best practices.

Implementation Steps:

Enforce Extension Policies:
Utilize centralized management tools (e.g., Group Policy, MDM, or browser-specific management consoles) to control which extensions can be installed and used.

Monitor and Audit Extensions:
Regularly review installed extensions to ensure compliance with organizational policies. Remove or block any unapproved or suspicious extensions.

Regular Updates and Reviews:
Ensure that all approved extensions are kept up-to-date and configure automatic updates where possible. Periodically review security guidelines and adjust policies as needed.

User Education:
Educate users about the risks of installing unapproved extensions and provide guidelines for verifying extension authenticity.

References:

- Google Chrome Enterprise - Manage Chrome Extensions

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: NGINX Server

Rec1. Remove unnecessary server headers C-NGINX-SERVER-CNT-03 High

Component: Browser

Rec1. Manage browser extensions securely C-BROWSER-CNT-10 Very high

NIST 800-53: AC-17 Remote Access

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: NGINX Server

1 Recommended countermeasures

Rec1. Implement modern SSL/TLS protocols and disable weak protocols C-NGINX-SERVER-CNT-02 Very high Not tested

State: Recommended

Description:

1. Obtain SSL Certificates:

a. Install the Certbot utility from Let's Encrypt, which is a popular choice for obtaining SSL certificates:

`sudo apt-get update`
`sudo apt-get install certbot python3-certbot-nginx`

b. Use Certbot to obtain an SSL certificate for your domain. Ensure your domain is correctly mapped to the server:

Report: SpaceHub

Compliance report - 2026-03-25T16:40:26.994993079Z

115 of 182

```
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com
```

Follow the on-screen instructions, confirming any prompts to allow Certbot to configure SSL for you.

2. **Configure NGINX for TLS 1.2 and TLS 1.3:**

a. Edit the NGINX configuration file:

```
sudo nano /etc/nginx/nginx.conf
```

b. Locate the 'http' block and ensure it includes the following directives to enable only TLS 1.2 and 1.3:

```
ssl_protocols TLSv1.2 TLSv1.3;
```

c. Edit any specific server block files within the 'sites-available' directory to reflect these changes:

```
sudo nano /etc/nginx/sites-available/yourdomain
```

Add or update the server block to include:

```
ssl_protocols TLSv1.2 TLSv1.3;
```

3. **Enable Strong Cipher Suites:**

a. Edit the ssl configuration file or add the settings to your existing configurations such as 'nginx.conf' or 'default.conf':

```
sudo nano /etc/nginx/conf.d/ssl.conf
```

b. Set the cipher suites, preferring strong and secure ciphers. For example:

```
ssl_ciphers 'ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256';
```

4. **Test the SSL Configuration:**

a. Run an NGINX syntax test:

```
sudo nginx -t
```

Ensure there are no errors. If there are errors, correct them based on the feedback provided.

b. Restart NGINX to apply the changes:

```
sudo systemctl restart nginx
```

c. Use SSL testing tools like SSL Labs' SSL Test to verify your SSL configuration is strong and only the desired protocols and ciphers are enabled. [Test your site at SSL Labs](#).

References

- [NGINX Server Security: Configuration of SSL](#)
- [Certbot: Instructions on getting certificates](#)
- [SSL Labs SSL Test](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. **Require https communication** C-KEYCLOAK-CNT-02 High Not tested

• **State:** Recommended

• **Description:**

Ensure Keycloak only serves sensitive endpoints over encrypted channels:

1. **Configure Realm SSL Mode**

- In the Admin Console, navigate to **Realm Settings** **Login**, set **Require SSL** to either:
 - **External requests only:** Allows HTTP from private IP ranges (localhost, 10.x.x.x, 192.168.x.x, etc.) but forces HTTPS for all other clients.
 - **All requests:** Enforces HTTPS for every request, regardless of source IP. [w/jw465150.gitbooks.io](#)

2. **Deploy valid TLS certificates**

- **Direct Keycloak TLS:** If terminating TLS in Keycloak itself, provide PEM files or a Java keystore on startup:

```
bin/kc.sh start \  
--https-certificate-file=/path/to/cert.pem \  
--https-certificate-key-file=/path/to/key.pem
```

Or point to a keystore:

```
bin/kc.sh start \  
--https-key-store-file=/path/to/keystore.p12
```
 - **Reverse-proxy TLS termination:** If using NGINX, Apache, or a load balancer in front of Keycloak:
 1. Install the certificate and key on the proxy.
 2. Terminate TLS there and forward traffic to Keycloak over HTTP (e.g. 127.0.0.1:8080).
 3. Ensure X-Forwarded-Proto is set so Keycloak recognizes the original scheme. [Keycloak](#)
3. **Verify enforcement**
- Attempt to access any Keycloak endpoint via http://; it should be blocked or redirected to https://.
 - Confirm cookies like KEYCLOAK_SESSION have the Secure flag set.

By enforcing SSL Mode at the realm level and deploying trusted TLS certificates, you eliminate unencrypted access, protecting credentials and tokens from interception.

References

- [Realm SSL Mode configuration \(Require SSL\)](#)
- [Configuring TLS for Keycloak certificates](#)

Component: PostgreSQL

1 Recommended countermeasures

Rec1. **Enforce TLS encryption for all connections** C-POSTGRESQL-CNT-POSTGRES-03 High Not tested

• **State:** Recommended

• **Description:**

Implement and regularly update TLS encryption for all connections to PostgreSQL to secure data in transit against eavesdropping and man-in-the-middle attacks. This control mandates that all client-to-server and inter-node communications are encrypted using strong TLS protocols (e.g., TLS 1.2 or TLS 1.3) and that proper certificate management is in place. Developers and DevOps engineers should integrate TLS configuration into the PostgreSQL deployment process and enforce these settings via centralized management tools to maintain a secure network environment.

Implementation Steps:

Enable TLS Encryption:

Configure PostgreSQL to require TLS for all incoming connections by setting the appropriate parameters in the PostgreSQL configuration file (e.g., ssl = on).

Configure Strong Cipher Suites:

Ensure that only secure TLS versions and cipher suites (e.g., TLS 1.2/1.3) are enabled, and disable deprecated protocols to enhance the encryption strength.

Manage Certificates Securely:

Use a centralized certificate management system to issue, renew, and manage SSL/TLS certificates for PostgreSQL, ensuring that certificates are always up-to-date and trusted.

Monitor and Audit TLS Usage:

Regularly review logs and perform security audits to verify that all connections are using TLS encryption, and adjust configurations as necessary to address emerging threats.

References:

- [PostgreSQL SSL Support Documentation](#)

Component: HashiCorp Vault

1 Recommended countermeasures

Rec1. Enforce TLS for HashiCorp Vault in production C-HASHICORP-VAULT-CNT-01

High

Not tested

State: Recommended

Description:

To safeguard the communication between clients and HashiCorp Vault, it is essential to implement TLS (Transport Layer Security). This encryption protocol ensures that the data exchanged is secure from interception and unauthorized access. Follow these steps to configure TLS effectively:

- Obtain a Valid SSL/TLS Certificate:**

Acquire an SSL/TLS certificate from a trusted Certificate Authority (CA). This certificate verifies the authenticity of the server and ensures that the data is encrypted during transmission.
- Configure HashiCorp Vault to Use the Certificate:**

Place the SSL/TLS certificate and the corresponding private key on the server where Vault is installed. In the Vault configuration file, specify the paths to these files under the TLS settings. For example:

```
tls_disable = 0
tls_cert_file = "/path/to/cert.pem"
tls_key_file = "/path/to/key.pem"
```
- Set Up Certificate Authorities (CAs):**

If using internal or self-signed certificates, ensure you configure the clients to trust these certificates by distributing the root CA certificate. Include either a CA bundle file or individual CA certificates in the configuration.

Example:

```
tls_ca_file = "/path/to/ca.pem"
```
- Enforce Secure Protocols and Ciphers:**

Configure and enforce the use of strong TLS protocols and ciphers. This enhances the security posture by keeping communications secure against known vulnerabilities.

```
tls_min_version = "tls12"
```
- Regularly Update and Maintain Certificates:**

Regular maintenance and updates of the certificates are crucial. Monitor the expiration of the certificates and renew them timely. Additionally, stay informed about the latest security practices regarding TLS. Proper TLS implementation protects the integrity and confidentiality of data in-transit, ensuring secure communication with HashiCorp Vault.

Component: Browser

1 Recommended countermeasures

Rec1. Utilize encrypted communication tools C-BROWSER-CNT-06

Very high

Not tested

State: Recommended

Description:

Implement and regularly update encrypted communication tools on all client machines to ensure that all browser communications are securely transmitted. This control enforces the use of secure protocols and encryption solutions, such as TLS and VPNs, to protect data in transit against interception and man-in-the-middle attacks.

Implementation Steps:

Enable Secure Protocols:
Configure browsers to default to secure protocols (e.g., HTTPS, TLS 1.2/1.3) and disable outdated, insecure versions.

Deploy VPN or Encrypted Tunnels:
Use reputable VPN solutions to encrypt browser traffic, especially when accessing untrusted networks, ensuring that data remains protected.

Monitor and Validate Encryption:
Regularly review encryption settings, validate certificate authenticity, and update encryption libraries to keep pace with emerging threats.

Integrate with Centralized Management:
Utilize enterprise tools to enforce encrypted communication configurations and monitor compliance across all client machines.

References:

- [OWASP Transport Layer Protection Cheat Sheet](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: NGINX Server

Rec1. Implement modern SSL/TLS protocols and disable weak protocols C-NGINX-SERVER-CNT-02

Very high

Component: Keycloak

Rec1. Require https communication C-KEYCLOAK-CNT-02

High

Component: PostgreSQL

Rec1. Enforce TLS encryption for all connections C-POSTGRESQL-CNT-POSTGRES-03

High

Component: HashiCorp Vault

Rec1. Enforce TLS for HashiCorp Vault in production C-HASHICORP-VAULT-CNT-01

High

Component: Browser

Rec1. Utilize encrypted communication tools C-BROWSER-CNT-06

Very high

NIST 800-53: AC-18 Wireless Access

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqP") countermeasures which tests have failed are also shown as non-compliant ones.

Component: NGINX Server

1 Recommended countermeasures

Rec1. Implement modern SSL/TLS protocols and disable weak protocols C-NGINX-SERVER-CNT-02

Very high

Not tested

State: Recommended

Description:

1. Obtain SSL Certificates:

a. Install the Certbot utility from Let's Encrypt, which is a popular choice for obtaining SSL certificates:

```
sudo apt-get update
sudo apt-get install certbot python3-certbot-nginx
```

b. Use Certbot to obtain an SSL certificate for your domain. Ensure your domain is correctly mapped to the server:

```
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com
```

Follow the on-screen instructions, confirming any prompts to allow Certbot to configure SSL for you.

2. Configure NGINX for TLS 1.2 and TLS 1.3:

a. Edit the NGINX configuration file:

Report: SpaceHub

Compliance report - 2026-03-25T16:40:26.994993079Z

117 of 182

- sudo nano /etc/nginx/nginx.conf
- b. Locate the 'http' block and ensure it includes the following directives to enable only TLS 1.2 and 1.3:
- ssl_protocols TLSv1.2 TLSv1.3;
- c. Edit any specific server block files within the 'sites-available' directory to reflect these changes:
- sudo nano /etc/nginx/sites-available/yourdomain
- Add or update the server block to include:
- ssl_protocols TLSv1.2 TLSv1.3;
3. Enable Strong Cipher Suites:
- a. Edit the ssl configuration file or add the settings to your existing configurations such as 'nginx.conf' or 'default.conf':
- sudo nano /etc/nginx/conf.d/ssl.conf
- b. Set the cipher suites, preferring strong and secure ciphers. For example:
- ssl_ciphers 'ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256';
4. Test the SSL Configuration:
- a. Run an NGINX syntax test:
- sudo nginx -t
- Ensure there are no errors. If there are errors, correct them based on the feedback provided.
- b. Restart NGINX to apply the changes:
- sudo systemctl restart nginx
- c. Use SSL testing tools like SSL Labs' SSL Test to verify your SSL configuration is strong and only the desired protocols and ciphers are enabled. [Test your site at SSL Labs.](#)

- References
- [NGINX Server Security: Configuration of SSL](#)
 - [Certbot: Instructions on getting certificates](#)
 - [SSL Labs SSL Test](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. Require https communication C-KEYCLOAK-CNT-02 High Not tested

- State: Recommended
- Description:
Ensure Keycloak only serves sensitive endpoints over encrypted channels:
 1. **Configure Realm SSL Mode**
 - In the Admin Console, navigate to **Realm Settings** **Login**, set **Require SSL** to either:
 - **External requests only:** Allows HTTP from private IP ranges (localhost, 10.x.x.x, 192.168.x.x, etc.) but forces HTTPS for all other clients.
 - **All requests:** Enforces HTTPS for every request, regardless of source IP. [wjwt465150.gitbooks.io](#)
 2. **Deploy valid TLS certificates**
 - **Direct Keycloak TLS:** If terminating TLS in Keycloak itself, provide PEM files or a Java keystore on startup:

```
bin/kc.sh start \  
--https-certificate-file=/path/to/cert.pem \  
--https-certificate-key-file=/path/to/key.pem
```

Or point to a keystore:

```
bin/kc.sh start \  
--https-key-store-file=/path/to/keystore.p12
```
 - **Reverse-proxy TLS termination:** If using NGINX, Apache, or a load balancer in front of Keycloak:
 1. Install the certificate and key on the proxy.
 2. Terminate TLS there and forward traffic to Keycloak over HTTP (e.g. 127.0.0.1:8080).
 3. Ensure X-Forwarded-Proto is set so Keycloak recognizes the original scheme. [Keycloak](#)
 3. **Verify enforcement**
 - Attempt to access any Keycloak endpoint via http://; it should be blocked or redirected to https://.
 - Confirm cookies like KEYCLOAK_SESSION have the Secure flag set.

By enforcing SSL Mode at the realm level and deploying trusted TLS certificates, you eliminate unencrypted access, protecting credentials and tokens from interception.

- References
- [Realm SSL Mode configuration \(Require SSL\)](#)
 - [Configuring TLS for Keycloak certificates](#)

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce TLS encryption for all connections C-POSTGRESQL-CNT-POSTGRES-03 High Not tested

- State: Recommended
 - Description:
Implement and regularly update TLS encryption for all connections to PostgreSQL to secure data in transit against eavesdropping and man-in-the-middle attacks. This control mandates that all client-to-server and inter-node communications are encrypted using strong TLS protocols (e.g., TLS 1.2 or TLS 1.3) and that proper certificate management is in place. Developers and DevOps engineers should integrate TLS configuration into the PostgreSQL deployment process and enforce these settings via centralized management tools to maintain a secure network environment.

Implementation Steps:

Enable TLS Encryption:
Configure PostgreSQL to require TLS for all incoming connections by setting the appropriate parameters in the PostgreSQL configuration file (e.g., ssl = on).

Configure Strong Cipher Suites:
Ensure that only secure TLS versions and cipher suites (e.g., TLS 1.2/1.3) are enabled, and disable deprecated protocols to enhance the encryption strength.

Manage Certificates Securely:
Use a centralized certificate management system to issue, renew, and manage SSL/TLS certificates for PostgreSQL, ensuring that certificates are always up-to-date and trusted.

Monitor and Audit TLS Usage:
Regularly review logs and perform security audits to verify that all connections are using TLS encryption, and adjust configurations as necessary to address emerging threats.
- References:
- [PostgreSQL SSL Support Documentation](#)

Component: HashiCorp Vault

1 Recommended countermeasures

Rec1. Enforce TLS for HashiCorp Vault in production C-HASHICORP-VAULT-CNT-01 High Not tested

- **State:** 🟢 Recommended
- **Description:**

To safeguard the communication between clients and HashiCorp Vault, it is essential to implement TLS (Transport Layer Security). This encryption protocol ensures that the data exchanged is secure from interception and unauthorized access. Follow these steps to configure TLS effectively:

 - **Obtain a Valid SSL/TLS Certificate:**

Acquire an SSL/TLS certificate from a trusted Certificate Authority (CA). This certificate verifies the authenticity of the server and ensures that the data is encrypted during transmission.
 - **Configure HashiCorp Vault to Use the Certificate:**

Place the SSL/TLS certificate and the corresponding private key on the server where Vault is installed. In the Vault configuration file, specify the paths to these files under the TLS settings. For example:

```
tls_disable = 0
tls_cert_file = "/path/to/cert.pem"
tls_key_file = "/path/to/key.pem"
```
 - **Set Up Certificate Authorities (CAs):**

If using internal or self-signed certificates, ensure you configure the clients to trust these certificates by distributing the root CA certificate. Include either a CA bundle file or individual CA certificates in the configuration.

Example:

```
tls_ca_file = "/path/to/ca.pem"
```
 - **Enforce Secure Protocols and Ciphers:**

Configure and enforce the use of strong TLS protocols and ciphers. This enhances the security posture by keeping communications secure against known vulnerabilities.

```
tls_min_version = "tls12"
```
 - **Regularly Update and Maintain Certificates:**

Regular maintenance and updates of the certificates are crucial. Monitor the expiration of the certificates and renew them timely. Additionally, stay informed about the latest security practices regarding TLS. Proper TLS implementation protects the integrity and confidentiality of data in-transit, ensuring secure communication with HashiCorp Vault.

Component: Browser

1 Recommended countermeasures

Rec1. Utilize encrypted communication tools C-BROWSER-CNT-06 Very high ○ Not tested

- **State:** 🟢 Recommended
- **Description:**

Implement and regularly update encrypted communication tools on all client machines to ensure that all browser communications are securely transmitted. This control enforces the use of secure protocols and encryption solutions, such as TLS and VPNs, to protect data in transit against interception and man-in-the-middle attacks.
- **Implementation Steps:**
 - **Enable Secure Protocols:**

Configure browsers to default to secure protocols (e.g., HTTPS, TLS 1.2/1.3) and disable outdated, insecure versions.
 - **Deploy VPN or Encrypted Tunnels:**

Use reputable VPN solutions to encrypt browser traffic, especially when accessing untrusted networks, ensuring that data remains protected.
 - **Monitor and Validate Encryption:**

Regularly review encryption settings, validate certificate authenticity, and update encryption libraries to keep pace with emerging threats.
 - **Integrate with Centralized Management:**

Utilize enterprise tools to enforce encrypted communication configurations and monitor compliance across all client machines.
- **References:**
 - [OWASP Transport Layer Protection Cheat Sheet](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: NGINX Server

Rec1. Implement modern SSL/TLS protocols and disable weak protocols C-NGINX-SERVER-CNT-02 Very high

Component: Keycloak

Rec1. Require https communication C-KEYCLOAK-CNT-02 🔴 High

Component: PostgreSQL

Rec1. Enforce TLS encryption for all connections C-POSTGRESQL-CNT-POSTGRES-03 🔴 High

Component: HashiCorp Vault

Rec1. Enforce TLS for HashiCorp Vault in production C-HASHICORP-VAULT-CNT-01 🔴 High

Component: Browser

Rec1. Utilize encrypted communication tools C-BROWSER-CNT-06 Very high

NIST 800-53: CA-3 Information Exchange

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: NGINX Server

1 Recommended countermeasures

Rec1. Implement modern SSL/TLS protocols and disable weak protocols C-NGINX-SERVER-CNT-02 Very high ○ Not tested

- **State:** 🟢 Recommended
- **Description:**
 - 1. **Obtain SSL Certificates:**
 - a. Install the Certbot utility from Let's Encrypt, which is a popular choice for obtaining SSL certificates:

```
sudo apt-get update
sudo apt-get install certbot python3-certbot-nginx
```
 - b. Use Certbot to obtain an SSL certificate for your domain. Ensure your domain is correctly mapped to the server:

```
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com
```
 - Follow the on-screen instructions, confirming any prompts to allow Certbot to configure SSL for you.
 - 2. **Configure NGINX for TLS 1.2 and TLS 1.3:**
 - a. Edit the NGINX configuration file:

```
sudo nano /etc/nginx/nginx.conf
```
 - b. Locate the 'http' block and ensure it includes the following directives to enable only TLS 1.2 and 1.3:

```
ssl_protocols TLSv1.2 TLSv1.3;
```

- c. Edit any specific server block files within the 'sites-available' directory to reflect these changes:
- ```
sudo nano /etc/nginx/sites-available/yourdomain
```
- Add or update the server block to include:
- ```
ssl_protocols TLSv1.2 TLSv1.3;
```
3. **Enable Strong Cipher Suites:**
- a. Edit the ssl configuration file or add the settings to your existing configurations such as 'nginx.conf' or 'default.conf':
- ```
sudo nano /etc/nginx/conf.d/ssl.conf
```
- b. Set the cipher suites, preferring strong and secure ciphers. For example:
- ```
ssl_ciphers 'ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256';
```
4. **Test the SSL Configuration:**
- a. Run an NGINX syntax test:
- ```
sudo nginx -t
```
- Ensure there are no errors. If there are errors, correct them based on the feedback provided.
- b. Restart NGINX to apply the changes:
- ```
sudo systemctl restart nginx
```
- c. Use SSL testing tools like SSL Labs' SSL Test to verify your SSL configuration is strong and only the desired protocols and ciphers are enabled. [Test your site at SSL Labs](#).

- References**
- [NGINX Server Security: Configuration of SSL](#)
 - [Certbot: Instructions on getting certificates](#)
 - [SSL Labs SSL Test](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. Require https communication C-KEYCLOAK-CNT-02 High Not tested

- **State:** Recommended
- **Description:**

Ensure Keycloak only serves sensitive endpoints over encrypted channels:

 1. **Configure Realm SSL Mode**
 - In the Admin Console, navigate to **Realm Settings** **Login**, set **Require SSL** to either:
 - **External requests only:** Allows HTTP from private IP ranges (localhost, 10.x.x.x, 192.168.x.x, etc.) but forces HTTPS for all other clients.
 - **All requests:** Enforces HTTPS for every request, regardless of source IP. [w/jw465150.gitbooks.io](#)
 2. **Deploy valid TLS certificates**
 - **Direct Keycloak TLS:** If terminating TLS in Keycloak itself, provide PEM files or a Java keystore on startup:

```
bin/kc.sh start \  
--https-certificate-file=/path/to/cert.pem \  
--https-certificate-key-file=/path/to/key.pem
```

Or point to a keystore:

```
bin/kc.sh start \  
--https-key-store-file=/path/to/keystore.p12
```
 - **Reverse-proxy TLS termination:** If using NGINX, Apache, or a load balancer in front of Keycloak:
 1. Install the certificate and key on the proxy.
 2. Terminate TLS there and forward traffic to Keycloak over HTTP (e.g. 127.0.0.1:8080).
 3. Ensure X-Forwarded-Proto is set so Keycloak recognizes the original scheme. [Keycloak](#)
 3. **Verify enforcement**
 - Attempt to access any Keycloak endpoint via http://; it should be blocked or redirected to https://.
 - Confirm cookies like KEYCLOAK_SESSION have the Secure flag set.

- By enforcing SSL Mode at the realm level and deploying trusted TLS certificates, you eliminate unencrypted access, protecting credentials and tokens from interception.
- References**
- [Realm SSL Mode configuration \(Require SSL\)](#)
 - [Configuring TLS for Keycloak certificates](#)

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce TLS encryption for all connections C-POSTGRESQL-CNT-POSTGRES-03 High Not tested

- **State:** Recommended
- **Description:**

Implement and regularly update TLS encryption for all connections to PostgreSQL to secure data in transit against eavesdropping and man-in-the-middle attacks. This control mandates that all client-to-server and inter-node communications are encrypted using strong TLS protocols (e.g., TLS 1.2 or TLS 1.3) and that proper certificate management is in place. Developers and DevOps engineers should integrate TLS configuration into the PostgreSQL deployment process and enforce these settings via centralized management tools to maintain a secure network environment.

Implementation Steps:

Enable TLS Encryption:
Configure PostgreSQL to require TLS for all incoming connections by setting the appropriate parameters in the PostgreSQL configuration file (e.g., ssl = on).

Configure Strong Cipher Suites:
Ensure that only secure TLS versions and cipher suites (e.g., TLS 1.2/1.3) are enabled, and disable deprecated protocols to enhance the encryption strength.

Manage Certificates Securely:
Use a centralized certificate management system to issue, renew, and manage SSL/TLS certificates for PostgreSQL, ensuring that certificates are always up-to-date and trusted.

Monitor and Audit TLS Usage:
Regularly review logs and perform security audits to verify that all connections are using TLS encryption, and adjust configurations as necessary to address emerging threats.
- **References:**
 - [PostgreSQL SSL Support Documentation](#)

Component: HashiCorp Vault

1 Recommended countermeasures

Rec1. Enforce TLS for HashiCorp Vault in production C-HASHICORP-VAULT-CNT-01 High Not tested

- **State:** Recommended
- **Description:**

To safeguard the communication between clients and HashiCorp Vault, it is essential to implement TLS (Transport Layer Security). This encryption protocol ensures that the data exchanged is secure from interception and unauthorized access. Follow these steps to configure TLS effectively:

 - **Obtain a Valid SSL/TLS Certificate:**
Acquire an SSL/TLS certificate from a trusted Certificate Authority (CA). This certificate verifies the authenticity of the server and ensures that the data is encrypted during transmission.

• **Configure HashiCorp Vault to Use the Certificate:**

Place the SSL/TLS certificate and the corresponding private key on the server where Vault is installed. In the Vault configuration file, specify the paths to these files under the TLS settings. For example:
tls_disable = 0
tls_cert_file = "/path/to/cert.pem"
tls_key_file = "/path/to/key.pem"

• **Set Up Certificate Authorities (CAs):**

If using internal or self-signed certificates, ensure you configure the clients to trust these certificates by distributing the root CA certificate. Include either a CA bundle file or individual CA certificates in the configuration.
Example:

tls_ca_file = "/path/to/ca.pem"

• **Enforce Secure Protocols and Ciphers:**

Configure and enforce the use of strong TLS protocols and ciphers. This enhances the security posture by keeping communications secure against known vulnerabilities.

tls_min_version = "tls12"

• **Regularly Update and Maintain Certificates:**

Regular maintenance and updates of the certificates are crucial. Monitor the expiration of the certificates and renew them timely. Additionally, stay informed about the latest security practices regarding TLS. Proper TLS implementation protects the integrity and confidentiality of data in-transit, ensuring secure communication with HashiCorp Vault.

Component: Browser

1 Recommended countermeasures

Rec1. Utilize encrypted communication tools C-BROWSER-CNT-06 Very high Not tested

• State: Recommended

• Description:

Implement and regularly update encrypted communication tools on all client machines to ensure that all browser communications are securely transmitted. This control enforces the use of secure protocols and encryption solutions, such as TLS and VPNs, to protect data in transit against interception and man-in-the-middle attacks.

Implementation Steps:

Enable Secure Protocols:

Configure browsers to default to secure protocols (e.g., HTTPS, TLS 1.2/1.3) and disable outdated, insecure versions.

Deploy VPN or Encrypted Tunnels:

Use reputable VPN solutions to encrypt browser traffic, especially when accessing untrusted networks, ensuring that data remains protected.

Monitor and Validate Encryption:

Regularly review encryption settings, validate certificate authenticity, and update encryption libraries to keep pace with emerging threats.

Integrate with Centralized Management:

Utilize enterprise tools to enforce encrypted communication configurations and monitor compliance across all client machines.

References:

- OWASP Transport Layer Protection Cheat Sheet

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: NGINX Server

Rec1. Implement modern SSL/TLS protocols and disable weak protocols C-NGINX-SERVER-CNT-02 Very high

Component: Keycloak

Rec1. Require https communication C-KEYCLOAK-CNT-02 High

Component: PostgreSQL

Rec1. Enforce TLS encryption for all connections C-POSTGRESQL-CNT-POSTGRES-03 High

Component: HashiCorp Vault

Rec1. Enforce TLS for HashiCorp Vault in production C-HASHICORP-VAULT-CNT-01 High

Component: Browser

Rec1. Utilize encrypted communication tools C-BROWSER-CNT-06 Very high

NIST 800-53: CA-9 Internal System Connections

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("Req") countermeasures which tests have failed are also shown as non-compliant ones.

Component: NGINX Server

1 Recommended countermeasures

Rec1. Implement modern SSL/TLS protocols and disable weak protocols C-NGINX-SERVER-CNT-02 Very high Not tested

• State: Recommended

• Description:

1. Obtain SSL Certificates:

a. Install the Certbot utility from Let's Encrypt, which is a popular choice for obtaining SSL certificates:

```
sudo apt-get update
sudo apt-get install certbot python3-certbot-nginx
```

b. Use Certbot to obtain an SSL certificate for your domain. Ensure your domain is correctly mapped to the server:

```
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com
```

Follow the on-screen instructions, confirming any prompts to allow Certbot to configure SSL for you.

2. Configure NGINX for TLS 1.2 and TLS 1.3:

a. Edit the NGINX configuration file:

```
sudo nano /etc/nginx/nginx.conf
```

b. Locate the 'http' block and ensure it includes the following directives to enable only TLS 1.2 and 1.3:

```
ssl_protocols TLSv1.2 TLSv1.3;
```

c. Edit any specific server block files within the 'sites-available' directory to reflect these changes:

```
sudo nano /etc/nginx/sites-available/yourdomain
```

Add or update the server block to include:

ssl_protocols TLSv1.2 TLSv1.3;

3. Enable Strong Cipher Suites:

a. Edit the ssl configuration file or add the settings to your existing configurations such as 'nginx.conf' or 'default.conf':

sudo nano /etc/nginx/conf.d/ssl.conf

b. Set the cipher suites, preferring strong and secure ciphers. For example:

ssl_ciphers 'ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256';

4. Test the SSL Configuration:

a. Run an NGINX syntax test:

sudo nginx -t

Ensure there are no errors. If there are errors, correct them based on the feedback provided.

b. Restart NGINX to apply the changes:

sudo systemctl restart nginx

c. Use SSL testing tools like SSL Labs' SSL Test to verify your SSL configuration is strong and only the desired protocols and ciphers are enabled. [Test your site at SSL Labs](#).

References

- [NGINX Server Security: Configuration of SSL](#)
- [Certbot: Instructions on getting certificates](#)
- [SSL Labs SSL Test](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. Require https communication C-KEYCLOAK-CNT-02 High Not tested

• State: Recommended

• Description:

Ensure Keycloak only serves sensitive endpoints over encrypted channels:

1. Configure Realm SSL Mode

- In the Admin Console, navigate to **Realm Settings** **Login**, set **Require SSL** to either:
 - **External requests only:** Allows HTTP from private IP ranges (localhost, 10.x.x.x, 192.168.x.x, etc.) but forces HTTPS for all other clients.
 - **All requests:** Enforces HTTPS for every request, regardless of source IP. [wjlw465150.gitbooks.io](#)

2. Deploy valid TLS certificates

- **Direct Keycloak TLS:** If terminating TLS in Keycloak itself, provide PEM files or a Java keystore on startup:

```
bin/kc.sh start \
--https-certificate-file=/path/to/cert.pem \
--https-certificate-key-file=/path/to/key.pem
Or point to a keystore:
bin/kc.sh start \
--https-key-store-file=/path/to/keystore.p12
```

- **Reverse-proxy TLS termination:** If using NGINX, Apache, or a load balancer in front of Keycloak:
 1. Install the certificate and key on the proxy.
 2. Terminate TLS there and forward traffic to Keycloak over HTTP (e.g. 127.0.0.1:8080).
 3. Ensure X-Forwarded-Proto is set so Keycloak recognizes the original scheme. [Keycloak](#)

3. Verify enforcement

- Attempt to access any Keycloak endpoint via http://; it should be blocked or redirected to https://.
- Confirm cookies like KEYCLOAK_SESSION have the Secure flag set.

By enforcing SSL Mode at the realm level and deploying trusted TLS certificates, you eliminate unencrypted access, protecting credentials and tokens from interception.

References

- [Realm SSL Mode configuration \(Require SSL\)](#)
- [Configuring TLS for Keycloak certificates](#)

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce TLS encryption for all connections C-POSTGRESQL-CNT-POSTGRES-03 High Not tested

• State: Recommended

• Description:

Implement and regularly update TLS encryption for all connections to PostgreSQL to secure data in transit against eavesdropping and man-in-the-middle attacks. This control mandates that all client-to-server and inter-node communications are encrypted using strong TLS protocols (e.g., TLS 1.2 or TLS 1.3) and that proper certificate management is in place. Developers and DevOps engineers should integrate TLS configuration into the PostgreSQL deployment process and enforce these settings via centralized management tools to maintain a secure network environment.

Implementation Steps:

Enable TLS Encryption:

Configure PostgreSQL to require TLS for all incoming connections by setting the appropriate parameters in the PostgreSQL configuration file (e.g., ssl = on).

Configure Strong Cipher Suites:

Ensure that only secure TLS versions and cipher suites (e.g., TLS 1.2/1.3) are enabled, and disable deprecated protocols to enhance the encryption strength.

Manage Certificates Securely:

Use a centralized certificate management system to issue, renew, and manage SSL/TLS certificates for PostgreSQL, ensuring that certificates are always up-to-date and trusted.

Monitor and Audit TLS Usage:

Regularly review logs and perform security audits to verify that all connections are using TLS encryption, and adjust configurations as necessary to address emerging threats.

References:

- [PostgreSQL SSL Support Documentation](#)

Component: HashiCorp Vault

1 Recommended countermeasures

Rec1. Enforce TLS for HashiCorp Vault in production C-HASHICORP-VAULT-CNT-01 High Not tested

• State: Recommended

• Description:

To safeguard the communication between clients and HashiCorp Vault, it is essential to implement TLS (Transport Layer Security). This encryption protocol ensures that the data exchanged is secure from interception and unauthorized access. Follow these steps to configure TLS effectively:

• Obtain a Valid SSL/TLS Certificate:

Acquire an SSL/TLS certificate from a trusted Certificate Authority (CA). This certificate verifies the authenticity of the server and ensures that the data is encrypted during transmission.

• Configure HashiCorp Vault to Use the Certificate:

Place the SSL/TLS certificate and the corresponding private key on the server where Vault is installed. In the Vault configuration file, specify the paths to these files under the TLS settings. For example:

tls_disable = 0
tls_cert_file = "/path/to/cert.pem"
tls_key_file = "/path/to/key.pem"

• Set Up Certificate Authorities (CAs):

If using internal or self-signed certificates, ensure you configure the clients to trust these certificates by distributing the root CA certificate. Include either a CA bundle file or individual CA certificates in the configuration.

Example:

tls_ca_file = "/path/to/ca.pem"

• **Enforce Secure Protocols and Ciphers:**

Configure and enforce the use of strong TLS protocols and ciphers. This enhances the security posture by keeping communications secure against known vulnerabilities.
tls_min_version = "tls12"

• **Regularly Update and Maintain Certificates:**

Regular maintenance and updates of the certificates are crucial. Monitor the expiration of the certificates and renew them timely. Additionally, stay informed about the latest security practices regarding TLS. Proper TLS implementation protects the integrity and confidentiality of data in-transit, ensuring secure communication with HashiCorp Vault.

Component: Browser

1 Recommended countermeasures

Rec1. Utilize encrypted communication tools C-BROWSER-CNT-06 Very high Not tested

- **State:** Recommended
- **Description:**
Implement and regularly update encrypted communication tools on all client machines to ensure that all browser communications are securely transmitted. This control enforces the use of secure protocols and encryption solutions, such as TLS and VPNs, to protect data in transit against interception and man-in-the-middle attacks.
Implementation Steps:
Enable Secure Protocols:
Configure browsers to default to secure protocols (e.g., HTTPS, TLS 1.2/1.3) and disable outdated, insecure versions.
Deploy VPN or Encrypted Tunnels:
Use reputable VPN solutions to encrypt browser traffic, especially when accessing untrusted networks, ensuring that data remains protected.
Monitor and Validate Encryption:
Regularly review encryption settings, validate certificate authenticity, and update encryption libraries to keep pace with emerging threats.
Integrate with Centralized Management:
Utilize enterprise tools to enforce encrypted communication configurations and monitor compliance across all client machines.
References:
 - [OWASP Transport Layer Protection Cheat Sheet](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: NGINX Server

Rec1. Implement modern SSL/TLS protocols and disable weak protocols C-NGINX-SERVER-CNT-02 Very high

Component: Keycloak

Rec1. Require https communication C-KEYCLOAK-CNT-02 High

Component: PostgreSQL

Rec1. Enforce TLS encryption for all connections C-POSTGRESQL-CNT-POSTGRES-03 High

Component: HashiCorp Vault

Rec1. Enforce TLS for HashiCorp Vault in production C-HASHICORP-VAULT-CNT-01 High

Component: Browser

Rec1. Utilize encrypted communication tools C-BROWSER-CNT-06 Very high

NIST 800-53: SC-8 TRANSMISSION CONFIDENTIALITY AND INTEGRITY

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: NGINX Server

1 Recommended countermeasures

Rec1. Implement modern SSL/TLS protocols and disable weak protocols C-NGINX-SERVER-CNT-02 Very high Not tested

- **State:** Recommended
- **Description:**
 1. **Obtain SSL Certificates:**
 - a. Install the Certbot utility from Let's Encrypt, which is a popular choice for obtaining SSL certificates:

```
sudo apt-get update
sudo apt-get install certbot python3-certbot-nginx
```
 - b. Use Certbot to obtain an SSL certificate for your domain. Ensure your domain is correctly mapped to the server:

```
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com
```

Follow the on-screen instructions, confirming any prompts to allow Certbot to configure SSL for you.
 2. **Configure NGINX for TLS 1.2 and TLS 1.3:**
 - a. Edit the NGINX configuration file:

```
sudo nano /etc/nginx/nginx.conf
```
 - b. Locate the 'http' block and ensure it includes the following directives to enable only TLS 1.2 and 1.3:

```
ssl_protocols TLSv1.2 TLSv1.3;
```
 - c. Edit any specific server block files within the 'sites-available' directory to reflect these changes:

```
sudo nano /etc/nginx/sites-available/yourdomain
```

Add or update the server block to include:

```
ssl_protocols TLSv1.2 TLSv1.3;
```
 3. **Enable Strong Cipher Suites:**
 - a. Edit the ssl configuration file or add the settings to your existing configurations such as 'nginx.conf' or 'default.conf':

sudo nano /etc/nginx/conf.d/ssl.conf

b. Set the cipher suites, preferring strong and secure ciphers. For example:

ssl_ciphers 'ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256';

4. Test the SSL Configuration:

a. Run an NGINX syntax test:

sudo nginx -t

Ensure there are no errors. If there are errors, correct them based on the feedback provided.

b. Restart NGINX to apply the changes:

sudo systemctl restart nginx

c. Use SSL testing tools like SSL Labs' SSL Test to verify your SSL configuration is strong and only the desired protocols and ciphers are enabled. [Test your site at SSL Labs.](#)

References

- [NGINX Server Security: Configuration of SSL](#)
- [Certbot: Instructions on getting certificates](#)
- [SSL Labs SSL Test](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. Require https communication C-KEYCLOAK-CNT-02 High ☐ Not tested

• State: Recommended

• Description:

Ensure Keycloak only serves sensitive endpoints over encrypted channels:

1. Configure Realm SSL Mode

- In the Admin Console, navigate to **Realm Settings** **Login**, set **Require SSL** to either:
 - **External requests only:** Allows HTTP from private IP ranges (localhost, 10.x.x.x, 192.168.x.x, etc.) but forces HTTPS for all other clients.
 - **All requests:** Enforces HTTPS for every request, regardless of source IP. [wjw465150.gitbooks.io](#)

2. Deploy valid TLS certificates

- **Direct Keycloak TLS:** If terminating TLS in Keycloak itself, provide PEM files or a Java keystore on startup:

```
bin/kc.sh start \  
--https-certificate-file=/path/to/cert.pem \  
--https-certificate-key-file=/path/to/key.pem  
Or point to a keystore:  
bin/kc.sh start \  
--https-key-store-file=/path/to/keystore.p12
```

- **Reverse-proxy TLS termination:** If using NGINX, Apache, or a load balancer in front of Keycloak:

1. Install the certificate and key on the proxy.
2. Terminate TLS there and forward traffic to Keycloak over HTTP (e.g. 127.0.0.1:8080).
3. Ensure X-Forwarded-Proto is set so Keycloak recognizes the original scheme. [Keycloak](#)

3. Verify enforcement

- Attempt to access any Keycloak endpoint via http://; it should be blocked or redirected to https://.
- Confirm cookies like KEYCLOAK_SESSION have the Secure flag set.

By enforcing SSL Mode at the realm level and deploying trusted TLS certificates, you eliminate unencrypted access, protecting credentials and tokens from interception.

References

- [Realm SSL Mode configuration \(Require SSL\)](#)
- [Configuring TLS for Keycloak certificates](#)

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce TLS encryption for all connections C-POSTGRESQL-CNT-POSTGRES-03 High ☐ Not tested

• State: Recommended

• Description:

Implement and regularly update TLS encryption for all connections to PostgreSQL to secure data in transit against eavesdropping and man-in-the-middle attacks. This control mandates that all client-to-server and inter-node communications are encrypted using strong TLS protocols (e.g., TLS 1.2 or TLS 1.3) and that proper certificate management is in place. Developers and DevOps engineers should integrate TLS configuration into the PostgreSQL deployment process and enforce these settings via centralized management tools to maintain a secure network environment.

Implementation Steps:

Enable TLS Encryption:

Configure PostgreSQL to require TLS for all incoming connections by setting the appropriate parameters in the PostgreSQL configuration file (e.g., ssl = on).

Configure Strong Cipher Suites:

Ensure that only secure TLS versions and cipher suites (e.g., TLS 1.2/1.3) are enabled, and disable deprecated protocols to enhance the encryption strength.

Manage Certificates Securely:

Use a centralized certificate management system to issue, renew, and manage SSL/TLS certificates for PostgreSQL, ensuring that certificates are always up-to-date and trusted.

Monitor and Audit TLS Usage:

Regularly review logs and perform security audits to verify that all connections are using TLS encryption, and adjust configurations as necessary to address emerging threats.

References:

- [PostgreSQL SSL Support Documentation](#)

Component: HashiCorp Vault

1 Recommended countermeasures

Rec1. Enforce TLS for HashiCorp Vault in production C-HASHICORP-VAULT-CNT-01 High ☐ Not tested

• State: Recommended

• Description:

To safeguard the communication between clients and HashiCorp Vault, it is essential to implement TLS (Transport Layer Security). This encryption protocol ensures that the data exchanged is secure from interception and unauthorized access. Follow these steps to configure TLS effectively:

• Obtain a Valid SSL/TLS Certificate:

Acquire an SSL/TLS certificate from a trusted Certificate Authority (CA). This certificate verifies the authenticity of the server and ensures that the data is encrypted during transmission.

• Configure HashiCorp Vault to Use the Certificate:

Place the SSL/TLS certificate and the corresponding private key on the server where Vault is installed. In the Vault configuration file, specify the paths to these files under the TLS settings. For example:

```
tls_disable = 0  
tls_cert_file = "/path/to/cert.pem"  
tls_key_file = "/path/to/key.pem"
```

• Set Up Certificate Authorities (CAs):

If using internal or self-signed certificates, ensure you configure the clients to trust these certificates by distributing the root CA certificate. Include either a CA bundle file or individual CA certificates in the configuration.

Example:

```
tls_ca_file = "/path/to/ca.pem"
```

• Enforce Secure Protocols and Ciphers:

Configure and enforce the use of strong TLS protocols and ciphers. This enhances the security posture by keeping communications secure against known vulnerabilities.

```
tls_min_version = "tls12"
```

• Regularly Update and Maintain Certificates:

Regular maintenance and updates of the certificates are crucial. Monitor the expiration of the certificates and renew them timely. Additionally, stay informed about the latest security practices regarding TLS.

Proper TLS implementation protects the integrity and confidentiality of data in-transit, ensuring secure communication with HashiCorp Vault.

Component: Browser

1 Recommended countermeasures

Rec1. Utilize encrypted communication tools C-BROWSER-CNT-06 Very high Not tested

- State: Recommended
- Description:

Implement and regularly update encrypted communication tools on all client machines to ensure that all browser communications are securely transmitted. This control enforces the use of secure protocols and encryption solutions, such as TLS and VPNs, to protect data in transit against interception and man-in-the-middle attacks.

Implementation Steps:

Enable Secure Protocols:
Configure browsers to default to secure protocols (e.g., HTTPS, TLS 1.2/1.3) and disable outdated, insecure versions.

Deploy VPN or Encrypted Tunnels:
Use reputable VPN solutions to encrypt browser traffic, especially when accessing untrusted networks, ensuring that data remains protected.

Monitor and Validate Encryption:
Regularly review encryption settings, validate certificate authenticity, and update encryption libraries to keep pace with emerging threats.

Integrate with Centralized Management:
Utilize enterprise tools to enforce encrypted communication configurations and monitor compliance across all client machines.

References:

 - OWASP Transport Layer Protection Cheat Sheet

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: NGINX Server

Rec1. Implement modern SSL/TLS protocols and disable weak protocols C-NGINX-SERVER-CNT-02 Very high

Component: Keycloak

Rec1. Require https communication C-KEYCLOAK-CNT-02 High

Component: PostgreSQL

Rec1. Enforce TLS encryption for all connections C-POSTGRESQL-CNT-POSTGRES-03 High

Component: HashiCorp Vault

Rec1. Enforce TLS for HashiCorp Vault in production C-HASHICORP-VAULT-CNT-01 High

Component: Browser

Rec1. Utilize encrypted communication tools C-BROWSER-CNT-06 Very high

NIST 800-53: SC-16 Transmission of Security and Privacy Attributes

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: NGINX Server

1 Recommended countermeasures

Rec1. Implement modern SSL/TLS protocols and disable weak protocols C-NGINX-SERVER-CNT-02 Very high Not tested

- State: Recommended
- Description:
 - Obtain SSL Certificates:
 - Install the Certbot utility from Let's Encrypt, which is a popular choice for obtaining SSL certificates:

```
sudo apt-get update
sudo apt-get install certbot python3-certbot-nginx
```
 - Use Certbot to obtain an SSL certificate for your domain. Ensure your domain is correctly mapped to the server:

```
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com
```

Follow the on-screen instructions, confirming any prompts to allow Certbot to configure SSL for you.
 - Configure NGINX for TLS 1.2 and TLS 1.3:
 - Edit the NGINX configuration file:

```
sudo nano /etc/nginx/nginx.conf
```
 - Locate the 'http' block and ensure it includes the following directives to enable only TLS 1.2 and 1.3:

```
ssl_protocols TLSv1.2 TLSv1.3;
```
 - Edit any specific server block files within the 'sites-available' directory to reflect these changes:

```
sudo nano /etc/nginx/sites-available/yourdomain
```

Add or update the server block to include:

```
ssl_protocols TLSv1.2 TLSv1.3;
```
 - Enable Strong Cipher Suites:
 - Edit the ssl configuration file or add the settings to your existing configurations such as 'nginx.conf' or 'default.conf':

```
sudo nano /etc/nginx/conf.d/ssl.conf
```
 - Set the cipher suites, preferring strong and secure ciphers. For example:

ssl_ciphers 'ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256';

4. Test the SSL Configuration:

a. Run an NGINX syntax test:

sudo nginx -t

Ensure there are no errors. If there are errors, correct them based on the feedback provided.

b. Restart NGINX to apply the changes:

sudo systemctl restart nginx

c. Use SSL testing tools like SSL Labs' SSL Test to verify your SSL configuration is strong and only the desired protocols and ciphers are enabled. [Test your site at SSL Labs.](#)

References

- [NGINX Server Security Configuration of SSL](#)
- [Certbot: Instructions on getting certificates](#)
- [SSL Labs SSL Test](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. Require https communication C-KEYCLOAK-CNT-02 High Not tested

• State: Recommended

• Description:

Ensure Keycloak only serves sensitive endpoints over encrypted channels:

1. Configure Realm SSL Mode

- In the Admin Console, navigate to **Realm Settings** **Login**, set **Require SSL** to either:
 - **External requests only:** Allows HTTP from private IP ranges (localhost, 10.x.x.x, 192.168.x.x, etc.) but forces HTTPS for all other clients.
 - **All requests:** Enforces HTTPS for every request, regardless of source IP. [wjwt465150.github.io](#)

2. Deploy valid TLS certificates

- **Direct Keycloak TLS:** If terminating TLS in Keycloak itself, provide PEM files or a Java keystore on startup:

```
bin/kc.sh start \  
--https-certificate-file=/path/to/cert.pem \  
--https-certificate-key-file=/path/to/key.pem  
Or point to a keystore:  
bin/kc.sh start \  
--https-key-store-file=/path/to/keystore.p12
```

- **Reverse-proxy TLS termination:** If using NGINX, Apache, or a load balancer in front of Keycloak:

1. Install the certificate and key on the proxy.
2. Terminate TLS there and forward traffic to Keycloak over HTTP (e.g. 127.0.0.1:8080).
3. Ensure X-Forwarded-Proto is set so Keycloak recognizes the original scheme. [Keycloak](#)

3. Verify enforcement

- Attempt to access any Keycloak endpoint via http://; it should be blocked or redirected to https://.
- Confirm cookies like KEYCLOAK_SESSION have the Secure flag set.

By enforcing SSL Mode at the realm level and deploying trusted TLS certificates, you eliminate unencrypted access, protecting credentials and tokens from interception.

References

- [Realm SSL Mode configuration \(Require SSL\)](#)
- [Configuring TLS for Keycloak certificates](#)

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce TLS encryption for all connections C-POSTGRESQL-CNT-POSTGRES-03 High Not tested

• State: Recommended

• Description:

Implement and regularly update TLS encryption for all connections to PostgreSQL to secure data in transit against eavesdropping and man-in-the-middle attacks. This control mandates that all client-to-server and inter-node communications are encrypted using strong TLS protocols (e.g., TLS 1.2 or TLS 1.3) and that proper certificate management is in place. Developers and DevOps engineers should integrate TLS configuration into the PostgreSQL deployment process and enforce these settings via centralized management tools to maintain a secure network environment.

Implementation Steps:

Enable TLS Encryption:

Configure PostgreSQL to require TLS for all incoming connections by setting the appropriate parameters in the PostgreSQL configuration file (e.g., ssl = on).

Configure Strong Cipher Suites:

Ensure that only secure TLS versions and cipher suites (e.g., TLS 1.2/1.3) are enabled, and disable deprecated protocols to enhance the encryption strength.

Manage Certificates Securely:

Use a centralized certificate management system to issue, renew, and manage SSL/TLS certificates for PostgreSQL, ensuring that certificates are always up-to-date and trusted.

Monitor and Audit TLS Usage:

Regularly review logs and perform security audits to verify that all connections are using TLS encryption, and adjust configurations as necessary to address emerging threats.

References:

- [PostgreSQL SSL Support Documentation](#)

Component: HashiCorp Vault

1 Recommended countermeasures

Rec1. Enforce TLS for HashiCorp Vault in production C-HASHICORP-VAULT-CNT-01 High Not tested

• State: Recommended

• Description:

To safeguard the communication between clients and HashiCorp Vault, it is essential to implement TLS (Transport Layer Security). This encryption protocol ensures that the data exchanged is secure from interception and unauthorized access. Follow these steps to configure TLS effectively:

• **Obtain a Valid SSL/TLS Certificate:**

Acquire an SSL/TLS certificate from a trusted Certificate Authority (CA). This certificate verifies the authenticity of the server and ensures that the data is encrypted during transmission.

• **Configure HashiCorp Vault to Use the Certificate:**

Place the SSL/TLS certificate and the corresponding private key on the server where Vault is installed. In the Vault configuration file, specify the paths to these files under the TLS settings. For example:

```
tls_disable = 0  
tls_cert_file = "/path/to/cert.pem"  
tls_key_file = "/path/to/key.pem"
```

• **Set Up Certificate Authorities (CAs):**

If using internal or self-signed certificates, ensure you configure the clients to trust these certificates by distributing the root CA certificate. Include either a CA bundle file or individual CA certificates in the configuration.

Example:

```
tls_ca_file = "/path/to/ca.pem"
```

• **Enforce Secure Protocols and Ciphers:**

Configure and enforce the use of strong TLS protocols and ciphers. This enhances the security posture by keeping communications secure against known vulnerabilities.

```
tls_min_version = "tls12"
```

• **Regularly Update and Maintain Certificates:**

Regular maintenance and updates of the certificates are crucial. Monitor the expiration of the certificates and renew them timely. Additionally, stay informed about the latest security practices regarding TLS.

Proper TLS implementation protects the integrity and confidentiality of data in-transit, ensuring secure communication with HashiCorp Vault.

Component: Browser

1 Recommended countermeasures

Rec1. Utilize encrypted communication tools

C-BROWSER-CNT-06

Very high

☒ Recommended

☐ Not tested

• State:

Recommended

• Description:

Implement and regularly update encrypted communication tools on all client machines to ensure that all browser communications are securely transmitted. This control enforces the use of secure protocols and encryption solutions, such as TLS and VPNs, to protect data in transit against interception and man-in-the-middle attacks.

Implementation Steps:

Enable Secure Protocols:

Configure browsers to default to secure protocols (e.g., HTTPS, TLS 1.2/1.3) and disable outdated, insecure versions.

Deploy VPN or Encrypted Tunnels:

Use reputable VPN solutions to encrypt browser traffic, especially when accessing untrusted networks, ensuring that data remains protected.

Monitor and Validate Encryption:

Regularly review encryption settings, validate certificate authenticity, and update encryption libraries to keep pace with emerging threats.

Integrate with Centralized Management:

Utilize enterprise tools to enforce encrypted communication configurations and monitor compliance across all client machines.

References:

OWASP Transport Layer Protection Cheat Sheet

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: NGINX Server

Rec1. Implement modern SSL/TLS protocols and disable weak protocols C-NGINX-SERVER-CNT-02

Very high

Component: Keycloak

Rec1. Require https communication C-KEYCLOAK-CNT-02

High

Component: PostgreSQL

Rec1. Enforce TLS encryption for all connections C-POSTGRESQL-CNT-POSTGRES-03

High

Component: HashiCorp Vault

Rec1. Enforce TLS for HashiCorp Vault in production C-HASHICORP-VAULT-CNT-01

High

Component: Browser

Rec1. Utilize encrypted communication tools C-BROWSER-CNT-06

Very high

NIST 800-53: SC-11 Trusted Path

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: NGINX Server

1 Recommended countermeasures

Rec1. Implement modern SSL/TLS protocols and disable weak protocols

C-NGINX-SERVER-CNT-02

Very high

☒ Recommended

☐ Not tested

• State:

Recommended

• Description:

1. Obtain SSL Certificates:

a. Install the Certbot utility from Let's Encrypt, which is a popular choice for obtaining SSL certificates:

```
sudo apt-get update
sudo apt-get install certbot python3-certbot-nginx
```

b. Use Certbot to obtain an SSL certificate for your domain. Ensure your domain is correctly mapped to the server:

```
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com
```

Follow the on-screen instructions, confirming any prompts to allow Certbot to configure SSL for you.

2. Configure NGINX for TLS 1.2 and TLS 1.3:

a. Edit the NGINX configuration file:

```
sudo nano /etc/nginx/nginx.conf
```

b. Locate the 'http' block and ensure it includes the following directives to enable only TLS 1.2 and 1.3:

```
ssl_protocols TLSv1.2 TLSv1.3;
```

c. Edit any specific server block files within the 'sites-available' directory to reflect these changes:

```
sudo nano /etc/nginx/sites-available/yourdomain
```

Add or update the server block to include:

```
ssl_protocols TLSv1.2 TLSv1.3;
```

3. Enable Strong Cipher Suites:

a. Edit the ssl configuration file or add the settings to your existing configurations such as 'nginx.conf' or 'default.conf':

```
sudo nano /etc/nginx/conf.d/ssl.conf
```

b. Set the cipher suites, preferring strong and secure ciphers. For example:

```
ssl_ciphers 'ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256';
```

4. Test the SSL Configuration:

a. Run an NGINX syntax test:

sudo nginx -t

- Ensure there are no errors. If there are errors, correct them based on the feedback provided.
- b. Restart NGINX to apply the changes:

sudo systemctl restart nginx

- c. Use SSL testing tools like SSL Labs' SSL Test to verify your SSL configuration is strong and only the desired protocols and ciphers are enabled. [Test your site at SSL Labs](#).

References

- [NGINX Server Security: Configuration of SSL](#)
- [Certbot: Instructions on getting certificates](#)
- [SSL Labs SSL Test](#)

Component: Keycloak

1 Recommended countermeasures

Rec1. Require https communication C-KEYCLOAK-CNT-02 High Not tested

- State: Recommended

Description:

Ensure Keycloak only serves sensitive endpoints over encrypted channels:

1. **Configure Realm SSL Mode**
 - In the Admin Console, navigate to **Realm Settings** **Login**, set **Require SSL** to either:
 - **External requests only:** Allows HTTP from private IP ranges (localhost, 10.x.x.x, 192.168.x.x, etc.) but forces HTTPS for all other clients.
 - **All requests:** Enforces HTTPS for every request, regardless of source IP. [wjw465150.github.io](#)
2. **Deploy valid TLS certificates**
 - **Direct Keycloak TLS:** If terminating TLS in Keycloak itself, provide PEM files or a Java keystore on startup:

```
bin/kc.sh start \  
--https-certificate-file=/path/to/cert.pem \  
--https-certificate-key-file=/path/to/key.pem  
Or point to a keystore:  
bin/kc.sh start \  
--https-key-store-file=/path/to/keystore.p12
```
 - **Reverse-proxy TLS termination:** If using NGINX, Apache, or a load balancer in front of Keycloak:
 1. Install the certificate and key on the proxy.
 2. Terminate TLS there and forward traffic to Keycloak over HTTP (e.g. 127.0.0.1:8080).
 3. Ensure X-Forwarded-Proto is set so Keycloak recognizes the original scheme. [Keycloak](#)
3. **Verify enforcement**
 - Attempt to access any Keycloak endpoint via http://; it should be blocked or redirected to https://.
 - Confirm cookies like KEYCLOAK_SESSION have the Secure flag set.

By enforcing SSL Mode at the realm level and deploying trusted TLS certificates, you eliminate unencrypted access, protecting credentials and tokens from interception.

References

- [Realm SSL Mode configuration \(Require SSL\)](#)
- [Configuring TLS for Keycloak certificates](#)

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce TLS encryption for all connections C-POSTGRESQL-CNT-POSTGRES-03 High Not tested

- State: Recommended

Description:

Implement and regularly update TLS encryption for all connections to PostgreSQL to secure data in transit against eavesdropping and man-in-the-middle attacks. This control mandates that all client-to-server and inter-node communications are encrypted using strong TLS protocols (e.g., TLS 1.2 or TLS 1.3) and that proper certificate management is in place. Developers and DevOps engineers should integrate TLS configuration into the PostgreSQL deployment process and enforce these settings via centralized management tools to maintain a secure network environment.

Implementation Steps:

Enable TLS Encryption:

Configure PostgreSQL to require TLS for all incoming connections by setting the appropriate parameters in the PostgreSQL configuration file (e.g., ssl = on).

Configure Strong Cipher Suites:

Ensure that only secure TLS versions and cipher suites (e.g., TLS 1.2/1.3) are enabled, and disable deprecated protocols to enhance the encryption strength.

Manage Certificates Securely:

Use a centralized certificate management system to issue, renew, and manage SSL/TLS certificates for PostgreSQL, ensuring that certificates are always up-to-date and trusted.

Monitor and Audit TLS Usage:

Regularly review logs and perform security audits to verify that all connections are using TLS encryption, and adjust configurations as necessary to address emerging threats.

References:

- [PostgreSQL SSL Support Documentation](#)

Component: HashiCorp Vault

1 Recommended countermeasures

Rec1. Enforce TLS for HashiCorp Vault in production C-HASHICORP-VAULT-CNT-01 High Not tested

- State: Recommended

Description:

To safeguard the communication between clients and HashiCorp Vault, it is essential to implement TLS (Transport Layer Security). This encryption protocol ensures that the data exchanged is secure from interception and unauthorized access. Follow these steps to configure TLS effectively:

Obtain a Valid SSL/TLS Certificate:

Acquire an SSL/TLS certificate from a trusted Certificate Authority (CA). This certificate verifies the authenticity of the server and ensures that the data is encrypted during transmission.

Configure HashiCorp Vault to Use the Certificate:

Place the SSL/TLS certificate and the corresponding private key on the server where Vault is installed. In the Vault configuration file, specify the paths to these files under the TLS settings. For example:

tls_disable = 0
tls_cert_file = "/path/to/cert.pem"
tls_key_file = "/path/to/key.pem"

Set Up Certificate Authorities (CAs):

If using internal or self-signed certificates, ensure you configure the clients to trust these certificates by distributing the root CA certificate. Include either a CA bundle file or individual CA certificates in the configuration.

Example:

tls_ca_file = "/path/to/ca.pem"

Enforce Secure Protocols and Ciphers:

Configure and enforce the use of strong TLS protocols and ciphers. This enhances the security posture by keeping communications secure against known vulnerabilities.

tls_min_version = "tls12"

Regularly Update and Maintain Certificates:

Regular maintenance and updates of the certificates are crucial. Monitor the expiration of the certificates and renew them timely. Additionally, stay informed about the latest security practices regarding TLS.

Proper TLS implementation protects the integrity and confidentiality of data in-transit, ensuring secure communication with HashiCorp Vault.

Component: Browser

1 Recommended countermeasures

Rec1. Utilize encrypted communication tools C-BROWSER-CNT-06 Very high Not tested

- State: Recommended

- **Description:**
Implement and regularly update encrypted communication tools on all client machines to ensure that all browser communications are securely transmitted. This control enforces the use of secure protocols and encryption solutions, such as TLS and VPNs, to protect data in transit against interception and man-in-the-middle attacks.
Implementation Steps:
Enable Secure Protocols:
Configure browsers to default to secure protocols (e.g., HTTPS, TLS 1.2/1.3) and disable outdated, insecure versions.
Deploy VPN or Encrypted Tunnels:
Use reputable VPN solutions to encrypt browser traffic, especially when accessing untrusted networks, ensuring that data remains protected.
Monitor and Validate Encryption:
Regularly review encryption settings, validate certificate authenticity, and update encryption libraries to keep pace with emerging threats.
Integrate with Centralized Management:
Utilize enterprise tools to enforce encrypted communication configurations and monitor compliance across all client machines.
References:
 - [OWASP Transport Layer Protection Cheat Sheet](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: NGINX Server

Rec1. Implement modern SSL/TLS protocols and disable weak protocols C-NGINX-SERVER-CNT-02 Very high

Component: Keycloak

Rec1. Require https communication C-KEYCLOAK-CNT-02 High

Component: PostgreSQL

Rec1. Enforce TLS encryption for all connections C-POSTGRESQL-CNT-POSTGRES-03 High

Component: HashiCorp Vault

Rec1. Enforce TLS for HashiCorp Vault in production C-HASHICORP-VAULT-CNT-01 High

Component: Browser

Rec1. Utilize encrypted communication tools C-BROWSER-CNT-06 Very high

NIST 800-53: AC-9 PREVIOUS LOGON NOTIFICATION

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High Not tested

- **State:** Recommended
- **Description:**
Configure PostgreSQL to require **Multi-Factor Authentication (MFA)** for database access by offloading authentication to an external Identity Provider (IdP) via protocols like **GSSAPI (Kerberos)**, **LDAP with StartTLS**, or **PAM (Pluggable Authentication Modules)**. Because PostgreSQL does not natively support MFA (like SMS or TOTP) directly within its core engine, this control involves integrating the database with a centralized authentication system that enforces MFA at the source. This ensures that a compromised database password alone is insufficient to gain access to sensitive data. The objective of this control is to prevent **Credential Theft and Unauthorized Access**. By requiring a second factor (such as a hardware token, mobile push, or biometric) via an enterprise IdP, you significantly harden the database against brute-force attacks and leaked administrative credentials.
Implementation guidance for developers and DevOps engineers:
 - **Leverage PAM for Local MFA:** On Linux hosts, configure PostgreSQL to use the **PAM** authentication method. You can then integrate PAM with modules like `pam_google_authenticator` or Duo Security to require a TOTP code during the login handshake.
 - *Edit `pg_hba.conf`:* host all all 0.0.0.0/0 pam pamservice=postgresql
 - **Integrate with LDAP/Active Directory:** Configure PostgreSQL to authenticate users via **LDAP**. Ensure that the LDAP server itself is configured to require MFA or is part of a Single Sign-On (SSO) chain that enforces it before granting a Kerberos ticket (GSSAPI).
 - **Use Certificate-Based Authentication:** Implement **Mutual TLS (mTLS)** where the client must present a valid SSL certificate signed by the internal CA. While technically "something you have," combining this with a password ("something you know") creates a multi-factor environment.
 - *Edit `pg_hba.conf`:* hostssl all all 0.0.0.0/0 cert clientcert=verify-full
 - **Deploy a Database Proxy:** Use a modern database proxy or "Cloud SQL Auth Proxy" (in GCP) or "IAM Database Authentication" (in AWS). These services allow users to log in using their Cloud Identity, which natively supports MFA, and then map that identity to a short-lived database token.
 - **Configure Connection Requirements:** Ensure that the `pg_hba.conf` file is strictly configured to only allow these high-security authentication methods, disabling simpler methods like trust, password, or md5.
 - **Monitor Authentication Logs:** Regularly audit the PostgreSQL logs for failed login attempts. Since MFA introduces an additional step, a spike in password successes followed by MFA failures is a high-fidelity indicator of a targeted credential attack.

This control ensures that your data layer is protected by the same rigorous identity standards used across the rest of your enterprise infrastructure.

References:
 - [PostgreSQL Documentation: Client Authentication Methods](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High Not tested

- **State:** Recommended
- **Description:**
To enhance the security posture of your MariaDB installation, it is crucial to implement several key account management practices. These include establishing stringent password policies, eliminating or modifying default accounts, enforcing the principle of least privilege, and enabling multi-factor authentication (MFA) where applicable. Follow these steps to effectively implement the security countermeasures:
 1. **Implement Strong Password Policies**
 - Edit the MariaDB configuration file `my.cnf`, typically located at `/etc/mysql/my.cnf` or `/etc/mysql/mariadb.conf.d/`.
 - Under the `[mysqld]` section, add or update the `default_password_lifetime` and `enforce_strong_password_policy` to suitable values:

```
[mysqld]
default_password_lifetime=90
enforce_strong_password_policy=ON
```
 - Restart MariaDB to apply changes:

```
sudo systemctl restart mariadb
```
 - Utilize the [PAM Authentication Plugin](#) to integrate with external password management systems if needed.
 2. **Remove or Change Default Accounts**

- Access the MariaDB server using a secure account with administrative privileges:
`mysql -u root -p`
- List all user accounts to identify default and unnecessary ones:
`SELECT User, Host FROM mysql.user;`
- For each default or unnecessary account, decide whether to remove or alter it. To remove an account, execute:
`DROP USER 'username'@'host';`
- To change default account passwords, use:
`ALTER USER 'username'@'host' IDENTIFIED BY 'newpassword';`
- Verify changes and ensure only necessary accounts remain:
`SELECT User, Host FROM mysql.user;`
- 3. **Enforce the Principle of Least Privilege**
 - For each user, review and adjust permissions to align with their specific job requirements:
`SHOW GRANTS FOR 'username'@'host';`
 - Revoke unnecessary privileges from users:
`REVOKE privilege ON database.table FROM 'username'@'host';`
 - Grant only the minimum required privileges:
`GRANT SELECT, INSERT ON database.table TO 'username'@'host';`
- 4. **Enable Multi-Factor Authentication (MFA)**
 - Evaluate your environment to determine where MFA can be integrated. Consider using plugins like [OpenIAM](#) for integrating MFA.
 - Implement the chosen MFA solution and configure it according to your organization's security policy.
 - Test authentication procedures to ensure they function correctly and securely with MFA in place.

- References
- [PAM Authentication Plugin](#)

Component: Card Reader

1 Recommended countermeasures

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High Not tested

- State: Recommended
- Description:
Configure the Card Reader's management interface to require strong authentication protocols, such as multi-factor authentication (MFA), and enforce robust password policies. Integrate with centralized identity management systems to control and monitor access, ensuring that only authorized personnel can interact with the device. Implement role-based access controls and secure channels for authentication transactions, and regularly audit user access and credentials to maintain compliance with security standards. This approach minimizes unauthorized access and protects sensitive cardholder data.
- References:
 - [NIST Digital Identity Guidelines](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High

Component: MariaDB

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High

Component: Card Reader

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High

NIST 800-53: AC-8 SYSTEM USE NOTIFICATION

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High Not tested

- State: Recommended
- Description:
Configure PostgreSQL to require **Multi-Factor Authentication (MFA)** for database access by offloading authentication to an external Identity Provider (IdP) via protocols like **GSSAPI (Kerberos)**, **LDAP with StartTLS**, or **PAM (Pluggable Authentication Modules)**. Because PostgreSQL does not natively support MFA (like SMS or TOTP) directly within its core engine, this control involves integrating the database with a centralized authentication system that enforces MFA at the source. This ensures that a compromised database password alone is insufficient to gain access to sensitive data. The objective of this control is to prevent **Credential Theft and Unauthorized Access**. By requiring a second factor (such as a hardware token, mobile push, or biometric) via an enterprise IdP, you significantly harden the database against brute-force attacks and leaked administrative credentials. Implementation guidance for developers and DevOps engineers:
 - **Leverage PAM for Local MFA:** On Linux hosts, configure PostgreSQL to use the **PAM** authentication method. You can then integrate PAM with modules like `pam_google_authenticator` or Duo Security to require a TOTP code during the login handshake.
 - `Edit pg_hba.conf:` host all all 0.0.0.0/0 pam pamservice=postgresql
 - **Integrate with LDAP/Active Directory:** Configure PostgreSQL to authenticate users via **LDAP**. Ensure that the LDAP server itself is configured to require MFA or is part of a Single Sign-On (SSO) chain that enforces it before granting a Kerberos ticket (GSSAPI).
 - **Use Certificate-Based Authentication:** Implement **Mutual TLS (mTLS)** where the client must present a valid SSL certificate signed by the internal CA. While technically "something you have," combining this with a password ("something you know") creates a multi-factor environment.
 - `Edit pg_hba.conf:` hostssl all all 0.0.0.0/0 cert clientcert=verify-full

- **Deploy a Database Proxy:** Use a modern database proxy or "Cloud SQL Auth Proxy" (in GCP) or "IAM Database Authentication" (in AWS). These services allow users to log in using their Cloud Identity, which natively supports MFA, and then map that identity to a short-lived database token.
- **Configure Connection Requirements:** Ensure that the pg_hba.conf file is strictly configured to only allow these high-security authentication methods, disabling simpler methods like trust, password, or md5.
- **Monitor Authentication Logs:** Regularly audit the PostgreSQL logs for failed login attempts. Since MFA introduces an additional step, a spike in password successes followed by MFA failures is a high-fidelity indicator of a targeted credential attack.

This control ensures that your data layer is protected by the same rigorous identity standards used across the rest of your enterprise infrastructure.

References:

- [PostgreSQL Documentation: Client Authentication Methods](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High Not tested

- State: Recommended

Description:

To enhance the security posture of your MariaDB installation, it is crucial to implement several key account management practices. These include establishing stringent password policies, eliminating or modifying default accounts, enforcing the principle of least privilege, and enabling multi-factor authentication (MFA) where applicable. Follow these steps to effectively implement the security countermeasures:

1. Implement Strong Password Policies

- Edit the MariaDB configuration file my.cnf, typically located at /etc/mysql/my.cnf or /etc/mysql/mariadb.conf.d/.
- Under the [mysqld] section, add or update the default_password_lifetime and enforce_strong_password_policy to suitable values:

```
[mysqld]
default_password_lifetime=90
enforce_strong_password_policy=ON
```

- Restart MariaDB to apply changes:

```
sudo systemctl restart mariadb
```

- Utilize the [PAM Authentication Plugin](#) to integrate with external password management systems if needed.

2. Remove or Change Default Accounts

- Access the MariaDB server using a secure account with administrative privileges:

```
mysql -u root -p
```

- List all user accounts to identify default and unnecessary ones:

```
SELECT User, Host FROM mysql.user;
```

- For each default or unnecessary account, decide whether to remove or alter it. To remove an account, execute:

```
DROP USER 'username'@'host';
```

- To change default account passwords, use:

```
ALTER USER 'username'@'host' IDENTIFIED BY 'newpassword';
```

- Verify changes and ensure only necessary accounts remain:

```
SELECT User, Host FROM mysql.user;
```

3. Enforce the Principle of Least Privilege

- For each user, review and adjust permissions to align with their specific job requirements:

```
SHOW GRANTS FOR 'username'@'host';
```

- Revoke unnecessary privileges from users:

```
REVOKE privilege ON database.table FROM 'username'@'host';
```

- Grant only the minimum required privileges:

```
GRANT SELECT, INSERT ON database.table TO 'username'@'host';
```

4. Enable Multi-Factor Authentication (MFA)

- Evaluate your environment to determine where MFA can be integrated. Consider using plugins like [OpenIAM](#) for integrating MFA.
- Implement the chosen MFA solution and configure it according to your organization's security policy.
- Test authentication procedures to ensure they function correctly and securely with MFA in place.

References

- [PAM Authentication Plugin](#)

Component: Card Reader

1 Recommended countermeasures

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High Not tested

- State: Recommended

Description:

Configure the Card Reader's management interface to require strong authentication protocols, such as multi-factor authentication (MFA), and enforce robust password policies. Integrate with centralized identity management systems to control and monitor access, ensuring that only authorized personnel can interact with the device. Implement role-based access controls and secure channels for authentication transactions, and regularly audit user access and credentials to maintain compliance with security standards. This approach minimizes unauthorized access and protects sensitive cardholder data.

References:

- [NIST Digital Identity Guidelines](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High

Component: MariaDB

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High

Component: Card Reader

NIST 800-53: AC-7 UNSUCCESSFUL LOGON ATTEMPTS

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High Not tested

State: Recommended

Description:

Configure PostgreSQL to require **Multi-Factor Authentication (MFA)** for database access by offloading authentication to an external Identity Provider (IdP) via protocols like **GSSAPI (Kerberos)**, **LDAP with StartTLS**, or **PAM (Pluggable Authentication Modules)**. Because PostgreSQL does not natively support MFA (like SMS or TOTP) directly within its core engine, this control involves integrating the database with a centralized authentication system that enforces MFA at the source. This ensures that a compromised database password alone is insufficient to gain access to sensitive data.

The objective of this control is to prevent **Credential Theft and Unauthorized Access**. By requiring a second factor (such as a hardware token, mobile push, or biometric) via an enterprise IdP, you significantly harden the database against brute-force attacks and leaked administrative credentials.

Implementation guidance for developers and DevOps engineers:

- Leverage PAM for Local MFA:** On Linux hosts, configure PostgreSQL to use the **PAM** authentication method. You can then integrate PAM with modules like `pam_google_authenticator` or Duo Security to require a TOTP code during the login handshake.
 - Edit `pg_hba.conf`: `host all all 0.0.0.0/0 pam pamservice=postgresql`
- Integrate with LDAP/Active Directory:** Configure PostgreSQL to authenticate users via **LDAP**. Ensure that the LDAP server itself is configured to require MFA or is part of a Single Sign-On (SSO) chain that enforces it before granting a Kerberos ticket (GSSAPI).
- Use Certificate-Based Authentication:** Implement **Mutual TLS (mTLS)** where the client must present a valid SSL certificate signed by the internal CA. While technically "something you have," combining this with a password ("something you know") creates a multi-factor environment.
 - Edit `pg_hba.conf`: `hostssl all all 0.0.0.0/0 cert clientcert=verify-full`
- Deploy a Database Proxy:** Use a modern database proxy or "Cloud SQL Auth Proxy" (in GCP) or "IAM Database Authentication" (in AWS). These services allow users to log in using their Cloud Identity, which natively supports MFA, and then map that identity to a short-lived database token.
- Configure Connection Requirements:** Ensure that the `pg_hba.conf` file is strictly configured to only allow these high-security authentication methods, disabling simpler methods like `trust`, `password`, or `md5`.
- Monitor Authentication Logs:** Regularly audit the PostgreSQL logs for failed login attempts. Since MFA introduces an additional step, a spike in password successes followed by MFA failures is a high-fidelity indicator of a targeted credential attack.

This control ensures that your data layer is protected by the same rigorous identity standards used across the rest of your enterprise infrastructure.

References:

- PostgreSQL Documentation: Client Authentication Methods

Component: MariaDB

1 Recommended countermeasures

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High Not tested

State: Recommended

Description:

To enhance the security posture of your MariaDB installation, it is crucial to implement several key account management practices. These include establishing stringent password policies, eliminating or modifying default accounts, enforcing the principle of least privilege, and enabling multi-factor authentication (MFA) where applicable. Follow these steps to effectively implement the security countermeasures:

- Implement Strong Password Policies**
 - Edit the MariaDB configuration file `my.cnf`, typically located at `/etc/mysql/my.cnf` or `/etc/mysql/mariadb.conf.d/`.
 - Under the `[mysqld]` section, add or update the `default_password_lifetime` and `enforce_strong_password_policy` to suitable values:

```
[mysqld]
default_password_lifetime=90
enforce_strong_password_policy=ON
```
 - Restart MariaDB to apply changes:

```
sudo systemctl restart mariadb
```
 - Utilize the [PAM Authentication Plugin](#) to integrate with external password management systems if needed.
- Remove or Change Default Accounts**
 - Access the MariaDB server using a secure account with administrative privileges:

```
mysql -u root -p
```
 - List all user accounts to identify default and unnecessary ones:

```
SELECT User, Host FROM mysql.user;
```
 - For each default or unnecessary account, decide whether to remove or alter it. To remove an account, execute:

```
DROP USER 'username'@'host';
```
 - To change default account passwords, use:

```
ALTER USER 'username'@'host' IDENTIFIED BY 'newpassword';
```
 - Verify changes and ensure only necessary accounts remain:

```
SELECT User, Host FROM mysql.user;
```
- Enforce the Principle of Least Privilege**
 - For each user, review and adjust permissions to align with their specific job requirements:

```
SHOW GRANTS FOR 'username'@'host';
```
 - Revoke unnecessary privileges from users:

```
REVOKE privilege ON database.table FROM 'username'@'host';
```
 - Grant only the minimum required privileges:

```
GRANT SELECT, INSERT ON database.table TO 'username'@'host';
```
- Enable Multi-Factor Authentication (MFA)**
 - Evaluate your environment to determine where MFA can be integrated. Consider using plugins like [OpenIAM](#) for integrating MFA.
 - Implement the chosen MFA solution and configure it according to your organization's security policy.

- Test authentication procedures to ensure they function correctly and securely with MFA in place.

- References
- [PAM Authentication Plugin](#)

Component: Card Reader

1 Recommended countermeasures

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High Not tested

- State: Recommended
- Description:
Configure the Card Reader's management interface to require strong authentication protocols, such as multi-factor authentication (MFA), and enforce robust password policies. Integrate with centralized identity management systems to control and monitor access, ensuring that only authorized personnel can interact with the device. Implement role-based access controls and secure channels for authentication transactions, and regularly audit user access and credentials to maintain compliance with security standards. This approach minimizes unauthorized access and protects sensitive cardholder data.
- References:
 - [NIST Digital Identity Guidelines](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High

Component: MariaDB

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High

Component: Card Reader

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High

NIST 800-53: IA-5 AUTHENTICATOR MANAGEMENT

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High Not tested

- State: Recommended
- Description:
Configure PostgreSQL to require **Multi-Factor Authentication (MFA)** for database access by offloading authentication to an external Identity Provider (IdP) via protocols like **GSSAPI (Kerberos)**, **LDAP with StartTLS**, or **PAM (Pluggable Authentication Modules)**. Because PostgreSQL does not natively support MFA (like SMS or TOTP) directly within its core engine, this control involves integrating the database with a centralized authentication system that enforces MFA at the source. This ensures that a compromised database password alone is insufficient to gain access to sensitive data. The objective of this control is to prevent **Credential Theft and Unauthorized Access**. By requiring a second factor (such as a hardware token, mobile push, or biometric) via an enterprise IdP, you significantly harden the database against brute-force attacks and leaked administrative credentials.
Implementation guidance for developers and DevOps engineers:
 - **Leverage PAM for Local MFA:** On Linux hosts, configure PostgreSQL to use the **PAM** authentication method. You can then integrate PAM with modules like `pam_google_authenticator` or Duo Security to require a TOTP code during the login handshake.
 - *Edit `pg_hba.conf`:* host all all 0.0.0.0/0 pam pamservice=postgresql
 - **Integrate with LDAP/Active Directory:** Configure PostgreSQL to authenticate users via **LDAP**. Ensure that the LDAP server itself is configured to require MFA or is part of a Single Sign-On (SSO) chain that enforces it before granting a Kerberos ticket (GSSAPI).
 - **Use Certificate-Based Authentication:** Implement **Mutual TLS (mTLS)** where the client must present a valid SSL certificate signed by the internal CA. While technically "something you have," combining this with a password ("something you know") creates a multi-factor environment.
 - *Edit `pg_hba.conf`:* hostssl all all 0.0.0.0/0 cert clientcert=verify-full
 - **Deploy a Database Proxy:** Use a modern database proxy or "Cloud SQL Auth Proxy" (in GCP) or "IAM Database Authentication" (in AWS). These services allow users to log in using their Cloud Identity, which natively supports MFA, and then map that identity to a short-lived database token.
 - **Configure Connection Requirements:** Ensure that the `pg_hba.conf` file is strictly configured to only allow these high-security authentication methods, disabling simpler methods like trust, password, or md5.
 - **Monitor Authentication Logs:** Regularly audit the PostgreSQL logs for failed login attempts. Since MFA introduces an additional step, a spike in password successes followed by MFA failures is a high-fidelity indicator of a targeted credential attack.This control ensures that your data layer is protected by the same rigorous identity standards used across the rest of your enterprise infrastructure.
- References:
 - [PostgreSQL Documentation: Client Authentication Methods](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High Not tested

- State: Recommended
- Description:
To enhance the security posture of your MariaDB installation, it is crucial to implement several key account management practices. These include establishing stringent password policies, eliminating or modifying default accounts, enforcing the principle of least privilege, and enabling multi-factor authentication (MFA) where applicable. Follow these steps to effectively implement the security countermeasures:
 1. **Implement Strong Password Policies**
 - Edit the MariaDB configuration file `my.cnf`, typically located at `/etc/mysql/my.cnf` or `/etc/mysql/mariadb.conf.d/`.
 - Under the `[mysqld]` section, add or update the `default_password_lifetime` and `enforce_strong_password_policy` to suitable values:

```
[mysqld]
default_password_lifetime=90
enforce_strong_password_policy=ON
```
 - Restart MariaDB to apply changes:

```
sudo systemctl restart mariadb
```
 - Utilize the [PAM Authentication Plugin](#) to integrate with external password management systems if needed.
 2. **Remove or Change Default Accounts**
 - Access the MariaDB server using a secure account with administrative privileges:

mysql -u root -p

- List all user accounts to identify default and unnecessary ones:
`SELECT User, Host FROM mysql.user;`
- For each default or unnecessary account, decide whether to remove or alter it. To remove an account, execute:
`DROP USER 'username'@'host';`
- To change default account passwords, use:
`ALTER USER 'username'@'host' IDENTIFIED BY 'newpassword';`
- Verify changes and ensure only necessary accounts remain:
`SELECT User, Host FROM mysql.user;`

3. Enforce the Principle of Least Privilege

- For each user, review and adjust permissions to align with their specific job requirements:
`SHOW GRANTS FOR 'username'@'host';`
- Revoke unnecessary privileges from users:
`REVOKE privilege ON database.table FROM 'username'@'host';`
- Grant only the minimum required privileges:
`GRANT SELECT, INSERT ON database.table TO 'username'@'host';`

4. Enable Multi-Factor Authentication (MFA)

- Evaluate your environment to determine where MFA can be integrated. Consider using plugins like [OpenIAM](#) for integrating MFA.
- Implement the chosen MFA solution and configure it according to your organization's security policy.
- Test authentication procedures to ensure they function correctly and securely with MFA in place.

References

- [PAM Authentication Plugin](#)

Component: Card Reader

1 Recommended countermeasures

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High Not tested

- State: Recommended
- Description: Configure the Card Reader's management interface to require strong authentication protocols, such as multi-factor authentication (MFA), and enforce robust password policies. Integrate with centralized identity management systems to control and monitor access, ensuring that only authorized personnel can interact with the device. Implement role-based access controls and secure channels for authentication transactions, and regularly audit user access and credentials to maintain compliance with security standards. This approach minimizes unauthorized access and protects sensitive cardholder data.
- References:
 - [NIST Digital Identity Guidelines](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High

Component: MariaDB

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High

Component: Card Reader

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High

NIST 800-53: IA-10 Adaptive Authentication

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High Not tested

- State: Recommended
- Description: Configure PostgreSQL to require **Multi-Factor Authentication (MFA)** for database access by offloading authentication to an external Identity Provider (IdP) via protocols like **GSSAPI (Kerberos)**, **LDAP with StartTLS**, or **PAM (Pluggable Authentication Modules)**. Because PostgreSQL does not natively support MFA (like SMS or TOTP) directly within its core engine, this control involves integrating the database with a centralized authentication system that enforces MFA at the source. This ensures that a compromised database password alone is insufficient to gain access to sensitive data. The objective of this control is to prevent **Credential Theft and Unauthorized Access**. By requiring a second factor (such as a hardware token, mobile push, or biometric) via an enterprise IdP, you significantly harden the database against brute-force attacks and leaked administrative credentials. Implementation guidance for developers and DevOps engineers:
 - **Leverage PAM for Local MFA:** On Linux hosts, configure PostgreSQL to use the **PAM** authentication method. You can then integrate PAM with modules like `pam_google_authenticator` or Duo Security to require a TOTP code during the login handshake.
 - *Edit `pg_hba.conf`:* host all 0.0.0.0/0 pam pamservice=postgresql
 - **Integrate with LDAP/Active Directory:** Configure PostgreSQL to authenticate users via **LDAP**. Ensure that the LDAP server itself is configured to require MFA or is part of a Single Sign-On (SSO) chain that enforces it before granting a Kerberos ticket (GSSAPI).
 - **Use Certificate-Based Authentication:** Implement **Mutual TLS (mTLS)** where the client must present a valid SSL certificate signed by the internal CA. While technically "something you have," combining this with a password ("something you know") creates a multi-factor environment.
 - *Edit `pg_hba.conf`:* hostssl all 0.0.0.0/0 cert clientcert=verify-full
 - **Deploy a Database Proxy:** Use a modern database proxy or "Cloud SQL Auth Proxy" (in GCP) or "IAM Database Authentication" (in AWS). These services allow users to log in using their Cloud Identity, which natively supports MFA, and then map that identity to a short-lived database token.
 - **Configure Connection Requirements:** Ensure that the `pg_hba.conf` file is strictly configured to only allow these high-security authentication methods, disabling simpler methods like trust, password, or md5.

- **Monitor Authentication Logs:** Regularly audit the PostgreSQL logs for failed login attempts. Since MFA introduces an additional step, a spike in password successes followed by MFA failures is a high-fidelity indicator of a targeted credential attack.

This control ensures that your data layer is protected by the same rigorous identity standards used across the rest of your enterprise infrastructure.

References:

- [PostgreSQL Documentation: Client Authentication Methods](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High Not tested

- State: Recommended

Description:

To enhance the security posture of your MariaDB installation, it is crucial to implement several key account management practices. These include establishing stringent password policies, eliminating or modifying default accounts, enforcing the principle of least privilege, and enabling multi-factor authentication (MFA) where applicable. Follow these steps to effectively implement the security countermeasures:

1. Implement Strong Password Policies

- Edit the MariaDB configuration file `my.cnf`, typically located at `/etc/mysql/my.cnf` or `/etc/mysql/mariadb.conf.d/`.
- Under the `[mysqld]` section, add or update the `default_password_lifetime` and `enforce_strong_password_policy` to suitable values:

```
[mysqld]
default_password_lifetime=90
enforce_strong_password_policy=ON
```

- Restart MariaDB to apply changes:

```
sudo systemctl restart mariadb
```

- Utilize the [PAM Authentication Plugin](#) to integrate with external password management systems if needed.

2. Remove or Change Default Accounts

- Access the MariaDB server using a secure account with administrative privileges:

```
mysql -u root -p
```

- List all user accounts to identify default and unnecessary ones:

```
SELECT User, Host FROM mysql.user;
```

- For each default or unnecessary account, decide whether to remove or alter it. To remove an account, execute:

```
DROP USER 'username'@'host';
```

- To change default account passwords, use:

```
ALTER USER 'username'@'host' IDENTIFIED BY 'newpassword';
```

- Verify changes and ensure only necessary accounts remain:

```
SELECT User, Host FROM mysql.user;
```

3. Enforce the Principle of Least Privilege

- For each user, review and adjust permissions to align with their specific job requirements:

```
SHOW GRANTS FOR 'username'@'host';
```

- Revoke unnecessary privileges from users:

```
REVOKE privilege ON database.table FROM 'username'@'host';
```

- Grant only the minimum required privileges:

```
GRANT SELECT, INSERT ON database.table TO 'username'@'host';
```

4. Enable Multi-Factor Authentication (MFA)

- Evaluate your environment to determine where MFA can be integrated. Consider using plugins like [OpenIAM](#) for integrating MFA.
- Implement the chosen MFA solution and configure it according to your organization's security policy.
- Test authentication procedures to ensure they function correctly and securely with MFA in place.

References

- [PAM Authentication Plugin](#)

Component: Card Reader

1 Recommended countermeasures

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High Not tested

- State: Recommended

Description:

Configure the Card Reader's management interface to require strong authentication protocols, such as multi-factor authentication (MFA), and enforce robust password policies. Integrate with centralized identity management systems to control and monitor access, ensuring that only authorized personnel can interact with the device. Implement role-based access controls and secure channels for authentication transactions, and regularly audit user access and credentials to maintain compliance with security standards. This approach minimizes unauthorized access and protects sensitive cardholder data.

References:

- [NIST Digital Identity Guidelines](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High

Component: MariaDB

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High

Component: Card Reader

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High

NIST 800-53: IA-3 Device Identification and Authentication

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL		
1 Recommended countermeasures		
Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High Not tested State: Recommended Description: Configure PostgreSQL to require Multi-Factor Authentication (MFA) for database access by offloading authentication to an external Identity Provider (IdP) via protocols like GSSAPI (Kerberos), LDAP with StartTLS, or PAM (Pluggable Authentication Modules). Because PostgreSQL does not natively support MFA (like SMS or TOTP) directly within its core engine, this control involves integrating the database with a centralized authentication system that enforces MFA at the source. This ensures that a compromised database password alone is insufficient to gain access to sensitive data. The objective of this control is to prevent Credential Theft and Unauthorized Access. By requiring a second factor (such as a hardware token, mobile push, or biometric) via an enterprise IdP, you significantly harden the database against brute-force attacks and leaked administrative credentials. Implementation guidance for developers and DevOps engineers: Leverage PAM for Local MFA: On Linux hosts, configure PostgreSQL to use the PAM authentication method. You can then integrate PAM with modules like <code>pam_google_authenticator</code> or Duo Security to require a TOTP code during the login handshake. <code>Edit pg_hba.conf:</code> host all 0.0.0.0/0 pam pamservice=postgresql Integrate with LDAP/Active Directory: Configure PostgreSQL to authenticate users via LDAP. Ensure that the LDAP server itself is configured to require MFA or is part of a Single Sign-On (SSO) chain that enforces it before granting a Kerberos ticket (GSSAPI). Use Certificate-Based Authentication: Implement Mutual TLS (mTLS) where the client must present a valid SSL certificate signed by the internal CA. While technically "something you have," combining this with a password ("something you know") creates a multi-factor environment. <code>Edit pg_hba.conf:</code> hostssl all 0.0.0.0/0 cert clientcert=verify-full Deploy a Database Proxy: Use a modern database proxy or "Cloud SQL Auth Proxy" (in GCP) or "IAM Database Authentication" (in AWS). These services allow users to log in using their Cloud Identity, which natively supports MFA, and then map that identity to a short-lived database token. Configure Connection Requirements: Ensure that the <code>pg_hba.conf</code> file is strictly configured to only allow these high-security authentication methods, disabling simpler methods like <code>trust</code>, <code>password</code>, or <code>md5</code>. Monitor Authentication Logs: Regularly audit the PostgreSQL logs for failed login attempts. Since MFA introduces an additional step, a spike in password successes followed by MFA failures is a high-fidelity indicator of a targeted credential attack. This control ensures that your data layer is protected by the same rigorous identity standards used across the rest of your enterprise infrastructure. References: - PostgreSQL Documentation: Client Authentication Methods		
Component: MariaDB		
1 Recommended countermeasures		
Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High Not tested State: Recommended Description: To enhance the security posture of your MariaDB installation, it is crucial to implement several key account management practices. These include establishing stringent password policies, eliminating or modifying default accounts, enforcing the principle of least privilege, and enabling multi-factor authentication (MFA) where applicable. Follow these steps to effectively implement the security countermeasures: Implement Strong Password Policies - Edit the MariaDB configuration file <code>my.cnf</code>, typically located at <code>/etc/mysql/my.cnf</code> or <code>/etc/mysql/mariadb.conf.d/</code>. - Under the <code>[mysqld]</code> section, add or update the <code>default_password_lifetime</code> and <code>enforce_strong_password_policy</code> to suitable values:<pre>[mysqld] default_password_lifetime=90 enforce_strong_password_policy=ON</pre> - Restart MariaDB to apply changes:<pre>sudo systemctl restart mariadb</pre> - Utilize the PAM Authentication Plugin to integrate with external password management systems if needed. Remove or Change Default Accounts - Access the MariaDB server using a secure account with administrative privileges:<pre>mysql -u root -p</pre> - List all user accounts to identify default and unnecessary ones:<pre>SELECT User, Host FROM mysql.user;</pre> - For each default or unnecessary account, decide whether to remove or alter it. To remove an account, execute:<pre>DROP USER 'username'@'host';</pre> - To change default account passwords, use:<pre>ALTER USER 'username'@'host' IDENTIFIED BY 'newpassword';</pre> - Verify changes and ensure only necessary accounts remain:<pre>SELECT User, Host FROM mysql.user;</pre> Enforce the Principle of Least Privilege - For each user, review and adjust permissions to align with their specific job requirements:<pre>SHOW GRANTS FOR 'username'@'host';</pre> - Revoke unnecessary privileges from users:<pre>REVOKE privilege ON database.table FROM 'username'@'host';</pre> - Grant only the minimum required privileges:<pre>GRANT SELECT, INSERT ON database.table TO 'username'@'host';</pre> Enable Multi-Factor Authentication (MFA) - Evaluate your environment to determine where MFA can be integrated. Consider using plugins like OpenIAM for integrating MFA. - Implement the chosen MFA solution and configure it according to your organization's security policy. - Test authentication procedures to ensure they function correctly and securely with MFA in place.		
References		

- [PAM Authentication Plugin](#)

Component: Card Reader

1 Recommended countermeasures

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High Not tested

- State: Recommended
- Description:
Configure the Card Reader's management interface to require strong authentication protocols, such as multi-factor authentication (MFA), and enforce robust password policies. Integrate with centralized identity management systems to control and monitor access, ensuring that only authorized personnel can interact with the device. Implement role-based access controls and secure channels for authentication transactions, and regularly audit user access and credentials to maintain compliance with security standards. This approach minimizes unauthorized access and protects sensitive cardholder data.
- References:
 - [NIST Digital Identity Guidelines](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High

Component: MariaDB

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High

Component: Card Reader

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High

NIST 800-53: IA-8 Identification and Authentication (non-organizational Users)

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High Not tested

- State: Recommended
- Description:
Configure PostgreSQL to require **Multi-Factor Authentication (MFA)** for database access by offloading authentication to an external Identity Provider (IdP) via protocols like **GSSAPI (Kerberos)**, **LDAP with StartTLS**, or **PAM (Pluggable Authentication Modules)**. Because PostgreSQL does not natively support MFA (like SMS or TOTP) directly within its core engine, this control involves integrating the database with a centralized authentication system that enforces MFA at the source. This ensures that a compromised database password alone is insufficient to gain access to sensitive data. The objective of this control is to prevent **Credential Theft and Unauthorized Access**. By requiring a second factor (such as a hardware token, mobile push, or biometric) via an enterprise IdP, you significantly harden the database against brute-force attacks and leaked administrative credentials. Implementation guidance for developers and DevOps engineers:
 - Leverage PAM for Local MFA:** On Linux hosts, configure PostgreSQL to use the **PAM** authentication method. You can then integrate PAM with modules like `pam_google_authenticator` or Duo Security to require a TOTP code during the login handshake.
 - Edit `pg_hba.conf`: host all all 0.0.0.0/0 pam pamservice=postgresql
 - Integrate with LDAP/Active Directory:** Configure PostgreSQL to authenticate users via **LDAP**. Ensure that the LDAP server itself is configured to require MFA or is part of a Single Sign-On (SSO) chain that enforces it before granting a Kerberos ticket (GSSAPI).
 - Use Certificate-Based Authentication:** Implement **Mutual TLS (mTLS)** where the client must present a valid SSL certificate signed by the internal CA. While technically "something you have," combining this with a password ("something you know") creates a multi-factor environment.
 - Edit `pg_hba.conf`: hostssl all all 0.0.0.0/0 cert clientcert=verify-full
 - Deploy a Database Proxy:** Use a modern database proxy or "Cloud SQL Auth Proxy" (in GCP) or "IAM Database Authentication" (in AWS). These services allow users to log in using their Cloud Identity, which natively supports MFA, and then map that identity to a short-lived database token.
 - Configure Connection Requirements:** Ensure that the `pg_hba.conf` file is strictly configured to only allow these high-security authentication methods, disabling simpler methods like trust, password, or md5.
 - Monitor Authentication Logs:** Regularly audit the PostgreSQL logs for failed login attempts. Since MFA introduces an additional step, a spike in password successes followed by MFA failures is a high-fidelity indicator of a targeted credential attack.This control ensures that your data layer is protected by the same rigorous identity standards used across the rest of your enterprise infrastructure.
- References:
 - [PostgreSQL Documentation: Client Authentication Methods](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High Not tested

- State: Recommended
- Description:
To enhance the security posture of your MariaDB installation, it is crucial to implement several key account management practices. These include establishing stringent password policies, eliminating or modifying default accounts, enforcing the principle of least privilege, and enabling multi-factor authentication (MFA) where applicable. Follow these steps to effectively implement the security countermeasures:
 - Implement Strong Password Policies**
 - Edit the MariaDB configuration file `my.cnf`, typically located at `/etc/mysql/my.cnf` or `/etc/mysql/mariadb.conf.d/`.
 - Under the `[mysqld]` section, add or update the `default_password_lifetime` and `enforce_strong_password_policy` to suitable values:

```
[mysqld]
default_password_lifetime=90
enforce_strong_password_policy=ON
```
 - Restart MariaDB to apply changes:

```
sudo systemctl restart mariadb
```
 - Utilize the [PAM Authentication Plugin](#) to integrate with external password management systems if needed.
 - Remove or Change Default Accounts**
 - Access the MariaDB server using a secure account with administrative privileges:

```
mysql -u root -p
```
 - List all user accounts to identify default and unnecessary ones:

```
SELECT User, Host FROM mysql.user;
```

- For each default or unnecessary account, decide whether to remove or alter it. To remove an account, execute:

```
DROP USER 'username'@'host';
```

- To change default account passwords, use:

```
ALTER USER 'username'@'host' IDENTIFIED BY 'newpassword';
```

- Verify changes and ensure only necessary accounts remain:

```
SELECT User, Host FROM mysql.user;
```

3. **Enforce the Principle of Least Privilege**

- For each user, review and adjust permissions to align with their specific job requirements:

```
SHOW GRANTS FOR 'username'@'host';
```

- Revoke unnecessary privileges from users:

```
REVOKE privilege ON database.table FROM 'username'@'host';
```

- Grant only the minimum required privileges:

```
GRANT SELECT, INSERT ON database.table TO 'username'@'host';
```

4. **Enable Multi-Factor Authentication (MFA)**

- Evaluate your environment to determine where MFA can be integrated. Consider using plugins like [OpenIAM](#) for integrating MFA.
- Implement the chosen MFA solution and configure it according to your organization's security policy.
- Test authentication procedures to ensure they function correctly and securely with MFA in place.

References

- [PAM Authentication Plugin](#)

Component: Card Reader

1 Recommended countermeasures

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High ☐ Not tested

- State: ☒ Recommended

• Description:

Configure the Card Reader's management interface to require strong authentication protocols, such as multi-factor authentication (MFA), and enforce robust password policies. Integrate with centralized identity management systems to control and monitor access, ensuring that only authorized personnel can interact with the device. Implement role-based access controls and secure channels for authentication transactions, and regularly audit user access and credentials to maintain compliance with security standards. This approach minimizes unauthorized access and protects sensitive cardholder data.

References:

- [NIST Digital Identity Guidelines](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High

Component: MariaDB

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High

Component: Card Reader

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High

NIST 800-53: IA-2 Identification and Authentication (organizational Users)

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High ☐ Not tested

- State: ☒ Recommended

• Description:

Configure PostgreSQL to require **Multi-Factor Authentication (MFA)** for database access by offloading authentication to an external Identity Provider (IdP) via protocols like **GSSAPI (Kerberos)**, **LDAP with StartTLS**, or **PAM (Pluggable Authentication Modules)**. Because PostgreSQL does not natively support MFA (like SMS or TOTP) directly within its core engine, this control involves integrating the database with a centralized authentication system that enforces MFA at the source. This ensures that a compromised database password alone is insufficient to gain access to sensitive data. The objective of this control is to prevent **Credential Theft and Unauthorized Access**. By requiring a second factor (such as a hardware token, mobile push, or biometric) via an enterprise IdP, you significantly harden the database against brute-force attacks and leaked administrative credentials.

Implementation guidance for developers and DevOps engineers:

- **Leverage PAM for Local MFA:** On Linux hosts, configure PostgreSQL to use the **PAM** authentication method. You can then integrate PAM with modules like `pam_google_authenticator` or Duo Security to require a TOTP code during the login handshake.
 - *Edit `pg_hba.conf`:* host all 0.0.0.0/0 pam pamservice=postgresql
- **Integrate with LDAP/Active Directory:** Configure PostgreSQL to authenticate users via **LDAP**. Ensure that the LDAP server itself is configured to require MFA or is part of a Single Sign-On (SSO) chain that enforces it before granting a Kerberos ticket (GSSAPI).
- **Use Certificate-Based Authentication:** Implement **Mutual TLS (mTLS)** where the client must present a valid SSL certificate signed by the internal CA. While technically "something you have," combining this with a password ("something you know") creates a multi-factor environment.
 - *Edit `pg_hba.conf`:* hostssl all 0.0.0.0/0 cert clientcert=verify-full
- **Deploy a Database Proxy:** Use a modern database proxy or "Cloud SQL Auth Proxy" (in GCP) or "IAM Database Authentication" (in AWS). These services allow users to log in using their Cloud Identity, which natively supports MFA, and then map that identity to a short-lived database token.
- **Configure Connection Requirements:** Ensure that the `pg_hba.conf` file is strictly configured to only allow these high-security authentication methods, disabling simpler methods like trust, password, or md5.
- **Monitor Authentication Logs:** Regularly audit the PostgreSQL logs for failed login attempts. Since MFA introduces an additional step, a spike in password successes followed by MFA failures is a high-fidelity indicator of a targeted credential attack.

This control ensures that your data layer is protected by the same rigorous identity standards used across the rest of your enterprise infrastructure.

- References:
- [PostgreSQL Documentation: Client Authentication Methods](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High Not tested

- State: Recommended
- Description:

To enhance the security posture of your MariaDB installation, it is crucial to implement several key account management practices. These include establishing stringent password policies, eliminating or modifying default accounts, enforcing the principle of least privilege, and enabling multi-factor authentication (MFA) where applicable. Follow these steps to effectively implement the security countermeasures:

 - 1. Implement Strong Password Policies**
 - Edit the MariaDB configuration file `my.cnf`, typically located at `/etc/mysql/my.cnf` or `/etc/mysql/mariadb.conf.d/`.
 - Under the `[mysqld]` section, add or update the `default_password_lifetime` and `enforce_strong_password_policy` to suitable values:

```
[mysqld]
default_password_lifetime=90
enforce_strong_password_policy=ON
```
 - Restart MariaDB to apply changes:

```
sudo systemctl restart mariadb
```
 - Utilize the [PAM Authentication Plugin](#) to integrate with external password management systems if needed.
 - 2. Remove or Change Default Accounts**
 - Access the MariaDB server using a secure account with administrative privileges:

```
mysql -u root -p
```
 - List all user accounts to identify default and unnecessary ones:

```
SELECT User, Host FROM mysql.user;
```
 - For each default or unnecessary account, decide whether to remove or alter it. To remove an account, execute:

```
DROP USER 'username'@'host';
```
 - To change default account passwords, use:

```
ALTER USER 'username'@'host' IDENTIFIED BY 'newpassword';
```
 - Verify changes and ensure only necessary accounts remain:

```
SELECT User, Host FROM mysql.user;
```
 - 3. Enforce the Principle of Least Privilege**
 - For each user, review and adjust permissions to align with their specific job requirements:

```
SHOW GRANTS FOR 'username'@'host';
```
 - Revoke unnecessary privileges from users:

```
REVOKE privilege ON database.table FROM 'username'@'host';
```
 - Grant only the minimum required privileges:

```
GRANT SELECT, INSERT ON database.table TO 'username'@'host';
```
 - 4. Enable Multi-Factor Authentication (MFA)**
 - Evaluate your environment to determine where MFA can be integrated. Consider using plugins like [OpenIAM](#) for integrating MFA.
 - Implement the chosen MFA solution and configure it according to your organization's security policy.
 - Test authentication procedures to ensure they function correctly and securely with MFA in place.

- References
- [PAM Authentication Plugin](#)

Component: Card Reader

1 Recommended countermeasures

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High Not tested

- State: Recommended
 - Description:

Configure the Card Reader's management interface to require strong authentication protocols, such as multi-factor authentication (MFA), and enforce robust password policies. Integrate with centralized identity management systems to control and monitor access, ensuring that only authorized personnel can interact with the device. Implement role-based access controls and secure channels for authentication transactions, and regularly audit user access and credentials to maintain compliance with security standards. This approach minimizes unauthorized access and protects sensitive cardholder data.
- References:
- [NIST Digital Identity Guidelines](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRES-CNT-POSTGRES-09 High

Component: MariaDB

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High

Component: Card Reader

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High

NIST 800-53: IA-4 Identifier Management

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement multi-factor authentication (MFA) using identity providers

C-POSTGRESQL-CNT-POSTGRES-09

High

Not tested

State: Recommended

Description:

Configure PostgreSQL to require **Multi-Factor Authentication (MFA)** for database access by offloading authentication to an external Identity Provider (IdP) via protocols like **GSSAPI (Kerberos)**, **LDAP with StartTLS**, or **PAM (Pluggable Authentication Modules)**. Because PostgreSQL does not natively support MFA (like SMS or TOTP) directly within its core engine, this control involves integrating the database with a centralized authentication system that enforces MFA at the source. This ensures that a compromised database password alone is insufficient to gain access to sensitive data. The objective of this control is to prevent **Credential Theft and Unauthorized Access**. By requiring a second factor (such as a hardware token, mobile push, or biometric) via an enterprise IdP, you significantly harden the database against brute-force attacks and leaked administrative credentials.

Implementation guidance for developers and DevOps engineers:

- Leverage PAM for Local MFA:** On Linux hosts, configure PostgreSQL to use the **PAM** authentication method. You can then integrate PAM with modules like `pam_google_authenticator` or Duo Security to require a TOTP code during the login handshake.
 - Edit `pg_hba.conf`: host all all 0.0.0.0/0 pam pamservice=postgresql
- Integrate with LDAP/Active Directory:** Configure PostgreSQL to authenticate users via **LDAP**. Ensure that the LDAP server itself is configured to require MFA or is part of a Single Sign-On (SSO) chain that enforces it before granting a Kerberos ticket (GSSAPI).
- Use Certificate-Based Authentication:** Implement **Mutual TLS (mTLS)** where the client must present a valid SSL certificate signed by the internal CA. While technically "something you have," combining this with a password ("something you know") creates a multi-factor environment.
 - Edit `pg_hba.conf`: hostssl all all 0.0.0.0/0 cert clientcert=verify-full
- Deploy a Database Proxy:** Use a modern database proxy or "Cloud SQL Auth Proxy" (in GCP) or "IAM Database Authentication" (in AWS). These services allow users to log in using their Cloud Identity, which natively supports MFA, and then map that identity to a short-lived database token.
- Configure Connection Requirements:** Ensure that the `pg_hba.conf` file is strictly configured to only allow these high-security authentication methods, disabling simpler methods like trust, password, or md5.
- Monitor Authentication Logs:** Regularly audit the PostgreSQL logs for failed login attempts. Since MFA introduces an additional step, a spike in password successes followed by MFA failures is a high-fidelity indicator of a targeted credential attack.

This control ensures that your data layer is protected by the same rigorous identity standards used across the rest of your enterprise infrastructure.

References:

- PostgreSQL Documentation: Client Authentication Methods

Component: MariaDB

1 Recommended countermeasures

Rec1. Enforce strong authentication and access control

C-MARIADB-CNT-01

High

Not tested

State: Recommended

Description:

To enhance the security posture of your MariaDB installation, it is crucial to implement several key account management practices. These include establishing stringent password policies, eliminating or modifying default accounts, enforcing the principle of least privilege, and enabling multi-factor authentication (MFA) where applicable. Follow these steps to effectively implement the security countermeasures:

- Implement Strong Password Policies**
 - Edit the MariaDB configuration file `my.cnf`, typically located at `/etc/mysql/my.cnf` or `/etc/mysql/mariadb.conf.d/`.
 - Under the `[mysqld]` section, add or update the `default_password_lifetime` and `enforce_strong_password_policy` to suitable values:

```
[mysqld]
default_password_lifetime=90
enforce_strong_password_policy=ON
```
 - Restart MariaDB to apply changes:

```
sudo systemctl restart mariadb
```
 - Utilize the [PAM Authentication Plugin](#) to integrate with external password management systems if needed.
- Remove or Change Default Accounts**
 - Access the MariaDB server using a secure account with administrative privileges:

```
mysql -u root -p
```
 - List all user accounts to identify default and unnecessary ones:

```
SELECT User, Host FROM mysql.user;
```
 - For each default or unnecessary account, decide whether to remove or alter it. To remove an account, execute:

```
DROP USER 'username'@'host';
```
 - To change default account passwords, use:

```
ALTER USER 'username'@'host' IDENTIFIED BY 'newpassword';
```
 - Verify changes and ensure only necessary accounts remain:

```
SELECT User, Host FROM mysql.user;
```
- Enforce the Principle of Least Privilege**
 - For each user, review and adjust permissions to align with their specific job requirements:

```
SHOW GRANTS FOR 'username'@'host';
```
 - Revoke unnecessary privileges from users:

```
REVOKE privilege ON database.table FROM 'username'@'host';
```
 - Grant only the minimum required privileges:

```
GRANT SELECT, INSERT ON database.table TO 'username'@'host';
```
- Enable Multi-Factor Authentication (MFA)**
 - Evaluate your environment to determine where MFA can be integrated. Consider using plugins like [OpenIAM](#) for integrating MFA.
 - Implement the chosen MFA solution and configure it according to your organization's security policy.
 - Test authentication procedures to ensure they function correctly and securely with MFA in place.

References

- [PAM Authentication Plugin](#)

Component: Card Reader

1 Recommended countermeasures

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High Not tested

State: Recommended

Description:

Configure the Card Reader's management interface to require strong authentication protocols, such as multi-factor authentication (MFA), and enforce robust password policies. Integrate with centralized identity management systems to control and monitor access, ensuring that only authorized personnel can interact with the device. Implement role-based access controls and secure channels for authentication transactions, and regularly audit user access and credentials to maintain compliance with security standards. This approach minimizes unauthorized access and protects sensitive cardholder data.

References:

- NIST Digital Identity Guidelines

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High

Component: MariaDB

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High

Component: Card Reader

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High

NIST 800-53: IA-12 Identity Proofing

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High Not tested

State: Recommended

Description:

Configure PostgreSQL to require **Multi-Factor Authentication (MFA)** for database access by offloading authentication to an external Identity Provider (IdP) via protocols like **GSSAPI (Kerberos)**, **LDAP with StartTLS**, or **PAM (Pluggable Authentication Modules)**. Because PostgreSQL does not natively support MFA (like SMS or TOTP) directly within its core engine, this control involves integrating the database with a centralized authentication system that enforces MFA at the source. This ensures that a compromised database password alone is insufficient to gain access to sensitive data. The objective of this control is to prevent **Credential Theft and Unauthorized Access**. By requiring a second factor (such as a hardware token, mobile push, or biometric) via an enterprise IdP, you significantly harden the database against brute-force attacks and leaked administrative credentials.

Implementation guidance for developers and DevOps engineers:

- Leverage PAM for Local MFA:** On Linux hosts, configure PostgreSQL to use the **PAM** authentication method. You can then integrate PAM with modules like `pam_google_authenticator` or Duo Security to require a TOTP code during the login handshake.
 - Edit `pg_hba.conf`: `host all all 0.0.0.0/0 pam pamservice=postgresql`
- Integrate with LDAP/Active Directory:** Configure PostgreSQL to authenticate users via **LDAP**. Ensure that the LDAP server itself is configured to require MFA or is part of a Single Sign-On (SSO) chain that enforces it before granting a Kerberos ticket (GSSAPI).
- Use Certificate-Based Authentication:** Implement **Mutual TLS (mTLS)** where the client must present a valid SSL certificate signed by the internal CA. While technically "something you have," combining this with a password ("something you know") creates a multi-factor environment.
 - Edit `pg_hba.conf`: `hostssl all all 0.0.0.0/0 cert clientcert=verify-full`
- Deploy a Database Proxy:** Use a modern database proxy or "Cloud SQL Auth Proxy" (in GCP) or "IAM Database Authentication" (in AWS). These services allow users to log in using their Cloud Identity, which natively supports MFA, and then map that identity to a short-lived database token.
- Configure Connection Requirements:** Ensure that the `pg_hba.conf` file is strictly configured to only allow these high-security authentication methods, disabling simpler methods like trust, password, or md5.
- Monitor Authentication Logs:** Regularly audit the PostgreSQL logs for failed login attempts. Since MFA introduces an additional step, a spike in password successes followed by MFA failures is a high-fidelity indicator of a targeted credential attack.

This control ensures that your data layer is protected by the same rigorous identity standards used across the rest of your enterprise infrastructure.

References:

- PostgreSQL Documentation: Client Authentication Methods

Component: MariaDB

1 Recommended countermeasures

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High Not tested

State: Recommended

Description:

To enhance the security posture of your MariaDB installation, it is crucial to implement several key account management practices. These include establishing stringent password policies, eliminating or modifying default accounts, enforcing the principle of least privilege, and enabling multi-factor authentication (MFA) where applicable. Follow these steps to effectively implement the security countermeasures:

1. Implement Strong Password Policies

- Edit the MariaDB configuration file `my.cnf`, typically located at `/etc/mysql/my.cnf` or `/etc/mysql/mariadb.conf.d/`.
- Under the `[mysqld]` section, add or update the `default_password_lifetime` and `enforce_strong_password_policy` to suitable values:

```
[mysqld]
default_password_lifetime=90
enforce_strong_password_policy=ON
```
- Restart MariaDB to apply changes:

```
sudo systemctl restart mariadb
```
- Utilize the [PAM Authentication Plugin](#) to integrate with external password management systems if needed.

2. Remove or Change Default Accounts

- Access the MariaDB server using a secure account with administrative privileges:

```
mysql -u root -p
```
- List all user accounts to identify default and unnecessary ones:

```
SELECT User, Host FROM mysql.user;
```

Report: SpaceHub

Compliance report - 2026-03-25T16:40:26.994993079Z

141 of 182

- For each default or unnecessary account, decide whether to remove or alter it. To remove an account, execute:

```
DROP USER 'username'@'host';
```

- To change default account passwords, use:

```
ALTER USER 'username'@'host' IDENTIFIED BY 'newpassword';
```

- Verify changes and ensure only necessary accounts remain:

```
SELECT User, Host FROM mysql.user;
```

3. Enforce the Principle of Least Privilege

- For each user, review and adjust permissions to align with their specific job requirements:

```
SHOW GRANTS FOR 'username'@'host';
```

- Revoke unnecessary privileges from users:

```
REVOKE privilege ON database.table FROM 'username'@'host';
```

- Grant only the minimum required privileges:

```
GRANT SELECT, INSERT ON database.table TO 'username'@'host';
```

4. Enable Multi-Factor Authentication (MFA)

- Evaluate your environment to determine where MFA can be integrated. Consider using plugins like [OpenIAM](#) for integrating MFA.
- Implement the chosen MFA solution and configure it according to your organization's security policy.
- Test authentication procedures to ensure they function correctly and securely with MFA in place.

References

- [PAM Authentication Plugin](#)

Component: Card Reader

1 Recommended countermeasures

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High Not tested

- State: Recommended

Description:

Configure the Card Reader's management interface to require strong authentication protocols, such as multi-factor authentication (MFA), and enforce robust password policies. Integrate with centralized identity management systems to control and monitor access, ensuring that only authorized personnel can interact with the device. Implement role-based access controls and secure channels for authentication transactions, and regularly audit user access and credentials to maintain compliance with security standards. This approach minimizes unauthorized access and protects sensitive cardholder data.

References:

- [NIST Digital Identity Guidelines](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High

Component: MariaDB

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High

Component: Card Reader

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High

NIST 800-53: IA-1 Policy and Procedures

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High Not tested

- State: Recommended

Description:

Configure PostgreSQL to require **Multi-Factor Authentication (MFA)** for database access by offloading authentication to an external Identity Provider (IdP) via protocols like **GSSAPI (Kerberos)**, **LDAP with StartTLS**, or **PAM (Pluggable Authentication Modules)**. Because PostgreSQL does not natively support MFA (like SMS or TOTP) directly within its core engine, this control involves integrating the database with a centralized authentication system that enforces MFA at the source. This ensures that a compromised database password alone is insufficient to gain access to sensitive data.

The objective of this control is to prevent **Credential Theft and Unauthorized Access**. By requiring a second factor (such as a hardware token, mobile push, or biometric) via an enterprise IdP, you significantly harden the database against brute-force attacks and leaked administrative credentials.

Implementation guidance for developers and DevOps engineers:

- **Leverage PAM for Local MFA:** On Linux hosts, configure PostgreSQL to use the **PAM** authentication method. You can then integrate PAM with modules like `pam_google_authenticator` or Duo Security to require a TOTP code during the login handshake.
 - `Edit pg_hba.conf:` host all all 0.0.0.0/0 pam pamservice=postgresql
- **Integrate with LDAP/Active Directory:** Configure PostgreSQL to authenticate users via **LDAP**. Ensure that the LDAP server itself is configured to require MFA or is part of a Single Sign-On (SSO) chain that enforces it before granting a Kerberos ticket (GSSAPI).
- **Use Certificate-Based Authentication:** Implement **Mutual TLS (mTLS)** where the client must present a valid SSL certificate signed by the internal CA. While technically "something you have," combining this with a password ("something you know") creates a multi-factor environment.
 - `Edit pg_hba.conf:` hostssl all all 0.0.0.0/0 cert clientcert=verify-full
- **Deploy a Database Proxy:** Use a modern database proxy or "Cloud SQL Auth Proxy" (in GCP) or "IAM Database Authentication" (in AWS). These services allow users to log in using their Cloud Identity, which natively supports MFA, and then map that identity to a short-lived database token.
- **Configure Connection Requirements:** Ensure that the `pg_hba.conf` file is strictly configured to only allow these high-security authentication methods, disabling simpler methods like trust, password, or md5.
- **Monitor Authentication Logs:** Regularly audit the PostgreSQL logs for failed login attempts. Since MFA introduces an additional step, a spike in password successes followed by MFA failures is a high-fidelity indicator of a targeted credential attack.

This control ensures that your data layer is protected by the same rigorous identity standards used across the rest of your enterprise infrastructure.

References:

- [PostgreSQL Documentation: Client Authentication Methods](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High Not tested

- State: Recommended
- Description:

To enhance the security posture of your MariaDB installation, it is crucial to implement several key account management practices. These include establishing stringent password policies, eliminating or modifying default accounts, enforcing the principle of least privilege, and enabling multi-factor authentication (MFA) where applicable. Follow these steps to effectively implement the security countermeasures:

 - 1. Implement Strong Password Policies**
 - Edit the MariaDB configuration file my.cnf, typically located at /etc/mysql/my.cnf or /etc/mysql/mariadb.conf.d/.
 - Under the [mysqld] section, add or update the default_password_lifetime and enforce_strong_password_policy to suitable values:

```
[mysqld]
default_password_lifetime=90
enforce_strong_password_policy=ON
```
 - Restart MariaDB to apply changes:

```
sudo systemctl restart mariadb
```
 - Utilize the [PAM Authentication Plugin](#) to integrate with external password management systems if needed.
 - 2. Remove or Change Default Accounts**
 - Access the MariaDB server using a secure account with administrative privileges:

```
mysql -u root -p
```
 - List all user accounts to identify default and unnecessary ones:

```
SELECT User, Host FROM mysql.user;
```
 - For each default or unnecessary account, decide whether to remove or alter it. To remove an account, execute:

```
DROP USER 'username'@'host';
```
 - To change default account passwords, use:

```
ALTER USER 'username'@'host' IDENTIFIED BY 'newpassword';
```
 - Verify changes and ensure only necessary accounts remain:

```
SELECT User, Host FROM mysql.user;
```
 - 3. Enforce the Principle of Least Privilege**
 - For each user, review and adjust permissions to align with their specific job requirements:

```
SHOW GRANTS FOR 'username'@'host';
```
 - Revoke unnecessary privileges from users:

```
REVOKE privilege ON database.table FROM 'username'@'host';
```
 - Grant only the minimum required privileges:

```
GRANT SELECT, INSERT ON database.table TO 'username'@'host';
```
 - 4. Enable Multi-Factor Authentication (MFA)**
 - Evaluate your environment to determine where MFA can be integrated. Consider using plugins like [OpenIAM](#) for integrating MFA.
 - Implement the chosen MFA solution and configure it according to your organization's security policy.
 - Test authentication procedures to ensure they function correctly and securely with MFA in place.

- References
- [PAM Authentication Plugin](#)

Component: Card Reader

1 Recommended countermeasures

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High Not tested

- State: Recommended
- Description:

Configure the Card Reader's management interface to require strong authentication protocols, such as multi-factor authentication (MFA), and enforce robust password policies. Integrate with centralized identity management systems to control and monitor access, ensuring that only authorized personnel can interact with the device. Implement role-based access controls and secure channels for authentication transactions, and regularly audit user access and credentials to maintain compliance with security standards. This approach minimizes unauthorized access and protects sensitive cardholder data.
- References:
 - [NIST Digital Identity Guidelines](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement multi-factor authentication (MFA) using identity providers C-POSTGRESQL-CNT-POSTGRES-09 High

Component: MariaDB

Rec1. Enforce strong authentication and access control C-MARIADB-CNT-01 High

Component: Card Reader

Rec1. Deploy robust authentication measures C-CARD-READER-CNT-04 High

NIST 800-53: PE-5 Access Control for Output Devices

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technologyC-CARD-READER-CNT-01

High

Not tested

State: Recommended

Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
References:

- NIST Special Publication 800-53

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technologyC-CARD-READER-CNT-01

High

NIST 800-53: PE-4 Access Control for Transmission

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technologyC-CARD-READER-CNT-01

High

Not tested

State: Recommended

Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
References:

- NIST Special Publication 800-53

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technologyC-CARD-READER-CNT-01

High

NIST 800-53: PE-17 Alternate Work Site

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technologyC-CARD-READER-CNT-01

High

Not tested

State: Recommended

Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
References:

- NIST Special Publication 800-53

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technologyC-CARD-READER-CNT-01

High

NIST 800-53: PE-20 Asset Monitoring and Tracking

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology

C-CARD-READER-CNT-01

High

Not tested

State: Recommended

Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
References:

- NIST Special Publication 800-53

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High

NIST 800-53: PE-22 Component Marking

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology

C-CARD-READER-CNT-01

High

Not tested

State: Recommended

Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
References:

- NIST Special Publication 800-53

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High

NIST 800-53: PE-16 Delivery and Removal

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology

C-CARD-READER-CNT-01

High

Not tested

State: Recommended

Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
References:

- NIST Special Publication 800-53

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High

NIST 800-53: PE-21 Electromagnetic Pulse Protection

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology

C-CARD-READER-CNT-01

High

Not tested

State: Recommended

Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
References:

- NIST Special Publication 800-53

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High

NIST 800-53: PE-12 Emergency Lighting

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology

C-CARD-READER-CNT-01

High

Not tested

State: Recommended

Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
References:

- NIST Special Publication 800-53

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High

NIST 800-53: PE-11 Emergency Power

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology

C-CARD-READER-CNT-01

High

Not tested

State: Recommended

Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
References:

- NIST Special Publication 800-53

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High

NIST 800-53: PE-10 Emergency Shutoff

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology

C-CARD-READER-CNT-01

High

Not tested

- State: 🟢 Recommended
- Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
References:
 - [NIST Special Publication 800-53](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 ⬆️ High

NIST 800-53: PE-14 Environmental Controls

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 ⬆️ High ○ Not tested

- State: 🟢 Recommended
- Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
References:
 - [NIST Special Publication 800-53](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 ⬆️ High

NIST 800-53: PE-23 Facility Location

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 ⬆️ High ○ Not tested

- State: 🟢 Recommended
- Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
References:
 - [NIST Special Publication 800-53](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 ⬆️ High

NIST 800-53: PE-13 Fire Protection

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 ⬆️ High ○ Not tested

- State: 🟢 Recommended
- Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This

layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.

References:

- [NIST Special Publication 800-53](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High

NIST 800-53: PE-19 Information Leakage

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High Not tested

- State: Recommended
 - Description: Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
- References:**
- [NIST Special Publication 800-53](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High

NIST 800-53: PE-18 Location of System Components

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High Not tested

- State: Recommended
 - Description: Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
- References:**
- [NIST Special Publication 800-53](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High

NIST 800-53: PE-6 Monitoring Physical Access

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High Not tested

- State: Recommended
 - Description: Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
- References:**
- [NIST Special Publication 800-53](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High

NIST 800-53: PE-2 Physical Access Authorizations

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High Not tested

State: Recommended

Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
References:

- NIST Special Publication 800-53

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High

NIST 800-53: PE-3 Physical Access Control

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High Not tested

State: Recommended

Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
References:

- NIST Special Publication 800-53

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High

NIST 800-53: PE-1 Policy and Procedures

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High Not tested

State: Recommended

Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
References:

- NIST Special Publication 800-53

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High

NIST 800-53: PE-9 Power Equipment and Cabling

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High Not tested

State: Recommended

Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
References:

- NIST Special Publication 800-53

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High

NIST 800-53: PE-8 Visitor Access Records

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High Not tested

State: Recommended

Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
References:

- NIST Special Publication 800-53

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High

NIST 800-53: PE-15 Water Damage Protection

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Card Reader

1 Recommended countermeasures

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High Not tested

State: Recommended

Description:
Apply tamper-evident seals on the Card Reader and secure the device within a locked or monitored environment to deter physical tampering and unauthorized modifications. Restrict physical access to authorized personnel only, and integrate surveillance systems to continuously monitor the area. Regularly inspect the tamper-evident seals and associated security controls to ensure their integrity, enabling early detection of any breaches. This layered physical security approach minimizes the risk of device compromise and helps maintain the overall security of card data.
References:

- NIST Special Publication 800-53

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Card Reader

Rec1. Implement physical security measures and tamper-evident technology C-CARD-READER-CNT-01 High

NIST 800-53: AC-3 ACCESS ENFORCEMENT

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Keycloak

1 Recommended countermeasures

Rec1. Enable brute force detection

C-KEYCLOAK-CNT-01

High

Not tested

State: Recommended

Description:
Prevent credential-guessing attacks by enforcing automatic lockouts after repeated failed logins in Keycloak:

- Navigate to Brute-Force Detection**
 - In the Keycloak Admin Console, go to **Realm Settings** **Security Defenses** **Brute Force Detection** and toggle **Enabled** to **ON** [Keycloak](#).
- Configure lockout thresholds**
 - Max Login Failures:** Number of consecutive failures before lockout (e.g. 5–10).
 - Quick Login Check:** Minimum time (ms) between attempts to trigger temporary lockout.
 - Wait Increment & Max Wait:** Initial and maximum temporary lockout durations (e.g. 30 s increment up to 15 min).
 - Permanent Lockout:** Optionally require administrator unlocking after a defined number of temporary lockouts.
- Save and validate**
 - Click **Save**.
 - Test by attempting logins beyond your thresholds and confirm user accounts are temporarily or permanently disabled as configured.

By enabling this control, you automatically block both rapid-fire and sustained brute-force attempts, reducing the risk of unauthorized access.

References

- [Keycloak Server Administration Guide: Brute force detection](#)
- [Red Hat Build of Keycloak Server Administration Guide: Brute Force Attacks](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Keycloak

Rec1. Enable brute force detection C-KEYCLOAK-CNT-01 High

NIST 800-53: AC-10 CONCURRENT SESSION CONTROL

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Keycloak

2 Recommended countermeasures

Rec1. Implement state cookie verification

C-KEYCLOAK-CNT-03

Medium

Not tested

State: Recommended

Description:
Keycloak's built-in CSRF defense relies on the "double-submit cookie" pattern: it issues a KEYCLOAK_STATE_CHECKER cookie and embeds the same value in all forms or query parameters, then verifies they match on submission. To ensure this protection remains in place:

- Keep CSRF defenses enabled**
 - Do **not** disable or remove the stateChecker functionality when customizing login themes or flows. By default, Keycloak issues the KEYCLOAK_STATE_CHECKER cookie and inserts it into hidden form fields or URLs.
- Preserve stateChecker fields in custom themes**
 - If you override any of Keycloak's FreeMarker templates (e.g. login.ftl, account.ftl), ensure the following snippet remains intact in all <form> blocks:
<input type="hidden" name="stateChecker" value="{cookie.KEYCLOAK_STATE_CHECKER}"/>
 - For any actions invoked via GET (e.g. OAuth2 authorization endpoints), verify the state parameter matches the cookie.
- Validate and monitor CSRF failures**
 - Review Keycloak logs for warning or error messages mentioning "stateChecker mismatch" to detect potential CSRF attempts or misconfigurations.
 - Test your realm by attempting submissions with incorrect or missing stateChecker values to confirm they are rejected.

By preserving the double-submit state checker in all templates and keeping CSRF defenses enabled, you guard against cross-site request forgery attacks without additional infrastructure.

References

- [Cross-Site Request Forgery \(CSRF\) mitigation in Keycloak](#)
- [OWASP CSRF Prevention Cheat Sheet](#)

Rec2. Shorten token lifespans and enable revocation

C-KEYCLOAK-CNT-05

Very high

Not tested

State: Recommended

Description:
Limit the window of validity for issued tokens and provide a means to revoke all outstanding tokens when needed:

- Set minimal token lifespans**
 - In the Admin Console, go to **Realm Settings** **Tokens (Timeouts)**.
 - Access Token Lifespan:** Reduce to the shortest acceptable duration (e.g. 5 minutes).
 - Refresh Token Lifespan:** Reduce to the shortest acceptable duration (e.g. 15 minutes). [Keycloak](#)
- Enforce Not-Before revocation**
 - Navigate to **Realm Settings** **Sessions** **Revocation**.
 - Click **Set to now** to mark the current timestamp, then **Push** the revocation policy to all registered clients.
 - All tokens issued before this timestamp become invalid, forcing clients to obtain new tokens issued thereafter [Keycloak](#).

By combining short-lived tokens with a realm-wide Not-Before policy, you minimize the impact of leaked credentials and ensure that compromised tokens can be rapidly invalidated.

References

- [Session and token timeouts](#)
- [Compromised access and refresh tokens \(Not-Before revocation\)](#)

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement robust authentication and role-based access control

C-POSTGRESQL-CNT-POSTGRES-01

High

Not tested

Report: SpaceHub

Compliance report - 2026-03-25T16:40:26.994993079Z

151 of 182

- State: 🟢 Recommended
- Description:
Implement and regularly update robust authentication and role-based access control for PostgreSQL to ensure that only verified users with appropriate privileges can access and manage the database. This control mandates the use of strong authentication mechanisms (such as multi-factor authentication and complex passwords) and granular role-based access control (RBAC) to restrict permissions according to the principle of least privilege. Developers and DevOps engineers should integrate these practices into the PostgreSQL deployment using centralized identity management systems and enforce policies via configuration management tools.
Implementation Steps:
Enforce Strong Authentication:
Configure PostgreSQL to require strong passwords and, where applicable, integrate with external identity providers (e.g., LDAP, Active Directory) that support multi-factor authentication.
Implement Role-Based Access Control (RBAC):
Define roles with specific permissions and assign users only the privileges necessary for their responsibilities. Regularly review and adjust roles to ensure they align with current security policies.
Centralize Authentication Management:
Leverage centralized identity management systems to streamline user provisioning, password policies, and access audits across all PostgreSQL instances.
Monitor and Audit:
Enable detailed logging of authentication attempts and role changes. Periodically audit access logs and RBAC configurations to ensure compliance and detect potential anomalies.
References:
 - [OWASP Authentication Cheat Sheet](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Keycloak

- Rec1. Implement state cookie verification C-KEYCLOAK-CNT-03 🟡 Medium
- Rec2. Shorten token lifespans and enable revocation C-KEYCLOAK-CNT-05 🔴 Very high

Component: PostgreSQL

- Rec1. Implement robust authentication and role-based access control C-POSTGRESQL-CNT-POSTGRES-01 🔴 High

NIST 800-53: IA-11 RE-AUTHENTICATION

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Keycloak

2 Recommended countermeasures

- Rec1. Implement state cookie verification C-KEYCLOAK-CNT-03 🟡 Medium 🔴 Not tested

- State: 🟢 Recommended
- Description:
Keycloak's built-in CSRF defense relies on the "double-submit cookie" pattern: it issues a KEYCLOAK_STATE_CHECKER cookie and embeds the same value in all forms or query parameters, then verifies they match on submission. To ensure this protection remains in place:
 1. **Keep CSRF defenses enabled**
 - Do **not** disable or remove the stateChecker functionality when customizing login themes or flows. By default, Keycloak issues the KEYCLOAK_STATE_CHECKER cookie and inserts it into hidden form fields or URLs.
 2. **Preserve stateChecker fields in custom themes**
 - If you override any of Keycloak's FreeMarker templates (e.g. login.ftl, account.ftl), ensure the following snippet remains intact in all <form> blocks:
<input type="hidden" name="stateChecker" value="\${cookie.KEYCLOAK_STATE_CHECKER!}" />
 - For any actions invoked via GET (e.g. OAuth2 authorization endpoints), verify the state parameter matches the cookie.
 3. **Validate and monitor CSRF failures**
 - Review Keycloak logs for warning or error messages mentioning "stateChecker mismatch" to detect potential CSRF attempts or misconfigurations.
 - Test your realm by attempting submissions with incorrect or missing stateChecker values to confirm they are rejected.
By preserving the double-submit state checker in all templates and keeping CSRF defenses enabled, you guard against cross-site request forgery attacks without additional infrastructure.
References
 - [Cross-Site Request Forgery \(CSRF\) mitigation in Keycloak](#)
 - [OWASP CSRF Prevention Cheat Sheet](#)

- Rec2. Shorten token lifespans and enable revocation C-KEYCLOAK-CNT-05 🔴 Very high 🔴 Not tested

- State: 🟢 Recommended
- Description:
Limit the window of validity for issued tokens and provide a means to revoke all outstanding tokens when needed:
 1. **Set minimal token lifespans**
 - In the Admin Console, go to **Realm Settings Tokens (Timeouts)**.
 - **Access Token Lifespan:** Reduce to the shortest acceptable duration (e.g. 5 minutes).
 - **Refresh Token Lifespan:** Reduce to the shortest acceptable duration (e.g. 15 minutes). [Keycloak](#)
 2. **Enforce Not-Before revocation**
 - Navigate to **Realm Settings Sessions Revocation**.
 - Click **Set to now** to mark the current timestamp, then **Push** the revocation policy to all registered clients.
 - All tokens issued before this timestamp become invalid, forcing clients to obtain new tokens issued thereafter [Keycloak](#).
By combining short-lived tokens with a realm-wide Not-Before policy, you minimize the impact of leaked credentials and ensure that compromised tokens can be rapidly invalidated.
References
 - [Session and token timeouts](#)
 - [Compromised access and refresh tokens \(Not-Before revocation\)](#)

Component: PostgreSQL

1 Recommended countermeasures

- Rec1. Implement robust authentication and role-based access control C-POSTGRESQL-CNT-POSTGRES-01 🔴 High 🔴 Not tested

- State: 🟢 Recommended
- Description:
Implement and regularly update robust authentication and role-based access control for PostgreSQL to ensure that only verified users with appropriate privileges can access and manage the database. This control mandates the use of strong authentication mechanisms (such as multi-factor authentication and complex passwords) and granular role-based access control (RBAC) to restrict permissions according to the principle of least privilege. Developers and DevOps engineers should integrate these practices into the PostgreSQL deployment using centralized identity management systems and enforce policies via configuration management tools.
Implementation Steps:
Enforce Strong Authentication:
Configure PostgreSQL to require strong passwords and, where applicable, integrate with external identity providers (e.g., LDAP, Active Directory) that support multi-factor authentication.
Implement Role-Based Access Control (RBAC):
Define roles with specific permissions and assign users only the privileges necessary for their responsibilities. Regularly review and adjust roles to ensure they align with current security policies.
Centralize Authentication Management:
Leverage centralized identity management systems to streamline user provisioning, password policies, and access audits across all PostgreSQL instances.
Monitor and Audit:
Enable detailed logging of authentication attempts and role changes. Periodically audit access logs and RBAC configurations to ensure compliance and detect potential anomalies.

- References:
- [OWASP Authentication Cheat Sheet](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Keycloak

- Rec1. Implement state cookie verification C-KEYCLOAK-CNT-03 Medium
- Rec2. Shorten token lifespans and enable revocation C-KEYCLOAK-CNT-05 Very high

Component: PostgreSQL

- Rec1. Implement robust authentication and role-based access control C-POSTGRESQL-CNT-POSTGRES-01 High

NIST 800-53: SC-23 SESSION AUTHENTICITY

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Keycloak

2 Recommended countermeasures

Rec1. Implement state cookie verification C-KEYCLOAK-CNT-03 Medium ☐ Not tested

State: ☒ Recommended

Description:

Keycloak's built-in CSRF defense relies on the "double-submit cookie" pattern: it issues a KEYCLOAK_STATE_CHECKER cookie and embeds the same value in all forms or query parameters, then verifies they match on submission. To ensure this protection remains in place:

- Keep CSRF defenses enabled**
 - Do **not** disable or remove the stateChecker functionality when customizing login themes or flows. By default, Keycloak issues the KEYCLOAK_STATE_CHECKER cookie and inserts it into hidden form fields or URLs.
- Preserve stateChecker fields in custom themes**
 - If you override any of Keycloak's FreeMarker templates (e.g. login.ftl, account.ftl), ensure the following snippet remains intact in all <form> blocks:
<input type="hidden" name="stateChecker" value="\${cookie.KEYCLOAK_STATE_CHECKER!}" />
 - For any actions invoked via GET (e.g. OAuth2 authorization endpoints), verify the state parameter matches the cookie.
- Validate and monitor CSRF failures**
 - Review Keycloak logs for warning or error messages mentioning "stateChecker mismatch" to detect potential CSRF attempts or misconfigurations.
 - Test your realm by attempting submissions with incorrect or missing stateChecker values to confirm they are rejected.

By preserving the double-submit state checker in all templates and keeping CSRF defenses enabled, you guard against cross-site request forgery attacks without additional infrastructure.

References

- [Cross-Site Request Forgery \(CSRF\) mitigation in Keycloak](#)
- [OWASP CSRF Prevention Cheat Sheet](#)

Rec2. Shorten token lifespans and enable revocation C-KEYCLOAK-CNT-05 Very high ☐ Not tested

State: ☒ Recommended

Description:

Limit the window of validity for issued tokens and provide a means to revoke all outstanding tokens when needed:

- Set minimal token lifespans**
 - In the Admin Console, go to **Realm Settings** **Tokens (Timeouts)**.
 - Access Token Lifespan:** Reduce to the shortest acceptable duration (e.g. 5 minutes).
 - Refresh Token Lifespan:** Reduce to the shortest acceptable duration (e.g. 15 minutes), [Keycloak](#)
- Enforce Not-Before revocation**
 - Navigate to **Realm Settings** **Sessions** **Revocation**.
 - Click **Set to now** to mark the current timestamp, then **Push** the revocation policy to all registered clients.
 - All tokens issued before this timestamp become invalid, forcing clients to obtain new tokens issued thereafter [Keycloak](#).

By combining short-lived tokens with a realm-wide Not-Before policy, you minimize the impact of leaked credentials and ensure that compromised tokens can be rapidly invalidated.

References

- [Session and token timeouts](#)
- [Compromised access and refresh tokens \(Not-Before revocation\)](#)

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement robust authentication and role-based access control C-POSTGRESQL-CNT-POSTGRES-01 High ☐ Not tested

State: ☒ Recommended

Description:

Implement and regularly update robust authentication and role-based access control for PostgreSQL to ensure that only verified users with appropriate privileges can access and manage the database. This control mandates the use of strong authentication mechanisms (such as multi-factor authentication and complex passwords) and granular role-based access control (RBAC) to restrict permissions according to the principle of least privilege. Developers and DevOps engineers should integrate these practices into the PostgreSQL deployment using centralized identity management systems and enforce policies via configuration management tools.

Implementation Steps:

Enforce Strong Authentication:
Configure PostgreSQL to require strong passwords and, where applicable, integrate with external identity providers (e.g., LDAP, Active Directory) that support multi-factor authentication.

Implement Role-Based Access Control (RBAC):
Define roles with specific permissions and assign users only the privileges necessary for their responsibilities. Regularly review and adjust roles to ensure they align with current security policies.

Centralize Authentication Management:
Leverage centralized identity management systems to streamline user provisioning, password policies, and access audits across all PostgreSQL instances.

Monitor and Audit:
Enable detailed logging of authentication attempts and role changes. Periodically audit access logs and RBAC configurations to ensure compliance and detect potential anomalies.

References:

- [OWASP Authentication Cheat Sheet](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Keycloak

- Rec1. Implement state cookie verification C-KEYCLOAK-CNT-03 Medium
- Rec2. Shorten token lifespans and enable revocation C-KEYCLOAK-CNT-05 Very high

Component: PostgreSQL

Rec1. Implement robust authentication and role-based access control C-POSTGRESQL-CNT-POSTGRES-01 High

NIST 800-53: SA-15 Development Process, Standards, and Tools

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Keycloak

1 Recommended countermeasures

Rec1. Set short code validity periods C-KEYCLOAK-CNT-06 Very high Not tested

State: Recommended

Description:

Limit the window during which an issued authorization code can be exchanged for tokens to reduce the risk of code interception or replay:

1. Open the Tokens (Timeouts) settings

- In the Keycloak Admin Console, navigate to **Realm Settings** **Tokens (Timeouts)**.

2. Set the client login timeout

- Locate **Client login timeout** (the maximum time a client has to complete the Authorization Code Flow).
- Reduce this value to a few seconds (e.g. 5 s-15 s), ensuring legitimate clients can still exchange the code but minimizing exposure time [wjw465150.gitbooks.io](#).

3. Save and verify

- Click **Save**.
- Test by initiating an OIDC login, copying the returned code parameter, waiting longer than your configured timeout, and attempting the token exchange again—it should fail with invalid_grant.

By enforcing a very short authorization code lifespan, you mitigate interception or replay attacks against the OAuth2 code grant [Keycloak](#).

References:

- [Keycloak Server Administration Guide: Session and Token Timeouts](#)
- [Keycloak Securing Applications Guide: Authorization Code Flow](#)

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Apply regular security patches and minor version updates C-POSTGRESQL-CNT-POSTGRES-12 Very high Not tested

State: Recommended

Description:

Establish a rigorous patch management process to apply security fixes and minor version updates to the PostgreSQL database engine and its associated extensions. PostgreSQL minor releases (e.g., updating from 15.1 to 15.2) are specifically designed to be binary-compatible and include critical bug fixes and security patches. By applying these updates promptly, you protect the database from known vulnerabilities (CVEs) that could lead to remote code execution, privilege escalation, or denial-of-service attacks.

The objective of this control is **Vulnerability Mitigation**. It ensures that the database remains resilient against evolving threats by maintaining the software at the most stable and secure version supported by the community. Implementation guidance for developers and DevOps engineers:

- Monitor Security Announcements:** Subscribe to the [PostgreSQL Announce mailing list](#) or monitor the official security page to stay informed about new releases and critical CVEs.
- Automate Patching in Cloud/Managed Services:** If using AWS RDS or GCP Cloud SQL, enable "Auto Minor Version Upgrade." This allows the provider to automatically apply patches during a pre-defined maintenance window.
- Implement a Staging-to-Production Pipeline:** For self-hosted instances, never apply patches directly to production. Use a CI/CD pipeline to apply the update in a staging environment first to verify that application queries and extensions (like PostGIS or pgaudit) remain functional.
- Use Package Managers for Consistency:** Utilize system package managers (e.g., apt for Debian/Ubuntu or dnf for RHEL/CentOS) to manage updates. This ensures that dependencies are correctly handled and that the installation follows a predictable structure.
- Plan for Minimal Downtime:** Use high-availability (HA) configurations with a standby server. Perform a "switchover" to make the standby the new primary, patch the old primary, and then rejoin it to the cluster to minimize service interruption.
- Verify Post-Patch Integrity:** After patching, verify the version using SELECT version(); and run a suite of smoke tests to ensure that critical application paths and database performance metrics are stable.
- Retire End-of-Life (EOL) Versions:** Monitor the [PostgreSQL Versioning Policy](#). Ensure a migration plan is in place for major versions before they reach their 5-year EOL mark, as they will no longer receive security updates.

References:

- [PostgreSQL Official: Security Information and CVEs](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Keycloak

Rec1. Set short code validity periods C-KEYCLOAK-CNT-06 Very high

Component: PostgreSQL

Rec1. Apply regular security patches and minor version updates C-POSTGRESQL-CNT-POSTGRES-12 Very high

NIST 800-53: SA-8 Security and Privacy Engineering Principles

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Keycloak

1 Recommended countermeasures

Rec1. Set short code validity periods C-KEYCLOAK-CNT-06 Very high Not tested

State: Recommended

Description:

Limit the window during which an issued authorization code can be exchanged for tokens to reduce the risk of code interception or replay:

1. Open the Tokens (Timeouts) settings

- In the Keycloak Admin Console, navigate to **Realm Settings** **Tokens (Timeouts)**.

2. Set the client login timeout

- Locate **Client login timeout** (the maximum time a client has to complete the Authorization Code Flow).
- Reduce this value to a few seconds (e.g. 5 s-15 s), ensuring legitimate clients can still exchange the code but minimizing exposure time [wjw465150.gitbooks.io](#).

3. Save and verify

- Click **Save**.

- Test by initiating an OIDC login, copying the returned code parameter, waiting longer than your configured timeout, and attempting the token exchange again—it should fail with invalid_grant.

By enforcing a very short authorization code lifespan, you mitigate interception or replay attacks against the OAuth2 code grant [Keycloak](#).

References

- [Keycloak Server Administration Guide: Session and Token Timeouts](#)
- [Keycloak Securing Applications Guide: Authorization Code Flow](#)

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Apply regular security patches and minor version updates C-POSTGRESQL-CNT-POSTGRES-12 Very high ☐ Not tested

- State:  Recommended

• Description:

Establish a rigorous patch management process to apply security fixes and minor version updates to the PostgreSQL database engine and its associated extensions. PostgreSQL minor releases (e.g., updating from 15.1 to 15.2) are specifically designed to be binary-compatible and include critical bug fixes and security patches. By applying these updates promptly, you protect the database from known vulnerabilities (CVEs) that could lead to remote code execution, privilege escalation, or denial-of-service attacks.

The objective of this control is **Vulnerability Mitigation**. It ensures that the database remains resilient against evolving threats by maintaining the software at the most stable and secure version supported by the community. Implementation guidance for developers and DevOps engineers:

- **Monitor Security Announcements:** Subscribe to the [PostgreSQL Announce mailing list](#) or monitor the official security page to stay informed about new releases and critical CVEs.
- **Automate Patching in Cloud/Managed Services:** If using AWS RDS or GCP Cloud SQL, enable "Auto Minor Version Upgrade." This allows the provider to automatically apply patches during a pre-defined maintenance window.
- **Implement a Staging-to-Production Pipeline:** For self-hosted instances, never apply patches directly to production. Use a CI/CD pipeline to apply the update in a staging environment first to verify that application queries and extensions (like PostGIS or pgaudit) remain functional.
- **Use Package Managers for Consistency:** Utilize system package managers (e.g., apt for Debian/Ubuntu or dnf for RHEL/CentOS) to manage updates. This ensures that dependencies are correctly handled and that the installation follows a predictable structure.
- **Plan for Minimal Downtime:** Use high-availability (HA) configurations with a standby server. Perform a "switchover" to make the standby the new primary, patch the old primary, and then rejoin it to the cluster to minimize service interruption.
- **Verify Post-Patch Integrity:** After patching, verify the version using SELECT version(); and run a suite of smoke tests to ensure that critical application paths and database performance metrics are stable.
- **Retire End-of-Life (EOL) Versions:** Monitor the [PostgreSQL Versioning Policy](#). Ensure a migration plan is in place for major versions before they reach their 5-year EOL mark, as they will no longer receive security updates.

References:

- [PostgreSQL Official: Security Information and CVEs](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Keycloak

Rec1. Set short code validity periods C-KEYCLOAK-CNT-06 Very high

Component: PostgreSQL

Rec1. Apply regular security patches and minor version updates C-POSTGRESQL-CNT-POSTGRES-12 Very high

NIST 800-53: SA-3 System Development Life Cycle

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Keycloak

1 Recommended countermeasures

Rec1. Set short code validity periods C-KEYCLOAK-CNT-06 Very high ☐ Not tested

- State:  Recommended

• Description:

Limit the window during which an issued authorization code can be exchanged for tokens to reduce the risk of code interception or replay:

1. **Open the Tokens (Timeouts) settings**
 - In the Keycloak Admin Console, navigate to **Realm Settings** **Tokens (Timeouts)**.
2. **Set the client login timeout**
 - Locate **Client login timeout** (the maximum time a client has to complete the Authorization Code Flow).
 - Reduce this value to a few seconds (e.g. 5 s–15 s), ensuring legitimate clients can still exchange the code but minimizing exposure time [w/jw465150.gitbooks.io](#).
3. **Save and verify**
 - Click **Save**.
 - Test by initiating an OIDC login, copying the returned code parameter, waiting longer than your configured timeout, and attempting the token exchange again—it should fail with invalid_grant.

By enforcing a very short authorization code lifespan, you mitigate interception or replay attacks against the OAuth2 code grant [Keycloak](#).

References

- [Keycloak Server Administration Guide: Session and Token Timeouts](#)
- [Keycloak Securing Applications Guide: Authorization Code Flow](#)

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Apply regular security patches and minor version updates C-POSTGRESQL-CNT-POSTGRES-12 Very high ☐ Not tested

- State:  Recommended

• Description:

Establish a rigorous patch management process to apply security fixes and minor version updates to the PostgreSQL database engine and its associated extensions. PostgreSQL minor releases (e.g., updating from 15.1 to 15.2) are specifically designed to be binary-compatible and include critical bug fixes and security patches. By applying these updates promptly, you protect the database from known vulnerabilities (CVEs) that could lead to remote code execution, privilege escalation, or denial-of-service attacks.

The objective of this control is **Vulnerability Mitigation**. It ensures that the database remains resilient against evolving threats by maintaining the software at the most stable and secure version supported by the community. Implementation guidance for developers and DevOps engineers:

- **Monitor Security Announcements:** Subscribe to the [PostgreSQL Announce mailing list](#) or monitor the official security page to stay informed about new releases and critical CVEs.
- **Automate Patching in Cloud/Managed Services:** If using AWS RDS or GCP Cloud SQL, enable "Auto Minor Version Upgrade." This allows the provider to automatically apply patches during a pre-defined maintenance window.
- **Implement a Staging-to-Production Pipeline:** For self-hosted instances, never apply patches directly to production. Use a CI/CD pipeline to apply the update in a staging environment first to verify that application queries and extensions (like PostGIS or pgaudit) remain functional.
- **Use Package Managers for Consistency:** Utilize system package managers (e.g., apt for Debian/Ubuntu or dnf for RHEL/CentOS) to manage updates. This ensures that dependencies are correctly handled and that the installation follows a predictable structure.
- **Plan for Minimal Downtime:** Use high-availability (HA) configurations with a standby server. Perform a "switchover" to make the standby the new primary, patch the old primary, and then rejoin it to the cluster to minimize service interruption.
- **Verify Post-Patch Integrity:** After patching, verify the version using SELECT version(); and run a suite of smoke tests to ensure that critical application paths and database performance metrics are stable.

- **Retire End-of-Life (EOL) Versions:** Monitor the [PostgreSQL Versioning Policy](#). Ensure a migration plan is in place for major versions before they reach their 5-year EOL mark, as they will no longer receive security updates.

References:

- [PostgreSQL Official: Security Information and CVEs](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Keycloak

Rec1. Set short code validity periods C-KEYCLOAK-CNT-06 Very high

Component: PostgreSQL

Rec1. Apply regular security patches and minor version updates C-POSTGRESQL-CNT-POSTGRES-12 Very high

NIST 800-53: SI-3 Malicious Code Protection

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: Keycloak

1 Recommended countermeasures

Rec1. Set short code validity periods C-KEYCLOAK-CNT-06 Very high Not tested

- **State:** Recommended
- **Description:**

Limit the window during which an issued authorization code can be exchanged for tokens to reduce the risk of code interception or replay:

1. **Open the Tokens (Timeouts) settings**
 - In the Keycloak Admin Console, navigate to **Realm Settings** **Tokens (Timeouts)**.
2. **Set the client login timeout**
 - Locate **Client login timeout** (the maximum time a client has to complete the Authorization Code Flow).
 - Reduce this value to a few seconds (e.g. 5 s–15 s), ensuring legitimate clients can still exchange the code but minimizing exposure time [www.465150.gitbooks.io](#).
3. **Save and verify**
 - Click **Save**.
 - Test by initiating an OIDC login, copying the returned code parameter, waiting longer than your configured timeout, and attempting the token exchange again—it should fail with invalid_grant.

By enforcing a very short authorization code lifespan, you mitigate interception or replay attacks against the OAuth2 code grant [Keycloak](#).

References

- [Keycloak Server Administration Guide: Session and Token Timeouts](#)
- [Keycloak Securing Applications Guide: Authorization Code Flow](#)

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Apply regular security patches and minor version updates C-POSTGRESQL-CNT-POSTGRES-12 Very high Not tested

- **State:** Recommended
- **Description:**

Establish a rigorous patch management process to apply security fixes and minor version updates to the PostgreSQL database engine and its associated extensions. PostgreSQL minor releases (e.g., updating from 15.1 to 15.2) are specifically designed to be binary-compatible and include critical bug fixes and security patches. By applying these updates promptly, you protect the database from known vulnerabilities (CVEs) that could lead to remote code execution, privilege escalation, or denial-of-service attacks.

The objective of this control is **Vulnerability Mitigation**. It ensures that the database remains resilient against evolving threats by maintaining the software at the most stable and secure version supported by the community. Implementation guidance for developers and DevOps engineers:

- **Monitor Security Announcements:** Subscribe to the [PostgreSQL Announce mailing list](#) or monitor the official security page to stay informed about new releases and critical CVEs.
- **Automate Patching in Cloud/Managed Services:** If using AWS RDS or GCP Cloud SQL, enable "Auto Minor Version Upgrade." This allows the provider to automatically apply patches during a pre-defined maintenance window.
- **Implement a Staging-to-Production Pipeline:** For self-hosted instances, never apply patches directly to production. Use a CI/CD pipeline to apply the update in a staging environment first to verify that application queries and extensions (like PostGIS or pgaudit) remain functional.
- **Use Package Managers for Consistency:** Utilize system package managers (e.g., apt for Debian/Ubuntu or dnf for RHEL/CentOS) to manage updates. This ensures that dependencies are correctly handled and that the installation follows a predictable structure.
- **Plan for Minimal Downtime:** Use high-availability (HA) configurations with a standby server. Perform a "switchover" to make the standby the new primary, patch the old primary, and then rejoin it to the cluster to minimize service interruption.
- **Verify Post-Patch Integrity:** After patching, verify the version using SELECT version(); and run a suite of smoke tests to ensure that critical application paths and database performance metrics are stable.
- **Retire End-of-Life (EOL) Versions:** Monitor the [PostgreSQL Versioning Policy](#). Ensure a migration plan is in place for major versions before they reach their 5-year EOL mark, as they will no longer receive security updates.

References:

- [PostgreSQL Official: Security Information and CVEs](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: Keycloak

Rec1. Set short code validity periods C-KEYCLOAK-CNT-06 Very high

Component: PostgreSQL

Rec1. Apply regular security patches and minor version updates C-POSTGRESQL-CNT-POSTGRES-12 Very high

NIST 800-53: SC-5 DENIAL-OF-SERVICE PROTECTION

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement rate limiting and resource throttling C-POSTGRESQL-CNT-POSTGRES-07 High Not tested

- State: Recommended
- Description: Implement and regularly update rate limiting and resource throttling for PostgreSQL to protect against denial-of-service (DoS) attacks and resource exhaustion. This control ensures that query rates and system resource usage (CPU, memory, disk I/O) are constrained within defined limits, preserving database availability and performance even under heavy load or attack conditions. Developers and DevOps engineers should integrate rate limiting into the query processing and utilize OS-level or middleware resource controls, continuously monitoring system performance to adjust thresholds as needed.
Implementation Steps:
Define Acceptable Thresholds: Establish baseline limits for the number of queries, connections, and resource usage (CPU, memory, I/O) that the system can handle under normal operation.
Implement Query Rate Limiting: Use PostgreSQL settings or middleware solutions to limit the rate at which queries are accepted from clients, preventing overload during traffic spikes.
Configure Resource Throttling: Apply OS-level resource controls (e.g., cgroups on Linux) or similar mechanisms to restrict the CPU and memory usage of PostgreSQL processes, ensuring that no single query or connection can monopolize system resources.
Monitor and Audit: Continuously track performance metrics and log query activities. Regularly audit resource consumption and adjust thresholds to maintain optimal performance and security.
References:
 - PostgreSQL Performance Tips

Component: MariaDB

1 Recommended countermeasures

Rec1. Enforce resource limits and query restrictions C-MARIADB-CNT-06 High Not tested

- State: Recommended
- Description: To enhance the security and resilience of your MariaDB deployment, it is critical to configure settings that prevent excessive resource consumption and mitigate the risk of malicious traffic. Follow the steps below to configure these parameters effectively:
Step 1: Configure max_connections
 - Access your MariaDB server configuration file, typically located at /etc/mysql/my.cnf or /etc/my.cnf.
 - Find the section labeled [mysqld] and set the max_connections parameter. This limits the total number of simultaneous client connections allowed. For example:

```
[mysqld]
max_connections = 100
```
 - Save and close the configuration file once edited.
 - Restart the MariaDB service to apply the changes with the command:

```
sudo systemctl restart mariadb
```  
Step 2: Configure max_user_connections
 - Within the same configuration file, ensure you have set a limit on the number of simultaneous connections each user can have:

```
[mysqld]
max_user_connections = 10
```
 - Save your changes and restart the MariaDB service as outlined in Step 1.
Step 3: Set Query Timeouts
 - Adjust the query timeout settings to prevent long-running queries consuming excessive resources. Set the wait_timeout and the interactive_timeout to appropriate values:

```
[mysqld]
wait_timeout = 600
interactive_timeout = 600
```
 - Keep adjustments aligned with your application's needs and conduct testing to ensure performance is not negatively impacted.
 - Restart the MariaDB service to apply these settings.
Step 4: Implement Firewall Rules
 - To protect against malicious traffic, ensure a firewall is configured to only allow expected and secure traffic to the MariaDB server port, typically 3306. For example, use iptables or ufw for this purpose:

```
sudo ufw allow from [YOUR_NETWORK_IP_RANGE] to any port 3306
```
 - Block traffic from suspicious or known malicious IP addresses unless absolutely necessary.
 - Review and update firewall rules regularly as per best practices.
Once these settings are configured, monitor the server performance and ensure stability by reviewing logs and database metrics. Regular audits and updates will keep your MariaDB deployment secure and efficient.
References
 - MariaDB Server System Variables
 - Configuring MariaDB with Option Files

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement rate limiting and resource throttling C-POSTGRESQL-CNT-POSTGRES-07 High

Component: MariaDB

Rec1. Enforce resource limits and query restrictions C-MARIADB-CNT-06 High

NIST 800-53: AU-4 Audit Log Storage Capacity

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enable comprehensive auditing and logging of database activity C-POSTGRESQL-CNT-POSTGRES-10 Medium Not tested

- State: Recommended
- Description: Configure PostgreSQL to capture a detailed audit trail of all database activities, including successful and failed connection attempts, schema changes (DDL), and sensitive data access (DML). While PostgreSQL provides standard error logging, **comprehensive auditing** involves enabling extended logging parameters or utilizing extensions like **pgaudit**. This control ensures that there is a non-repudiable record of "who did what and when," which is vital for incident response, forensic investigations, and meeting regulatory compliance requirements (such as PCI-DSS, SOC2, or HIPAA).

The objective of this control is to provide **Visibility and Accountability**. By capturing the specific queries executed and the users involved, security teams can detect unauthorized data exfiltration, identify accidental misconfigurations, and reconstruct events following a security breach.

- Implementation guidance for developers and DevOps engineers:
- Install and Enable pgaudit:** For production environments, the standard log_statement = 'all' setting is often too noisy and lacks detail. Install the pgaudit extension to allow for granular auditing of specific classes of actions.
 - Configuration:* Add pgaudit to shared_preload_libraries in postgresql.conf and restart the service.
 - Configure Audit Classes:** Define what needs to be tracked. Use pgaudit.log = 'write, ddl, role' to capture data modifications, schema changes, and authorization changes (GRANT/REVOKE), while avoiding the performance overhead of logging every SELECT query.
 - Log Connection Metadata:** Ensure basic connection details are captured by setting the following in postgresql.conf:
 - log_connections = on
 - log_disconnections = on
 - log_hostname = on
 - Standardize Log Prefixes:** Use a consistent log_line_prefix to ensure logs contain the timestamp, user name, database name, and process ID (PID).
 - Example:* log_line_prefix = '%m [%p] %u@%d: '
 - Offload Logs to a Centralized SIEM:** Do not store audit logs solely on the database server. Use tools like **Fluentd**, **Logstash**, or **CloudWatch Logs** to stream PostgreSQL logs to a centralized Security Information and Event Management (SIEM) system for real-time alerting and long-term retention.
 - Implement Log Rotation:** To prevent the disk from filling up, configure strict log rotation policies using log_rotation_age and log_rotation_size.
 - Audit Privileged Users:** Apply specific auditing rules to superuser accounts. Since these accounts can bypass many security checks, every action they take should be logged regardless of the global configuration.

References:

- [PostgreSQL Documentation: Error Reporting and Logging](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Enable logging and auditing C-MARIADB-CNT-05

Medium

Not tested

State: Recommended

Description:

To enhance the security of your MariaDB installation, it's crucial to configure the MariaDB audit plugin to log critical activities. This will enable you to monitor authentication attempts, query executions, and privilege changes, helping you identify unauthorized or malicious access attempts. Follow these steps to implement this security countermeasure effectively:

1. Install the MariaDB Audit Plugin:

- Start by logging into your MariaDB server as a user with sufficient privileges to modify plugins.
- Execute the following command to install the audit plugin: `INSTALL PLUGIN server_audit SONAME 'server_audit.so';`
- Confirm the installation by checking that the plugin is listed under installed plugins: `SHOW PLUGINS;`

2. Configure Audit Logging:

- Edit the MariaDB server configuration file, typically located at `/etc/my.cnf` or `/etc/mysql/my.cnf`.
- Add or update the following configuration entries under the `[mysqld]` section: `server_audit_events='CONNECT,QUERY,TABLE' server_audit_logging=ON`
- `server_audit_file_path=/var/log/mariadb/server_audit.log` `server_audit_output_type=FILE`
- This configuration will log connection attempts ('CONNECT'), queries executed ('QUERY'), and tables accessed ('TABLE') to the specified log file.

3. Secure the Log Files:

- Set appropriate file permissions to ensure that only authorized users can access the log files: `chown mysql:mysql /var/log/mariadb/server_audit.log` `chmod 640 /var/log/mariadb/server_audit.log`

4. Enable Remote Logging (Optional):

- If centralizing logs, consider log forwarding using a syslog server by setting: `server_audit_output_type=SYSLOG`

5. Regular Log Review and Monitoring:

- Establish a routine to review the audit logs for any signs of suspicious activity, such as multiple failed login attempts or unauthorized query executions.
- Employ automated tools to parse and analyze logs for significant security events, if possible.

6. Update and Maintain the Audit Configuration:

- Regularly update the audit configurations based on emerging threats and requirements.
- Keep the MariaDB server and audit plugin up-to-date to benefit from the latest security patches and features.

References

- [MariaDB Audit Plugin Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enable comprehensive auditing and logging of database activity C-POSTGRESQL-CNT-POSTGRES-10

Medium

Component: MariaDB

Rec1. Enable logging and auditing C-MARIADB-CNT-05

Medium

NIST 800-53: AU-12 Audit Record Generation

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enable comprehensive auditing and logging of database activity C-POSTGRESQL-CNT-POSTGRES-10

Medium

Not tested

State: Recommended

Description:

Configure PostgreSQL to capture a detailed audit trail of all database activities, including successful and failed connection attempts, schema changes (DDL), and sensitive data access (DML). While PostgreSQL provides standard error logging, **comprehensive auditing** involves enabling extended logging parameters or utilizing extensions like **pgaudit**. This control ensures that there is a non-repudiable record of "who did what and when," which is vital for incident response, forensic investigations, and meeting regulatory compliance requirements (such as PCI-DSS, SOC2, or HIPAA).

The objective of this control is to provide **Visibility and Accountability**. By capturing the specific queries executed and the users involved, security teams can detect unauthorized data exfiltration, identify accidental misconfigurations, and reconstruct events following a security breach.

Implementation guidance for developers and DevOps engineers:

- Install and Enable pgaudit:** For production environments, the standard log_statement = 'all' setting is often too noisy and lacks detail. Install the pgaudit extension to allow for granular auditing of specific classes of actions.
 - Configuration:* Add pgaudit to shared_preload_libraries in postgresql.conf and restart the service.
- Configure Audit Classes:** Define what needs to be tracked. Use pgaudit.log = 'write, ddl, role' to capture data modifications, schema changes, and authorization changes (GRANT/REVOKE), while avoiding the performance overhead of logging every SELECT query.
- Log Connection Metadata:** Ensure basic connection details are captured by setting the following in postgresql.conf:
 - log_connections = on
 - log_disconnections = on
 - log_hostname = on
- Standardize Log Prefixes:** Use a consistent log_line_prefix to ensure logs contain the timestamp, user name, database name, and process ID (PID).
 - Example:* log_line_prefix = '%m [%p] %u@%d: '
- Offload Logs to a Centralized SIEM:** Do not store audit logs solely on the database server. Use tools like **Fluentd**, **Logstash**, or **CloudWatch Logs** to stream PostgreSQL logs to a centralized Security Information and Event Management (SIEM) system for real-time alerting and long-term retention.
- Implement Log Rotation:** To prevent the disk from filling up, configure strict log rotation policies using log_rotation_age and log_rotation_size.
- Audit Privileged Users:** Apply specific auditing rules to superuser accounts. Since these accounts can bypass many security checks, every action they take should be logged regardless of the global configuration.

References:

- [PostgreSQL Documentation: Error Reporting and Logging](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Enable logging and auditing C-MARIADB-CNT-05 Medium Not tested

- **State:** Recommended
- **Description:**

To enhance the security of your MariaDB installation, it's crucial to configure the MariaDB audit plugin to log critical activities. This will enable you to monitor authentication attempts, query executions, and privilege changes, helping you identify unauthorized or malicious access attempts. Follow these steps to implement this security countermeasure effectively:

1. Install the MariaDB Audit Plugin:

 - Start by logging into your MariaDB server as a user with sufficient privileges to modify plugins.
 - Execute the following command to install the audit plugin: `INSTALL PLUGIN server_audit SONAME 'server_audit.so';`
 - Confirm the installation by checking that the plugin is listed under installed plugins: `SHOW PLUGINS;`

2. Configure Audit Logging:

 - Edit the MariaDB server configuration file, typically located at `/etc/my.cnf` or `/etc/mysql/my.cnf`.
 - Add or update the following configuration entries under the `[mysqld]` section: `server_audit_events='CONNECT,QUERY,TABLE' server_audit_logging=ON`
 - `server_audit_file_path=/var/log/mariadb/server_audit.log` `server_audit_output_type=FILE`
 - This configuration will log connection attempts ('CONNECT'), queries executed ('QUERY'), and tables accessed ('TABLE') to the specified log file.

3. Secure the Log Files:

 - Set appropriate file permissions to ensure that only authorized users can access the log files: `chown mysql:mysql /var/log/mariadb/server_audit.log` `chmod 640 /var/log/mariadb/server_audit.log`

4. Enable Remote Logging (Optional):

 - If centralizing logs, consider log forwarding using a syslog server by setting: `server_audit_output_type=SYSLOG`

5. Regular Log Review and Monitoring:

 - Establish a routine to review the audit logs for any signs of suspicious activity, such as multiple failed login attempts or unauthorized query executions.
 - Employ automated tools to parse and analyze logs for significant security events, if possible.

6. Update and Maintain the Audit Configuration:

 - Regularly update the audit configurations based on emerging threats and requirements.
 - Keep the MariaDB server and audit plugin up-to-date to benefit from the latest security patches and features.
- **References**
 - [MariaDB Audit Plugin Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enable comprehensive auditing and logging of database activity C-POSTGRESQL-CNT-POSTGRES-10 Medium

Component: MariaDB

Rec1. Enable logging and auditing C-MARIADB-CNT-05 Medium

NIST 800-53: AU-7 Audit Record Reduction and Report Generation

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enable comprehensive auditing and logging of database activity C-POSTGRESQL-CNT-POSTGRES-10 Medium Not tested

- **State:** Recommended
- **Description:**

Configure PostgreSQL to capture a detailed audit trail of all database activities, including successful and failed connection attempts, schema changes (DDL), and sensitive data access (DML). While PostgreSQL provides standard error logging, **comprehensive auditing** involves enabling extended logging parameters or utilizing extensions like **pgaudit**. This control ensures that there is a non-repudiable record of "who did what and when," which is vital for incident response, forensic investigations, and meeting regulatory compliance requirements (such as PCI-DSS, SOC2, or HIPAA).

The objective of this control is to provide **Visibility and Accountability**. By capturing the specific queries executed and the users involved, security teams can detect unauthorized data exfiltration, identify accidental misconfigurations, and reconstruct events following a security breach.

Implementation guidance for developers and DevOps engineers:

 - **Install and Enable pgaudit:** For production environments, the standard `log_statement = 'all'` setting is often too noisy and lacks detail. Install the `pgaudit` extension to allow for granular auditing of specific classes of actions.
 - *Configuration:* Add `pgaudit` to `shared_preload_libraries` in `postgresql.conf` and restart the service.
 - **Configure Audit Classes:** Define what needs to be tracked. Use `pgaudit.log = 'write, ddl, role'` to capture data modifications, schema changes, and authorization changes (GRANT/REVOKE), while avoiding the performance overhead of logging every SELECT query.
 - **Log Connection Metadata:** Ensure basic connection details are captured by setting the following in `postgresql.conf`:
 - `log_connections = on`
 - `log_disconnections = on`
 - `log_hostname = on`
 - **Standardize Log Prefixes:** Use a consistent `log_line_prefix` to ensure logs contain the timestamp, user name, database name, and process ID (PID).
 - *Example:* `log_line_prefix = '%m [%p] %u@%d: '`
 - **Offload Logs to a Centralized SIEM:** Do not store audit logs solely on the database server. Use tools like **Fluentd**, **Logstash**, or **CloudWatch Logs** to stream PostgreSQL logs to a centralized Security Information and Event Management (SIEM) system for real-time alerting and long-term retention.
 - **Implement Log Rotation:** To prevent the disk from filling up, configure strict log rotation policies using `log_rotation_age` and `log_rotation_size`.
 - **Audit Privileged Users:** Apply specific auditing rules to superuser accounts. Since these accounts can bypass many security checks, every action they take should be logged regardless of the global configuration.
- **References:**
 - [PostgreSQL Documentation: Error Reporting and Logging](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Enable logging and auditing C-MARIADB-CNT-05 Medium Not tested

- **State:** Recommended
- **Description:**

To enhance the security of your MariaDB installation, it's crucial to configure the MariaDB audit plugin to log critical activities. This will enable you to monitor authentication attempts, query executions, and privilege changes, helping you identify unauthorized or malicious access attempts. Follow these steps to implement this security countermeasure effectively:

1. Install the MariaDB Audit Plugin:

 - Start by logging into your MariaDB server as a user with sufficient privileges to modify plugins.
 - Execute the following command to install the audit plugin: `INSTALL PLUGIN server_audit SONAME 'server_audit.so';`
 - Confirm the installation by checking that the plugin is listed under installed plugins: `SHOW PLUGINS;`

2. Configure Audit Logging:

 - Edit the MariaDB server configuration file, typically located at `/etc/my.cnf` or `/etc/mysql/my.cnf`.

- Add or update the following configuration entries under the `[mysqld]` section: `''' [mysqld] server_audit_events='CONNECT,QUERY,TABLE' server_audit_logging=ON server_audit_file_path=/var/log/mariadb/server_audit.log server_audit_output_type=FILE`

- This configuration will log connection attempts ('CONNECT'), queries executed ('QUERY'), and tables accessed ('TABLE') to the specified log file.

3. Secure the Log Files:

- Set appropriate file permissions to ensure that only authorized users can access the log files: `chown mysql:mysql /var/log/mariadb/server_audit.log chmod 640 /var/log/mariadb/server_audit.log`

4. Enable Remote Logging (Optional):

- If centralizing logs, consider log forwarding using a syslog server by setting: `server_audit_output_type=SYSLOG`

5. Regular Log Review and Monitoring:

- Establish a routine to review the audit logs for any signs of suspicious activity, such as multiple failed login attempts or unauthorized query executions.

- Employ automated tools to parse and analyze logs for significant security events, if possible.

6. Update and Maintain the Audit Configuration:

- Regularly update the audit configurations based on emerging threats and requirements.

- Keep the MariaDB server and audit plugin up-to-date to benefit from the latest security patches and features.

References

- [MariaDB Audit Plugin Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enable comprehensive auditing and logging of database activity C-POSTGRESQL-CNT-POSTGRES-10 Medium

Component: MariaDB

Rec1. Enable logging and auditing C-MARIADB-CNT-05 Medium

NIST 800-53: AU-11 Audit Record Retention

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enable comprehensive auditing and logging of database activity C-POSTGRESQL-CNT-POSTGRES-10 Medium Not tested

- State:** Recommended
- Description:**

Configure PostgreSQL to capture a detailed audit trail of all database activities, including successful and failed connection attempts, schema changes (DDL), and sensitive data access (DML). While PostgreSQL provides standard error logging, **comprehensive auditing** involves enabling extended logging parameters or utilizing extensions like **pgaudit**. This control ensures that there is a non-repudiable record of "who did what and when," which is vital for incident response, forensic investigations, and meeting regulatory compliance requirements (such as PCI-DSS, SOC2, or HIPAA).

The objective of this control is to provide **Visibility and Accountability**. By capturing the specific queries executed and the users involved, security teams can detect unauthorized data exfiltration, identify accidental misconfigurations, and reconstruct events following a security breach.

Implementation guidance for developers and DevOps engineers:

 - Install and Enable pgaudit:** For production environments, the standard `log_statement = 'all'` setting is often too noisy and lacks detail. Install the `pgaudit` extension to allow for granular auditing of specific classes of actions.
 - Configuration:* Add `pgaudit` to `shared_preload_libraries` in `postgresql.conf` and restart the service.
 - Configure Audit Classes:** Define what needs to be tracked. Use `pgaudit.log = 'write, ddl, role'` to capture data modifications, schema changes, and authorization changes (GRANT/REVOKE), while avoiding the performance overhead of logging every SELECT query.
 - Log Connection Metadata:** Ensure basic connection details are captured by setting the following in `postgresql.conf`:
 - `log_connections = on`
 - `log_disconnections = on`
 - `log_hostname = on`
 - Standardize Log Prefixes:** Use a consistent `log_line_prefix` to ensure logs contain the timestamp, user name, database name, and process ID (PID).
 - Example:* `log_line_prefix = '%m [%p] %u@%d: '`
 - Offload Logs to a Centralized SIEM:** Do not store audit logs solely on the database server. Use tools like **Fluentd**, **Logstash**, or **CloudWatch Logs** to stream PostgreSQL logs to a centralized Security Information and Event Management (SIEM) system for real-time alerting and long-term retention.
 - Implement Log Rotation:** To prevent the disk from filling up, configure strict log rotation policies using `log_rotation_age` and `log_rotation_size`.
 - Audit Privileged Users:** Apply specific auditing rules to superuser accounts. Since these accounts can bypass many security checks, every action they take should be logged regardless of the global configuration.

References:

- [PostgreSQL Documentation: Error Reporting and Logging](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Enable logging and auditing C-MARIADB-CNT-05 Medium Not tested

- State:** Recommended
 - Description:**

To enhance the security of your MariaDB installation, it's crucial to configure the MariaDB audit plugin to log critical activities. This will enable you to monitor authentication attempts, query executions, and privilege changes, helping you identify unauthorized or malicious access attempts. Follow these steps to implement this security countermeasure effectively:

1. Install the MariaDB Audit Plugin:

 - Start by logging into your MariaDB server as a user with sufficient privileges to modify plugins.
 - Execute the following command to install the audit plugin: `INSTALL PLUGIN server_audit SONAME 'server_audit.so';`
 - Confirm the installation by checking that the plugin is listed under installed plugins: `SHOW PLUGINS;`

2. Configure Audit Logging:

 - Edit the MariaDB server configuration file, typically located at `/etc/my.cnf` or `/etc/mysql/my.cnf`.
 - Add or update the following configuration entries under the `[mysqld]` section: `''' [mysqld] server_audit_events='CONNECT,QUERY,TABLE' server_audit_logging=ON server_audit_file_path=/var/log/mariadb/server_audit.log server_audit_output_type=FILE`
 - This configuration will log connection attempts ('CONNECT'), queries executed ('QUERY'), and tables accessed ('TABLE') to the specified log file.

3. Secure the Log Files:

 - Set appropriate file permissions to ensure that only authorized users can access the log files: `chown mysql:mysql /var/log/mariadb/server_audit.log chmod 640 /var/log/mariadb/server_audit.log`

4. Enable Remote Logging (Optional):

 - If centralizing logs, consider log forwarding using a syslog server by setting: `server_audit_output_type=SYSLOG`

5. Regular Log Review and Monitoring:

 - Establish a routine to review the audit logs for any signs of suspicious activity, such as multiple failed login attempts or unauthorized query executions.
 - Employ automated tools to parse and analyze logs for significant security events, if possible.

6. Update and Maintain the Audit Configuration:

 - Regularly update the audit configurations based on emerging threats and requirements.
 - Keep the MariaDB server and audit plugin up-to-date to benefit from the latest security patches and features.
- References**
- [MariaDB Audit Plugin Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enable comprehensive auditing and logging of database activity C-POSTGRESQL-CNT-POSTGRES-10 Medium

Component: MariaDB

Rec1. Enable logging and auditing C-MARIADB-CNT-05 Medium

NIST 800-53: AU-6 Audit Record Review, Analysis, and Reporting

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enable comprehensive auditing and logging of database activity C-POSTGRESQL-CNT-POSTGRES-10 Medium Not tested

• State: Recommended

• Description:

Configure PostgreSQL to capture a detailed audit trail of all database activities, including successful and failed connection attempts, schema changes (DDL), and sensitive data access (DML). While PostgreSQL provides standard error logging, **comprehensive auditing** involves enabling extended logging parameters or utilizing extensions like **pgaudit**. This control ensures that there is a non-repudiable record of "who did what and when," which is vital for incident response, forensic investigations, and meeting regulatory compliance requirements (such as PCI-DSS, SOC2, or HIPAA).

The objective of this control is to provide **Visibility and Accountability**. By capturing the specific queries executed and the users involved, security teams can detect unauthorized data exfiltration, identify accidental misconfigurations, and reconstruct events following a security breach.

Implementation guidance for developers and DevOps engineers:

- **Install and Enable pgaudit:** For production environments, the standard log_statement = 'all' setting is often too noisy and lacks detail. Install the pgaudit extension to allow for granular auditing of specific classes of actions.
 - *Configuration:* Add pgaudit to shared_preload_libraries in postgresql.conf and restart the service.
- **Configure Audit Classes:** Define what needs to be tracked. Use pgaudit.log = 'write, ddl, role' to capture data modifications, schema changes, and authorization changes (GRANT/REVOKE), while avoiding the performance overhead of logging every SELECT query.
- **Log Connection Metadata:** Ensure basic connection details are captured by setting the following in postgresql.conf:
 - log_connections = on
 - log_disconnections = on
 - log_hostname = on
- **Standardize Log Prefixes:** Use a consistent log_line_prefix to ensure logs contain the timestamp, user name, database name, and process ID (PID).
 - *Example:* log_line_prefix = '%m [%p] %u@%d: '
- **Offload Logs to a Centralized SIEM:** Do not store audit logs solely on the database server. Use tools like **Fluentd**, **Logstash**, or **CloudWatch Logs** to stream PostgreSQL logs to a centralized Security Information and Event Management (SIEM) system for real-time alerting and long-term retention.
- **Implement Log Rotation:** To prevent the disk from filling up, configure strict log rotation policies using log_rotation_age and log_rotation_size.
- **Audit Privileged Users:** Apply specific auditing rules to superuser accounts. Since these accounts can bypass many security checks, every action they take should be logged regardless of the global configuration.

References:

- [PostgreSQL Documentation: Error Reporting and Logging](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Enable logging and auditing C-MARIADB-CNT-05 Medium Not tested

• State: Recommended

• Description:

To enhance the security of your MariaDB installation, it's crucial to configure the MariaDB audit plugin to log critical activities. This will enable you to monitor authentication attempts, query executions, and privilege changes, helping you identify unauthorized or malicious access attempts. Follow these steps to implement this security countermeasure effectively:

1. **Install the MariaDB Audit Plugin:**
 - Start by logging into your MariaDB server as a user with sufficient privileges to modify plugins.
 - Execute the following command to install the audit plugin: `INSTALL PLUGIN server_audit SONAME 'server_audit.so';`
 - Confirm the installation by checking that the plugin is listed under installed plugins: `SHOW PLUGINS;`
2. **Configure Audit Logging:**
 - Edit the MariaDB server configuration file, typically located at '/etc/my.cnf' or '/etc/mysqld/my.cnf'.
 - Add or update the following configuration entries under the '[mysqld]' section: `[mysqld] server_audit_events='CONNECT,QUERY,TABLE' server_audit_logging=ON`
 - `server_audit_file_path=/var/log/mariadb/server_audit.log` `server_audit_output_type=FILE`
 - This configuration will log connection attempts ('CONNECT'), queries executed ('QUERY'), and tables accessed ('TABLE') to the specified log file.
3. **Secure the Log Files:**
 - Set appropriate file permissions to ensure that only authorized users can access the log files: `chown mysql:mysql /var/log/mariadb/server_audit.log` `chmod 640 /var/log/mariadb/server_audit.log`
4. **Enable Remote Logging (Optional):**
 - If centralizing logs, consider log forwarding using a syslog server by setting: `server_audit_output_type=SYSLOG`
5. **Regular Log Review and Monitoring:**
 - Establish a routine to review the audit logs for any signs of suspicious activity, such as multiple failed login attempts or unauthorized query executions.
 - Employ automated tools to parse and analyze logs for significant security events, if possible.
6. **Update and Maintain the Audit Configuration:**
 - Regularly update the audit configurations based on emerging threats and requirements.
 - Keep the MariaDB server and audit plugin up-to-date to benefit from the latest security patches and features.

References

- [MariaDB Audit Plugin Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enable comprehensive auditing and logging of database activity C-POSTGRESQL-CNT-POSTGRES-10 Medium

Component: MariaDB

Rec1. Enable logging and auditing C-MARIADB-CNT-05 Medium

NIST 800-53: AU-16 Cross-organizational Audit Logging

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enable comprehensive auditing and logging of database activity

C-POSTGRESQL-CNT-POSTGRES-10

Medium

Not tested

State: Recommended

Description:

Configure PostgreSQL to capture a detailed audit trail of all database activities, including successful and failed connection attempts, schema changes (DDL), and sensitive data access (DML). While PostgreSQL provides standard error logging, **comprehensive auditing** involves enabling extended logging parameters or utilizing extensions like **pgaudit**. This control ensures that there is a non-repudiable record of "who did what and when," which is vital for incident response, forensic investigations, and meeting regulatory compliance requirements (such as PCI-DSS, SOC2, or HIPAA).

The objective of this control is to provide **Visibility and Accountability**. By capturing the specific queries executed and the users involved, security teams can detect unauthorized data exfiltration, identify accidental misconfigurations, and reconstruct events following a security breach.

Implementation guidance for developers and DevOps engineers:

- Install and Enable pgaudit:** For production environments, the standard log_statement = 'all' setting is often too noisy and lacks detail. Install the pgaudit extension to allow for granular auditing of specific classes of actions.
 - Configuration:* Add pgaudit to shared_preload_libraries in postgresql.conf and restart the service.
- Configure Audit Classes:** Define what needs to be tracked. Use pgaudit.log = 'write, ddl, role' to capture data modifications, schema changes, and authorization changes (GRANT/REVOKE), while avoiding the performance overhead of logging every SELECT query.
- Log Connection Metadata:** Ensure basic connection details are captured by setting the following in postgresql.conf:
 - log_connections = on
 - log_disconnections = on
 - log_hostname = on
- Standardize Log Prefixes:** Use a consistent log_line_prefix to ensure logs contain the timestamp, user name, database name, and process ID (PID).
 - Example:* log_line_prefix = '%m [%p] %u@%d: '
- Offload Logs to a Centralized SIEM:** Do not store audit logs solely on the database server. Use tools like **Fluentd**, **Logstash**, or **CloudWatch Logs** to stream PostgreSQL logs to a centralized Security Information and Event Management (SIEM) system for real-time alerting and long-term retention.
- Implement Log Rotation:** To prevent the disk from filling up, configure strict log rotation policies using log_rotation_age and log_rotation_size.
- Audit Privileged Users:** Apply specific auditing rules to superuser accounts. Since these accounts can bypass many security checks, every action they take should be logged regardless of the global configuration.

References:

- PostgreSQL Documentation: Error Reporting and Logging

Component: MariaDB

1 Recommended countermeasures

Rec1. Enable logging and auditing

C-MARIADB-CNT-05

Medium

Not tested

State: Recommended

Description:

To enhance the security of your MariaDB installation, it's crucial to configure the MariaDB audit plugin to log critical activities. This will enable you to monitor authentication attempts, query executions, and privilege changes, helping you identify unauthorized or malicious access attempts. Follow these steps to implement this security countermeasure effectively:

1. Install the MariaDB Audit Plugin:

- Start by logging into your MariaDB server as a user with sufficient privileges to modify plugins.
- Execute the following command to install the audit plugin: `INSTALL PLUGIN server_audit SONAME 'server_audit.so';`
- Confirm the installation by checking that the plugin is listed under installed plugins: `SHOW PLUGINS;`

2. Configure Audit Logging:

- Edit the MariaDB server configuration file, typically located at `/etc/my.cnf` or `/etc/mysql/my.cnf`.
- Add or update the following configuration entries under the `[mysqld]` section: `server_audit_events='CONNECT,QUERY,TABLE' server_audit_logging=ON server_audit_file_path='/var/log/mariadb/server_audit.log' server_audit_output_type=FILE`
- This configuration will log connection attempts ('CONNECT'), queries executed ('QUERY'), and tables accessed ('TABLE') to the specified log file.

3. Secure the Log Files:

- Set appropriate file permissions to ensure that only authorized users can access the log files: `chown mysql:mysql /var/log/mariadb/server_audit.log` `chmod 640 /var/log/mariadb/server_audit.log`

4. Enable Remote Logging (Optional):

- If centralizing logs, consider log forwarding using a syslog server by setting: `server_audit_output_type=SYSLOG`

5. Regular Log Review and Monitoring:

- Establish a routine to review the audit logs for any signs of suspicious activity, such as multiple failed login attempts or unauthorized query executions.
- Employ automated tools to parse and analyze logs for significant security events, if possible.

6. Update and Maintain the Audit Configuration:

- Regularly update the audit configurations based on emerging threats and requirements.
- Keep the MariaDB server and audit plugin up-to-date to benefit from the latest security patches and features.

References

- MariaDB Audit Plugin Documentation

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enable comprehensive auditing and logging of database activity C-POSTGRESQL-CNT-POSTGRES-10 Medium

Component: MariaDB

Rec1. Enable logging and auditing C-MARIADB-CNT-05 Medium

NIST 800-53: AU-1 Policy and Procedures

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enable comprehensive auditing and logging of database activity

C-POSTGRESQL-CNT-POSTGRES-10

Medium

Not tested

State: Recommended

Description:

Configure PostgreSQL to capture a detailed audit trail of all database activities, including successful and failed connection attempts, schema changes (DDL), and sensitive data access (DML). While PostgreSQL provides standard error logging, **comprehensive auditing** involves enabling extended logging parameters or utilizing extensions like **pgaudit**. This control ensures that there is a non-repudiable record of "who did what and when," which is vital for incident response, forensic investigations, and meeting regulatory compliance requirements (such as PCI-DSS, SOC2, or HIPAA).

The objective of this control is to provide **Visibility and Accountability**. By capturing the specific queries executed and the users involved, security teams can detect unauthorized data exfiltration, identify accidental misconfigurations, and reconstruct events following a security breach.

Implementation guidance for developers and DevOps engineers:

- Install and Enable pgaudit:** For production environments, the standard log_statement = 'all' setting is often too noisy and lacks detail. Install the pgaudit extension to allow for granular auditing of specific classes of actions.
 - Configuration:* Add pgaudit to shared_preload_libraries in postgresql.conf and restart the service.
- Configure Audit Classes:** Define what needs to be tracked. Use pgaudit.log = 'write, ddl, role' to capture data modifications, schema changes, and authorization changes (GRANT/REVOKE), while avoiding the performance overhead of logging every SELECT query.
- Log Connection Metadata:** Ensure basic connection details are captured by setting the following in postgresql.conf:

- log_connections = on
 - log_disconnections = on
 - log_hostname = on
 - **Standardize Log Prefixes:** Use a consistent log_line_prefix to ensure logs contain the timestamp, user name, database name, and process ID (PID).
 - Example: log_line_prefix = '%m [%p] %u@%d: '
 - **Offload Logs to a Centralized SIEM:** Do not store audit logs solely on the database server. Use tools like **Fluentd**, **Logstash**, or **CloudWatch Logs** to stream PostgreSQL logs to a centralized Security Information and Event Management (SIEM) system for real-time alerting and long-term retention.
 - **Implement Log Rotation:** To prevent the disk from filling up, configure strict log rotation policies using log_rotation_age and log_rotation_size.
 - **Audit Privileged Users:** Apply specific auditing rules to superuser accounts. Since these accounts can bypass many security checks, every action they take should be logged regardless of the global configuration.
- References:
- [PostgreSQL Documentation: Error Reporting and Logging](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Enable logging and auditing C-MARIADB-CNT-05

Medium

Not tested

State: Recommended

Description:

To enhance the security of your MariaDB installation, it's crucial to configure the MariaDB audit plugin to log critical activities. This will enable you to monitor authentication attempts, query executions, and privilege changes, helping you identify unauthorized or malicious access attempts. Follow these steps to implement this security countermeasure effectively:

1. Install the MariaDB Audit Plugin:

- Start by logging into your MariaDB server as a user with sufficient privileges to modify plugins.
- Execute the following command to install the audit plugin: `INSTALL PLUGIN server_audit SONAME 'server_audit.so';`
- Confirm the installation by checking that the plugin is listed under installed plugins: `SHOW PLUGINS;`

2. Configure Audit Logging:

- Edit the MariaDB server configuration file, typically located at `/etc/my.cnf` or `/etc/mysql/my.cnf`.
- Add or update the following configuration entries under the `[mysqld]` section: `server_audit_events='CONNECT,QUERY,TABLE' server_audit_logging=ON server_audit_file_path='/var/log/mariadb/server_audit.log' server_audit_output_type=FILE`
- This configuration will log connection attempts ('CONNECT'), queries executed ('QUERY'), and tables accessed ('TABLE') to the specified log file.

3. Secure the Log Files:

- Set appropriate file permissions to ensure that only authorized users can access the log files: `chown mysql:mysql /var/log/mariadb/server_audit.log` `chmod 640 /var/log/mariadb/server_audit.log`

4. Enable Remote Logging (Optional):

- If centralizing logs, consider log forwarding using a syslog server by setting: `server_audit_output_type=SYSLOG`

5. Regular Log Review and Monitoring:

- Establish a routine to review the audit logs for any signs of suspicious activity, such as multiple failed login attempts or unauthorized query executions.
- Employ automated tools to parse and analyze logs for significant security events, if possible.

6. Update and Maintain the Audit Configuration:

- Regularly update the audit configurations based on emerging threats and requirements.
- Keep the MariaDB server and audit plugin up-to-date to benefit from the latest security patches and features.

References

- [MariaDB Audit Plugin Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enable comprehensive auditing and logging of database activity C-POSTGRESQL-CNT-POSTGRES-10

Medium

Component: MariaDB

Rec1. Enable logging and auditing C-MARIADB-CNT-05

Medium

NIST 800-53: AU-5 Response to Audit Logging Process Failures

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enable comprehensive auditing and logging of database activity C-POSTGRESQL-CNT-POSTGRES-10

Medium

Not tested

State: Recommended

Description:

Configure PostgreSQL to capture a detailed audit trail of all database activities, including successful and failed connection attempts, schema changes (DDL), and sensitive data access (DML). While PostgreSQL provides standard error logging, **comprehensive auditing** involves enabling extended logging parameters or utilizing extensions like **pgaudit**. This control ensures that there is a non-repudiable record of "who did what and when," which is vital for incident response, forensic investigations, and meeting regulatory compliance requirements (such as PCI-DSS, SOC2, or HIPAA).

The objective of this control is to provide **Visibility and Accountability**. By capturing the specific queries executed and the users involved, security teams can detect unauthorized data exfiltration, identify accidental misconfigurations, and reconstruct events following a security breach.

Implementation guidance for developers and DevOps engineers:

- **Install and Enable pgaudit:** For production environments, the standard log_statement = 'all' setting is often too noisy and lacks detail. Install the pgaudit extension to allow for granular auditing of specific classes of actions.
 - Configuration: Add pgaudit to shared_preload_libraries in postgresql.conf and restart the service.
- **Configure Audit Classes:** Define what needs to be tracked. Use pgaudit.log = 'write, ddl, role' to capture data modifications, schema changes, and authorization changes (GRANT/REVOKE), while avoiding the performance overhead of logging every SELECT query.
- **Log Connection Metadata:** Ensure basic connection details are captured by setting the following in postgresql.conf:
 - log_connections = on
 - log_disconnections = on
 - log_hostname = on
- **Standardize Log Prefixes:** Use a consistent log_line_prefix to ensure logs contain the timestamp, user name, database name, and process ID (PID).
 - Example: log_line_prefix = '%m [%p] %u@%d: '
- **Offload Logs to a Centralized SIEM:** Do not store audit logs solely on the database server. Use tools like **Fluentd**, **Logstash**, or **CloudWatch Logs** to stream PostgreSQL logs to a centralized Security Information and Event Management (SIEM) system for real-time alerting and long-term retention.
- **Implement Log Rotation:** To prevent the disk from filling up, configure strict log rotation policies using log_rotation_age and log_rotation_size.
- **Audit Privileged Users:** Apply specific auditing rules to superuser accounts. Since these accounts can bypass many security checks, every action they take should be logged regardless of the global configuration.

References

- [PostgreSQL Documentation: Error Reporting and Logging](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Enable logging and auditing C-MARIADB-CNT-05

Medium

Not tested

State: Recommended

- Description:

To enhance the security of your MariaDB installation, it's crucial to configure the MariaDB audit plugin to log critical activities. This will enable you to monitor authentication attempts, query executions, and privilege changes, helping you identify unauthorized or malicious access attempts. Follow these steps to implement this security countermeasure effectively:

1. Install the MariaDB Audit Plugin:

- Start by logging into your MariaDB server as a user with sufficient privileges to modify plugins.

- Execute the following command to install the audit plugin: `INSTALL PLUGIN server_audit SONAME 'server_audit.so';`

- Confirm the installation by checking that the plugin is listed under installed plugins: `SHOW PLUGINS;`

2. Configure Audit Logging:

- Edit the MariaDB server configuration file, typically located at `/etc/my.cnf` or `/etc/mysql/my.cnf`.

- Add or update the following configuration entries under the `[mysqld]` section: `server_audit_events='CONNECT,QUERY,TABLE'` `server_audit_logging=ON`

`server_audit_file_path=/var/log/mariadb/server_audit.log` `server_audit_output_type=FILE`

- This configuration will log connection attempts ('CONNECT'), queries executed ('QUERY'), and tables accessed ('TABLE') to the specified log file.

3. Secure the Log Files:

- Set appropriate file permissions to ensure that only authorized users can access the log files: `chown mysql:mysql /var/log/mariadb/server_audit.log` `chmod 640 /var/log/mariadb/server_audit.log`

4. Enable Remote Logging (Optional):

- If centralizing logs, consider log forwarding using a syslog server by setting: `server_audit_output_type=SYSLOG`

5. Regular Log Review and Monitoring:

- Establish a routine to review the audit logs for any signs of suspicious activity, such as multiple failed login attempts or unauthorized query executions.

- Employ automated tools to parse and analyze logs for significant security events, if possible.

6. Update and Maintain the Audit Configuration:

- Regularly update the audit configurations based on emerging threats and requirements.

- Keep the MariaDB server and audit plugin up-to-date to benefit from the latest security patches and features.

References

MariaDB Audit Plugin Documentation

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enable comprehensive auditing and logging of database activity C-POSTGRESQL-CNT-POSTGRES-10

Medium

Component: MariaDB

Rec1. Enable logging and auditing C-MARIADB-CNT-05

Medium

NIST 800-53: SI-11 Error Handling

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enable comprehensive auditing and logging of database activity C-POSTGRESQL-CNT-POSTGRES-10

Medium

Not tested

State: Recommended

Description:

Configure PostgreSQL to capture a detailed audit trail of all database activities, including successful and failed connection attempts, schema changes (DDL), and sensitive data access (DML). While PostgreSQL provides standard error logging, **comprehensive auditing** involves enabling extended logging parameters or utilizing extensions like **pgaudit**. This control ensures that there is a non-repudiable record of "who did what and when," which is vital for incident response, forensic investigations, and meeting regulatory compliance requirements (such as PCI-DSS, SOC2, or HIPAA). The objective of this control is to provide **Visibility and Accountability**. By capturing the specific queries executed and the users involved, security teams can detect unauthorized data exfiltration, identify accidental misconfigurations, and reconstruct events following a security breach.

Implementation guidance for developers and DevOps engineers:

Install and Enable pgaudit:

For production environments, the standard log_statement = 'all' setting is often too noisy and lacks detail. Install the pgaudit extension to allow for granular auditing of specific classes of actions.

Configuration: Add pgaudit to shared_preload_libraries in postgresql.conf and restart the service.

Configure Audit Classes:

Define what needs to be tracked. Use pgaudit.log = 'write, ddl, role' to capture data modifications, schema changes, and authorization changes (GRANT/REVOKE), while avoiding the performance overhead of logging every SELECT query.

Log Connection Metadata:

Ensure basic connection details are captured by setting the following in postgresql.conf:

log_connections = on

log_disconnections = on

log_hostname = on

Standardize Log Prefixes:

Use a consistent log_line_prefix to ensure logs contain the timestamp, user name, database name, and process ID (PID).

Example: log_line_prefix = '%m [%p] %u@%d: '

Offload Logs to a Centralized SIEM:

Do not store audit logs solely on the database server. Use tools like **Fluentd**, **Logstash**, or **CloudWatch Logs** to stream PostgreSQL logs to a centralized Security Information and Event Management (SIEM) system for real-time alerting and long-term retention.

Implement Log Rotation:

To prevent the disk from filling up, configure strict log rotation policies using log_rotation_age and log_rotation_size.

Audit Privileged Users:

Apply specific auditing rules to superuser accounts. Since these accounts can bypass many security checks, every action they take should be logged regardless of the global configuration.

References:

PostgreSQL Documentation: Error Reporting and Logging

Component: MariaDB

1 Recommended countermeasures

Rec1. Enable logging and auditing C-MARIADB-CNT-05

Medium

Not tested

State: Recommended

Description:

To enhance the security of your MariaDB installation, it's crucial to configure the MariaDB audit plugin to log critical activities. This will enable you to monitor authentication attempts, query executions, and privilege changes, helping you identify unauthorized or malicious access attempts. Follow these steps to implement this security countermeasure effectively:

1. Install the MariaDB Audit Plugin:

- Start by logging into your MariaDB server as a user with sufficient privileges to modify plugins.

- Execute the following command to install the audit plugin: `INSTALL PLUGIN server_audit SONAME 'server_audit.so';`

- Confirm the installation by checking that the plugin is listed under installed plugins: `SHOW PLUGINS;`

2. Configure Audit Logging:

- Edit the MariaDB server configuration file, typically located at `/etc/my.cnf` or `/etc/mysql/my.cnf`.

- Add or update the following configuration entries under the `[mysqld]` section: `server_audit_events='CONNECT,QUERY,TABLE'` `server_audit_logging=ON`

`server_audit_file_path=/var/log/mariadb/server_audit.log` `server_audit_output_type=FILE`

- This configuration will log connection attempts ('CONNECT'), queries executed ('QUERY'), and tables accessed ('TABLE') to the specified log file.

3. Secure the Log Files:

- Set appropriate file permissions to ensure that only authorized users can access the log files: `chown mysql:mysql /var/log/mariadb/server_audit.log` `chmod 640 /var/log/mariadb/server_audit.log`

4. Enable Remote Logging (Optional):

- If centralizing logs, consider log forwarding using a syslog server by setting: `server_audit_output_type=SYSLOG`

5. Regular Log Review and Monitoring:

- Establish a routine to review the audit logs for any signs of suspicious activity, such as multiple failed login attempts or unauthorized query executions.

- Employ automated tools to parse and analyze logs for significant security events, if possible.

6. Update and Maintain the Audit Configuration:

- Regularly update the audit configurations based on emerging threats and requirements.
- Keep the MariaDB server and audit plugin up-to-date to benefit from the latest security patches and features.

References

- [MariaDB Audit Plugin Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enable comprehensive auditing and logging of database activity C-POSTGRESQL-CNT-POSTGRES-10 Medium

Component: MariaDB

Rec1. Enable logging and auditing C-MARIADB-CNT-05 Medium

NIST 800-53: CP-11 Alternate Communications Protocols

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08 Very high Not tested

- State: Recommended
- Description:
Implement and regularly update secure backup procedures with encryption and access controls for PostgreSQL to protect backup data against unauthorized access and tampering. This control ensures that all backup files are encrypted using strong cryptographic standards (e.g., AES-256) and that access to these backups is restricted to authorized personnel only. Developers and DevOps engineers should integrate secure backup mechanisms into their PostgreSQL deployment process using automated backup tools and centralized management systems to enforce encryption and access policies consistently.
Implementation Steps:
Encrypt Backup Files:
Configure backup tools to encrypt PostgreSQL backup files using robust encryption algorithms before storage or transmission.
Enforce Access Controls:
Use OS-level permissions, cloud storage access policies, or centralized management tools to restrict backup file access to authorized users only.
Automate Backup Processes:
Integrate automated backup solutions that enforce encryption and access controls, ensuring consistent and secure backup routines across all PostgreSQL instances.
Monitor and Audit Backup Procedures:
Regularly review backup logs and conduct audits to verify that encryption and access controls are maintained and that backup data remains secure.
References:
 - [PostgreSQL Backup and Restore Documentation](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04 High Not tested

- State: Recommended
- Description:
To improve the security of MariaDB backups, follow these steps to ensure that backups are encrypted, access is controlled, and a secure backup retention policy is implemented. Additionally, regularly test the integrity of these backups to ensure they can be restored when necessary.
 - Encrypt Database Backups:**
 - Use the built-in mariabackup tool for creating backups. During the backup process, employ encryption to protect backup data.
 - Select a robust encryption algorithm, such as AES-256, to ensure the data within backups is secured against unauthorized access.
 - Ensure the backup scripts or commands implement encryption flags, e.g., by using the --encrypt option in command executions.
 - Store Backups in Access-Controlled Locations:**
 - Designate specific, secure locations for storing backups. These locations should have stringent access controls, such as being in a separate network segment or specific access-controlled directory.
 - Ensure that only authorized personnel have access to these storage locations. Implement role-based access control (RBAC) to restrict access.
 - Maintain a log of access attempts and successful access to backup locations, allowing for auditing and alerting on unauthorized access attempts.
 - Implement Secure Backup Retention Policy:**
 - Define a retention policy that clearly specifies the duration for which backups should be retained. This policy should comply with regulatory and organizational requirements.
 - Use automation tools to manage backup rotations, ensuring old backups are securely deleted after their retention period expires.
 - Regularly review and update the retention policy to align with evolving data retention needs or regulatory changes.
 - Regularly Test Backup Integrity:**
 - Schedule periodic tests to restore data from backups to verify the data integrity and process efficacy.
 - Establish a testing protocol that includes both manual and automated testing to ensure that backups are not corrupted and can be reliably restored.
 - Document results of integrity tests and any issues encountered to continually improve your backup strategy.
References
 - [MariaDB Backup and Restore Overview](#)
 - [MariaDB Backup Tool Overview](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08 Very high

Component: MariaDB

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04 High

NIST 800-53: CP-7 Alternate Processing Site

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08

Very high

Not tested

State: Recommended

Description:

Implement and regularly update secure backup procedures with encryption and access controls for PostgreSQL to protect backup data against unauthorized access and tampering. This control ensures that all backup files are encrypted using strong cryptographic standards (e.g., AES-256) and that access to these backups is restricted to authorized personnel only. Developers and DevOps engineers should integrate secure backup mechanisms into their PostgreSQL deployment process using automated backup tools and centralized management systems to enforce encryption and access policies consistently.

Implementation Steps:

Encrypt Backup Files:
Configure backup tools to encrypt PostgreSQL backup files using robust encryption algorithms before storage or transmission.

Enforce Access Controls:
Use OS-level permissions, cloud storage access policies, or centralized management tools to restrict backup file access to authorized users only.

Automate Backup Processes:
Integrate automated backup solutions that enforce encryption and access controls, ensuring consistent and secure backup routines across all PostgreSQL instances.

Monitor and Audit Backup Procedures:
Regularly review backup logs and conduct audits to verify that encryption and access controls are maintained and that backup data remains secure.

References:

- PostgreSQL Backup and Restore Documentation

Component: MariaDB

1 Recommended countermeasures

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04

High

Not tested

State: Recommended

Description:

To improve the security of MariaDB backups, follow these steps to ensure that backups are encrypted, access is controlled, and a secure backup retention policy is implemented. Additionally, regularly test the integrity of these backups to ensure they can be restored when necessary.

- Encrypt Database Backups:**
 - Use the built-in mariabackup tool for creating backups. During the backup process, employ encryption to protect backup data.
 - Select a robust encryption algorithm, such as AES-256, to ensure the data within backups is secured against unauthorized access.
 - Ensure the backup scripts or commands implement encryption flags, e.g., by using the --encrypt option in command executions.
- Store Backups in Access-Controlled Locations:**
 - Designate specific, secure locations for storing backups. These locations should have stringent access controls, such as being in a separate network segment or specific access-controlled directory.
 - Ensure that only authorized personnel have access to these storage locations. Implement role-based access control (RBAC) to restrict access.
 - Maintain a log of access attempts and successful access to backup locations, allowing for auditing and alerting on unauthorized access attempts.
- Implement Secure Backup Retention Policy:**
 - Define a retention policy that clearly specifies the duration for which backups should be retained. This policy should comply with regulatory and organizational requirements.
 - Use automation tools to manage backup rotations, ensuring old backups are securely deleted after their retention period expires.
 - Regularly review and update the retention policy to align with evolving data retention needs or regulatory changes.
- Regularly Test Backup Integrity:**
 - Schedule periodic tests to restore data from backups to verify the data integrity and process efficacy.
 - Establish a testing protocol that includes both manual and automated testing to ensure that backups are not corrupted and can be reliably restored.
 - Document results of integrity tests and any issues encountered to continually improve your backup strategy.

References

- MariaDB Backup and Restore Overview
- MariaDB Backup Tool Overview

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08

Very high

Component: MariaDB

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04

High

NIST 800-53: CP-6 Alternate Storage Site

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08

Very high

Not tested

State: Recommended

Description:

Implement and regularly update secure backup procedures with encryption and access controls for PostgreSQL to protect backup data against unauthorized access and tampering. This control ensures that all backup files are encrypted using strong cryptographic standards (e.g., AES-256) and that access to these backups is restricted to authorized personnel only. Developers and DevOps engineers should integrate secure backup mechanisms into their PostgreSQL deployment process using automated backup tools and centralized management systems to enforce encryption and access policies consistently.

Implementation Steps:

Encrypt Backup Files:
Configure backup tools to encrypt PostgreSQL backup files using robust encryption algorithms before storage or transmission.

Enforce Access Controls:
Use OS-level permissions, cloud storage access policies, or centralized management tools to restrict backup file access to authorized users only.

Automate Backup Processes:
Integrate automated backup solutions that enforce encryption and access controls, ensuring consistent and secure backup routines across all PostgreSQL instances.

Monitor and Audit Backup Procedures:
Regularly review backup logs and conduct audits to verify that encryption and access controls are maintained and that backup data remains secure.

References:

- PostgreSQL Backup and Restore Documentation

Component: MariaDB

1 Recommended countermeasures

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04 High Not tested

- State: Recommended
 - Description:

To improve the security of MariaDB backups, follow these steps to ensure that backups are encrypted, access is controlled, and a secure backup retention policy is implemented. Additionally, regularly test the integrity of these backups to ensure they can be restored when necessary.

 - Encrypt Database Backups:**
 - Use the built-in mariabackup tool for creating backups. During the backup process, employ encryption to protect backup data.
 - Select a robust encryption algorithm, such as AES-256, to ensure the data within backups is secured against unauthorized access.
 - Ensure the backup scripts or commands implement encryption flags, e.g., by using the --encrypt option in command executions.
 - Store Backups in Access-Controlled Locations:**
 - Designate specific, secure locations for storing backups. These locations should have stringent access controls, such as being in a separate network segment or specific access-controlled directory.
 - Ensure that only authorized personnel have access to these storage locations. Implement role-based access control (RBAC) to restrict access.
 - Maintain a log of access attempts and successful access to backup locations, allowing for auditing and alerting on unauthorized access attempts.
 - Implement Secure Backup Retention Policy:**
 - Define a retention policy that clearly specifies the duration for which backups should be retained. This policy should comply with regulatory and organizational requirements.
 - Use automation tools to manage backup rotations, ensuring old backups are securely deleted after their retention period expires.
 - Regularly review and update the retention policy to align with evolving data retention needs or regulatory changes.
 - Regularly Test Backup Integrity:**
 - Schedule periodic tests to restore data from backups to verify the data integrity and process efficacy.
 - Establish a testing protocol that includes both manual and automated testing to ensure that backups are not corrupted and can be reliably restored.
 - Document results of integrity tests and any issues encountered to continually improve your backup strategy.
- References
- [MariaDB Backup and Restore Overview](#)
 - [MariaDB Backup Tool Overview](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08 Very high

Component: MariaDB

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04 High

NIST 800-53: CP-13 Alternative Security Mechanisms

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

- Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08 Very high Not tested
- State: Recommended
 - Description:

Implement and regularly update secure backup procedures with encryption and access controls for PostgreSQL to protect backup data against unauthorized access and tampering. This control ensures that all backup files are encrypted using strong cryptographic standards (e.g., AES-256) and that access to these backups is restricted to authorized personnel only. Developers and DevOps engineers should integrate secure backup mechanisms into their PostgreSQL deployment process using automated backup tools and centralized management systems to enforce encryption and access policies consistently.

Implementation Steps:

Encrypt Backup Files:
Configure backup tools to encrypt PostgreSQL backup files using robust encryption algorithms before storage or transmission.

Enforce Access Controls:
Use OS-level permissions, cloud storage access policies, or centralized management tools to restrict backup file access to authorized users only.

Automate Backup Processes:
Integrate automated backup solutions that enforce encryption and access controls, ensuring consistent and secure backup routines across all PostgreSQL instances.

Monitor and Audit Backup Procedures:
Regularly review backup logs and conduct audits to verify that encryption and access controls are maintained and that backup data remains secure.
- References:
- [PostgreSQL Backup and Restore Documentation](#)

Component: MariaDB

1 Recommended countermeasures

- Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04 High Not tested
- State: Recommended
 - Description:

To improve the security of MariaDB backups, follow these steps to ensure that backups are encrypted, access is controlled, and a secure backup retention policy is implemented. Additionally, regularly test the integrity of these backups to ensure they can be restored when necessary.

 - Encrypt Database Backups:**
 - Use the built-in mariabackup tool for creating backups. During the backup process, employ encryption to protect backup data.
 - Select a robust encryption algorithm, such as AES-256, to ensure the data within backups is secured against unauthorized access.
 - Ensure the backup scripts or commands implement encryption flags, e.g., by using the --encrypt option in command executions.
 - Store Backups in Access-Controlled Locations:**
 - Designate specific, secure locations for storing backups. These locations should have stringent access controls, such as being in a separate network segment or specific access-controlled directory.
 - Ensure that only authorized personnel have access to these storage locations. Implement role-based access control (RBAC) to restrict access.
 - Maintain a log of access attempts and successful access to backup locations, allowing for auditing and alerting on unauthorized access attempts.
 - Implement Secure Backup Retention Policy:**
 - Define a retention policy that clearly specifies the duration for which backups should be retained. This policy should comply with regulatory and organizational requirements.
 - Use automation tools to manage backup rotations, ensuring old backups are securely deleted after their retention period expires.
 - Regularly review and update the retention policy to align with evolving data retention needs or regulatory changes.
 - Regularly Test Backup Integrity:**
 - Schedule periodic tests to restore data from backups to verify the data integrity and process efficacy.
 - Establish a testing protocol that includes both manual and automated testing to ensure that backups are not corrupted and can be reliably restored.
 - Document results of integrity tests and any issues encountered to continually improve your backup strategy.
- References
- [MariaDB Backup and Restore Overview](#)
 - [MariaDB Backup Tool Overview](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08 Very high

Component: MariaDB

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04 High

NIST 800-53: CP-2 Contingency Plan

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08Very highNot tested

State: Recommended

Description:

Implement and regularly update secure backup procedures with encryption and access controls for PostgreSQL to protect backup data against unauthorized access and tampering. This control ensures that all backup files are encrypted using strong cryptographic standards (e.g., AES-256) and that access to these backups is restricted to authorized personnel only. Developers and DevOps engineers should integrate secure backup mechanisms into their PostgreSQL deployment process using automated backup tools and centralized management systems to enforce encryption and access policies consistently.

Implementation Steps:

Encrypt Backup Files:

Configure backup tools to encrypt PostgreSQL backup files using robust encryption algorithms before storage or transmission.

Enforce Access Controls:

Use OS-level permissions, cloud storage access policies, or centralized management tools to restrict backup file access to authorized users only.

Automate Backup Processes:

Integrate automated backup solutions that enforce encryption and access controls, ensuring consistent and secure backup routines across all PostgreSQL instances.

Monitor and Audit Backup Procedures:

Regularly review backup logs and conduct audits to verify that encryption and access controls are maintained and that backup data remains secure.

References:

PostgreSQL Backup and Restore Documentation

Component: MariaDB

1 Recommended countermeasures

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04HighNot tested

State: Recommended

Description:

To improve the security of MariaDB backups, follow these steps to ensure that backups are encrypted, access is controlled, and a secure backup retention policy is implemented. Additionally, regularly test the integrity of these backups to ensure they can be restored when necessary.

1. Encrypt Database Backups:

Use the built-in mariabackup tool for creating backups. During the backup process, employ encryption to protect backup data.

Select a robust encryption algorithm, such as AES-256, to ensure the data within backups is secured against unauthorized access.

Ensure the backup scripts or commands implement encryption flags, e.g., by using the --encrypt option in command executions.

2. Store Backups in Access-Controlled Locations:

Designate specific, secure locations for storing backups. These locations should have stringent access controls, such as being in a separate network segment or specific access-controlled directory.

Ensure that only authorized personnel have access to these storage locations. Implement role-based access control (RBAC) to restrict access.

Maintain a log of access attempts and successful access to backup locations, allowing for auditing and alerting on unauthorized access attempts.

3. Implement Secure Backup Retention Policy:

Define a retention policy that clearly specifies the duration for which backups should be retained. This policy should comply with regulatory and organizational requirements.

Use automation tools to manage backup rotations, ensuring old backups are securely deleted after their retention period expires.

Regularly review and update the retention policy to align with evolving data retention needs or regulatory changes.

4. Regularly Test Backup Integrity:

Schedule periodic tests to restore data from backups to verify the data integrity and process efficacy.

Establish a testing protocol that includes both manual and automated testing to ensure that backups are not corrupted and can be reliably restored.

Document results of integrity tests and any issues encountered to continually improve your backup strategy.

References

MariaDB Backup and Restore Overview

MariaDB Backup Tool Overview

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08 Very high

Component: MariaDB

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04 High

NIST 800-53: CP-4 Contingency Plan Testing

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08Very highNot tested

Report: SpaceHub

Compliance report - 2026-03-25T16:40:26.994993079Z

168 of 182

- **State:** 🟢 Recommended
- **Description:**
Implement and regularly update secure backup procedures with encryption and access controls for PostgreSQL to protect backup data against unauthorized access and tampering. This control ensures that all backup files are encrypted using strong cryptographic standards (e.g., AES-256) and that access to these backups is restricted to authorized personnel only. Developers and DevOps engineers should integrate secure backup mechanisms into their PostgreSQL deployment process using automated backup tools and centralized management systems to enforce encryption and access policies consistently.
Implementation Steps:
Encrypt Backup Files:
Configure backup tools to encrypt PostgreSQL backup files using robust encryption algorithms before storage or transmission.
Enforce Access Controls:
Use OS-level permissions, cloud storage access policies, or centralized management tools to restrict backup file access to authorized users only.
Automate Backup Processes:
Integrate automated backup solutions that enforce encryption and access controls, ensuring consistent and secure backup routines across all PostgreSQL instances.
Monitor and Audit Backup Procedures:
Regularly review backup logs and conduct audits to verify that encryption and access controls are maintained and that backup data remains secure.
References:
 - [PostgreSQL Backup and Restore Documentation](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04 🔴 High ⬜ Not tested

- **State:** 🟢 Recommended
- **Description:**
To improve the security of MariaDB backups, follow these steps to ensure that backups are encrypted, access is controlled, and a secure backup retention policy is implemented. Additionally, regularly test the integrity of these backups to ensure they can be restored when necessary.
 - 1. Encrypt Database Backups:**
 - Use the built-in mariabackup tool for creating backups. During the backup process, employ encryption to protect backup data.
 - Select a robust encryption algorithm, such as AES-256, to ensure the data within backups is secured against unauthorized access.
 - Ensure the backup scripts or commands implement encryption flags, e.g., by using the --encrypt option in command executions.
 - 2. Store Backups in Access-Controlled Locations:**
 - Designate specific, secure locations for storing backups. These locations should have stringent access controls, such as being in a separate network segment or specific access-controlled directory.
 - Ensure that only authorized personnel have access to these storage locations. Implement role-based access control (RBAC) to restrict access.
 - Maintain a log of access attempts and successful access to backup locations, allowing for auditing and alerting on unauthorized access attempts.
 - 3. Implement Secure Backup Retention Policy:**
 - Define a retention policy that clearly specifies the duration for which backups should be retained. This policy should comply with regulatory and organizational requirements.
 - Use automation tools to manage backup rotations, ensuring old backups are securely deleted after their retention period expires.
 - Regularly review and update the retention policy to align with evolving data retention needs or regulatory changes.
 - 4. Regularly Test Backup Integrity:**
 - Schedule periodic tests to restore data from backups to verify the data integrity and process efficacy.
 - Establish a testing protocol that includes both manual and automated testing to ensure that backups are not corrupted and can be reliably restored.
 - Document results of integrity tests and any issues encountered to continually improve your backup strategy.
References
 - [MariaDB Backup and Restore Overview](#)
 - [MariaDB Backup Tool Overview](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08 🟢 Very high

Component: MariaDB

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04 🔴 High

NIST 800-53: CP-3 Contingency Training

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

- Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08 🟢 Very high ⬜ Not tested
- **State:** 🟢 Recommended
 - **Description:**
Implement and regularly update secure backup procedures with encryption and access controls for PostgreSQL to protect backup data against unauthorized access and tampering. This control ensures that all backup files are encrypted using strong cryptographic standards (e.g., AES-256) and that access to these backups is restricted to authorized personnel only. Developers and DevOps engineers should integrate secure backup mechanisms into their PostgreSQL deployment process using automated backup tools and centralized management systems to enforce encryption and access policies consistently.
Implementation Steps:
Encrypt Backup Files:
Configure backup tools to encrypt PostgreSQL backup files using robust encryption algorithms before storage or transmission.
Enforce Access Controls:
Use OS-level permissions, cloud storage access policies, or centralized management tools to restrict backup file access to authorized users only.
Automate Backup Processes:
Integrate automated backup solutions that enforce encryption and access controls, ensuring consistent and secure backup routines across all PostgreSQL instances.
Monitor and Audit Backup Procedures:
Regularly review backup logs and conduct audits to verify that encryption and access controls are maintained and that backup data remains secure.
References:
 - [PostgreSQL Backup and Restore Documentation](#)

Component: MariaDB

1 Recommended countermeasures

- Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04 🔴 High ⬜ Not tested
- **State:** 🟢 Recommended
 - **Description:**
To improve the security of MariaDB backups, follow these steps to ensure that backups are encrypted, access is controlled, and a secure backup retention policy is implemented. Additionally, regularly test the integrity of these backups to ensure they can be restored when necessary.

1. **Encrypt Database Backups:**
 - Use the built-in mariabackup tool for creating backups. During the backup process, employ encryption to protect backup data.
 - Select a robust encryption algorithm, such as AES-256, to ensure the data within backups is secured against unauthorized access.
 - Ensure the backup scripts or commands implement encryption flags, e.g., by using the --encrypt option in command executions.
2. **Store Backups in Access-Controlled Locations:**
 - Designate specific, secure locations for storing backups. These locations should have stringent access controls, such as being in a separate network segment or specific access-controlled directory.
 - Ensure that only authorized personnel have access to these storage locations. Implement role-based access control (RBAC) to restrict access.
 - Maintain a log of access attempts and successful access to backup locations, allowing for auditing and alerting on unauthorized access attempts.
3. **Implement Secure Backup Retention Policy:**
 - Define a retention policy that clearly specifies the duration for which backups should be retained. This policy should comply with regulatory and organizational requirements.
 - Use automation tools to manage backup rotations, ensuring old backups are securely deleted after their retention period expires.
 - Regularly review and update the retention policy to align with evolving data retention needs or regulatory changes.
4. **Regularly Test Backup Integrity:**
 - Schedule periodic tests to restore data from backups to verify the data integrity and process efficacy.
 - Establish a testing protocol that includes both manual and automated testing to ensure that backups are not corrupted and can be reliably restored.
 - Document results of integrity tests and any issues encountered to continually improve your backup strategy.

- References
- MariaDB Backup and Restore Overview
 - MariaDB Backup Tool Overview

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08 Very high

Component: MariaDB

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04 High

NIST 800-53: CP-1 Policy and Procedures

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08 Very high Not tested

State: Recommended

Description:

Implement and regularly update secure backup procedures with encryption and access controls for PostgreSQL to protect backup data against unauthorized access and tampering. This control ensures that all backup files are encrypted using strong cryptographic standards (e.g., AES-256) and that access to these backups is restricted to authorized personnel only. Developers and DevOps engineers should integrate secure backup mechanisms into their PostgreSQL deployment process using automated backup tools and centralized management systems to enforce encryption and access policies consistently.

Implementation Steps:

Encrypt Backup Files:
Configure backup tools to encrypt PostgreSQL backup files using robust encryption algorithms before storage or transmission.

Enforce Access Controls:
Use OS-level permissions, cloud storage access policies, or centralized management tools to restrict backup file access to authorized users only.

Automate Backup Processes:
Integrate automated backup solutions that enforce encryption and access controls, ensuring consistent and secure backup routines across all PostgreSQL instances.

Monitor and Audit Backup Procedures:
Regularly review backup logs and conduct audits to verify that encryption and access controls are maintained and that backup data remains secure.

References:

- PostgreSQL Backup and Restore Documentation

Component: MariaDB

1 Recommended countermeasures

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04 High Not tested

State: Recommended

Description:

To improve the security of MariaDB backups, follow these steps to ensure that backups are encrypted, access is controlled, and a secure backup retention policy is implemented. Additionally, regularly test the integrity of these backups to ensure they can be restored when necessary.

1. **Encrypt Database Backups:**

- Use the built-in mariabackup tool for creating backups. During the backup process, employ encryption to protect backup data.
- Select a robust encryption algorithm, such as AES-256, to ensure the data within backups is secured against unauthorized access.
- Ensure the backup scripts or commands implement encryption flags, e.g., by using the --encrypt option in command executions.

2. **Store Backups in Access-Controlled Locations:**

- Designate specific, secure locations for storing backups. These locations should have stringent access controls, such as being in a separate network segment or specific access-controlled directory.
- Ensure that only authorized personnel have access to these storage locations. Implement role-based access control (RBAC) to restrict access.
- Maintain a log of access attempts and successful access to backup locations, allowing for auditing and alerting on unauthorized access attempts.

3. **Implement Secure Backup Retention Policy:**

- Define a retention policy that clearly specifies the duration for which backups should be retained. This policy should comply with regulatory and organizational requirements.
- Use automation tools to manage backup rotations, ensuring old backups are securely deleted after their retention period expires.
- Regularly review and update the retention policy to align with evolving data retention needs or regulatory changes.

4. **Regularly Test Backup Integrity:**

- Schedule periodic tests to restore data from backups to verify the data integrity and process efficacy.
- Establish a testing protocol that includes both manual and automated testing to ensure that backups are not corrupted and can be reliably restored.
- Document results of integrity tests and any issues encountered to continually improve your backup strategy.

References

- MariaDB Backup and Restore Overview
- MariaDB Backup Tool Overview

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08 Very high

Component: MariaDB

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04 High

NIST 800-53: CP-12 Safe Mode

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08

Very high

☐ Not tested

State: Recommended

Description:
Implement and regularly update secure backup procedures with encryption and access controls for PostgreSQL to protect backup data against unauthorized access and tampering. This control ensures that all backup files are encrypted using strong cryptographic standards (e.g., AES-256) and that access to these backups is restricted to authorized personnel only. Developers and DevOps engineers should integrate secure backup mechanisms into their PostgreSQL deployment process using automated backup tools and centralized management systems to enforce encryption and access policies consistently.
Implementation Steps:
Encrypt Backup Files:
Configure backup tools to encrypt PostgreSQL backup files using robust encryption algorithms before storage or transmission.
Enforce Access Controls:
Use OS-level permissions, cloud storage access policies, or centralized management tools to restrict backup file access to authorized users only.
Automate Backup Processes:
Integrate automated backup solutions that enforce encryption and access controls, ensuring consistent and secure backup routines across all PostgreSQL instances.
Monitor and Audit Backup Procedures:
Regularly review backup logs and conduct audits to verify that encryption and access controls are maintained and that backup data remains secure.
References:

- PostgreSQL Backup and Restore Documentation

Component: MariaDB

1 Recommended countermeasures

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04

High

☐ Not tested

State: Recommended

Description:
To improve the security of MariaDB backups, follow these steps to ensure that backups are encrypted, access is controlled, and a secure backup retention policy is implemented. Additionally, regularly test the integrity of these backups to ensure they can be restored when necessary.

- Encrypt Database Backups:**
 - Use the built-in mariabackup tool for creating backups. During the backup process, employ encryption to protect backup data.
 - Select a robust encryption algorithm, such as AES-256, to ensure the data within backups is secured against unauthorized access.
 - Ensure the backup scripts or commands implement encryption flags, e.g., by using the --encrypt option in command executions.
- Store Backups in Access-Controlled Locations:**
 - Designate specific, secure locations for storing backups. These locations should have stringent access controls, such as being in a separate network segment or specific access-controlled directory.
 - Ensure that only authorized personnel have access to these storage locations. Implement role-based access control (RBAC) to restrict access.
 - Maintain a log of access attempts and successful access to backup locations, allowing for auditing and alerting on unauthorized access attempts.
- Implement Secure Backup Retention Policy:**
 - Define a retention policy that clearly specifies the duration for which backups should be retained. This policy should comply with regulatory and organizational requirements.
 - Use automation tools to manage backup rotations, ensuring old backups are securely deleted after their retention period expires.
 - Regularly review and update the retention policy to align with evolving data retention needs or regulatory changes.
- Regularly Test Backup Integrity:**
 - Schedule periodic tests to restore data from backups to verify the data integrity and process efficacy.
 - Establish a testing protocol that includes both manual and automated testing to ensure that backups are not corrupted and can be reliably restored.
 - Document results of integrity tests and any issues encountered to continually improve your backup strategy.

References

- MariaDB Backup and Restore Overview
- MariaDB Backup Tool Overview

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08 Very high

Component: MariaDB

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04 High

NIST 800-53: CP-9 System Backup

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08

Very high

☐ Not tested

State: Recommended

Description:
Implement and regularly update secure backup procedures with encryption and access controls for PostgreSQL to protect backup data against unauthorized access and tampering. This control ensures that all backup files are encrypted using strong cryptographic standards (e.g., AES-256) and that access to these backups is restricted to authorized personnel only. Developers and DevOps engineers should integrate secure backup mechanisms into their PostgreSQL deployment process using automated backup tools and centralized management systems to enforce encryption and access policies consistently.
Implementation Steps:

Encrypt Backup Files:
Configure backup tools to encrypt PostgreSQL backup files using robust encryption algorithms before storage or transmission.

Enforce Access Controls:
Use OS-level permissions, cloud storage access policies, or centralized management tools to restrict backup file access to authorized users only.

Automate Backup Processes:
Integrate automated backup solutions that enforce encryption and access controls, ensuring consistent and secure backup routines across all PostgreSQL instances.

Monitor and Audit Backup Procedures:
Regularly review backup logs and conduct audits to verify that encryption and access controls are maintained and that backup data remains secure.

References:

- [PostgreSQL Backup and Restore Documentation](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04 High Not tested

- State: Recommended
- Description:
To improve the security of MariaDB backups, follow these steps to ensure that backups are encrypted, access is controlled, and a secure backup retention policy is implemented. Additionally, regularly test the integrity of these backups to ensure they can be restored when necessary.
 - Encrypt Database Backups:**
 - Use the built-in mariabackup tool for creating backups. During the backup process, employ encryption to protect backup data.
 - Select a robust encryption algorithm, such as AES-256, to ensure the data within backups is secured against unauthorized access.
 - Ensure the backup scripts or commands implement encryption flags, e.g., by using the --encrypt option in command executions.
 - Store Backups in Access-Controlled Locations:**
 - Designate specific, secure locations for storing backups. These locations should have stringent access controls, such as being in a separate network segment or specific access-controlled directory.
 - Ensure that only authorized personnel have access to these storage locations. Implement role-based access control (RBAC) to restrict access.
 - Maintain a log of access attempts and successful access to backup locations, allowing for auditing and alerting on unauthorized access attempts.
 - Implement Secure Backup Retention Policy:**
 - Define a retention policy that clearly specifies the duration for which backups should be retained. This policy should comply with regulatory and organizational requirements.
 - Use automation tools to manage backup rotations, ensuring old backups are securely deleted after their retention period expires.
 - Regularly review and update the retention policy to align with evolving data retention needs or regulatory changes.
 - Regularly Test Backup Integrity:**
 - Schedule periodic tests to restore data from backups to verify the data integrity and process efficacy.
 - Establish a testing protocol that includes both manual and automated testing to ensure that backups are not corrupted and can be reliably restored.
 - Document results of integrity tests and any issues encountered to continually improve your backup strategy.

- References
- [MariaDB Backup and Restore Overview](#)
 - [MariaDB Backup Tool Overview](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08 Very high

Component: MariaDB

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04 High

NIST 800-53: CP-10 System Recovery and Reconstitution

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08 Very high Not tested

- State: Recommended
 - Description:
Implement and regularly update secure backup procedures with encryption and access controls for PostgreSQL to protect backup data against unauthorized access and tampering. This control ensures that all backup files are encrypted using strong cryptographic standards (e.g., AES-256) and that access to these backups is restricted to authorized personnel only. Developers and DevOps engineers should integrate secure backup mechanisms into their PostgreSQL deployment process using automated backup tools and centralized management systems to enforce encryption and access policies consistently.
- Implementation Steps:**
- Encrypt Backup Files:**
Configure backup tools to encrypt PostgreSQL backup files using robust encryption algorithms before storage or transmission.
- Enforce Access Controls:**
Use OS-level permissions, cloud storage access policies, or centralized management tools to restrict backup file access to authorized users only.
- Automate Backup Processes:**
Integrate automated backup solutions that enforce encryption and access controls, ensuring consistent and secure backup routines across all PostgreSQL instances.
- Monitor and Audit Backup Procedures:**
Regularly review backup logs and conduct audits to verify that encryption and access controls are maintained and that backup data remains secure.
- References:**
- [PostgreSQL Backup and Restore Documentation](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04 High Not tested

- State: Recommended
- Description:
To improve the security of MariaDB backups, follow these steps to ensure that backups are encrypted, access is controlled, and a secure backup retention policy is implemented. Additionally, regularly test the integrity of these backups to ensure they can be restored when necessary.
 - Encrypt Database Backups:**
 - Use the built-in mariabackup tool for creating backups. During the backup process, employ encryption to protect backup data.
 - Select a robust encryption algorithm, such as AES-256, to ensure the data within backups is secured against unauthorized access.
 - Ensure the backup scripts or commands implement encryption flags, e.g., by using the --encrypt option in command executions.
 - Store Backups in Access-Controlled Locations:**
 - Designate specific, secure locations for storing backups. These locations should have stringent access controls, such as being in a separate network segment or specific access-controlled directory.

- Ensure that only authorized personnel have access to these storage locations. Implement role-based access control (RBAC) to restrict access.
- Maintain a log of access attempts and successful access to backup locations, allowing for auditing and alerting on unauthorized access attempts.

3. Implement Secure Backup Retention Policy:

- Define a retention policy that clearly specifies the duration for which backups should be retained. This policy should comply with regulatory and organizational requirements.
- Use automation tools to manage backup rotations, ensuring old backups are securely deleted after their retention period expires.
- Regularly review and update the retention policy to align with evolving data retention needs or regulatory changes.

4. Regularly Test Backup Integrity:

- Schedule periodic tests to restore data from backups to verify the data integrity and process efficacy.
- Establish a testing protocol that includes both manual and automated testing to ensure that backups are not corrupted and can be reliably restored.
- Document results of integrity tests and any issues encountered to continually improve your backup strategy.

References

- [MariaDB Backup and Restore Overview](#)
- [MariaDB Backup Tool Overview](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08

Very high

Component: MariaDB

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04

High

NIST 800-53: CP-8 Telecommunications Services

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08

Very high

Not tested

State: Recommended

Description:

Implement and regularly update secure backup procedures with encryption and access controls for PostgreSQL to protect backup data against unauthorized access and tampering. This control ensures that all backup files are encrypted using strong cryptographic standards (e.g., AES-256) and that access to these backups is restricted to authorized personnel only. Developers and DevOps engineers should integrate secure backup mechanisms into their PostgreSQL deployment process using automated backup tools and centralized management systems to enforce encryption and access policies consistently.

Implementation Steps:

Encrypt Backup Files:

Configure backup tools to encrypt PostgreSQL backup files using robust encryption algorithms before storage or transmission.

Enforce Access Controls:

Use OS-level permissions, cloud storage access policies, or centralized management tools to restrict backup file access to authorized users only.

Automate Backup Processes:

Integrate automated backup solutions that enforce encryption and access controls, ensuring consistent and secure backup routines across all PostgreSQL instances.

Monitor and Audit Backup Procedures:

Regularly review backup logs and conduct audits to verify that encryption and access controls are maintained and that backup data remains secure.

References:

- [PostgreSQL Backup and Restore Documentation](#)

Component: MariaDB

1 Recommended countermeasures

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04

High

Not tested

State: Recommended

Description:

To improve the security of MariaDB backups, follow these steps to ensure that backups are encrypted, access is controlled, and a secure backup retention policy is implemented. Additionally, regularly test the integrity of these backups to ensure they can be restored when necessary.

1. Encrypt Database Backups:

- Use the built-in mariabackup tool for creating backups. During the backup process, employ encryption to protect backup data.
- Select a robust encryption algorithm, such as AES-256, to ensure the data within backups is secured against unauthorized access.
- Ensure the backup scripts or commands implement encryption flags, e.g., by using the --encrypt option in command executions.

2. Store Backups in Access-Controlled Locations:

- Designate specific, secure locations for storing backups. These locations should have stringent access controls, such as being in a separate network segment or specific access-controlled directory.
- Ensure that only authorized personnel have access to these storage locations. Implement role-based access control (RBAC) to restrict access.
- Maintain a log of access attempts and successful access to backup locations, allowing for auditing and alerting on unauthorized access attempts.

3. Implement Secure Backup Retention Policy:

- Define a retention policy that clearly specifies the duration for which backups should be retained. This policy should comply with regulatory and organizational requirements.
- Use automation tools to manage backup rotations, ensuring old backups are securely deleted after their retention period expires.
- Regularly review and update the retention policy to align with evolving data retention needs or regulatory changes.

4. Regularly Test Backup Integrity:

- Schedule periodic tests to restore data from backups to verify the data integrity and process efficacy.
- Establish a testing protocol that includes both manual and automated testing to ensure that backups are not corrupted and can be reliably restored.
- Document results of integrity tests and any issues encountered to continually improve your backup strategy.

References

- [MariaDB Backup and Restore Overview](#)
- [MariaDB Backup Tool Overview](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement secure backup procedures with encryption and access controls C-POSTGRESQL-CNT-POSTGRES-08

Very high

Component: MariaDB

Rec1. Secure backups with encryption and access controls C-MARIADB-CNT-04

High

NIST 800-53: SI-13 Predictable Failure Prevention

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Implement secure backup procedures with encryption and access controls

C-POSTGRESQL-CNT-POSTGRES-08

Very high

☐ Not tested

State: ☒ Recommended

Description:
Implement and regularly update secure backup procedures with encryption and access controls for PostgreSQL to protect backup data against unauthorized access and tampering. This control ensures that all backup files are encrypted using strong cryptographic standards (e.g., AES-256) and that access to these backups is restricted to authorized personnel only. Developers and DevOps engineers should integrate secure backup mechanisms into their PostgreSQL deployment process using automated backup tools and centralized management systems to enforce encryption and access policies consistently.
Implementation Steps:
Encrypt Backup Files:
Configure backup tools to encrypt PostgreSQL backup files using robust encryption algorithms before storage or transmission.
Enforce Access Controls:
Use OS-level permissions, cloud storage access policies, or centralized management tools to restrict backup file access to authorized users only.
Automate Backup Processes:
Integrate automated backup solutions that enforce encryption and access controls, ensuring consistent and secure backup routines across all PostgreSQL instances.
Monitor and Audit Backup Procedures:
Regularly review backup logs and conduct audits to verify that encryption and access controls are maintained and that backup data remains secure.
References:

- PostgreSQL Backup and Restore Documentation

Component: MariaDB

1 Recommended countermeasures

Rec1. Secure backups with encryption and access controls

C-MARIADB-CNT-04

High

☐ Not tested

State: ☒ Recommended

Description:
To improve the security of MariaDB backups, follow these steps to ensure that backups are encrypted, access is controlled, and a secure backup retention policy is implemented. Additionally, regularly test the integrity of these backups to ensure they can be restored when necessary.

- Encrypt Database Backups:**
 - Use the built-in mariabackup tool for creating backups. During the backup process, employ encryption to protect backup data.
 - Select a robust encryption algorithm, such as AES-256, to ensure the data within backups is secured against unauthorized access.
 - Ensure the backup scripts or commands implement encryption flags, e.g., by using the --encrypt option in command executions.
- Store Backups in Access-Controlled Locations:**
 - Designate specific, secure locations for storing backups. These locations should have stringent access controls, such as being in a separate network segment or specific access-controlled directory.
 - Ensure that only authorized personnel have access to these storage locations. Implement role-based access control (RBAC) to restrict access.
 - Maintain a log of access attempts and successful access to backup locations, allowing for auditing and alerting on unauthorized access attempts.
- Implement Secure Backup Retention Policy:**
 - Define a retention policy that clearly specifies the duration for which backups should be retained. This policy should comply with regulatory and organizational requirements.
 - Use automation tools to manage backup rotations, ensuring old backups are securely deleted after their retention period expires.
 - Regularly review and update the retention policy to align with evolving data retention needs or regulatory changes.
- Regularly Test Backup Integrity:**
 - Schedule periodic tests to restore data from backups to verify the data integrity and process efficacy.
 - Establish a testing protocol that includes both manual and automated testing to ensure that backups are not corrupted and can be reliably restored.
 - Document results of integrity tests and any issues encountered to continually improve your backup strategy.

References

- MariaDB Backup and Restore Overview
- MariaDB Backup Tool Overview

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Implement secure backup procedures with encryption and access controls

C-POSTGRESQL-CNT-POSTGRES-08

Very high

Component: MariaDB

Rec1. Secure backups with encryption and access controls

C-MARIADB-CNT-04

High

NIST 800-53: PM-21 Accounting of Disclosures

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce secure file permissions on PostgreSQL database files

C-POSTGRESQL-CNT-POSTGRES-06

High

☐ Not tested

State: ☒ Recommended

Description:
Implement and regularly update strict file system permissions for PostgreSQL database files to ensure that only authorized users and processes can access or modify them. This control minimizes the risk of unauthorized data tampering and exposure by using OS-level security settings (such as chmod/chown on Linux) to restrict access to sensitive files. Developers and DevOps engineers should integrate these practices into their deployment procedures, using centralized configuration management tools to enforce and monitor secure file permissions across all database servers.
References:

- OWASP Secure Coding Practices - Access Control

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Report: SpaceHub

Compliance report - 2026-03-25T16:40:26.994993079Z

174 of 182

Component: PostgreSQL

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High

NIST 800-53: PM-24 Data Integrity Board

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High Not tested

State: Recommended

Description:
Implement and regularly update strict file system permissions for PostgreSQL database files to ensure that only authorized users and processes can access or modify them. This control minimizes the risk of unauthorized data tampering and exposure by using OS-level security settings (such as chmod/chown on Linux) to restrict access to sensitive files. Developers and DevOps engineers should integrate these practices into their deployment procedures, using centralized configuration management tools to enforce and monitor secure file permissions across all database servers.
References:

- OWASP Secure Coding Practices - Access Control

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High

NIST 800-53: PM-20 Dissemination of Privacy Program Information

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High Not tested

State: Recommended

Description:
Implement and regularly update strict file system permissions for PostgreSQL database files to ensure that only authorized users and processes can access or modify them. This control minimizes the risk of unauthorized data tampering and exposure by using OS-level security settings (such as chmod/chown on Linux) to restrict access to sensitive files. Developers and DevOps engineers should integrate these practices into their deployment procedures, using centralized configuration management tools to enforce and monitor secure file permissions across all database servers.
References:

- OWASP Secure Coding Practices - Access Control

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High

NIST 800-53: PM-25 Minimization of Personally Identifiable Information Used in Testing, Training, and Research

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High Not tested

State: Recommended

Description:
Implement and regularly update strict file system permissions for PostgreSQL database files to ensure that only authorized users and processes can access or modify them. This control minimizes the risk of unauthorized data tampering and exposure by using OS-level security settings (such as chmod/chown on Linux) to restrict access to sensitive files. Developers and DevOps engineers should integrate these practices into their deployment procedures, using centralized configuration management tools to enforce and monitor secure file permissions across all database servers.
References:

- OWASP Secure Coding Practices - Access Control

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High

NIST 800-53: PM-22 Personally Identifiable Information Quality Management

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High Not tested

State: Recommended

Description:
Implement and regularly update strict file system permissions for PostgreSQL database files to ensure that only authorized users and processes can access or modify them. This control minimizes the risk of unauthorized data tampering and exposure by using OS-level security settings (such as chmod/chown on Linux) to restrict access to sensitive files. Developers and DevOps engineers should integrate these practices into their deployment procedures, using centralized configuration management tools to enforce and monitor secure file permissions across all database servers.
References:

- OWASP Secure Coding Practices - Access Control

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High

NIST 800-53: PM-19 Privacy Program Leadership Role

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High Not tested

State: Recommended

Description:
Implement and regularly update strict file system permissions for PostgreSQL database files to ensure that only authorized users and processes can access or modify them. This control minimizes the risk of unauthorized data tampering and exposure by using OS-level security settings (such as chmod/chown on Linux) to restrict access to sensitive files. Developers and DevOps engineers should integrate these practices into their deployment procedures, using centralized configuration management tools to enforce and monitor secure file permissions across all database servers.
References:

- OWASP Secure Coding Practices - Access Control

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High

NIST 800-53: PM-18 Privacy Program Plan

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High Not tested

State: Recommended

Description:
Implement and regularly update strict file system permissions for PostgreSQL database files to ensure that only authorized users and processes can access or modify them. This control minimizes the risk of unauthorized data tampering and exposure by using OS-level security settings (such as chmod/chown on Linux) to restrict access to sensitive files. Developers and DevOps engineers should integrate these practices into their deployment procedures, using centralized configuration management tools to enforce and monitor secure file permissions across all database servers.
References:

- OWASP Secure Coding Practices - Access Control

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High

NIST 800-53: PM-27 Privacy Reporting

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce secure file permissions on PostgreSQL database files

C-POSTGRESQL-CNT-POSTGRES-06

High

Not tested

State: Recommended

Description:
Implement and regularly update strict file system permissions for PostgreSQL database files to ensure that only authorized users and processes can access or modify them. This control minimizes the risk of unauthorized data tampering and exposure by using OS-level security settings (such as chmod/chown on Linux) to restrict access to sensitive files. Developers and DevOps engineers should integrate these practices into their deployment procedures, using centralized configuration management tools to enforce and monitor secure file permissions across all database servers.
References:

- OWASP Secure Coding Practices - Access Control

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enforce secure file permissions on PostgreSQL database files

C-POSTGRESQL-CNT-POSTGRES-06

High

NIST 800-53: PT-2 Authority to Process Personally Identifiable Information

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce secure file permissions on PostgreSQL database files

C-POSTGRESQL-CNT-POSTGRES-06

High

Not tested

State: Recommended

Description:
Implement and regularly update strict file system permissions for PostgreSQL database files to ensure that only authorized users and processes can access or modify them. This control minimizes the risk of unauthorized data tampering and exposure by using OS-level security settings (such as chmod/chown on Linux) to restrict access to sensitive files. Developers and DevOps engineers should integrate these practices into their deployment procedures, using centralized configuration management tools to enforce and monitor secure file permissions across all database servers.
References:

- OWASP Secure Coding Practices - Access Control

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enforce secure file permissions on PostgreSQL database files

C-POSTGRESQL-CNT-POSTGRES-06

High

NIST 800-53: PT-8 Computer Matching Requirements

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce secure file permissions on PostgreSQL database files

C-POSTGRESQL-CNT-POSTGRES-06

High

Not tested

State: Recommended

Description:
Implement and regularly update strict file system permissions for PostgreSQL database files to ensure that only authorized users and processes can access or modify them. This control minimizes the risk of unauthorized data tampering and exposure by using OS-level security settings (such as chmod/chown on Linux) to restrict access to sensitive files. Developers and DevOps engineers should integrate these practices into their deployment procedures, using centralized configuration management tools to enforce and monitor secure file permissions across all database servers.
References:

- OWASP Secure Coding Practices - Access Control

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enforce secure file permissions on PostgreSQL database files

C-POSTGRESQL-CNT-POSTGRES-06

High

NIST 800-53: PT-4 Consent

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06

High

Not tested

State: Recommended

Description:
Implement and regularly update strict file system permissions for PostgreSQL database files to ensure that only authorized users and processes can access or modify them. This control minimizes the risk of unauthorized data tampering and exposure by using OS-level security settings (such as chmod/chown on Linux) to restrict access to sensitive files. Developers and DevOps engineers should integrate these practices into their deployment procedures, using centralized configuration management tools to enforce and monitor secure file permissions across all database servers.
References:

- OWASP Secure Coding Practices - Access Control

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High

NIST 800-53: PT-3 Personally Identifiable Information Processing Purposes

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06

High

Not tested

State: Recommended

Description:
Implement and regularly update strict file system permissions for PostgreSQL database files to ensure that only authorized users and processes can access or modify them. This control minimizes the risk of unauthorized data tampering and exposure by using OS-level security settings (such as chmod/chown on Linux) to restrict access to sensitive files. Developers and DevOps engineers should integrate these practices into their deployment procedures, using centralized configuration management tools to enforce and monitor secure file permissions across all database servers.
References:

- OWASP Secure Coding Practices - Access Control

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High

NIST 800-53: PT-1 Policy and Procedures

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06

High

Not tested

State: Recommended

Description:
Implement and regularly update strict file system permissions for PostgreSQL database files to ensure that only authorized users and processes can access or modify them. This control minimizes the risk of unauthorized data tampering and exposure by using OS-level security settings (such as chmod/chown on Linux) to restrict access to sensitive files. Developers and DevOps engineers should integrate these practices into their deployment procedures, using centralized configuration management tools to enforce and monitor secure file permissions across all database servers.
References:

- OWASP Secure Coding Practices - Access Control

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High

NIST 800-53: PT-5 Privacy Notice

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce secure file permissions on PostgreSQL database files

C-POSTGRESQL-CNT-POSTGRES-06

High

Not tested

State: Recommended

Description:
Implement and regularly update strict file system permissions for PostgreSQL database files to ensure that only authorized users and processes can access or modify them. This control minimizes the risk of unauthorized data tampering and exposure by using OS-level security settings (such as chmod/chown on Linux) to restrict access to sensitive files. Developers and DevOps engineers should integrate these practices into their deployment procedures, using centralized configuration management tools to enforce and monitor secure file permissions across all database servers.
References:

- OWASP Secure Coding Practices - Access Control

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High

NIST 800-53: PT-7 Specific Categories of Personally Identifiable Information

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce secure file permissions on PostgreSQL database files

C-POSTGRESQL-CNT-POSTGRES-06

High

Not tested

State: Recommended

Description:
Implement and regularly update strict file system permissions for PostgreSQL database files to ensure that only authorized users and processes can access or modify them. This control minimizes the risk of unauthorized data tampering and exposure by using OS-level security settings (such as chmod/chown on Linux) to restrict access to sensitive files. Developers and DevOps engineers should integrate these practices into their deployment procedures, using centralized configuration management tools to enforce and monitor secure file permissions across all database servers.
References:

- OWASP Secure Coding Practices - Access Control

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High

NIST 800-53: PT-6 System of Records Notice

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce secure file permissions on PostgreSQL database files

C-POSTGRESQL-CNT-POSTGRES-06

High

Not tested

State: Recommended

Description:
Implement and regularly update strict file system permissions for PostgreSQL database files to ensure that only authorized users and processes can access or modify them. This control minimizes the risk of unauthorized data tampering and exposure by using OS-level security settings (such as chmod/chown on Linux) to restrict access to sensitive files. Developers and DevOps engineers should integrate these practices into their deployment procedures, using centralized configuration management tools to enforce and monitor secure file permissions across all database servers.
References:

- OWASP Secure Coding Practices - Access Control

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 High

NIST 800-53: SI-19 De-identification

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce secure file permissions on PostgreSQL database files

C-POSTGRESQL-CNT-POSTGRES-06

High

Not tested

- State: 🟢 Recommended
- Description:
Implement and regularly update strict file system permissions for PostgreSQL database files to ensure that only authorized users and processes can access or modify them. This control minimizes the risk of unauthorized data tampering and exposure by using OS-level security settings (such as chmod/chown on Linux) to restrict access to sensitive files. Developers and DevOps engineers should integrate these practices into their deployment procedures, using centralized configuration management tools to enforce and monitor secure file permissions across all database servers.
References:
 - [OWASP Secure Coding Practices - Access Control](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 🔴 High

NIST 800-53: SI-18 Personally Identifiable Information Quality Operations

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 🔴 High ⬜ Not tested

- State: 🟢 Recommended
- Description:
Implement and regularly update strict file system permissions for PostgreSQL database files to ensure that only authorized users and processes can access or modify them. This control minimizes the risk of unauthorized data tampering and exposure by using OS-level security settings (such as chmod/chown on Linux) to restrict access to sensitive files. Developers and DevOps engineers should integrate these practices into their deployment procedures, using centralized configuration management tools to enforce and monitor secure file permissions across all database servers.
References:
 - [OWASP Secure Coding Practices - Access Control](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Enforce secure file permissions on PostgreSQL database files C-POSTGRESQL-CNT-POSTGRES-06 🔴 High

NIST 800-53: SC-22 Architecture and Provisioning for Name/address Resolution Service

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Harden postgresql configuration and restrict network access C-POSTGRESQL-CNT-POSTGRES-04 🔴 Very high ⬜ Not tested

- State: 🟢 Recommended
- Description:
Implement and regularly update a hardened PostgreSQL configuration and restrict network access to protect sensitive data and prevent unauthorized access. This control ensures that PostgreSQL is deployed with secure settings—disabling unnecessary features and default behaviors—and that network access is limited to trusted hosts using firewall rules, security groups, or VLAN segmentation. Developers and DevOps engineers should integrate these practices into the PostgreSQL deployment and maintenance process, ensuring that configurations are reviewed regularly and network access policies are enforced via centralized management tools.
Implementation Steps:
Harden PostgreSQL Configuration:
Review and update the PostgreSQL configuration (e.g., postgresql.conf and pg_hba.conf) to disable unnecessary services, enforce strong security parameters, and enable robust logging and auditing.
Restrict Network Access:
Deploy PostgreSQL in private network segments and use firewall rules or cloud security groups to allow access only from authorized IP addresses or subnets.
Regularly Audit and Update:
Perform periodic security audits and configuration reviews to ensure that both the database settings and network access policies remain aligned with evolving security best practices.
References:
 - [PostgreSQL Security Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Harden postgresql configuration and restrict network access C-POSTGRESQL-CNT-POSTGRES-04 🔴 Very high

NIST 800-53: SC-7 Boundary Protection

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Harden postgresql configuration and restrict network access C-POSTGRESQL-CNT-POSTGRES-04 Very high Not tested

- State: Recommended
- Description: Implement and regularly update a hardened PostgreSQL configuration and restrict network access to protect sensitive data and prevent unauthorized access. This control ensures that PostgreSQL is deployed with secure settings—disabling unnecessary features and default behaviors—and that network access is limited to trusted hosts using firewall rules, security groups, or VLAN segmentation. Developers and DevOps engineers should integrate these practices into the PostgreSQL deployment and maintenance process, ensuring that configurations are reviewed regularly and network access policies are enforced via centralized management tools.
Implementation Steps:
Harden PostgreSQL Configuration: Review and update the PostgreSQL configuration (e.g., postgresql.conf and pg_hba.conf) to disable unnecessary services, enforce strong security parameters, and enable robust logging and auditing.
Restrict Network Access: Deploy PostgreSQL in private network segments and use firewall rules or cloud security groups to allow access only from authorized IP addresses or subnets.
Regularly Audit and Update: Perform periodic security audits and configuration reviews to ensure that both the database settings and network access policies remain aligned with evolving security best practices.
References:
 - [PostgreSQL Security Documentation](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Harden postgresql configuration and restrict network access C-POSTGRESQL-CNT-POSTGRES-04 Very high

NIST 800-53: SI-10 INFORMATION INPUT VALIDATION

Non-compliant countermeasures

"Non-compliant" are those countermeasures that are recommended ("Rec"), rejected ("Rej") or not applicable ("N/A") in the model, but required by the standard. Required ("ReqF") countermeasures which tests have failed are also shown as non-compliant ones.

Component: PostgreSQL

1 Recommended countermeasures

Rec1. Use parameterized queries and validate inputs C-POSTGRESQL-CNT-POSTGRES-02 High Not tested

- State: Recommended
- Description: Implement and regularly update secure coding practices for PostgreSQL by using parameterized queries and validating inputs to prevent injection attacks and ensure data integrity. This control ensures that all user-supplied data is properly sanitized and that SQL statements are constructed safely, reducing the risk of SQL injection and unauthorized data manipulation. Developers and DevOps engineers should integrate input validation and parameterized query techniques into their development workflows and conduct regular code reviews to maintain robust security practices.
Implementation Steps:
Adopt Parameterized Queries: Ensure that all database queries are constructed using parameterized queries or prepared statements to separate query logic from data inputs, preventing injection of malicious SQL.
Validate and Sanitize Inputs: Integrate input validation routines to check user-supplied data against expected formats and sanitize any potentially dangerous characters or patterns.
Integrate Secure Coding Practices: Incorporate secure coding guidelines into the development process and use automated tools to scan for injection vulnerabilities and improper input handling.
Conduct Regular Code Reviews and Testing: Perform periodic code reviews and security testing (e.g., penetration testing and static analysis) to verify that input validation and parameterization are consistently applied.
References:
 - [OWASP SQL Injection Prevention Cheat Sheet](#)

Recommended countermeasures

Below are the recommended countermeasures ("Rec") by component and threat for standard reference.

Component: PostgreSQL

Rec1. Use parameterized queries and validate inputs C-POSTGRESQL-CNT-POSTGRES-02 High

End of Compliance report